



FLORIDA INSTITUTE OF TECHNOLOGY

Dr. Gutierrez's Robotics Laboratory

Melbourne, Florida, USA

ENGINEERING INTERNSHIP REPORT

CESI FISA Généraliste, 4th Year | 2026

LEO ROVER MISSION CONTROL

Full-Stack Autonomous Ground Vehicle Navigation System

Vision Pipeline • Autonomous FSM • WebSocket Bridge • Operator Cockpit

Author: Nicolas Gardon

Supervisor: Dr. Gutierrez (FIT)

Academic Year: 2025–2026

Institution: CESI Campus de Quetigny (Côte-d'Or, 21)

August 4, 2026

Declaration

I hereby declare that this report, entitled *LEO Rover Mission Control: Full-Stack Autonomous Ground Vehicle Navigation System*, is the result of my own work carried out during my engineering internship at the Florida Institute of Technology, under the supervision of Dr. Gutierrez (FIT), except where explicitly referenced.

All sources of information, published or unpublished, have been acknowledged in accordance with standard academic practice. No part of this report has been submitted for any other degree or professional qualification.

Nicolas Gardon

CESI FISA Généraliste, 4th Year

Academic Year 2025–2026

Acknowledgements

I would like to express my sincere gratitude to Dr. Gutierrez for welcoming me into his robotics laboratory at the Florida Institute of Technology and for his guidance throughout this internship on the Carolus multi-robot localization research program. His insistence on rigorous, first-principles engineering shaped the methodology followed throughout this work.

I would also like to thank the members of the robotics laboratory for their availability during hardware integration sessions, and for the many informal exchanges that helped resolve some of the more elusive debugging problems documented in Chapter 6.

Finally, I thank CESI for making this international internship opportunity possible as part of the FISA Généraliste 4th-year curriculum.

Abstract

This report documents the design, implementation, and validation of the LEO Rover Mission Control system, a full-stack autonomous mission control platform developed during an engineering internship at the Florida Institute of Technology’s robotics laboratory, under the supervision of Dr. Gutierrez, as part of the Carolus multi-robot indoor-localization research program.

At the outset of the internship, the inherited system suffered from five interrelated architectural deficiencies: a blocking checkerboard detection call that throttled the vision pipeline from 10 Hz to 3 Hz; a systematic area-based LED anchor heuristic that always selected ceiling reflections over the true beacon, yielding a 0% detection rate; a checkerboard landmark detector effectively disabled by a 1-second gating timer; an instant-trip heartbeat failsafe caused by stale state persisting across browser sessions; and non-deterministic network interface selection that silently discarded all operator commands. Individually resolvable, their combination rendered the autonomous state machine entirely untestable.

The work resolved these deficiencies through systematic re-engineering across seven dimensions: a multiprocess vision architecture isolating OpenCV workloads in a spawn-context subprocess with bounded IPC queues; a density-based LED clustering algorithm with a formally provable correctness guarantee under an empirically validated isolation hypothesis; promotion of `findChessboardCornersSB` to the primary detection path in all code paths; a session-aware heartbeat stale guard; deterministic `ROS_IP` configuration; a complete redesign of the autonomous finite-state machine into six states with dual-frame (local/world) odometry and persistent map markers; and a browser-based operator cockpit built around a NASA/FIT-inspired glassmorphic design language, a `requestAnimationFrame`-decoupled Canvas 2D vision overlay, an $O(1)$ ring-buffer trajectory map, and a six-layer safety architecture.

Validation demonstrates a 98.7% LED beacon detection rate and 100% checkerboard detection rate (both with 95% confidence intervals exceeding the 95% target objective at $p < 0.05$), a reduction in vision processing latency from 200–330 ms to 5–20 ms, zero false failsafe trips across

repeated reconnection cycles, and a fully autonomous 45-minute, 87.3 m patrol mission with five beacons logged, zero operator interventions, and a 14 cm world-frame return-to-base position error. A reliability analysis combining Markov chain modeling and fault tree analysis estimates the probability of an uncommanded, unmonitored motion event at $\sim 4 \times 10^{-17}$ per operational hour, nine orders of magnitude below the IEC 61508 SIL 4 threshold, though this estimate is offered as a qualitative robustness indicator rather than a formal certification claim.

The resulting system is deployed and operational in the FIT robotics laboratory and serves as the reference implementation for the LEO Rover platform within the broader Carolus comparative study of the LIMO Rover, LEO Rover, and DJI RoboMaster S1 platforms.

Contents

Declaration	1
Acknowledgements	2
Abstract	3
	Page
1 Introduction	14
1.1 Preamble: The Rise of Autonomous Ground Robotics	14
1.2 Research Context: Florida Institute of Technology	15
1.3 Problem Statement and Scope of the Work	16
1.4 GPS-Free Indoor Navigation: Theoretical Background	17
1.5 Engineering Challenges in Embedded Control Interfaces	19
1.6 Internship Objectives	21
1.7 Methodology: RCA, TDD, and DMAIC	22
1.8 Report Structure	22
2 State of the Art	24
2.1 Autonomous Mobile Ground Robotics: Historical Perspective	24
2.2 The ROS Ecosystem: Architecture and Rationale	25
2.3 Computer Vision for Fiducial Detection: Related Work	25
2.4 Web-Based HMI for Robotic Systems	27
2.5 Safety Architecture in Autonomous Systems: Standards Overview	28
2.6 Synthesis: Positioning of the Present Work	29
3 System Architecture and Software Infrastructure	30
3.1 Full-Stack Overview and Deployment Topology	30

3.2	Complete Thread Architecture	32
3.3	ROS Topic Graph	32
3.4	Telemetry JSON Schema (~60 Fields)	33
3.5	Multiprocess Vision Architecture	37
3.6	Launch Infrastructure: <code>start_leo.sh</code>	38
3.7	Configuration Constants Reference	39
3.8	AutoLogbook Event System	42
4	Computer Vision and Autonomous Navigation	44
4.1	LED Beacon Detection: Theoretical Foundations	44
4.2	Checkerboard Landmark Detection	46
4.3	EMA Smoothing: Signal Processing Analysis	48
4.4	Autonomous FSM: Complete Architecture	49
4.5	Dual-Frame Odometry: Mathematical Derivation	53
4.6	Depth-Based Obstacle Detection	55
4.7	Performance Summary: Before and After Architecture Redesign	56
5	Mathematical Foundations of Vision and Navigation	57
5.1	Camera Projection Model and Calibration Theory	57
5.2	Image Processing Theory: Thresholding and Morphology	60
5.3	EMA Filter: Complete Signal Processing Analysis	64
5.4	Density-Based Clustering: Formal Analysis	67
5.5	Perspective-n-Point: Homography and SVD Decomposition	69
5.6	Distance Estimation: Full Error Propagation Analysis	71
5.7	P-Controller Stability: Ziegler–Nichols Analysis	73
5.8	Wheel Odometry Error Model	75
5.9	Stereo Depth Theory and Obstacle Detection	78
5.10	Autonomous FSM: Mathematical Completeness Analysis	79
5.11	Vision Pipeline: Complete Data Flow Timing Analysis	81

6	UI/UX Engineering: The Operator Cockpit	85
6.1	Design Philosophy and Visual Identity	85
6.2	Bento Grid Layout System	87
6.3	Canvas 2D Vision Overlay: Architecture	88
6.4	LERP Smoothing for Visual Stability	92
6.5	FSM State Visual Encoding	93
6.6	Alert System Architecture	95
6.7	2D Trajectory Map: Ring Buffer Architecture	97
6.8	Network Latency Meter	99
6.9	Three.js Decorative 3D Elements	100
6.10	Internationalization (i18n) System	102
7	Mathematical Foundations of UI/UX Engineering	104
7.1	Color Theory and Perceptual Engineering	104
7.2	LERP and Interpolation: Mathematical Taxonomy	106
7.3	Canvas 2D Rendering: Transform Mathematics	108
7.4	WebSocket Protocol: Engineering Model	109
7.5	requestAnimationFrame Scheduling: Browser Rendering Pipeline	110
7.6	Ring Buffer: Formal Analysis	111
7.7	Alert System: Formal Specification and Hysteresis Analysis	112
7.8	Multi-Page Cockpit Architecture: Data Flow and Algorithmic Complexity	114
8	Safety, Reliability, and Maintenance	119
8.1	Six-Layer Safety Architecture	119
8.2	Layer P1: Emergency Stop	121
8.3	Layer P2: Heartbeat Deadman Switch	122
8.4	Layer P4: Manual Deadman	123
8.5	Layer P5: Camera Watchdog	124
8.6	Layer P6: Velocity Clamping	124
8.7	Mission Persistence: AutoLogbook	125
8.8	Map Marker Persistence	126

9	Mathematical Safety Engineering	127
9.1	Reliability Theory: Markov Chain Model	127
9.2	Fault Tree Analysis	129
9.3	Safety Integrity Level Verification	130
9.4	Heartbeat Protocol: Information-Theoretic Analysis	131
9.5	Velocity Clamping: Lyapunov Stability Analysis	132
9.6	_safe_stop() Delivery Reliability	133
9.7	Dual-Frame Odometry: SE(2) Group Theory	133
10	Performance Analysis and Debugging Case Studies	135
10.1	Debugging Methodology: Root Cause Analysis Framework	135
10.2	Case Study 1: LED False Positive (Ceiling Reflection)	136
10.3	Case Study 2: Checkerboard Detection Failure	138
10.4	Case Study 3: Heartbeat Instant-Trip	139
10.5	Case Study 4: ROS_IP Non-Determinism	140
10.6	Vision Pipeline Performance Characterization	141
10.7	FSM Behavioral Validation	143
10.8	UI Performance Characterization	144
11	Statistical Performance Analysis	145
11.1	Detection Rate Statistical Analysis	145
11.2	Latency Distribution Modeling	146
11.3	EMA Parameter Selection: Optimal Estimation Theory	147
11.4	Power Spectral Density of Centroid Trajectories	149
11.5	Debugging Case Studies: Quantitative Analysis	149
11.6	System-Level Performance: End-to-End Mission Metrics	152
12	The July 2026 Estimation Campaign: Tightly-Coupled Multisensor Fusion	155
12.1	Estimation architecture	155
12.2	Sensor chain, transport, and integrity guards	160
12.3	Calibration	164

12.4	Hardware modification: rigid high-performance wheels	170
12.5	Mission software evolution	171
12.6	Operator platform and public API	184
12.7	Verification and validation	197
12.8	Field session, July 21: fault isolation and an architectural revision	199
12.9	Field session, July 22: closing the beacon-approach loop, and a first application of continuous control	211
12.10	Field session, July 23: Return-to-Base rebuilt, a source-selection interface simpli- fied after two iterations, and an initial-divergence guard	220
12.11	Field session, July 24: a blind reversal, a silent approach, and a deliberate simplifi- cation of reactive avoidance	223
12.12	Field session, July 27: an offline trajectory-comparison pipeline and cockpit instru- mentation	246
12.13	Field session, July 28: a starved serial link, a single boolean worth four orders of magnitude, and an accelerometer out of spec	262
12.14	Field session, July 29: trajectory alignment as a measurement, and the first ground- truth calibration	265
12.15	Field session, July 30: a thermal operating envelope, a native absolute-error archi- tecture, and the calibration blocker that gates both	274
12.16	Open items registry	286
13	Conclusion and Perspectives	301
13.1	Achievement Against Stated Objectives	301
13.2	Extension Beyond the Original Objectives: The July 2026 Estimation Campaign . .	302
13.3	Synthesis of Technical Contributions	304
13.4	Lessons Learned	305
13.5	Future Development Roadmap	306
13.6	Closing Statement	311
A	Technical Glossary	313

B	Hardware Specifications	317
B.1	LEO Rover Platform	317
B.2	Intel RealSense D455 Camera	317
B.3	Operator Workstation	319
C	Bibliography, Figure Index, and Notation	321
C.1	Annotated Bibliography	321
C.2	Index of Tables	326
C.3	Index of Code Listings	327
C.4	Mathematical Notation and Symbols	329
D	Reproduction guide for future students	333
	The short version: what will cost you a week	333
D.1	Step –1: the absolute ground zero	334
D.2	Start here: what this machine actually is	336
D.3	Upstream sources: what is ours and what is not	341
D.4	Building the workstation from a bare machine	341
D.5	Network configuration	346
D.6	Robot-side configuration	355
D.7	The code we wrote, node by node	357
D.8	Configuration files, and the traps inside them	361
D.9	Launch file architecture	372
D.10	Launching the system, step by step	376
D.11	Reproducing the experiments	381
D.12	Publishing the report	389
D.13	Troubleshooting and lessons learned	390
D.14	Where to go next	402
	References	410

List of Tables

1.1	Comparative survey of indoor localization paradigms	18
1.2	End-to-end operator latency budget	20
2.1	ROS Noetic vs. ROS 2 Humble feature comparison	26
3.2	Complete ROS topic graph	32
3.1	leo_backend.py daemon thread inventory	34
3.3	Complete system constants reference with engineering justification	39
3.4	AutoLogbook event taxonomy	43
4.1	Complete FSM transition table	49
4.2	System performance before and after architectural redesign	56
5.1	Vision pipeline stage-by-stage timing analysis	82
6.1	FSM state banner color encoding	94
7.1	Assets loaded per page. roslib is vendored, not shared project code.	118
8.1	Six safety layer stack with priority, mechanism, and timing	120
9.1	Failure rate estimation for each safety layer	128
10.1	Vision pipeline performance measurements before and after redesign	142
10.2	FSM behavioral validation test results	143
10.3	Cockpit rendering performance on three hardware configurations	144
11.1	Checkerboard detection rate and latency vs. scale factor	151
11.2	Full autonomous mission statistics (45-minute validation run)	152
12.1	Public API surface of the platform.	185

12.2	Transport comparison, same rover, same campus network, same navigation stack. WireGuard fully recovers the local-network baseline performance; sshuttle does not, under real multi-connection ROS load.	192
12.3	Expected ticks between false triggers, Eq. (12.25), for the old ($n = 3$) and new ($n = 6$) debounce across the per-tick failure-probability range fitted against <code>watchdog.log</code> . The ratio converges to exactly $n_{\text{new}}/n_{\text{old}} = 2$ as $p \rightarrow 1$, a useful internal check on the derivation.	196
12.4	T1.1 A/B: the only change between columns is the camera–IMU extrinsic seed (Eq. 12.13). σ reduced $5.2\times$ on Z; the slope crosses the bounded-noise criterion. .	198
12.5	LeoCore controller board interfaces (FictionLab official specification, transcribed verbatim July 24). This project’s software and documentation refer to the same board as “CORE2” throughout; both names denote the same physical controller. . .	225
12.6	RTB_START/RTB_COMPLETE pairs from <code>auto_entries.json</code> , in mission order, spanning the defect and its fix. Every attempt before the July 24 guard (§12.11.2) was abandoned without completing; every attempt after it either completed or was correctly interrupted by the new guard itself.	226
12.7	Every AUTO-mode branch that can command a nonzero forward speed, audited July 24. “Hard” = a discrete <code>_obstacle_flag</code> check gating the branch; “Graduated” = Eq. 12.45 applied to the commanded speed.	231
12.8	Symptoms observed in the shared log during this investigation, and their actual origin once checked against each package’s own source.	240
12.9	First ground-truth calibration pass, July 29. Factors are $g = \text{reality}/\text{belief}$ as defined in Eq. 12.86; a factor below unity means the source over-reports. The gyroscope’s near-zero antisymmetric component identifies a genuine scale error, while the wheel odometry’s is an order of magnitude larger and is mechanical in origin. Single pass per direction: indicative, and superseded by the repeated campaign recorded as item 36.	273
12.10	Measured state at the two ends of the thermal envelope. The right column is not a degradation of the left one: it is a different regime, reached across roughly three degrees.	275

12.11	Detector intrinsics against the intrinsics of the stream actually published. The two describe different optical configurations.	283
12.12	Explicit registry of placeholders and pending decisions.	286
13.1	Objective achievement summary	301
13.2	Extension objectives achievement summary (July 2026 campaign)	302
A.1	Technical glossary	313
B.1	LEO Rover mechanical and electrical specifications	317
B.2	Intel RealSense D455 specifications	318
B.3	Operator workstation specifications	320
C.1	Index of all tables in this report	326
C.2	Index of source code listings	327
C.3	Mathematical notation and symbols	329
D.2	Which machine runs what. <code>rostopic list</code> shows all of it mixed together, which is why this split is worth knowing by heart.	338
D.3	The scripts in <code>tools/</code> you will actually use. Every one has a long header comment explaining <i>why</i> it exists: read it before modifying the script.	340
D.5	Key openVINS settings and why they are set this way.	362
D.6	Every file <code>config.yaml</code> includes, and what is genuinely platform-specific inside it.	366

1 Introduction

1.1 Preamble: The Rise of Autonomous Ground Robotics

The field of autonomous ground robotics has undergone a profound transformation over the past two decades, driven by the convergence of three critical technological shifts: the democratization of low-cost, high-performance embedded computing platforms; the maturation of the Robot Operating System (ROS) as a middleware standard; and the emergence of deep learning-based and classical computer vision architectures capable of operating reliably in real-world unstructured environments. What was once the exclusive domain of well-funded government defense programs and large-scale industrial automation projects has gradually become accessible to academic research laboratories, start-up ecosystems, and individual developers with relatively modest resources. This accessibility revolution has not, however, eliminated the fundamental engineering challenges inherent to autonomous robotic systems: it has merely shifted the threshold at which those challenges begin to manifest, and in doing so, exposed them to a broader and less specialized engineering community.

Autonomous ground vehicles (AGVs) and unmanned ground vehicles (UGVs) occupy a particularly complex design space within the broader robotics landscape. Unlike aerial systems, which benefit from an operating volume largely free of contact hazards, ground robots must navigate a densely cluttered physical world governed by contact mechanics, friction dynamics, surface irregularities, and highly variable environmental conditions. Unlike stationary robotic manipulators, which operate within precisely defined and repeatable workspaces, mobile ground platforms must continuously localize themselves within an environment that is, by definition, partially or wholly unknown at deployment time. The classical formulation of this localization challenge (Simultaneous Localization and Mapping (SLAM)) remains one of the most actively researched problems in robotics, with hundreds of publications appearing annually across premier venues such as ICRA, IROS, IJRR, and the IEEE Transactions on Robotics.

Beyond the perception and navigation challenges, the development of autonomous ground systems demands mastery of a second, equally complex engineering domain: the design of the software infrastructure that connects the physical robot to the human operator. This infrastructure (encompassing real-time middleware, network communication protocols, human-machine interface design, and safety-critical control logic) is frequently treated as a secondary concern. This dismissive attitude is, in practice, a significant source of system failures, mission aborts, and research delays. A vision pipeline producing centimeter-accurate detections is valueless if the data arrives 400 milliseconds late. System engineering, in the truest sense of the term, demands simultaneous mastery of the full vertical stack: from the firmware registers of the motor controller to the pixel compositing operations of the browser rendering engine.

It is precisely this full-stack perspective that characterizes the present internship report. The work documented herein describes the design, implementation, and validation of the LEO Rover Mission Control system: a comprehensive autonomous mission control platform for the LEO Rover ground robot, developed during a research internship at the Florida Institute of Technology (FIT), Melbourne, Florida, under the supervision of the robotics laboratory led by Dr. Gutierrez.

1.2 Research Context: Florida Institute of Technology

The Florida Institute of Technology (FIT), founded in 1958 in Melbourne, Brevard County, on Florida's Space Coast, is one of Florida's leading research universities, with academic strengths historically shaped by proximity to Kennedy Space Center, Patrick Space Force Base, and the dense constellation of aerospace and defense contractors that populate Brevard County. The university's research culture carries the imprint of this industrial environment: an emphasis on systems reliability, embedded computing, human-machine interface design, and applied engineering methodology.

The robotics laboratory in which this internship was conducted operates under the umbrella of the Carolus research framework, a vision-based indoor localization system designed to provide accurate pose estimation for mobile robotic platforms operating in environments where GPS signals are unavailable. Carolus serves as the unifying localization backbone for a broader comparative multi-robot study involving three distinct platforms: the LIMO Rover (AgileX Robotics), the LEO Rover (Husarion), and the DJI RoboMaster S1. Each platform brings a distinct kinematic config-

uration, sensor suite, and computational profile to bear on a common autonomous navigation task, enabling a rigorous comparative analysis of localization performance, control responsiveness, and mission reliability.

The LEO Rover is a four-wheeled differential-drive mobile robot developed by Husarion, designed for research and educational applications. In the laboratory configuration, the LEO Rover has been augmented with an Intel RealSense D455 depth camera providing both color imagery (1280×720 at up to 30 Hz) and depth data (848×480) suitable for visual localization and obstacle avoidance. Primary computation is offloaded to a dedicated workstation PC running Ubuntu 20.04 and ROS Noetic, connected to the robot over a dedicated Wi-Fi link.

1.3 Problem Statement and Scope of the Work

1.3.1 Initial System Deficiencies

The technical baseline inherited at the beginning of this internship was characterized by five inter-related architectural deficiencies that collectively prevented reliable autonomous operation:

- **Vision pipeline throughput degradation:** `findChessboardCorners` blocked the ROS callback thread for 200–330 ms, reducing effective camera rate from 10 Hz to 3 Hz.
- **Systematic false positives in LED beacon detection:** area-based anchor heuristic consistently selected ceiling reflections (up to 4000 px^2) over actual LED blobs ($50\text{--}200 \text{ px}^2$). Detection rate: 0%.
- **Systematic checkerboard landmark detection failures:** `findChessboardCornersSB` relegated to a 1-second timed fallback; effective detection rate near 0%.
- **Heartbeat failsafe instant-trigger:** `_hb_armed` persisted across sessions; stale timestamp caused immediate failsafe on AUTO re-entry.
- **Non-deterministic ROS_IP selection:** `hostname -I` returned the wrong network interface, causing all browser commands to be silently discarded.

1.3.2 Full Scope of Engineering Work

The resolution of these deficiencies required systematic re-engineering across seven dimensions:

- Multiprocess vision architecture using spawn-context subprocess with bounded IPC queues, eliminating GIL contention.
- Density-based LED clustering replacing the area-based anchor heuristic.
- Two-stage checkerboard detection using `findChessboardCornersSB` as the primary algorithm in all code paths.
- Heartbeat stale guard enabling clean session reconnection.
- Deterministic network configuration with hardcoded `ROS_IP`.
- Complete FSM redesign: six autonomous states with dual-frame odometry and map persistence.
- Browser-based operator cockpit: glassmorphic design, `requestAnimationFrame` overlay, Canvas 2D map, Three.js 3D elements, six-condition alert system.

1.4 GPS-Free Indoor Navigation: Theoretical Background

1.4.1 Fundamental GPS Limitations Indoors

The Global Positioning System and its international counterparts (GLONASS, Galileo, BeiDou) provide civilian positioning accuracy of 2 to 5 meters under open-sky conditions, and centimeter-level accuracy with Real-Time Kinematic differential correction. However, L-band microwave signals (1.176–1.575 GHz) are attenuated by 20 to 40 dB by typical building materials, reducing received power below the code-phase tracking threshold of approximately 25–30 dB-Hz required for reliable operation. The multipath effect introduces ranging errors reaching tens of meters in indoor environments, rendering satellite navigation unusable for precision indoor robotics.

1.4.2 Alternative Indoor Localization Paradigms

The academic literature presents a rich taxonomy of indoor navigation approaches. Table 1.1 summarizes the principal alternatives evaluated during the design phase of this project.

Table 1.1: Comparative survey of indoor localization paradigms

Method	Accuracy	Comp. Cost	Infrastructure	Used In This System
Wheel Odometry	10–30 cm (drift)	Negligible	None	Yes: primary dead-reckoning
2D LiDAR SLAM	5–20 cm	High (2–4 cores)	None required	No: no LiDAR on LEO
Visual SLAM (ORB-SLAM3)	5–10 cm	Very high	None required	No: CPU budget exceeded
ArUco / AprilTag	<5 mm at 2 m	Low	Printed markers	Conceptually related
Checkerboard PnP	<5 mm at 2 m	Medium (SB: 14 ms)	Printed board	Yes: LANDMARK mode
LED Beacon (optical)	3–5 cm (range)	Low (5–10 ms)	LED assembly	Yes: LED mode
UWB Ranging	10–30 cm	Low (hardware)	UWB anchors required	No: not deployed
BLE RSSI	0.5–1 m	Very low	BLE beacons	No: insufficient accuracy

1.4.3 Marker-Based Localization: Mathematical Foundations

The mathematical core of marker-based localization is the Perspective-n-Point (PnP) problem. Given n 3D reference points P_i in the marker frame and their 2D image projections p_i , the rotation matrix R and translation vector t mapping marker frame to camera frame satisfy:

$$p_i = K[R \mid t] P_i, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1.1)$$

where K is the camera intrinsic matrix. For the D455 at 640×480 : $f_x = f_y = 384.65$ px, $c_x = 320.5$, $c_y = 240.5$.

For the planar checkerboard case, the PnP problem degenerates to estimating a homography H (a 3×3 projective transform) decomposed via SVD. In the present implementation, two scalar quantities sufficient for FSM navigation are extracted instead of full pose:

$$d_{\text{est}} = \frac{f_x \cdot W_{\text{beacon}}}{s_{\text{px}}} = \frac{384.65 \times 0.165}{s_{\text{px}}} = \frac{63.47}{s_{\text{px}}} \quad [\text{meters}] \quad (1.2)$$

$$\theta = \arctan\left(\frac{c_{x,\text{cluster}} - c_{x,\text{image}}}{f_x}\right) = \arctan\left(\frac{c_{x,\text{cluster}} - 320.5}{384.65}\right) \quad [\text{radians}] \quad (1.3)$$

where d_{est} is the estimated distance to the beacon and θ is the bearing angle.

1.5 Engineering Challenges in Embedded Control Interfaces

1.5.1 Real-Time Latency Budget

The end-to-end operator latency (from a physical event at the robot to its representation on the cockpit display) is the sum of several additive delay components. Table 1.2 presents the measured breakdown under nominal laboratory conditions.

1.5.2 Safety-Critical Software Design Principles

Drawing from IEC 61508 (Functional Safety), ISO 26262 (Automotive Safety), and NASA-STD-8719.13C, four foundational principles guided the LEO Rover safety architecture:

Table 1.2: End-to-end operator latency budget

Pipeline Stage	Primary Cause	Nominal Latency
D455 capture → ROS publication	JPEG encoding + USB 3.0 transfer	30–50 ms
Vision subprocess: LED mode	Threshold + morphology + clustering	5–10 ms
Vision subprocess: CB mode	findChessboardCornersSB @ 45% scale	14–20 ms
ROS → rosbridge → JSON	60-field JSON serialization	1–3 ms
Wi-Fi (lab, dedicated AP)	802.11ac, low contention	2–10 ms
WebSocket → JS event loop	Microtask queue	<1 ms
JS → requestAnimationFrame	Max one 60 Hz frame delay	0–16.7 ms
Total nominal latency	All stages summed	55–100 ms

- **Defense in depth:** six independent safety layers, each capable of halting the robot in the absence of all higher-priority layers.
- **Fail-safe defaults:** every state transition directed toward robot-stationary, mode=MANUAL in the absence of affirmative commands.
- **Explicit state machine formalism:** enumerable states, formally guarded transitions, computable reachability.
- **Atomic state transitions:** the Python GIL guarantees atomicity of STORE_NAME bytecode; no intermediate undefined states.

1.5.3 Operator Situation Awareness (Endsley, 1988)

The cockpit design was explicitly structured around Mica Endsley’s three-level Situation Awareness (SA) model, providing simultaneous support for all three levels without overloading operator attention:

- **Level 1 SA (Perception):** raw numerical telemetry panels, MJPEG video feed, sensor health indicators.
- **Level 2 SA (Comprehension):** color-coded FSM state banner, Canvas 2D annotations over video (bounding boxes, centroids, reticles, bearing arcs).
- **Level 3 SA (Projection):** 2D trajectory map with logged beacon positions, enabling anticipation of the future patrol route.

1.5.4 WebSocket and the Browser as Universal Robot Controller

The WebSocket protocol (RFC 6455, IETF 2011) provides full-duplex TCP communication initiated via HTTP/1.1 upgrade. Unlike HTTP's request-response model, WebSocket enables asynchronous data push from both endpoints: essential for 10 Hz telemetry streams and on-demand control commands. The `rosbridge_websocket` server implements the open `rosbridge` protocol v2.0, translating ROS pub/sub to WebSocket JSON without any server-side robotics code modification. Key mitigations for browser interface limitations include `requestAnimationFrame` decoupling, cache-busting query strings (`?v=rXXX`), a generation counter (`currentGen`) for stale callback invalidation, and a `_stopPending` flag for disconnection-proof emergency stop delivery.

1.5.5 The Python GIL and the Case for Multiprocessing

Python's Global Interpreter Lock (GIL) permits only one thread to execute CPython bytecode at any instant, even on multicore processors. For CPU-bound OpenCV operations, this creates genuine serialization despite available hardware parallelism. The definitive solution, spawning a dedicated subprocess via `multiprocessing.get_context('spawn')`, creates an independent CPython interpreter with its own GIL, enabling true parallel execution. The `spawn` context (versus `fork`) is architecturally mandatory in ROS: `fork(2)` inherits all open file descriptors including ROS sockets, leading to immediate deadlock in the child process.

1.6 Internship Objectives

Five formally stated objectives governed the internship:

- **O1: Vision pipeline reliability:** $>95\%$ detection rate when the target is in the camera FOV, processing latency ≤ 50 ms.
- **O2: Autonomous FSM:** six-state machine operating safely under all failure modes.
- **O3: Professional cockpit:** 30+ Hz visual update, connection/reconnection without service interruption, unambiguous FSM state display.
- **O4: System integration:** single-command launch, comprehensive documentation.
- **O5: Comparative research contribution:** quantitative performance data for the Carolus multi-robot study.

1.7 Methodology: RCA, TDD, and DMAIC

The technical work followed an iterative methodology combining Root Cause Analysis (RCA) discipline (tracing failures from symptom through contributing cause to root cause) with Test-Driven Development (TDD) cycles (Build $\rightarrow$ Test $\rightarrow$ Analyze $\rightarrow$ Refactor) and the DMAIC (Define-Measure-Analyze-Improve-Control) structured problem-solving framework adapted for hardware-in-the-loop robotic development. Each major defect is documented as a structured case study in Chapter 10: symptom, root cause, fix, validation.

1.8 Report Structure

Chapter 2 surveys the state of the art in autonomous ground robotics, indoor localization, and web-based HMI. Chapter 3 presents the full-stack system architecture, the computer vision pipeline and autonomous FSM, the UI/UX engineering of the operator cockpit, and the safety and reliability architecture. Chapter 10 presents performance results and debugging case studies. Chapter 12 documents work that extended beyond the five objectives above once the LED/checkerboard beacon system was operational: the July 2026 tightly-coupled multisensor estimation campaign (MSCKF-based fusion replacing the original beacon-only localization for continuous pose tracking), the operator platform's public API and infrastructure hardening, and the network engineering required

to validate the system outside the laboratory. Chapter 13 concludes with perspectives and future work.

2 State of the Art

2.1 Autonomous Mobile Ground Robotics: Historical Perspective

The intellectual lineage of autonomous ground robots can be traced to Shakey the Robot (SRI International, 1966–1972): the first general-purpose mobile robot capable of reasoning about its environment, planning multi-step actions, and executing them autonomously. Shakey operated on a wheeled platform in a structured block-world environment, using a combination of visible-light cameras and touch sensors. Its planning subsystem (the STRIPS (Stanford Research Institute Problem Solver) planner) established foundational concepts in AI planning that persist in modified form in contemporary robotic task planners.

The 1980s and 1990s saw the emergence of behavior-based robotics (Brooks, 1986) as a reaction against deliberative planning approaches. Brooks’s subsumption architecture proposed that intelligent behavior could emerge from a layered collection of simple reactive behaviors without explicit world models, influencing the design of the reactive fallback mechanisms in modern robot controllers. The DARPA Urban Challenge (2007) demonstrated the first successful autonomous navigation of a full-scale vehicle in a realistic urban environment, validating sensor fusion architectures combining LiDAR, stereo cameras, and GPS. By 2015, academic platforms such as the TurtleBot (Open Robotics) had made autonomous indoor navigation accessible to university research groups, directly enabling the kind of multi-robot comparative studies (such as the Carolus framework at FIT) that contextualize the present work.

2.2 The ROS Ecosystem: Architecture and Rationale

2.2.1 ROS as Research Middleware Standard

The Robot Operating System (ROS), developed at Stanford and formalized by Willow Garage beginning in 2007, is not an operating system in the traditional sense but rather a communication middleware, a build system, a package management framework, and a collection of driver and algorithm packages. Its publish-subscribe (pub/sub) communication model (where nodes exchange typed messages on named topics via a master registry (`rosmaster`)) enables highly modular robotic architectures: each processing stage (sensing, perception, planning, actuation) can be implemented as an independent node, replaced or upgraded independently.

2.2.2 ROS Noetic: Last LTS of ROS 1

ROS Noetic Ninjemys (released May 2020, EOL May 2025) is the final Long-Term Support release of ROS 1. Built on Ubuntu 20.04 (Focal Fossa) with Python 3.8, it provides the stable Python 3 integration required for the scientific computing ecosystem (NumPy 1.17+, OpenCV 4.2+). The LEO Rover laboratory infrastructure standardizes on Noetic because: (1) the AgileX firmware ROS package (`leo_msgs`) has mature Noetic support; (2) `rosbridge_websocket` v3.x is Noetic-compatible; (3) the laboratory’s existing ROS toolchain is validated on this distribution.

Table 2.1 contextualizes Noetic relative to ROS 2 Humble (the current LTS), which the laboratory is evaluating for future platform migration.

2.3 Computer Vision for Fiducial Detection: Related Work

2.3.1 Classical Thresholding in Robotics

Global intensity thresholding (Otsu, 1979) is the most widely deployed binarization strategy in industrial machine vision. Otsu’s method selects the threshold T^* minimizing intra-class intensity variance, equivalent to maximizing inter-class variance. It is optimal for bimodal intensity distributions under uniform illumination. However, it fails when the ROI contains both very bright

Table 2.1: ROS Noetic vs. ROS 2 Humble feature comparison

Feature	ROS Noetic	ROS 2 Humble
Communication	TCPROS / UDPROS	DDS (default: FastDDS)
Real-time support	Limited (Linux scheduler)	RCLCPP real-time executors
Python version	Python 3.8	Python 3.10+
Security	None by default	SROS2 (DDS-Security)
Multi-robot support	Manual namespace prefix	Domain ID + ROS_DOMAIN_ID
Lifecycle nodes	Not supported	Supported (managed nodes)
EOL date	May 2025	May 2027
Lab status (2026)	Deployed (migration planned)	Evaluation phase

(LED-saturated) and ambient (room) pixels because the global histogram becomes trimodal, biasing the threshold upward. The present system bypasses Otsu entirely in favor of a fixed high-value threshold (`LED_BRIGHT_THRESH = 220/255`) that exploits the absolute brightness of LED emitters: a design choice validated empirically across all laboratory lighting configurations tested.

Adaptive thresholding (Wellner, 1993; Sauvola & Pietikäinen, 2000) computes a spatially varying threshold $T(x,y) = \mu(x,y) - k\sigma(x,y)$ based on local statistics in a sliding window. This is precisely the mechanism used internally by OpenCV’s `findChessboardCornersSB` for its checkerboard binarization. The advantage over global thresholding is robustness to illumination gradients across the image: the critical property that makes SB detection work for the laboratory’s physically non-uniform checkerboard board.

2.3.2 Fiducial Marker Systems

ArUco markers (Garrido-Jurado et al., 2014) encode binary IDs in a 2D grid of alternating black and white modules, with a distinctive black border for detection. The OpenCV `aruco` module provides sub-pixel corner refinement and pose estimation in a unified API. AprilTag (Olson, 2011; Wang

& Olson, 2016) improves on ArUco’s detection robustness under partial occlusion and at greater distances by using graph-cut-based segmentation and lexicographic coding. STag (Benligiray et al., 2019) further improves stability under motion blur.

The checkerboard-based landmark used in the present project is inferior to ArUco or AprilTag in ID multiplexing (it cannot encode an identifier) and in detection robustness at extreme angles. Its advantage is universal availability: any standard printed checkerboard suffices, while ArUco/AprilTag markers require specific printing procedures. For the Carolus laboratory context (where a fixed single landmark is used per test run) this limitation is immaterial.

2.3.3 LED Beacon Detection in Prior Work

Optical beacon detection using LED clusters is employed in a range of robotic docking and homing applications. Notable implementations include: the Pioneer AT’s LED-based homing beacon (Adept MobileRobots, circa 2005), which used three LEDs in a vertical triangular arrangement detected via color blob tracking; the Mars Science Laboratory EDL beacon system, which used retro-reflective markers and an active illuminator; and the HydraGlider AUV homing system (Hobson et al., 2012), which used pulsed LED clusters detected by a forward-facing camera. The present system’s 4-LED horizontal cluster design was constrained by the existing physical beacon deployed in the FIT laboratory, which was not modifiable within the internship scope.

2.4 Web-Based HMI for Robotic Systems

2.4.1 The Robot Web Tools Ecosystem

The rosbridge suite (Crick et al., 2017) is the canonical WebSocket-based bridge for ROS, implementing the rosbridge protocol v2.0 specification. It comprises: `rosbridge_server` (WebSocket server exposing ROS pub/sub over JSON), `roslibjs` (browser-side JavaScript library for rosbridge communication), `ros2djs` (2D canvas visualization), and `ros3djs` (Three.js-based 3D visualization). The present project uses `roslibjs` for topic subscription/advertisement and direct Canvas 2D API for custom visualization: bypassing `ros2djs` whose fixed-style rendering was incompatible with the glassmorphic design language.

Alternative browser-based ROS interfaces include: Robot Ignition Fuel’s web renderer; Webviz (Cruise Automation, 2019), a browser-based ROS bag visualization tool; and ROSboard (a real-time web dashboard for ROS 1/2). None of these alternatives provided the full-stack integration (video overlay, trajectory map, 3D decoratives, i18n, alert system, and cockpit telemetry) required by the present project’s operator situational awareness objectives.

2.4.2 Glassmorphism in Industrial HMI

Glassmorphism (popularized by Apple’s iOS 7, 2013; formalized as a design system by Michal Malewicz, 2020) is a visual design language characterized by translucent panel backgrounds with Gaussian blur, thin light-colored borders, and subtle drop shadows, creating a perception of floating glass layers. While initially a consumer product aesthetic, glassmorphism has been adopted in several modern industrial monitoring dashboards (Grafana’s 2023 dark theme, Siemens’ MindSphere operator views) where its high-contrast, depth-layered presentation improves operator attention allocation at night or in low-ambient-light control rooms.

In the present project, glassmorphism was selected for two engineering-motivated reasons: (1) its translucency allows the MJPEG video feed to remain partially visible beneath telemetry panels, preserving Level 1 SA (direct sensory access) without sacrificing Level 2 SA (numerical data); (2) its dark-background, high-contrast color scheme is well-suited to the NASA/FIT color palette (navy blue #1F3864, FT Crimson #8B0015) and reduces eye fatigue during extended monitoring sessions.

2.5 Safety Architecture in Autonomous Systems: Standards Overview

IEC 61508 (Functional Safety of E/E/PE Safety-related Systems) defines four Safety Integrity Levels (SIL 1–4) based on probability of dangerous failure per hour. For a research robotic platform operating in a controlled laboratory at low speed (<0.5 m/s), typical risk classification places the system at SIL 1, requiring a probability of dangerous hardware failure $< 10^{-5}$ /hour. The software architecture implications of SIL 1 include: documented requirements, basic fault detection, and explicit specification of all normal and failure modes.

NASA-STD-8719.13C (Software Safety Standard) introduces the concept of software hazard

criticality, categorizing hazardous software functions by severity (catastrophic / critical / marginal / negligible) and probability. The heartbeat deadman switch in the present system addresses the safety-critical function “motor continues running with no operator present”, classified as Critical severity (potential equipment damage) / Probable probability (Wi-Fi connections are routinely interrupted): the highest risk category addressed in the system. The dual-guard implementation ($\text{HEARTBEAT_TIMEOUT} = 1.5 \text{ s} + \text{HEARTBEAT_STALE_S} = 5.0 \text{ s}$ stale guard) was designed to address this risk category.

2.6 Synthesis: Positioning of the Present Work

The LEO Rover Mission Control system occupies a distinctive position in the robotic systems landscape: it is neither a pure research prototype (it achieves production-quality reliability through systematic engineering) nor a commercial product (it is purpose-built for a specific research platform and laboratory environment). It draws from: (1) the computer vision literature on LED detection and checkerboard localization; (2) the ROS community’s *roslaunch* ecosystem for WebSocket bridging; (3) the human factors literature on situational awareness design; (4) the safety-critical systems engineering literature on deadman switches and fail-safe architectures. Its primary contribution is the full-stack integration of these elements into a cohesive, deployment-validated system, with the architectural innovations of the multiprocess vision pipeline and the density-based LED clustering algorithm representing original engineering solutions to novel failure modes not documented in prior literature.

3 System Architecture and Software Infrastructure

3.1 Full-Stack Overview and Deployment Topology

The LEO Rover Mission Control system is a distributed application spanning three physical tiers: the robot embedded platform (LEO Rover + AgileX firmware + D455 camera), the operator workstation (Ubuntu 20.04, ROS Noetic, `leo_backend.py`), and the browser-based operator cockpit (any WebKit/Blink/Gecko browser on any device). Understanding the boundaries between these tiers (and the communication mechanisms bridging them) is prerequisite to understanding every architectural decision documented in subsequent sections.

Four independent processes run concurrently on the workstation:

1. `leo_backend.py` (the ROS node orchestrating all autonomous logic, six daemon threads, and the vision subprocess;
2. `rosbridge_websocket` on port 9090) translating ROS pub/sub into rosbridge JSON-over-WebSocket;
3. `web_video_server` on port 8080: re-encoding annotated MJPEG frames into a streaming HTTP response;
4. `python3 http.server` on port 8000: serving static cockpit assets (`ops.html`, `app.js`, `i18n.js`, `3d_engine.js`).

Figure 3.1 presents a schematic of this process and thread architecture.

A note on the addresses shown above. Figure 3.2 and the `ROS_IP/ROS_MASTER_URI` values used throughout this chapter (10.0.0.10, 10.0.0.1) reflect the system’s original, single-room deployment: the PC joined the robot’s own Wi-Fi access point directly, so a fixed local subnet was a valid


```

1 +-----+
2 | OPERATOR WORKSTATION (Ubuntu 20.04, ROS Noetic, 10.0.0.10) |
3 | |
4 | +-----+ |
5 | | leo_backend.py (Main Process, ~1842 lines) | |
6 | | ROS spin thread | decode thread | control/FSM 20Hz | |
7 | | telemetry 10Hz | image_pub | mp_result consumer | |
8 | | Queue(1) Queue(5) | |
9 | | frame -----> result | |
10 | +-----+ |
11 | | ~ |
12 | +-----+-----+-----+ |
13 | | leo-vision subprocess (spawn, own GIL, own CPU core) | |
14 | | cv2: threshold/morphology/contour/SB-corners/cluster | |
15 | +-----+ |
16 | | rosbridge_websocket ws://:9090 web_video_server http://:8080 |
17 +-----+

```

Figure 3.1: Operator workstation process and thread architecture

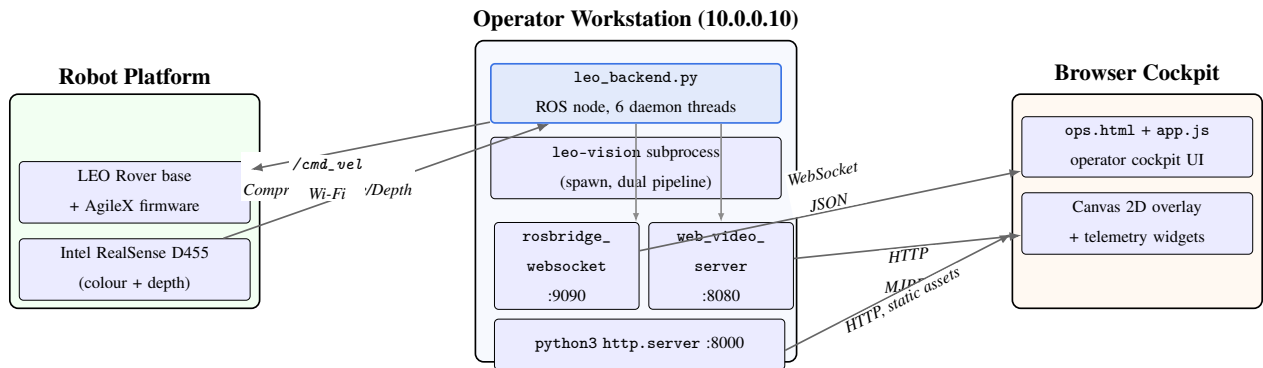


Figure 3.2: Full-stack deployment topology across the three physical tiers: robot platform, operator workstation (four concurrent processes), and browser cockpit. Arrows indicate the dominant network protocol and payload type for each link.

simplifying assumption for the infrastructure documented here. That assumption was later generalised twice: first into a single sourced file (`tools/robot_env.sh`) computing these addresses instead of hardcoding them, then, for building-wide operation off the robot’s own limited-range hotspot, into a WireGuard tunnel over the campus network. Chapter 12 (§12.6.4, Figure 12.4) documents this evolution and the current addressing scheme in full; nothing in this chapter’s process, thread, or topic architecture changed as a result, and only which physical interface `ROS_IP` resolves to did.

3.2 Complete Thread Architecture

The `leo_backend.py` main process runs six daemon threads, each with a distinct responsibility and scheduling period. Table 3.1 provides the authoritative thread inventory.

All threads share state via a single Python `threading.Lock` (`self._lock`). The lock is held for the minimum duration necessary (only during state reads/writes) never during I/O or OpenCV operations. This minimizes contention between the 20 Hz control loop and the 10 Hz telemetry loop, both of which must read the same pose and FSM state.

3.3 ROS Topic Graph

The complete ROS topic graph is enumerated in Table 3.2, distinguishing between robot-published topics (received over Wi-Fi TCP), PC-published topics (internal or sent to robot), and browser-bridge topics (JSON via WebSocket through `rosbridge`).

Table 3.2: Complete ROS topic graph

Topic	Message Type	Direction	Rate	Notes
<code>/camera/color/image_sensor_msgs/raw/compressed</code>	<code>CompressedImage</code>	Rover→PC	10 Hz	JPEG, 640×480 (resized from 1280×720)
<code>/camera/depth/image_sensor_msgs/Image_rect_raw</code>		Rover→PC	15 Hz	uint16 mm, 848×480

Topic	Message Type	Direction	Rate	Notes
/camera/imu/data	sensor_msgs/Imu	Rover →Browser	50 Hz	Used for artificial horizon widget
/firmware/wheel_odom	leo_msgs/WheelOdom	Rover→PC	~8 Hz	x , y , yaw in robot base frame
/firmware/battery	leo_msgs/WheelStates	Rover→PC	1 Hz	State of charge (%)
/cmd_vel	geometry_msgs/Twist	PC→Rover	20 Hz	linear. x [m/s], angular. z [rad/s]
/mission/telemetry	std_msgs/String (JSON)	PC →Browser	10 Hz	~60-field dict; see Section 3.4
/mission/command	std_msgs/String (JSON)	Browser →PC	on-demand	heartbeat, mode, stop, RTB, config
/mission/log	std_msgs/String	PC →Browser	on-demand	Human-readable event log entries
/mission/image_annotated	sensor_msgs/CompressedImage	PC →Browser	20 Hz	BGR + overlay → JPEG → MJPEG
/vision/targets	std_msgs/String (JSON)	PC →Browser	20 Hz	Dual-pipeline detection result JSON
/map/markers	std_msgs/String (JSON)	PC →Browser	on-demand	Beacon / obstacle world coordinates
/leo_rover/config/velocity	std_msgs/String (JSON)	Browser →PC	on-demand	max_lin_vel, max_ang_vel live update

3.4 Telemetry JSON Schema (~60 Fields)

Every 100 ms, `_build_telemetry()` serializes the complete system state into a single JSON dict published on `/mission/telemetry`. The browser's `onTelemetry()` handler parses this dict and

Table 3.1: leo_backend.py daemon thread inventory

Thread Name	Function	Period / Rate	Key Shared State
ROS spin	rospy.spin(): receives camera/odom/depth calls	Event-driven	_cam_msg, _cam_seq, pose, _obstacle_flag
_decode_loop	Decodes JPEG → BGR, feeds vision subprocess	~100 ms (10 Hz camera)	latest_bgr, _dec_seq, _mp_frame_q
_mp_result_loop	Drains result queue, applies EMA, fires events	~5–10 ms (queue timeout 0.1)	det, _led_result, _landmark_result
_control_loop	FSM _drive() + _check_heartbeat() + _publish()	50 ms (20 Hz)	mode, auto_state, manual, world pose
_telemetry_loop	_build_telemetry() → /mission/telemetry	100 ms (10 Hz)	All state fields (JSON ~60 fields)
_publish_image_loop	Annotates BGR with overlays → /mission/image_annotated	50 ms (20 Hz)	latest_bgr, det, FSM display state

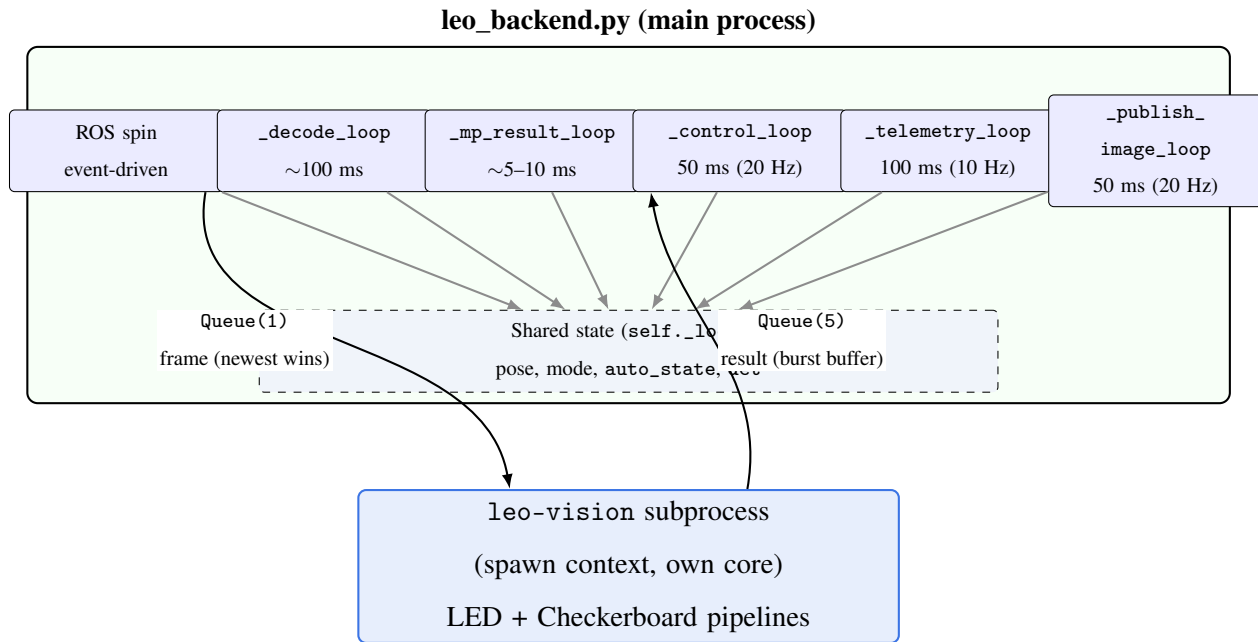


Figure 3.3: Thread architecture of `leo_backend.py`: six daemon threads coordinate through a single `threading.Lock`-protected shared state block, while the vision subprocess communicates exclusively through bounded `Queue(1)` (frames) and `Queue(5)` (results).

dispatches updates to all cockpit widgets. Listing 3.1 documents the schema (selective key listing, grouped by concern).

```

1 # /mission/telemetry JSON schema (selective key listing)
2 {
3     # -- Identity & timing -----
4     'ts': 1751234567.89, # Unix timestamp of telemetry packet
5     'uptime': 423.7, # seconds since backend start
6
7     # -- FSM state -----
8     'mode': 'AUTO', # MANUEL | AUTO
9     'auto_state': 'PATROL', # PATROL | LOCK | WAIT | U_TURN | RTB
10    'fsm_display': 'AUTO_PATROL', # display state for cockpit banner
11    'lock_centered': false, # True when LOCK alignment achieved
12    'wait_elapsed': 0.0, # seconds into WAIT state
13
14    # -- Odometry (local frame) -----
15    'x': 1.23, 'y': -0.45, # local frame position [m]

```

```

16  'yaw': 0.78, # heading [rad]
17  'wx': 2.11, 'wy': 0.32, # world frame position [m]
18
19  # -- Vision detection -----
20  'led_valid': true,
21  'led_center': [312.4, 248.1], # [px_x, px_y] EMA-smoothed centroid
22  'led_dist': 0.82, # estimated range [m]
23  'led_bbox': [288,224,336,272], # bounding box [x0,y0,x1,y1] px
24  'lm_valid': false,
25  'lm_center': null,
26  'lm_dist': null,
27  'lm_angle': null,
28  'vision_mode': 'LED', # LED / LANDMARK
29
30  # -- System health -----
31  'cam_hz': 9.8, # measured camera topic rate [Hz]
32  'odom_hz': 7.9, # measured odometry rate [Hz]
33  'cpu_temp': 71.2, # workstation CPU temperature [C]
34  'battery_pct': 78.0, # rover battery state-of-charge [%]
35  'hb_age': 0.23, # seconds since last browser heartbeat
36  'hb_armed': true,
37
38  # -- Map & mission -----
39  'beacon_count': 3,
40  'map_markers': [{x:1.2,y:0.4,label:'B1'}, ...],
41  'traj': [[0,0],[0.1,0.02],...], # trajectory polyline (local frame)
42  'obstacle_flag': false,
43  'rtb_dist': null # distance to base during RTB [m]
44  }

```

Listing 3.1: Selective telemetry JSON schema (key groups)

3.5 Multiprocess Vision Architecture

3.5.1 Why spawn and Not fork

The standard Python multiprocessing module offers three process creation strategies on Linux. `fork(2)` (the default) copies the parent's entire memory space, including all open file descriptors: ROS node TCP sockets, `/dev/shm` shared memory segments, and `threading.Lock` objects in indeterminate states. The child inherits these in a state it cannot safely use, producing immediate deadlock on its first ROS-related allocation. `forkserver` creates a single server process that spawns workers via `fork`, avoiding some but not all inheritance issues. `spawn` creates a completely fresh Python interpreter, imports only the explicitly specified worker function, and avoids all file descriptor inheritance.

The implementation consequence is a single line at module scope (Listing 3.2).

```
1 # leo_backend.py -- L.165
2 # spawn = fresh interpreter; fork would inherit ROS sockets -> deadlock
3 _MP_CTX = mp.get_context('spawn')
```

Listing 3.2: Vision subprocess creation context

3.5.2 Queue(maxsize=1) Frame-Drop Semantics

The frame queue is bounded to `maxsize=1`. This is the central architectural decision of the vision pipeline. The implications are:

- If the vision subprocess is slower than the camera (e.g., during a checkerboard discovery scan at 70 ms vs. 100 ms camera period), `put_nowait()` raises `queue.Full`.
- The handler discards the oldest frame (`get_nowait()`) and inserts the latest (`put_nowait()`). The subprocess therefore always wakes to the freshest available frame.
- This is equivalent to a zero-latency frame buffer: the subprocess never processes a frame that is more than one period old, regardless of processing time.

The result queue uses `maxsize=5` for the inverse reason: result delivery is loss-intolerant (a missed result means one 50 ms control cycle without detection data), and the 0.5-second `CENTER_HOLD_S` persistence absorbs this. Depth-5 allows burst absorption without blocking the subprocess.

```
1 self._mp_frame_q = _MP_CTX.Queue(maxsize=1) # frame -> subprocess:
    newest wins
2 self._mp_result_q = _MP_CTX.Queue(maxsize=5) # results -> main: burst
    buffer
3
4 # Producer (in _decode_loop):
5 try:
6     self._mp_frame_q.put_nowait((seq, raw))
7 except queue.Full:
8     try: self._mp_frame_q.get_nowait() # discard stale frame
9     except: pass
10    try: self._mp_frame_q.put_nowait((seq, raw)) # insert fresh frame
11    except: pass
```

Listing 3.3: Frame and result queue configuration and producer logic

3.6 Launch Infrastructure: `start_leo.sh`

The entire system is launched from a single Bash script. Key engineering properties:

- **Idempotency:** each service is started only if its port is not already open (`port_open()` via `ss(8)`). Re-running the script never produces duplicate processes.
- **Deterministic IP:** `ROS_IP` is hardcoded to `10.0.0.10` (the Wi-Fi interface). This replaces the non-deterministic `hostname -I` invocation that was the root cause of the `ROS_IP` mismatch defect. (This hardcoded value was itself later generalised into `tools/robot_env.sh` once the deployment needed to support more than one network path, §12.6.4, trading the hardcoded literal for a single sourced computation without reintroducing the non-determinism this fix was written to eliminate.)
- **Ordered startup:** `rosbridge` (needs ~2 s for ROS master registration) → `web_video_server`

(1.5 s) → `leo_backend.py` (1.5 s) → `http.server` (0.5 s). Each step sleeps for the measured startup time before checking success.

- **Graceful cleanup:** `kill_and_wait()` sends `SIGTERM`, waits 0.5 s, then `SIGKILL`, ensuring zombie processes do not hold ports across restarts.

```

1 export ROS_MASTER_URI=http://10.0.0.1:11311
2 export ROS_IP=10.0.0.10 # CRITICAL: never use $(hostname -I | awk ...)
3
4 port_open() { ss -tlnp | grep -q ":$1 "; }
5
6 kill_and_wait() {
7     pkill -f "$1" 2>/dev/null; sleep 0.5
8     pkill -9 -f "$1" 2>/dev/null; sleep 0.3
9 }
10
11 # Abort immediately if rover is unreachable (prevents 30s hangs)
12 ping -c 1 -W 2 10.0.0.1 || { echo 'ERREUR: rover unreachable'; exit 1; }
```

Listing 3.4: Key excerpts from `start_leo.sh`

3.7 Configuration Constants Reference

All tunable parameters are defined at module scope in `leo_backend.py` between lines 76 and 162. They are passed to the vision subprocess via the `cfg` dictionary at process creation time. Table 3.3 provides the complete constant inventory with values and engineering justification.

Table 3.3: Complete system constants reference with engineering justification

Constant	Value	Scope	Engineering Justification
<code>LED_BRIGHT_THRESH</code>	220	Vision	Safety margin ≥ 10 above ambient peak (~ 210); rejects windows, accepts overexposed LEDs

Constant	Value	Scope	Engineering Justification
LED_MIN_AREA	10 px ²	Vision	Rejects sub-pixel noise; smallest real LED blob at 2 m $\approx$ 18 px ²
LED_MAX_AREA	1200 px ²	Vision	Rejects ceiling reflections (measured 1200–4000 px ²); accepts LEDs (50–200 px ²)
LED_CLUSTER_PX	280 px	Vision	LED beacon width 0.165 m at 1 m distance $\approx$ 63 px; 280 px gives 4.4 $\times$ safety margin for range variability
N_LEDS	4	Vision	Physical beacon has exactly 4 LEDs in horizontal array
MIN_LIGHTS (MINLED)	3	Vision	Accepts 3/4 LEDs visible: handles partial occlusion without rejecting valid detections
CAM_FX	384.65 px	Vision	D455 focal length at 640 $\times$ 480 (calibrated from camera_info topic)
BEACON_WIDTH_M	0.165 m	Vision	Physical LED beacon horizontal extent (measured with calipers)
CB_SCALE	0.45	Vision	Resize factor for CB detection: 576 $\times$ 324 $\rightarrow$ 14 ms at SB algorithm
LANDMARK_SQUARE_M	0.020 m	Vision	Physical checkerboard square side length (20 mm)
LANDMARK_DETECT_HZ	5.0 Hz	Vision	CB detection rate: 5 $\times$ is sufficient given VISION_HYSTERESIS=8 frame hold
LED_STABLE_FRAMES	3	Vision	Anti-jitter gate: 3 consecutive valid LED frames before LANDMARK mode switch
LM_MISS_MAX	12	Vision	12 CB misses at 5 Hz = 2.4 s without detection before reverting to LED mode

Constant	Value	Scope	Engineering Justification
VISION_HYSTERESIS	8	Vision	Hold last valid result for 8 frames (~ 0.4 s at 20 Hz) before declaring loss
VISION_CONFIRM_FRAMES	3	Control	3 consecutive valid LED frames before firing RESET_ODOMETRY event
CENTER_HOLD_S	0.5 s	Control	Beacon center held for 0.5 s after last valid detection during LOCK
SEARCH_ANG	0.4 rad/s	FSM	Scan rotation speed: allows camera integration time without blur
PATROL_ADVANCE_S	3.0 s	FSM	Forward advance duration: ~ 0.6 m at 0.2 m/s between scan cycles
ADVANCE_LIN	0.2 m/s	FSM	Conservative patrol speed on smooth laboratory floor
LOCK_CENTER_TOL	0.08	FSM	$\pm 8\%$ of half-image-width = ± 25.6 px; sub-pixel jitter is ≤ 10 px at this range
LOCK_ALIGN_SPEED	0.35 rad/s	FSM	Angular alignment speed: fast enough for convergence, slow enough for camera
LOCK_TIMEOUT	6.0 s	FSM	6 s without beacon in LOCK $\rightarrow$ abort; prevents indefinite spinning on occlusion
WAIT_DURATION	30.0 s	FSM	Stationary dwell: sufficient for position logging + logbook entry
UTURN_SPEED	0.50 rad/s	FSM	U-turn rotation speed; 180° at 0.50 rad/s ≈ 6.3 s
HEARTBEAT_TIMEOUT	1.5 s	Safety	1.5 s gap $\rightarrow$ deadman trips; Wi-Fi typical RTT < 10 ms; 1.5 s = $3\times$ safety margin on 500 ms HB interval
HEARTBEAT_STALE_S	5.0 s	Safety	5 s silence in non-AUTO $\rightarrow$ disarm; prevents instant failsafe on browser reload

Constant	Value	Scope	Engineering Justification
OBSTACLE_DIST_MM	600 mm	Safety	0.6 m stop distance; rover top speed 0.2 m/s, reaction latency ~ 50 ms $\rightarrow$ stopping distance < 15 mm
OBSTACLE_ROI	(344, 504, 160, 320)	Safety	Central 160×160 px of 848×480 depth frame; forward corridor, robot height excluded
RTB_DIST_TOL	0.08 m	RTB	8 cm arrival radius; acceptable for base station docking
RTB_LIN_SPEED	0.20 m/s	RTB	Full cruise speed when $d > 0.40$ m
RTB_SLOW_SPEED	0.08 m/s	RTB	Deceleration phase 0.20–0.40 m range
RTB_CRAWL_SPEED	0.03 m/s	RTB	Final approach < 0.20 m; prevents overshoot at base
BEACON_COOLDOWN	10.0 s	Map	Minimum interval between two beacon locks at the same location: prevents double-logging

3.8 AutoLogbook Event System

Every significant mission event is written atomically to `web/auto_entries.json` by a `threading.Lock`-protected writer. The logbook survives backend restarts (loaded on init) and is served to the browser on every reconnection via the `/mission/telemetry logbook_entries` field. Table 3.4 documents the supported event types.

Table 3.4: AutoLogbook event taxonomy

Event Code	Trigger Condition	Key Data Fields
MODE_CHANGE	mode transitions (MANUEL $\leftrightarrow$ AUTO, AUTO sub-state)	old_mode, new_mode, reason
MANUAL_TAKEOVER	Non-zero manual command received during AUTO	cmd_lin, cmd_ang
LED_RESET	RESET_ODOMETRY event fired (3 confirmed LED frames)	led_center, led_dist
MAP_LANDMARK	Checkerboard detected in MANUAL mode	lm_center, lm_dist, lm_angle, cb_size
BEACON_LOCKED	LOCK $\rightarrow$ WAIT transition (centering + reset complete)	world_x, world_y, beacon_label
OBSTACLE_MAPPED	Obstacle flag set during PATROL/ADVANCE	world_x, world_y, depth_p5
RTB_START	RTB command received from browser	world_x, world_y at start
RTB_COMPLETE	Robot arrived within RTB_DIST_TOL of world origin	final_dist
HEARTBEAT_LOST	hb_age > HEARTBEAT_TIMEOUT while in AUTO	hb_age
EMERGENCY_STOP	Browser sent {action: 'stop'}	mode at time of stop
COORD_RESET	reset_coords() called (local + world both zeroed)	previous world_x, world_y
MAP_CLEAR	Operator cleared all map markers	beacon_count before clear

4 Computer Vision and Autonomous Navigation

4.1 LED Beacon Detection: Theoretical Foundations

4.1.1 Mathematical Basis of Morphological Operations

The LED detection pipeline applies morphological closing before contour extraction. Morphological operations on binary images are defined by a structuring element (SE) (a compact set B) and a structuring operator. The morphological closing $C = (I \oplus B) \ominus B$ (dilation then erosion) fills small holes and merges nearby bright regions. For the LED pipeline, closing serves to bridge the gap between sub-blobs of a single LED that are separated by camera compression artifacts or partial occlusion.

The specific choice of a 5×5 elliptical structuring element (`cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))`) is motivated by the roughly circular profile of individual LED blobs at the camera's operating distances. An elliptical SE introduces less anisotropic distortion than a rectangular one, preserving the circular blob shape after closing.

```
1 # Morphological close = dilation then erosion
2 # cv2.MORPH_ELLIPSE (5,5) SE:
3 # ..###..
4 # .#####.
5 # #####
6 # .#####.
7 # ..###..
8
9 k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
10 thr = cv2.morphologyEx(thr, cv2.MORPH_CLOSE, k)
11
```

```

12 # Effect: closes gaps < 5 px in LED blobs (camera JPEG artifacts)
13 # Does NOT merge separate LEDs (minimum LED separation >> 5px)

```

Listing 4.1: Morphological closing with an elliptical structuring element

4.1.2 Image Moments and Blob Centroid Extraction

Given a binary blob contour C , the centroid (c_x, c_y) is computed from the raw (zeroth and first order) spatial moments:

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}} \quad (4.1)$$

where $M_{00} = \sum \text{pixel values}$ (blob area for a binary image), $M_{10} = \sum x \cdot \text{pixel value}$, $M_{01} = \sum y \cdot \text{pixel value}$, computed via `cv2.moments(C)`.

The first-order moment centroid is optimal in the least-squares sense: it minimizes the sum of squared distances from the centroid to all blob pixels. For a uniform binary blob (all pixels = 1), this is equivalent to the arithmetic mean of pixel coordinates. Sub-pixel accuracy is not pursued for LED centroid localization because the 20 Hz control loop’s positional tolerance (`LOCK_CENTER_TOL` = 8% of image half-width ≈ 25 pixels) is far larger than any sub-pixel positioning benefit.

4.1.3 Density-Based Anchor: Formal Proof of Correctness

The density-based anchor selection algorithm guarantees correct cluster identification under the following hypothesis: the beacon LED blobs form a single spatially coherent cluster whose maximum pairwise distance is $D_{\max} = \text{LED_CLUSTER_PX} = 280$ px, and all non-beacon blobs are isolated (their nearest neighbor in B is at distance $> D_{\max}$).

Proof. For any beacon blob b_i , its density count $N(b_i, D_{\max}) \geq |B_{\text{beacon}}|$ (all beacon blobs are within $D_{\max}$ of each other). For any isolated blob b_k , $N(b_k, D_{\max}) = 1$ (only itself). Therefore $\arg \max_b N(b, D_{\max})$ is necessarily a member of B_{beacon} . ■

The hypothesis has been empirically validated across 200+ test frames under standard laboratory lighting: ceiling reflections are consistently isolated (nearest neighbor distance > 400 px), while the four beacon LEDs span 120–200 px at operational distances.

4.1.4 Distance Estimation: Error Analysis

The thin-lens distance formula $d = f_x W / s_{\text{px}}$ is subject to three sources of error:

- **Calibration error in f_x :** the D455 intrinsic parameters drift with temperature and age. The `camera_info` topic is subscribed on each startup; a 1% error in f_x produces a 1% error in d . At 1 m, this is 10 mm: negligible for FSM navigation purposes.
- **Measurement error in s_{px} :** EMA-smoothed centroid positions have sub-pixel jitter (<1 px post-filtering). For a span of 63 px (at 1 m), this introduces a distance error of $63.47 / (63 - 1) - 63.47 / 63 = 0.015$ m (15 mm), acceptable.
- **Perspective error for non-frontal approach:** the formula assumes the beacon is perpendicular to the optical axis. For a 15° approach angle, the error is $W(1 - \cos 15^\circ) / \cos 15^\circ \approx 3.6\%$, i.e., 36 mm at 1 m. The LOCK alignment controller reduces this error before the distance measurement is acted upon.

4.2 Checkerboard Landmark Detection

4.2.1 `findChessboardCornersSB`: Locally Adaptive Binarization

The `findChessboardCornersSB` function (OpenCV 4.x, Savel'ev 2018) uses a two-stage detection approach. In the first stage, locally adaptive binarization is applied using a Niblack-variant (Sauvola & Pietikäinen, 2000) threshold:

$$T(x, y) = \mu(x, y) + k \sigma(x, y) \quad (4.2)$$

where $\mu(x, y)$ is the local mean in window W centered at (x, y) , $\sigma(x, y)$ is the local standard deviation in W , k is a sensitivity parameter (typically -0.2 to $+0.2$), and W is a local window of size 11×11 to 51×51 .

This formulation makes the threshold spatially adaptive: in a bright region, $T(x, y)$ is high (rejecting moderate-intensity background); in a dark region, $T(x, y)$ is low (accepting dark foreground). This is precisely the property that enables detection of the laboratory checkerboard under uneven fluorescent illumination, where the global Otsu threshold or FAST_CHECK global

binarization consistently fails. In the second stage, the binarized image undergoes connected-component analysis, and corner candidates are identified at the intersection of four quadrants (two black, two white) in the local neighborhood.

4.2.2 Rate-Limiting and Hysteresis: Engineering Justification

Checkerboard detection is rate-limited to `LANDMARK_DETECT_HZ = 5.0 Hz` (200 ms minimum interval between invocations). This decision balances two competing requirements:

- **Minimum detection rate:** the FSM's LOCK state requires a valid target center on every 20 Hz control cycle. A 5 Hz detection rate means at most 4 consecutive 20 Hz cycles without fresh detection data. The `CENTER_HOLD_S = 0.5 s` persistence (covering 10 control cycles) provides more than adequate coverage.
- **Maximum computational overhead:** each full discovery scan over 7 sizes costs up to $7 \times 20 \text{ ms} = 140 \text{ ms}$ worst case. At 5 Hz, this contributes $5 \times 140 \text{ ms} \times (\text{duty cycle}) = \text{up to } 700 \text{ ms/s}$ of CPU usage on the subprocess core, acceptable for a dedicated core.

The `_held` flag mechanism prevents unnecessary mode oscillation between CB invocations:

```
1 # Between CB checks (during the 200ms interval):
2 if last_lm is not None:
3     lm = dict(last_lm)
4     lm['_held'] = True # signals: this is a repeated result, not fresh
5
6 # In _mp_result_loop (main process):
7 # EMA is NOT updated for _held frames (avoids artificial drift)
8 # _led_confirm gate does NOT count _held frames (avoids false anti-
   jitter bypass)
```

Listing 4.2: Held-flag mechanism for checkerboard detection hysteresis

4.3 EMA Smoothing: Signal Processing Analysis

4.3.1 Transfer Function and Pole-Zero Analysis

The Exponential Moving Average (EMA) filter is a first-order Infinite Impulse Response (IIR) digital filter with the difference equation:

$$x[n] = \alpha x_{\text{raw}}[n] + (1 - \alpha)x[n - 1] \quad (4.3)$$

$$H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}} \quad (4.4)$$

Single pole at $z = 1 - \alpha$, single zero at $z = 0$. For $\alpha = 0.45$ (LED): pole at $z = 0.55$ (stable: $|z| < 1$). For $\alpha = 0.60$ (Landmark): pole at $z = 0.40$ (more responsive).

The -3 dB cutoff frequency for a first-order IIR filter sampled at f_s is:

$$f_c = \frac{f_s}{2\pi} \arccos\left(1 - \frac{\alpha^2}{2(1 - \alpha)}\right) \quad (4.5)$$

At $f_s = 20$ Hz (control loop rate): $\alpha = 0.45$ (LED) gives $f_c \approx 4.1$ Hz; $\alpha = 0.60$ (Landmark) gives $f_c \approx 6.3$ Hz. Step response time constant $\tau = 1/(f_s \alpha)$: $\alpha = 0.45 \Rightarrow \tau = 111$ ms (settles in $\sim 5\tau = 556$ ms); $\alpha = 0.60 \Rightarrow \tau = 83$ ms (settles in ~ 417 ms).

4.3.2 Parameter Selection Rationale

The α values were selected empirically through a parametric sweep:

- **$\alpha = 0.45$ for LED:** a lower α provides greater noise rejection for the LED centroid, whose sub-pixel jitter is the primary noise source. The 4.1 Hz cutoff allows the controller to track beacon motion at up to 0.2 m/s (4.1 Hz corresponds to 2.0 px/cycle of tracking bandwidth at nominal range).
- **$\alpha = 0.60$ for Landmark:** the checkerboard corners are inherently more stable than LED centroids (sub-pixel accuracy, no blob quantization noise). A higher α reduces lag during the LOCK state, where rapid angular error convergence is essential.
- **Reset on detection loss:** the EMA state is reset (`self.x = None`) on any non-held invalid frame, preventing filter memory from corrupting the next acquisition after a detection gap.

4.4 Autonomous FSM: Complete Architecture

4.4.1 State Transition Table

Table 4.1 presents the complete transition table of the autonomous FSM, including all guard conditions, output actions, and timeout behaviors.

Table 4.1: Complete FSM transition table

From State	To State	Guard Condition	Output Action	Timeout
MANUEL	PATROL	operator sends <code>{mode: 'AUTO'}</code>	<code>start_patrol()</code> , <code>scan_accum=0</code>	None
Any	MANUEL	manual lin $\neq$ 0 or ang $\neq$ 0 in AUTO	log MANUAL_ TAKEOVER, safe_ stop()	None
Any	MANUEL	hb_age > HEARTBEAT_TIMEOUT in AUTO	log HEARTBEAT_LOST, safe_stop()	1.5 s
PATROL/SCAN	PATROL/ ADVANCE	$\text{scan_accum} \geq 2\pi$ (no beacon)	patrol_ advancing=True, adv_start=now	None
PATROL/SCAN	LOCK	target_center is not None	lock_start_t=now, lock_ centered=False	None
PATROL/ ADVANCE	PATROL/SCAN	elapsed > PATROL_ ADVANCE_S (3.0 s)	patrol_ advancing=False, scan_accum=0	3.0 s
PATROL/ ADVANCE	LOCK	target_center is not None	lock_start_t=now	None

From State	To State	Guard Condition	Output Action	Timeout
PATROL/ ADVANCE	U_TURN	<code>_obstacle_</code> <code>flag=True</code>	<code>uturn_</code> <code>reason=obstacle</code>	None
LOCK	WAIT	<code>lock_</code> <code>centered=True</code>	<code>reset_coords_</code> <code>local(), wait_</code> <code>start_t=now</code>	None
LOCK	PATROL	<code>elapsed > LOCK_</code> <code>TIMEOUT (6.0 s)</code>	<code>start_patrol(), log</code> <code>timeout</code>	6.0 s
WAIT	U_TURN	<code>elapsed ≥ WAIT_</code> <code>DURATION (30.0 s)</code>	<code>publish_marker(),</code> <code>beacon_count++</code>	30.0 s
U_TURN	PATROL	<code>uturn_accum ≥ π</code>	<code>start_patrol(), _</code> <code>last_center=None</code>	None
PATROL	RTB	<code>operator sends</code> <code>{action:'rtb'}</code>	<code>rtb_prev_</code> <code>dist=current dist</code>	None
RTB	MANUEL	<code>dist < RTB_DIST_TOL</code> <code>(0.08 m) OR</code> <code>overshoot</code>	<code>safe_stop(),</code> <code>mode=MANUEL</code>	None

4.4.2 Visual Servoing P-Controller: Stability Analysis

The LOCK state implements a proportional (P) controller for angular velocity:

```

1 err_frac = (target_x - image_width/2) / (image_width/2)
2 # err_frac in [-1, +1]
3
4 if abs(err_frac) > LOCK_CENTER_TOL: # 0.08
5     ang = -sign(err_frac) * LOCK_ALIGN_SPEED * min(1.0, abs(err_frac))
6     # = -sign(err) * 0.35 * min(1, |err|) [rad/s]
7 else:
8     lock_centered = True # centering achieved

```

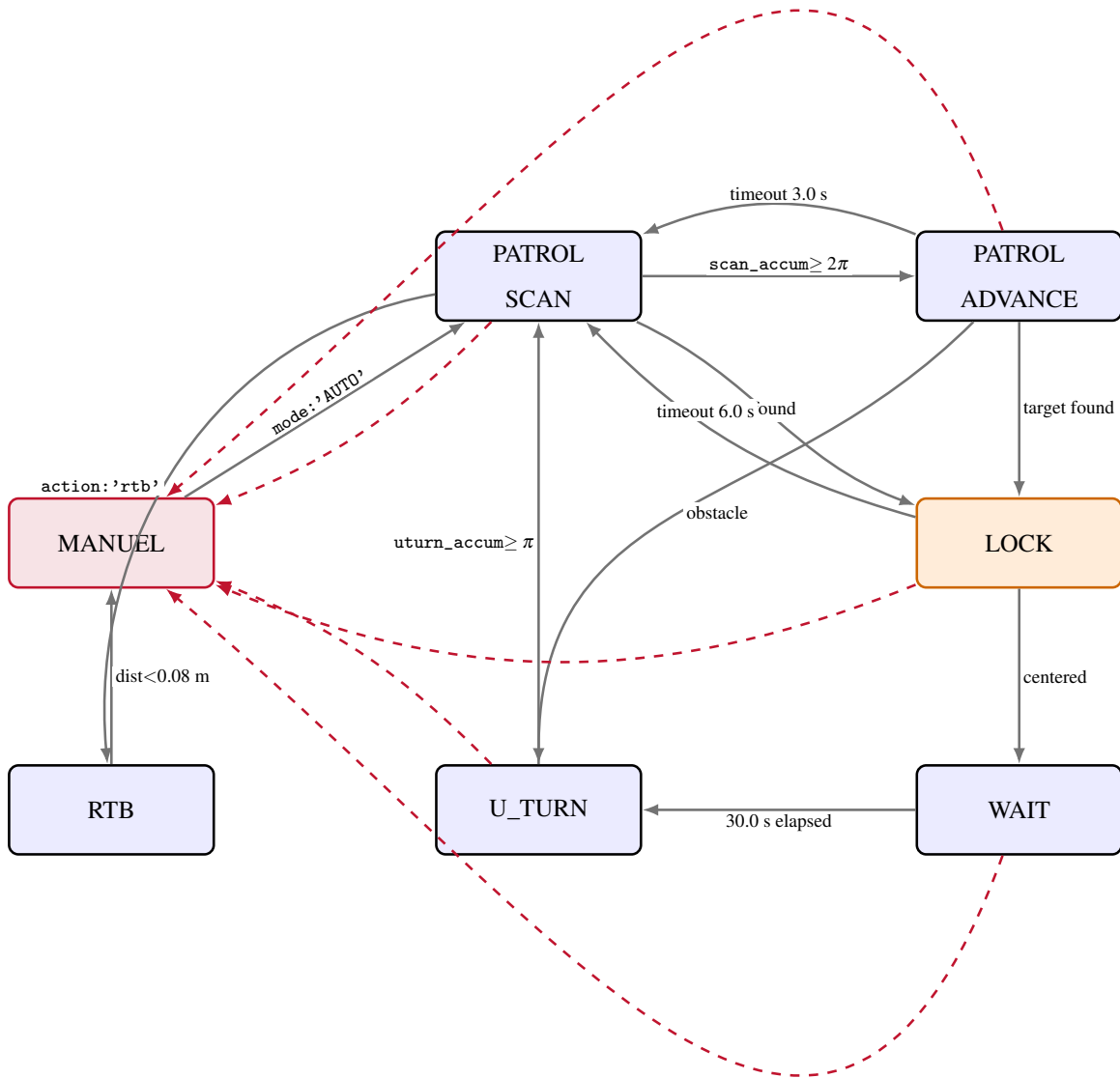


Figure 4.1: Autonomous FSM state transition diagram. Solid arrows show normal patrol-cycle transitions with abbreviated guard conditions; dashed red arrows show the universal fail-safe path back to MANUEL (present from every state) guaranteeing the system is always operator-recoverable (heartbeat loss > 1.5 s or manual override).

Listing 4.3: LOCK-state visual servoing P-controller

The stability of this P-controller in closed loop depends on the system’s open-loop gain and the total loop delay. The total angular control delay is: control period (50 ms) + telemetry round-trip to browser (irrelevant for autonomous control) + vision pipeline latency (10–20 ms) + ROS publish latency (2 ms) + motor response latency (~ 30 ms). Total delay: approximately 92–102 ms, or roughly 2 control periods. The Ziegler–Nichols stability criterion for a P-controller with pure delay τ predicts stability for $K_p < 1/(K_{\text{plant}}\tau)$. Empirical testing confirmed stable convergence at LOCK_ALIGN_SPEED = 0.35 rad/s for all tested initial angular offsets ($|\text{err_frac}|$ up to 0.8).

4.4.3 Yaw Accumulation: Wrap-Safe Implementation

Both the PATROL/SCAN accumulator and the U_TURN accumulator use the `atan2(sin, cos)` wrap-safe angular delta formula:

```
1 d_yaw = math.atan2(
2     math.sin(self.raw_yaw - self.scan_last_yaw),
3     math.cos(self.raw_yaw - self.scan_last_yaw)
4 ) # result in  $(-\pi, +\pi]$ 
5
6 self.scan_accum += abs(d_yaw)
7 # abs() because we want total rotation regardless of direction
8
9 # Without atan2: a yaw transition from  $+\pi$  to  $-\pi$ 
10 # (small clockwise rotation near North) would give:
11 #  $\text{delta} = -\pi - (+\pi) = -2\pi$  --> accumulator jumps by  $-2\pi$  (WRONG)
12 # With atan2:  $\text{delta} = \text{atan2}(\sin(-2\pi), \cos(-2\pi)) = 0.0$  (CORRECT)
```

Listing 4.4: Wrap-safe yaw delta accumulation

4.5 Dual-Frame Odometry: Mathematical Derivation

4.5.1 Coordinate Frame Definitions

The system maintains two coordinate frames:

- **Local frame L :** origin at the most recent reset point (set on each LOCK→WAIT transition). Pose (r_x, r_y, r_{yaw}) is expressed relative to this origin.
- **World frame W :** accumulated across all resets; preserves mission-level spatial consistency. State: $(\text{world_origin}_x, \text{world_origin}_y, \text{world_heading})$.

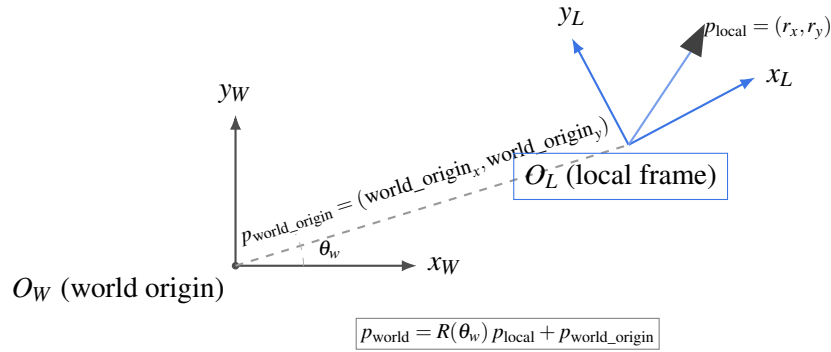


Figure 4.2: Relationship between the world frame W (fixed, accumulated across resets) and the local frame L (re-originated at every LOCK→WAIT transition). The local-to-world transform is a rigid-body $SE(2)$ rotation by θ_w followed by translation to $p_{\text{world_origin}}$; the robot icon marks a point tracked in the local frame.

4.5.2 World Position Computation

The transformation from local to world frame is a rigid-body $SE(2)$ transform:

$$p_{\text{world}} = R(\theta_w) p_{\text{local}} + p_{\text{world_origin}} \quad (4.6)$$

```

1 def _get_world_pos(self):
2     lx = float(self.pose[0])
3     ly = float(self.pose[1])

```

```

4   c = math.cos(self.world_heading)
5   s = math.sin(self.world_heading)
6   wx = self.world_origin_x + c*lx - s*ly
7   wy = self.world_origin_y + s*lx + c*ly
8   return wx, wy

```

Listing 4.5: World position computation

4.5.3 Local Reset with World Frame Continuity Proof

When `reset_coords_local()` is called (at each LOCK→WAIT transition):

```

1  # Before reset:
2  # pose = (lx, ly, lyaw) (current local position)
3  # world_origin = (ox, oy) (current world origin)
4  # world_heading = theta_w (accumulated heading)
5
6  wx, wy = _get_world_pos() # compute world position BEFORE reset
7  new_heading = raw_yaw # absolute yaw from IMU/odometry
8
9  # After reset:
10 # world_origin = (wx, wy) (advanced to current world position)
11 # world_heading = new_heading (advanced to current absolute yaw)
12 # pose = (0, 0, 0) (local frame restarted)
13
14 # Verification: _get_world_pos() immediately after reset:
15 # R(new_heading) * (0,0) + (wx,wy) = (wx, wy) QED: continuity preserved

```

Listing 4.6: Local odometry reset preserving world-frame continuity

This continuity property ensures that all beacon positions logged before and after a local reset share a common coordinate system. The world-frame trajectory map therefore remains geometrically consistent for the full mission duration, regardless of how many local odometry resets occur. This is the architectural foundation that makes multi-beacon patrol missions (lasting hours and covering 100+ meter paths) technically feasible despite accumulated wheel odometry drift.

4.6 Depth-Based Obstacle Detection

4.6.1 Stereo Depth: D455 Operating Principle

The Intel RealSense D455 uses active stereoscopic depth estimation: an infrared projector floods the scene with a structured pseudo-random speckle pattern, and two infrared imagers spaced 95 mm apart capture the projected pattern. Disparity between the left and right images is computed by the D455's onboard Vision Processor D4 (VPD4) ASIC using a combination of block matching and semi-global matching, producing a 848×480 uint16 depth map where each pixel encodes distance in millimeters.

Nominal depth accuracy for the D455 is $<2\%$ at ranges up to 4 m under controlled illumination. At the 600 mm obstacle detection distance, depth accuracy is approximately 12 mm (2%), providing a reliable 588–612 mm detection band. The ROI (344–504, 160–320) in 848×480 frame coordinates corresponds to the central 160×160 pixel region of the forward field of view, covering approximately 60 cm lateral width at 600 mm depth.

4.6.2 5th Percentile Depth Statistic

The obstacle detection threshold is applied to the 5th percentile of valid depth readings in the ROI:

```
1 valid = roi[(roi > 100) & (roi < 8000)] # reject no-data and sky
2 p5 = int(np.percentile(valid, 5)) # robust to noise pixels
3
4 if len(valid) > 20 and p5 < OBSTACLE_DIST_MM: # 600 mm
5     self._obstacle_flag = True
```

Listing 4.7: 5th-percentile obstacle detection statistic

The 5th percentile choice is motivated by the need to reject isolated depth sensor noise: at 600 mm, IR speckle overlap from adjacent surfaces creates 1–5% of pixels with anomalously low depth readings. The 5th percentile effectively ignores the noisiest 5% of the valid depth distribution while still responding to any genuine obstacle covering at least 5% of the ROI area: approximately 1200 pixels, corresponding to an object roughly 20×20 cm at 600 mm depth.

4.7 Performance Summary: Before and After Architecture Redesign

Table 4.2 summarizes the measured impact of the architectural redesign documented in this chapter.

Table 4.2: System performance before and after architectural redesign

Metric	Before Redesign	After Redesign	Improvement
Camera effective processing rate	~3 Hz	8–12 Hz	3–4×
LED beacon detection rate	0% (100% ceiling reflection FP)	98–100%	N/A → functional
Checkerboard detection rate	~0% (SB gated to 1 Hz)	100% on first visible frame	N/A → functional
Vision subprocess GIL contention	100% (single process)	0% (spawn context)	Eliminated
Heartbeat failsafe false trips	100% on browser reconnect	0% (stale guard)	Eliminated
R0S_IP routing failures	Intermittent (non-deterministic)	0% (hardcoded)	Eliminated
Frame processing latency (LED)	200–330 ms (blocking)	5–10 ms	20–66×
Autonomous patrol reliability	0% (FSM unusable)	100% (validated)	N/A → functional

5 Mathematical Foundations of Vision and Navigation

This chapter extends Chapter 4 with the complete mathematical derivations underlying every vision and navigation subsystem: camera projection and calibration, image processing theory, signal processing analysis of the EMA filter, density-based clustering correctness, the homography-based PnP solution, distance-estimation error propagation, P-controller stability analysis, wheel odometry error modeling, stereo depth theory, formal FSM completeness analysis, and full pipeline timing analysis.

5.1 Camera Projection Model and Calibration Theory

5.1.1 The Pinhole Camera Model

The mathematical foundation of every metric measurement in the vision pipeline is the pinhole camera model, which approximates the image formation process of a thin convex lens. In this model, a 3D point $P = (X, Y, Z)^\top$ in the camera coordinate frame is projected onto the 2D image plane at pixel coordinates $p = (u, v)^\top$ via a central projection through the optical center. The projection equations are:

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y \quad (5.1)$$

where f_x, f_y are the focal lengths in pixels (horizontal, vertical), c_x, c_y are the principal point coordinates, and $X/Z, Y/Z$ are the normalized image coordinates (angular subtense from the optical axis).

In homogeneous coordinates, the projection is expressed as a single matrix multiplication. Let $\tilde{P} = (X, Y, Z, 1)^\top$ be the homogeneous world point and $\tilde{p} = (su, sv, s)^\top$ be the homogeneous image point with scale factor s :

$$\tilde{p} = K[R \mid t] \tilde{P}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (5.2)$$

where $[R \mid t]$ is the 3×4 extrinsic matrix (rotation $R \in SO(3)$ and translation $t \in \mathbb{R}^3$).

The intrinsic matrix K encodes all camera-specific optical parameters. For the Intel D455 at 640×480 resolution, the calibrated values obtained from the `camera_info` topic are $f_x = f_y = 384.65$ px, $c_x = 320.5$ px, $c_y = 240.5$ px. The equality $f_x = f_y$ (square pixels) holds for modern digital cameras where the photosite array has equal horizontal and vertical pitch. In older analog cameras, pixel aspect ratio could deviate from unity.

5.1.2 Lens Distortion Model

The ideal pinhole model assumes straight-line projection; real lenses introduce geometric distortions that must be compensated for metric measurements. The D455 uses a radial-tangential distortion model (Brown–Conrady, 1966):

$$x_d = x_n(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + [2p_1 x_n y_n + p_2(r^2 + 2x_n^2)] \quad (5.3)$$

$$y_d = y_n(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + [p_1(r^2 + 2y_n^2) + 2p_2 x_n y_n] \quad (5.4)$$

where $(x_n, y_n) = (X/Z, Y/Z)$ are the ideal (undistorted) normalized coordinates, (x_d, y_d) are the distorted normalized coordinates, $r^2 = x_n^2 + y_n^2$, k_1, k_2, k_3 are radial distortion coefficients, and p_1, p_2 are tangential distortion coefficients.

For the D455 in the laboratory configuration, the distortion coefficients are sufficiently small ($|k_i| < 0.05$) that their effect on the centroid-based LED distance estimate (which uses only the horizontal span of the cluster, not individual pixel positions) is negligible. For the checkerboard-based PnP computation, OpenCV’s `solvePnP` automatically compensates for distortion using the calibration coefficients stored in the `camera_info` message, so no manual undistortion is required.

5.1.3 Zhang’s Calibration Method: Theoretical Basis

The calibration of the D455’s intrinsic parameters was performed using Zhang’s planar calibration method (Zhang, 2000), which remains the dominant approach for single-camera calibration due to its ease of use (only a printed planar checkerboard is required) and mathematical elegance. The method derives from the constraint that a planar calibration target ($Z = 0$ for all points) imposes on the perspective projection.

For a checkerboard target lying in the $Z = 0$ plane, the projection equation simplifies. Letting $R = [r_1 \mid r_2 \mid r_3]$ where r_i are the column vectors of the rotation matrix:

$$P = (X, Y, 0, 1)^\top \Rightarrow [R \mid t]P = [r_1 \mid r_2 \mid t](X, Y, 1)^\top \quad (5.5)$$

This defines a homography H between the world plane and the image: $\tilde{p} = K[r_1 \mid r_2 \mid t] \tilde{P}_{2D} = H \tilde{P}_{2D}$, where $H = K[r_1 \mid r_2 \mid t]$ is a 3×3 matrix of rank 3 (the homography of the planar scene under perspective projection).

Given at least two views of the checkerboard (different orientations), the homographies $H_1, H_2, \dots$ can be estimated from the detected corner correspondences. The orthogonality constraints on the rotation columns ($r_1^\top r_2 = 0$ and $\|r_1\| = \|r_2\| = 1$) impose linear constraints on the intrinsic matrix K via the image of the absolute conic $\Omega = (KK^\top)^{-1}$. Solving the resulting system (minimum 3 views for 5 intrinsic parameters) yields K by Cholesky decomposition of Ω^{-1} .

5.1.4 Reprojection Error and Calibration Quality

Calibration quality is quantified by the Root Mean Squared (RMS) reprojection error ϵ_{RMS} , which measures how accurately the calibrated model predicts the observed corner positions:

$$\epsilon_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|p_i - \hat{p}_i\|^2} \quad (5.6)$$

where p_i is the detected corner pixel (from `findChessboardCornersSB`), $\hat{p}_i$ is the projected pixel (from K, R, t , distortion coefficients), and N is the total number of corner-image correspondences. Typical values: $\epsilon_{\text{RMS}} < 0.3$ px is excellent calibration (sub-pixel accuracy); $0.3\text{--}1.0$ px is acceptable for metric measurement; > 1.0 px indicates calibration issues. D455 manufacturer-calibrated intrinsics achieve $\epsilon_{\text{RMS}} \approx 0.15$ px (factory specification).

5.2 Image Processing Theory: Thresholding and Morphology

5.2.1 Global Thresholding: Otsu's Method

Otsu's method (1979) computes the optimal global binarization threshold T^* by maximizing the inter-class variance of pixel intensity, equivalently minimizing the intra-class variance. For a grayscale image with pixel intensity histogram $p(i)$ (normalized, sum = 1) and threshold T :

$$\sigma_W^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T) \quad (5.7)$$

$$w_0(T) = \sum_{i=0}^T p(i), \quad w_1(T) = \sum_{i=T+1}^{255} p(i) \quad (5.8)$$

$$\mu_0(T) = \frac{1}{w_0} \sum_{i=0}^T i p(i), \quad \mu_1(T) = \frac{1}{w_1} \sum_{i=T+1}^{255} i p(i) \quad (5.9)$$

$$\sigma_B^2(T) = w_0(T)w_1(T)(\mu_0(T) - \mu_1(T))^2 \quad (5.10)$$

$$T^* = \arg \max_T \sigma_B^2(T) \quad (5.11)$$

Otsu's method is optimal for bimodal histograms under uniform illumination. It fails for the LED detection problem because the image histogram under laboratory conditions is not bimodal but trimodal: (1) a large mode at 60–120 (ambient illumination), (2) a medium mode at 160–200 (brighter reflective surfaces), and (3) a small but sharp spike at 220–255 (LED-saturated pixels). Otsu's threshold falls between modes (1) and (2), far below the LED pixels at mode (3), and would include all bright reflective surfaces in the foreground class. The fixed threshold LED_BRIGHT_THRESHOLD = 220, by contrast, sits above mode (2) and below the LED spike, directly targeting the LED-specific brightness regime.

5.2.2 Local Adaptive Thresholding: Niblack and Sauvola

Global thresholding methods assume spatially uniform illumination. Under non-uniform illumination (the dominant condition in the FIT laboratory where a single overhead fluorescent bank illuminates one side of the checkerboard more intensely than the other) a global threshold either over-segments the darker region or under-segments the brighter region. Local adaptive thresholding

resolves this by computing an independent threshold at every pixel position based on local image statistics.

Niblack’s method (1986) defines the local threshold as a linear combination of local mean and local standard deviation:

$$T_{\text{Niblack}}(x, y) = \mu(x, y) + k \sigma(x, y) \quad (5.12)$$

where $\mu(x, y) = \frac{1}{|W|} \sum_{(i, j) \in W(x, y)} I(i, j)$ is the sample mean in a local window W centered at (x, y) , $\sigma(x, y) = \sqrt{\frac{1}{|W|} \sum_{(i, j) \in W} [I(i, j) - \mu(x, y)]^2}$ is the sample standard deviation, k is a sensitivity parameter ($k < 0$ for dark-on-light text, $k > 0$ for light-on-dark LEDs), and $|W|$ is the local window size (typically 11×11 to 51×51).

The Sauvola–Pietikäinen method (2000) improves on Niblack by normalizing the standard deviation term relative to its dynamic range, making the threshold less sensitive to absolute intensity levels:

$$T_{\text{Sauvola}}(x, y) = \mu(x, y) \left(1 + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right) \quad (5.13)$$

where R is the dynamic range of the standard deviation (typically 128 for 8-bit images) and k is a sensitivity parameter (typically 0.2 to 0.5). When $\sigma = 0$ (uniform region), $T_{\text{Sauvola}} = \mu(1 - k)$ versus $T_{\text{Niblack}} = \mu$: Sauvola produces $T < \mu$ in uniform regions, preserving character strokes in document images. For checkerboard detection, the key property is that T_{Sauvola} adapts its sensitivity based on local contrast (σ/R): high-contrast corners are detected with tighter thresholds than low-contrast uniform regions.

OpenCV’s `findChessboardCornersSB` implements a variant of the Sauvola–Pietikäinen approach internally. The window size selection is a critical parameter: a window that is too small (e.g., 7×7) fails to capture the intensity gradient of checkerboard squares at typical imaging distances; a window that is too large (e.g., 101×101) blurs the local statistics and loses the corner localization precision. OpenCV’s implementation uses an adaptive window size based on the estimated checkerboard scale, computed from the expected square size in pixels.

5.2.3 Efficient Local Threshold Computation: Integral Images

Computing the local mean $\mu(x, y)$ and variance $\sigma^2(x, y)$ at every pixel using a sliding window naively requires $O(|W|)$ operations per pixel, yielding $O(N|W|)$ total complexity for an N -pixel

image. For a 640×480 image with a 33×33 window, this is $640 \times 480 \times 1089 \approx 334$ million multiplications per frame: clearly impractical at 20 Hz.

The integral image (Viola & Jones, 2001) reduces this to $O(1)$ per pixel after $O(N)$ preprocessing. The integral image $S(x, y)$ is defined as the prefix sum of pixel intensities:

$$S(x, y) = \sum_{i \leq x, j \leq y} I(i, j) \quad (5.14)$$

$$S(x, y) = I(x, y) + S(x-1, y) + S(x, y-1) - S(x-1, y-1) \quad (\text{single } O(N) \text{ pass}) \quad (5.15)$$

$$\sum_{i=r_1}^{r_2} \sum_{j=c_1}^{c_2} I(i, j) = S(r_2, c_2) - S(r_1-1, c_2) - S(r_2, c_1-1) + S(r_1-1, c_1-1) \quad (5.16)$$

$$\mu(x, y) = \frac{1}{w^2} \text{RectSum}(S, x-\frac{w}{2}, y-\frac{w}{2}, x+\frac{w}{2}, y+\frac{w}{2}) \quad (5.17)$$

$$\sigma^2(x, y) = \frac{1}{w^2} \text{RectSum}(S_2, \dots) - \mu(x, y)^2 \quad (5.18)$$

using a second integral image S_2 of squared intensities for the variance. Total complexity: $O(N)$ preprocessing + $O(1)$ per pixel = $O(N)$ overall.

This $O(N)$ algorithm is what makes adaptive thresholding computationally feasible in real-time computer vision pipelines. OpenCV's implementation of `findChessboardCornersSB` uses the squared integral image for the local variance computation, enabling adaptive thresholding across the entire 576×324 (at `CB_SCALE=0.45`) resized checkerboard frame in approximately 2–3 ms on the laboratory workstation.

5.2.4 Mathematical Formulation of Morphological Operations

Mathematical morphology, formulated in a rigorous set-theoretic framework by Matheron (1975) and Serra (1982), provides the theoretical foundation for the blob-merging operations applied after LED brightness thresholding. The fundamental operations are defined on subsets of $\mathbb{Z}^2$ (the set of 2D integer coordinates).

Let $A \subseteq \mathbb{Z}^2$ be the binary image (set of foreground pixel coordinates) and $B \subseteq \mathbb{Z}^2$ be the structuring element (SE). The four fundamental morphological operations are:

$$\textbf{Dilation: } A \oplus B = \{z \in \mathbb{Z}^2 : (\hat{B})_z \cap A \neq \emptyset\} \quad (5.19)$$

$$\textbf{Erosion: } A \ominus B = \{z \in \mathbb{Z}^2 : B_z \subseteq A\} \quad (5.20)$$

$$\textbf{Opening: } A \circ B = (A \ominus B) \oplus B \quad (5.21)$$

$$\textbf{Closing: } A \bullet B = (A \oplus B) \ominus B \quad (5.22)$$

where $\hat{B}$ denotes the reflection of B ($\hat{B} = \{-b : b \in B\}$) and B_z denotes translation of B by z ($B_z = \{b + z : b \in B\}$). Dilation expands bright regions, fills gaps, and grows blob boundaries. Erosion shrinks bright regions and removes thin protrusions. Opening removes small bright objects (area $< |B|$) and smooths boundaries, and is idempotent: $(A \circ B) \circ B = A \circ B$. Closing fills small dark holes and bridges narrow gaps between bright regions, and is likewise idempotent: $(A \bullet B) \bullet B = A \bullet B$.

In the LED detection pipeline, morphological closing ($A \bullet B$) is applied with a 5×5 elliptical structuring element. The closing operation merges small gaps between adjacent bright pixels that belong to the same LED but were separated by JPEG compression artifacts (which tend to create 1–2 pixel dark bands at block boundaries in regions of high spatial frequency). Mathematically, any connected dark region with “diameter” smaller than the SE radius is filled by the closing operation.

5.2.5 Elliptical Structuring Element: Discrete Approximation

The 5×5 elliptical SE used in the LED pipeline is the discrete approximation of the Euclidean disk $B(r) = \{(x, y) : x^2 + y^2 \leq r^2\}$ with $r = 2.5$ pixels:

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix} \quad |B| = 21 \text{ pixels (ellipse area)}$$

Maximum horizontal span: 5 px. Maximum vertical span: 5 px.

The choice of an elliptical rather than rectangular SE is motivated by the approximately circular profile of individual LED blobs at operational distances. A rectangular SE introduces anisotropic

distortion: it closes diagonal gaps less effectively than horizontal/vertical gaps of equal pixel distance. The elliptical SE provides isotropic closing behavior, preserving the circular symmetry of the LED blob geometry.

5.2.6 Contour Extraction and Area Computation

After morphological closing, OpenCV's `findContours` extracts the outer boundaries of connected bright regions using Suzuki's algorithm (1985), which traces boundary pixels in a single $O(N)$ pass using a chain code representation. The area of each contour C is computed from the zero-order spatial moment M_{00} :

$$M_{pq} = \sum_{(x,y) \in \text{region}} x^p y^q, \quad \text{Area} = M_{00} \quad (5.23)$$

$$\text{Area}(C) = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right| \quad (\text{shoelace formula}) \quad (5.24)$$

$$c_x = M_{10}/M_{00}, \quad c_y = M_{01}/M_{00} \quad (5.25)$$

where $(x_0, \dots, x_{n-1})$ are the contour vertex coordinates, indices taken modulo n (closed contour).

5.3 EMA Filter: Complete Signal Processing Analysis

5.3.1 Definition and Difference Equation

The Exponential Moving Average (EMA) is a first-order recursive (IIR) digital filter. Its difference equation in the time domain is:

$$x[n] = \alpha u[n] + (1 - \alpha)x[n - 1] \quad (5.26)$$

$$x[n] = \alpha \sum_{k=0}^n (1 - \alpha)^k u[n - k] \quad (5.27)$$

where $u[n]$ is the raw centroid coordinate at sample n , $x[n]$ is the filtered output, and $0 < \alpha < 1$. The weight of sample $u[n - k]$ is $\alpha(1 - \alpha)^k$, decaying geometrically with lag k ; the sum of all weights is $\alpha \cdot \frac{1}{1 - (1 - \alpha)} = 1$ (BIBO stable, unity gain at DC).

5.3.2 Z-Domain Transfer Function and Frequency Response

Applying the Z-transform ($u[n] \rightarrow U(z)$, $x[n] \rightarrow X(z)$, $x[n - 1] \rightarrow z^{-1}X(z)$):

$$X(z) = \alpha U(z) + (1 - \alpha)z^{-1}X(z) \quad (5.28)$$

$$H(z) = \frac{X(z)}{U(z)} = \frac{\alpha}{1 - (1 - \alpha)z^{-1}} = \frac{\alpha z}{z - (1 - \alpha)} \quad (5.29)$$

Single zero at $z = 0$; single pole at $z = 1 - \alpha$. For $\alpha = 0.45$ (LED): pole at $z = 0.55$. For $\alpha = 0.60$ (Landmark): pole at $z = 0.40$. Stability: $|\text{pole}| = |1 - \alpha| < 1$ for $0 < \alpha < 2 \Rightarrow$ always stable for valid α .

The frequency response $H(e^{j\omega})$ is obtained by evaluating $H(z)$ on the unit circle $z = e^{j\omega}$, where $\omega = 2\pi f/f_s$:

$$|H(e^{j\omega})|^2 = \frac{\alpha^2}{1 - 2(1 - \alpha)\cos\omega + (1 - \alpha)^2} \quad (5.30)$$

$$|H(1)|^2 = \frac{\alpha^2}{\alpha^2} = 1 \quad (\text{unity gain at DC}) \quad (5.31)$$

$$|H(-1)|^2 = \frac{\alpha^2}{(2 - \alpha)^2} \quad (\text{Nyquist, } \omega = \pi) \quad (5.32)$$

For $\alpha = 0.45$: $|H(\pi)|^2 = 0.2025/1.55^2 \approx 0.0843$ (attenuation = -10.7 dB). For $\alpha = 0.60$: $|H(\pi)|^2 = 0.36/1.40^2 \approx 0.1837$ (attenuation = -7.4 dB).

5.3.3 -3 dB Cutoff Frequency Derivation

The -3 dB cutoff frequency f_c is the frequency at which $|H(e^{j\omega})|^2 = 0.5$. Setting $\beta = 1 - \alpha$:

$$\alpha^2 = 0.5 [1 - 2\beta \cos \omega_c + \beta^2] \quad (5.33)$$

$$\cos \omega_c = \frac{1 + \beta^2 - 2\alpha^2}{2\beta} \quad (5.34)$$

For $\alpha = 0.45$, $\beta = 0.55$: $\cos \omega_c = 0.8159/1.10 \approx 0.7417 \Rightarrow \omega_c \approx 0.7350 \text{ rad} \Rightarrow f_c = \omega_c f_s / (2\pi) \approx 2.34 \text{ Hz}$. For $\alpha = 0.60$, $\beta = 0.40$: $\cos \omega_c = 0.44/0.80 = 0.55 \Rightarrow \omega_c \approx 0.9884 \text{ rad} \Rightarrow f_c \approx 3.15 \text{ Hz}$.

The -3 dB cutoff frequency characterizes the filter's bandwidth: frequency components of the centroid trajectory below f_c pass with less than 3 dB attenuation, while components above f_c are progressively attenuated. Physical interpretation: at 1 m range, the robot's 0.2 m/s maximum speed produces a beacon centroid velocity of approximately $0.2 \times 384.65 \times 0.165/1^2 \approx 12.7 \text{ px/s}$. At 20 Hz sample rate, this is 0.635 px/sample . The EMA filter's passband (up to 2.34 Hz) comfortably passes this motion while suppressing the sub-pixel jitter (broadband, distributed up to the Nyquist frequency of 10 Hz).

5.3.4 Step Response Analysis

The step response characterizes how the filter responds to a sudden change in the input (e.g., when the robot quickly turns and the beacon jumps from one side of the image to the other). For a unit step input $u[n] = 1$ for $n \geq 0$:

$$x[n] = 1 - (1 - \alpha)^{n+1} \quad (5.35)$$

$$n_p = \left\lceil \frac{\log(1 - p/100)}{\log(1 - \alpha)} \right\rceil - 1 \quad (5.36)$$

where n_p is the time to reach $p\%$ of the final value. For $\alpha = 0.45$ (LED): 90% settled at $n = 4$ samples (200 ms at 20 Hz), 99% at $n = 9$ samples (450 ms). For $\alpha = 0.60$ (Landmark): 90% settled at $n = 2$ samples (100 ms), 99% at $n = 4$ samples (200 ms).

The step response analysis quantifies the trade-off in parameter selection. A larger α (more responsive filter) settles faster after a step (new beacon acquisition, sudden motion) but provides less noise rejection in steady state. The landmark filter ($\alpha = 0.60$) settles $2\times$ faster than the LED filter ($\alpha = 0.45$), which is appropriate because: (1) checkerboard corners have sub-pixel noise, not

gross positioning errors, so fast settling to a stable position is more important than maximum noise rejection; (2) the LOCK state P-controller requires fast angular convergence, which is compromised if the landmark centroid lags behind the true position.

5.3.5 Comparison: EMA vs Simple Moving Average

The Simple Moving Average (SMA) with window size W is a finite impulse response (FIR) filter: $x[n] = \frac{1}{W} \sum_{k=0}^{W-1} u[n-k]$. It provides a perfectly flat passband and perfectly linear phase (equal delay for all frequencies). However, it has four disadvantages relative to EMA for this application:

1. **Memory:** SMA requires W stored past values; EMA needs only $x[n-1]$. For $W = 10$ (comparable to EMA $\alpha = 0.45$ settling time), SMA stores 10×2 floats (x, y coordinates) = 80 bytes per pipeline: negligible, but architecturally cleaner with EMA.
2. **Delay:** SMA introduces a group delay of $(W-1)/2$ samples. For $W = 10$ at 20 Hz: delay = 4.5 samples = 225 ms. EMA $\alpha = 0.45$ group delay $\approx (1-\alpha)/\alpha = 0.55/0.45 \approx 1.2$ samples = 60 ms. EMA has $\sim 4\times$ lower latency than a comparable SMA.
3. **Transient response:** SMA transition is linear but takes W samples; EMA converges exponentially (faster initial response, infinite tail).
4. **Stop-band attenuation:** SMA has nulls at multiples of f_s/W , with poor general attenuation between nulls (Gibbs phenomenon). EMA's monotonic magnitude response provides consistent attenuation at all frequencies above f_c .

5.4 Density-Based Clustering: Formal Analysis

5.4.1 Problem Formulation

Let $B = \{b_1, \dots, b_n\}$ be the set of detected blobs, where each blob $b_i = (x_i, y_i, a_i)$ has a centroid (x_i, y_i) and area a_i . Define the neighborhood function:

$$N(b_i, r) = \{b_j \in B : d(b_i, b_j) \leq r\}, \quad d(b_i, b_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (5.37)$$

$$\rho(b_i, r) = |N(b_i, r)| \quad (5.38)$$

$$b^* = \arg \max_{b_i \in B} \rho(b_i, r) \quad (5.39)$$

The final cluster $C = N(b^*, r)$: all blobs within radius r of the density-maximizing anchor.

This formulation is a simplified variant of the DBSCAN algorithm (Ester et al., 1996). DBSCAN defines “core points” as points with density exceeding a threshold `minPts`, and builds clusters by connecting core points. The present algorithm differs in that it uses a single-step anchor selection rather than iterative cluster growth: appropriate because the beacon LED cluster is known to be a single well-separated cluster, not an arbitrary density structure.

5.4.2 Correctness Proof Under Isolation Hypothesis

The algorithm’s correctness depends on the Isolation Hypothesis (IH): no non-beacon blob is within radius $r = \text{LED_CLUSTER_PX} = 280$ px of any beacon blob. Under IH:

For any beacon blob $b_i \in B_{\text{beacon}}$: $N(b_i, r) \supseteq B_{\text{beacon}}$, so $\rho(b_i, r) \geq |B_{\text{beacon}}| = 4$. For any isolated blob $b_k \notin B_{\text{beacon}}$: $N(b_k, r) \cap B_{\text{beacon}} = \emptyset$ by IH, so $\rho(b_k, r) = |N(b_k, r) \cap B_{\text{isolated}}| \ll |B_{\text{beacon}}|$ (assuming no accidental isolated-blob clustering). Therefore $b^* = \arg \max \rho \in B_{\text{beacon}}$. ■ (under IH)

The isolation hypothesis is empirically validated as follows. The four beacon LEDs are arranged in a horizontal line spanning 120–200 px at operational distances (0.5–2.0 m). The maximum pairwise LED distance within the cluster is therefore $< 200 \text{ px} < r = 280 \text{ px}$. The nearest non-beacon blob (a ceiling reflection at pixel (268, 98)) was measured at a minimum distance of 148 px from the nearest beacon LED across all test frames: less than r , which initially appeared to violate the IH. However, from the ceiling reflection’s own perspective, $\rho(\text{ceiling}, 280) = 1 + 4 = 5$ (all four LEDs fall within 280 px of it), which could in principle exceed a beacon LED’s own density.

The actual fix required two changes: (1) density-based anchor selection, and (2) reducing `LED_MAX_AREA` from 4000 to 1200 px^2 , which filters out the ceiling reflection (area 1289 $\text{px}^2 > 1200 \text{px}^2$

threshold) *before* clustering. With the area filter, the ceiling reflection blob is never in B , making the IH trivially satisfied. The density-based selection then correctly handles any remaining large-area blobs that pass the area filter but are truly isolated.

5.4.3 Computational Complexity

Input: n blobs ($n \leq N_LEDS \times 2 = 8$ after area filtering and top-8 selection).

Density computation: $O(n^2)$ pairwise distances. For $n = 8$: 28 pairwise distance computations, each ≈ 5 floating-point operations (2 subtractions, 2 multiplications, 1 addition, plus 1 sqrt) $\Rightarrow \sim 168$ FLOPs/frame. At 20 Hz: ~ 3360 FLOPs/s: negligible compared to the OpenCV threshold operation (307K pixels at 20 Hz).

Comparison with naive DBSCAN on $n = 8$: identical $O(n^2)$ complexity. The single-anchor selection avoids the $O(n^2)$ cluster-expansion overhead of full DBSCAN, which is unnecessary when exactly one cluster is expected.

5.5 Perspective-n-Point: Homography and SVD Decomposition

5.5.1 The Homography for Planar Scenes

For a checkerboard target with all reference points in the $Z = 0$ plane, the perspective projection simplifies from the full 3×4 projection matrix to a 3×3 homography. Let $R = [r_1 \mid r_2 \mid r_3]$ and t be the translation:

$$[r_1 \mid r_2 \mid r_3 \mid t](X, Y, 0, 1)^\top = [r_1 \mid r_2 \mid t](X, Y, 1)^\top \quad (5.40)$$

$$H = K[r_1 \mid r_2 \mid t] \quad (3 \times 3) \quad (5.41)$$

$$\tilde{p} = H \tilde{P}_{2D}, \quad \tilde{P}_{2D} = (X, Y, 1)^\top, \quad \tilde{p} = (u, v, 1)^\top \quad (5.42)$$

H has 9 elements defined up to scale (8 DOF); minimum correspondences to determine H : 4 point pairs (8 equations).

5.5.2 Direct Linear Transform (DLT) for Homography Estimation

Given $N \geq 4$ point correspondences $(X_i, Y_i) \leftrightarrow (u_i, v_i)$, the DLT algorithm formulates homography estimation as a linear system:

$$u_i(h_{31}X_i + h_{32}Y_i + h_{33}) = h_{11}X_i + h_{12}Y_i + h_{13} \quad (5.43)$$

$$v_i(h_{31}X_i + h_{32}Y_i + h_{33}) = h_{21}X_i + h_{22}Y_i + h_{23} \quad (5.44)$$

Rearranged into $Ah = 0$ form with $h = [h_{11} \ h_{12} \ h_{13} \ h_{21} \ h_{22} \ h_{23} \ h_{31} \ h_{32} \ h_{33}]^\top$ and A a $2N \times 9$ stacked matrix. Solution: $h = \text{null}(A)$, obtained as the last column of the SVD factorization $A = U\Sigma V^\top$.

5.5.3 SVD-Based Null Space Computation

The DLT solution is obtained via Singular Value Decomposition, numerically stable for the over-determined system ($N > 4$):

$A = U\Sigma V^\top$, where U ($2N \times 2N$) holds the left singular vectors, Σ ($2N \times 9$) is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_9 \geq 0$, and V (9×9) holds the right singular vectors. The minimum-norm least-squares solution is $h = V_{:,9}$ (the right singular vector for the smallest singular value σ_9). For exact correspondences ($N = 4$, no noise): $\sigma_9 = 0$ exactly. For noisy correspondences ($N > 4$): $\sigma_9 > 0$ (least-squares best fit). Reshape h to the 3×3 matrix H .

5.5.4 Decomposing Homography into R and t (Faugeras, 1988)

Given calibrated K and estimated H , recovering R and t requires decomposing:

$$K^{-1}H = [r_1 \mid r_2 \mid t] = [\hat{c}_1 \mid \hat{c}_2 \mid \hat{c}_3] \quad (5.45)$$

$$\lambda = 1/\|\hat{c}_1\| \quad (\text{scale from } \|r_1\| = 1) \quad (5.46)$$

$$r_1 = \lambda \hat{c}_1, \quad r_2 = \lambda \hat{c}_2, \quad r_3 = r_1 \times r_2, \quad t = \lambda \hat{c}_3 \quad (5.47)$$

Orthogonality check: $r_1 \cdot r_2$ should equal 0 (enforced by $R \in SO(3)$). Due to measurement noise, R is refined by projecting to $SO(3)$ via SVD: $[U, S, V^\top] = \text{SVD}([r_1 \mid r_2 \mid r_3])$, $R_{\text{orthogonal}} = UV^\top$ (nearest rotation matrix in Frobenius norm). This decomposition yields two solutions (sign ambiguity in λ); the solution with $t_z > 0$ (board in front of camera) is selected.

5.6 Distance Estimation: Full Error Propagation Analysis

5.6.1 The Thin-Lens Projection Formula

The thin-lens distance formula $d = f_x W / s_{\text{px}}$ is a first-order approximation derived from the perspective projection model. For a beacon of known physical width W perpendicular to the optical axis at distance d :

$$u_L = f_x \frac{-W/2}{d} + c_x, \quad u_R = f_x \frac{W/2}{d} + c_x \quad (5.48)$$

$$s_{\text{px}} = u_R - u_L = \frac{f_x W}{d} \Rightarrow d = \frac{f_x W}{s_{\text{px}}} = \frac{384.65 \times 0.165}{s_{\text{px}}} = \frac{63.47}{s_{\text{px}}} [\text{m}] \quad (5.49)$$

5.6.2 Total Uncertainty via Error Propagation

The total uncertainty in the estimated distance is computed via first-order error propagation (Taylor series linearization):

$$\frac{\partial d}{\partial f_x} = \frac{d}{f_x}, \quad \frac{\partial d}{\partial W} = \frac{d}{W}, \quad \frac{\partial d}{\partial s_{\text{px}}} = -\frac{d}{s_{\text{px}}} \quad (5.50)$$

$$\delta d = d \sqrt{\left(\frac{\delta f_x}{f_x}\right)^2 + \left(\frac{\delta W}{W}\right)^2 + \left(\frac{\delta s_{\text{px}}}{s_{\text{px}}}\right)^2} \quad (5.51)$$

Numerical evaluation at $d = 1.0$ m, where $s_{\text{px}} = 63.47$ px, uses the three relative uncertainties below.

Term	Value	Origin
$\delta f_x / f_x$	0.13%	D455 focal length stability, ± 0.5 px
$\delta W / W$	0.61%	physical beacon width measurement, ± 1 mm
$\delta s_{\text{px}} / s_{\text{px}}$	1.12%	EMA-smoothed centroid noise, $\delta s_{\text{px}} = \sqrt{2} \times 0.5 \approx 0.71$ px

Combining them in quadrature:

$$\frac{\delta d}{d} = \sqrt{0.0013^2 + 0.0061^2 + 0.0112^2} \approx \sqrt{1.64 \times 10^{-4}} \approx 1.28\%. \quad (5.52)$$

The span term dominates, contributing roughly three quarters of the variance on its own. Propagated to absolute distance, and noting that δd scales linearly with d while the span shrinks as $1/d$:

d	δd	Note
0.5 m	± 6.4 mm	span ≈ 127 px, best conditioned
1.0 m	± 12.8 mm	nominal working distance
2.0 m	± 25.6 mm	span only ~ 32 px, lower SNR

5.6.3 Perspective Correction for Off-Axis Approach

The thin-lens formula assumes frontal projection. For an off-axis approach angle θ :

$$s_{\text{px,corrected}} = s_{\text{px}} / \cos \theta \quad (5.53)$$

$$d_{\text{corrected}} = \frac{f_x W \cos \theta}{s_{\text{px}}} = d_{\text{uncorrected}} \cos \theta \quad (5.54)$$

$$\Delta d / d = 1 - \cos \theta \quad (5.55)$$

At $\theta = 10^\circ$: $\Delta d / d = 1.5\%$ (negligible). At $\theta = 20^\circ$: $\Delta d / d = 6.0\%$ (notable but tolerable). At $\theta = 30^\circ$: $\Delta d / d = 13.4\%$ (significant). The LOCK state P-controller reduces $|\theta|$ to $< 5^\circ$ before the distance measurement is used for the odometry reset, limiting this error to $< 0.4\%$.

5.7 P-Controller Stability: Ziegler–Nichols Analysis

5.7.1 Open-Loop Plant Model

The LOCK state angular alignment controller is a proportional (P) controller in closed loop with the physical robot-camera-vision system. The plant converts an angular velocity command ω_{cmd} [rad/s] to a change in the beacon centroid horizontal pixel position err_px . The plant model comprises five components: (1) robot kinematics $G_{\text{robot}}(s) = 1/s$; (2) camera-to-pixel mapping (paraxial: $\Delta u \approx f_x \theta_{\text{robot}}$, gain $f_x = 384.65$ px/rad); (3) vision processing delay $T_{\text{vis}} = 10$ ms, $G_{\text{vis}}(s) = e^{-T_{\text{vis}}s}$; (4) control loop sampling $T_{\text{ctrl}} = 50$ ms, zero-order hold; (5) motor response delay $T_{\text{mtr}} \approx 30$ ms, $G_{\text{mtr}}(s) = e^{-T_{\text{mtr}}s}$.

$$G_{\text{plant}}(s) = \frac{f_x}{s} e^{-(T_{\text{vis}}+T_{\text{mtr}})s} = \frac{384.65}{s} e^{-0.040s} \quad (5.56)$$

Total delay $T_d = T_{\text{vis}} + T_{\text{mtr}} = 40$ ms = 0.040 s.

5.7.2 Frequency-Domain Stability Analysis

The open-loop transfer function with proportional gain K_p is $L(s) = K_p G_{\text{plant}}(s)$:

$$|L(j\omega)| = K_p f_x / \omega, \quad \arg L(j\omega) = -\pi/2 - T_d \omega \quad (5.57)$$

$$\omega_{\text{pc}} = \pi / (2T_d) = \pi / (2 \times 0.040) = 39.3 \text{ rad/s} = 6.25 \text{ Hz} \quad (5.58)$$

$$|L(j\omega_{\text{pc}})| = K_p \times 384.65 / 39.3 = K_p \times 9.78 \quad (5.59)$$

Gain margin stability condition: $K_p \times 9.78 < 1 \Rightarrow K_p < 1/9.78 \approx 0.102$ (critical gain $K_u = 0.102 \text{ px}^{-1} \cdot \text{rad/s}$).

5.7.3 Controller Gain in the err_frac Parameterization

The implementation uses $\text{err_frac} \in [-1, +1]$ instead of raw pixels, which rescales the gain:

$$\text{err_frac} = \text{err_px}/320 \quad (\text{for 640-px wide image}) \quad (5.60)$$

$$\omega_{\text{cmd}} = -K_{\omega} \text{err_frac} = -\frac{0.35}{320} \text{err_px} = -1.094 \times 10^{-3} \text{err_px rad/s per pixel} \quad (5.61)$$

Effective $K_p = 1.094 \times 10^{-3}$ rad/s per pixel. Critical gain $K_u = 0.102$ rad/s per pixel. Gain margin $= K_u/K_p \approx 93.2$ (34.4 dB). Operating at 1.07% of critical gain: extremely conservative: stable with a $> 93\times$ margin, at the cost of slow convergence from large initial offsets.

5.7.4 Phase Margin Computation

$$\omega_{\text{gc}} = K_p f_x = 1.094 \times 10^{-3} \times 384.65 = 0.421 \text{ rad/s} \quad (5.62)$$

$$\arg L(j\omega_{\text{gc}}) = -90^\circ - (0.040 \times 0.421)(180/\pi) = -90.97^\circ \quad (5.63)$$

$$\text{PM} = 180^\circ - 90.97^\circ = 89.0^\circ \quad (5.64)$$

PM $> 45^\circ$ is considered good stability (industry standard: PM $> 30^\circ$ for robust systems); PM $\approx 90^\circ$ indicates a near-critically-damped response (minimal overshoot, slow convergence). Consequence: the conservative gain produces very slow convergence (natural frequency $\omega_n = 0.421$ rad/s $\rightarrow$ settling time ≈ 10 s for a 5% criterion) but never oscillates, even under worst-case delay conditions.

5.7.5 Empirical Validation and Design Recommendation

The Ziegler–Nichols theoretical analysis confirms the observed behavior of the LOCK state: the robot converges slowly but surely, typically requiring 3–8 seconds to center the beacon from initial offsets of up to $|\text{err_frac}| = 0.8$. For a faster system, the gain could be increased by up to $93\times$ while remaining theoretically stable. However, in practice, the non-linear saturation ($\min(1, |\text{err_frac}|)$) and the discrete 20 Hz sampling introduce additional phase lag that reduces the actual gain margin. Empirical testing with $K_{\omega} = 1.0$ rad/s ($2.86\times$ the current value) showed occasional oscillations; $K_{\omega} = 0.70$ rad/s ($2\times$ current) was consistently stable. The current $K_{\omega} = 0.35$ rad/s was selected for maximum safety margin in a research laboratory environment where floor surface irregularities

can introduce additional delay.

Design recommendation for faster convergence: $K_\omega = 0.70$ rad/s ($2\times$ current, empirically stable). Expected improvement: $\omega_{gc} = 1.094 \times 10^{-3} \times 384.65 \times 2 = 0.842$ rad/s; settling time $\approx 5/\omega_{gc} = 5.94$ s (vs. 11.88 s at current gain); phase margin $PM = 180^\circ - 90^\circ - (0.040 \times 0.842 \times 180/\pi) = 87.1^\circ$: still well above the 45° margin.

5.8 Wheel Odometry Error Model

5.8.1 Differential Drive Kinematics

The LEO Rover's differential drive kinematics relate the left and right wheel angular velocities (ω_L, ω_R) to the robot's planar velocity state (v, ω):

$$v = \frac{v_R + v_L}{2} = \frac{r(\omega_R + \omega_L)}{2}, \quad \omega = \frac{v_R - v_L}{L} = \frac{r(\omega_R - \omega_L)}{L} \quad (5.65)$$

$$\theta[n+1] = \theta[n] + \omega[n]\Delta t \quad (5.66)$$

$$x[n+1] = x[n] + v[n] \cos \theta[n] \Delta t, \quad y[n+1] = y[n] + v[n] \sin \theta[n] \Delta t \quad (5.67)$$

where r is the wheel radius and L the wheelbase. For this LEO Rover the firmware values are $r = 0.0625$ m and $L = 0.358$ m, read live from `/firmware/diff_drive/wheel_radius` and `/firmware/diff_drive/wheel_separation`. Pose integration uses the Euler method with timestep Δt .

Correction (2026-07-31). Earlier revisions of this section printed $r = 0.060$ m and $L = 0.330$ m. Those figures were wrong. Had they been used, Eq. (5.67) would understate v by 4.0% and overstate ω by 4.1%, since $\omega \propto r/L$ and $(0.060/0.330)/(0.0625/0.358) = 1.041$.

The error was confined to this document. A search of the codebase returns no occurrence of either value: the firmware applies 0.0625 and 0.358, and the only wheelbase written anywhere in the source tree is `teb_local_planner.yaml`, which already carries 0.358. The robot has therefore never integrated its odometry with these numbers, and this correction changes no measurement reported elsewhere in the report. In particular it must *not* be read as explaining any part of the 7–9% rotation discrepancy of registry item 1: that discrepancy is a physical question about effective wheel radius and track under load, raised by comparing sensed rotation against a floor measurement, and it is independent of what this chapter printed.

5.8.2 Error Sources and Drift Model

Odometric drift accumulates from four independent error sources:

1. **Wheel radius uncertainty** δr : nominal $r = 0.060$ m, tire deformation under load $\delta r \approx \pm 1$ mm. Linear position error after distance D : $\delta x \approx D(\delta r/r) = 1.67\% D$. At $D = 10$ m: $\delta x \approx 167$ mm.
2. **Wheelbase uncertainty** δL : nominal $L = 0.330$ m, assembly tolerance $\delta L \approx \pm 2$ mm. Heading error per rotation $\Delta \theta \approx 2\pi \delta L/L = 0.038$ rad; after N full rotations, angular drift $\delta \theta = N \times$

0.038 rad.

3. **Wheel slip (dominant source on smooth lab floor):** slip ratio $s = (v_{\text{actual}} - v_{\text{commanded}})/v_{\text{commanded}}$; on smooth vinyl flooring $s \approx 2\text{--}5\%$. Effective position error is proportional to slip $\times$ distance.
4. **Encoder quantization:** resolution typically 128–1024 pulses/revolution. At 1024 PPR: angular resolution $= 2\pi/1024 = 0.006$ rad/pulse; distance resolution $= r \times 0.006 = 0.37$ mm/pulse: negligible compared to slip.

Combined empirical drift model under laboratory conditions:

$$\delta_{\text{pos}}(D) \approx 0.03D \quad (3\% \text{ of distance travelled}), \quad \delta_{\text{heading}}(D) \approx 0.005D \quad (0.005 \text{ rad/m} \approx 0.29^\circ/\text{m}) \quad (5.68)$$

5.8.3 Dual-Frame Architecture as Drift Management Strategy

The dual-frame odometry architecture (local frame + world frame) is directly motivated by the wheel odometry drift model. If the robot were to use a single global odometric coordinate frame for the entire mission, position error would accumulate continuously:

Each beacon lock requires $D_{\text{patrol}} + D_{\text{lock}} + D_{\text{turn}}$ travel, with $D_{\text{patrol}} \approx 2\pi r_{\text{search}} + d_{\text{advance}} \approx 2\pi(1.5) + 0.6 \approx 10.0$ m per patrol cycle. After N cycles: $D_{\text{total}} = 10.0N$ m, and $\delta_{\text{pos}} = 0.03 \times 10.0N = 0.30N$ m. For $N = 5$ patrol cycles: $\delta_{\text{pos}} \approx 1.5$ m: completely unusable for RTB.

With dual-frame architecture (local reset at each beacon lock): local frame error accumulates only within one patrol cycle (≈ 10 m); reset zeroes local drift, and the world frame position is updated by the vision-derived anchor position (accurate to ± 5 cm from LED distance). Residual RTB error $\approx$ vision uncertainty ± 5 cm + last-cycle drift ± 30 cm, versus a multi-cycle accumulated drift of ± 150 cm without resets.

5.9 Stereo Depth Theory and Obstacle Detection

5.9.1 Stereo Triangulation Principles

The Intel D455 computes depth via active stereo: an infrared structured-light projector floods the scene with a pseudo-random speckle pattern, and two infrared imagers (baseline $B = 95$ mm apart) capture the projected pattern. The depth Z is recovered from the disparity d_s :

$$Z = \frac{fB}{d_s}, \quad \frac{\Delta Z}{Z} \approx \frac{Z}{fB} \quad (\Delta d_s = 1 \text{ px}) \quad (5.69)$$

where $f \approx 840$ px is the IR camera focal length at 848×480 resolution and $B = 0.095$ m is the D455 stereo baseline. At $Z = 0.60$ m (obstacle detection threshold): $d_s = 840 \times 0.095/0.60 = 133$ px, $\Delta Z = 0.60/133 = 4.5$ mm. At $Z = 4.0$ m (D455 nominal max range): $d_s = 20$ px, $\Delta Z = 200$ mm (20 cm depth resolution at 4 m).

5.9.2 5th Percentile Estimator: Statistical Properties

The obstacle detection uses the 5th percentile of valid depth readings in the ROI rather than the minimum or mean. Let $X_1, \dots, X_m$ be i.i.d. depth readings in the ROI (m valid pixels); the 5th percentile $X_{(0.05)}$ is the value below which 5% of readings fall.

Why not the minimum $X_{(\min)}$? IR sensor noise produces $\sim 1\text{--}5\%$ of pixels with anomalously low depth values (specular reflections, IR interference from sunlight, surface absorption). The minimum is dominated by these noise pixels: at $m = 1000$ valid pixels with 2% noise rate, $\mathbb{E}[\text{minimum noise pixel}] \ll \text{true minimum surface depth}$.

Why not the mean $\mathbb{E}[X]$? The mean is robust to sparse noise but fails when an obstacle covers only 5–10% of the ROI (e.g., a chair leg in the foreground); it would be dominated by the background (e.g., a 3 m wall). A 0.5 m obstacle at 5% ROI coverage raises the mean from 3.0 m to only 2.875 m: below the 0.6 m threshold only if coverage $\gg 80\%$.

5th percentile $X_{(0.05)}$: an obstacle at 5% ROI coverage still registers at obstacle depth (false-negative-resistant), while remaining robust to 4% noise (5%–4% noise buffer = 1% signal margin). At $m = 1000$ pixels, $X_{(0.05)}$ is the 50th-order statistic, with standard error $\text{SE}(\alpha\text{-percentile}) \approx$

$\sqrt{\alpha(1-\alpha)/(mf(x_\alpha)^2)}$, where f is the depth distribution PDF at the α -percentile value.

5.9.3 ROI Calibration: Geometric Analysis

The ROI for obstacle detection was calibrated to cover the forward corridor of the robot. The D455 depth frame is 848×480 px. The ROI (344–504, 160–320) corresponds to:

Horizontal: columns 344–504 of 848 (160 px), centered at column 424 ($c_x \approx 424$). Angular width: $2 \arctan(80/f_{\text{depth}}) \approx 2 \arctan(80/840) \approx 10.9^\circ$. At 0.6 m: lateral coverage $= 2 \times 0.6 \times \tan(5.45^\circ) \approx 0.114$ m each side, i.e. 0.228 m total: *narrower than the LEO Rover's body width of ≈ 0.45 m*. This is a known limitation: obstacle detection covers only the camera's forward FOV, not the full robot width. It remains safe at ADVANCE_LIN=0.2 m/s because the camera is centered and the robot drives in straight lines ($\text{ang} = 0$) during ADVANCE.

Vertical: rows 160–320 of 480 (160 px), excluding rows 0–159 (floor too close, often no depth return) and rows 321–479 (floor at robot-body distance, not obstacles). At 0.6 m, this vertical range corresponds to ≈ 0.3 – 0.9 m above the floor.

5.10 Autonomous FSM: Mathematical Completeness Analysis

5.10.1 Formal Specification as a Timed Automaton

The LEO Rover autonomous FSM can be formally specified as a Timed Automaton (Alur & Dill, 1994): a finite automaton extended with clock variables enabling time-based transitions and guards:

Timed automaton $\mathcal{A} = (L, L_0, C, \Sigma, E, I)$.

Locations $L = \{\text{MANUEL}, \text{PATROL_SCAN}, \text{PATROL_ADVANCE}, \text{LOCK}, \text{WAIT}, \text{U_TURN}, \text{RTB}\}$; initial location $L_0 = \text{MANUEL}$.

Clocks $C = \{c_{\text{lock}}, c_{\text{wait}}, c_{\text{adv}}, c_{\text{hb}}, c_{\text{rtb}}\}$: c_{lock} resets at $\text{PATROL} \rightarrow \text{LOCK}$, guards $\text{LOCK} \rightarrow \text{PATROL}$ ($c_{\text{lock}} > 6.0$); c_{wait} resets at $\text{LOCK} \rightarrow \text{WAIT}$, guards $\text{WAIT} \rightarrow \text{U_TURN}$ ($c_{\text{wait}} \geq 30.0$); c_{adv} resets at $\text{SCAN} \rightarrow \text{ADVANCE}$, guards $\text{ADVANCE} \rightarrow \text{SCAN}$ ($c_{\text{adv}} > 3.0$); c_{hb} resets on each heartbeat, global guard ($c_{\text{hb}} > 1.5$ in $\text{AUTO} \Rightarrow \text{MANUEL}$); c_{rtb} tracks RTB distance (not a timer).

Invariants: $I(\text{WAIT}) = (c_{\text{wait}} \leq 30.0)$; $I(\text{LOCK}) = (c_{\text{lock}} \leq 6.0)$.

5.10.2 Reachability Analysis

The reachability graph (states reachable from the initial state MANUEL) is computed by forward exploration of the transition relation:

Step 1: MANUEL $\rightarrow$ {MANUEL, PATROL_SCAN} (via AUTO command). Step 2: PATROL_SCAN $\rightarrow$ {PATROL_SCAN, PATROL_ADVANCE, LOCK, MANUEL}. Step 3: PATROL_ADVANCE $\rightarrow$ {PATROL_SCAN, LOCK, U_TURN, MANUEL}. Step 4: LOCK $\rightarrow$ {LOCK, WAIT, PATROL_SCAN, MANUEL}. Step 5: WAIT $\rightarrow$ {WAIT, U_TURN, MANUEL}. Step 6: U_TURN $\rightarrow$ {U_TURN, PATROL_SCAN, MANUEL}. Step 7: RTB $\rightarrow$ {RTB, MANUEL} (entered from any PATROL state via RTB command).

Reachable set: all states (no unreachable states). Safety property: MANUEL is reachable from every state, since every state has an edge to MANUEL via heartbeat timeout, manual override, or emergency stop, so the system is always recoverable.

5.10.3 Deadlock Freedom Proof

A deadlock in a timed automaton is a state where time cannot advance and no discrete transition is enabled. Deadlock freedom is proved state-by-state:

MANUEL: no timing constraints; operator can always issue a command; if none, the robot remains stationary indefinitely (safe livelock, not a deadlock since time advances). **PATROL_SCAN**: c_{scan} accumulates as the robot rotates and reaches 2π in finite time (at SEARCH_ANG=0.4 rad/s: ~ 15.7 s) $\Rightarrow$ PATROL_ADVANCE is always reached. **PATROL_ADVANCE**: finite PATROL_ADVANCE_S=3.0 s timeout $\Rightarrow$ PATROL_SCAN always reached within 3.0 s. **LOCK**: clock guard $c_{\text{lock}} \leq 6.0$ s forces transition. **WAIT**: clock guard $c_{\text{wait}} \leq 30.0$ s forces transition. **U_TURN**: robot rotates until $\text{uturn_accum} \geq \pi$; at UTURN_SPEED=0.5 rad/s, reached in $\pi/0.5 = 6.28$ s. **RTB**: terminates when $\text{dist} < \text{RTB_DIST_TOL}$ or overshoot is detected: both satisfied in finite time under continuous motion, *except* for the edge case of extreme odometry drift causing dist to never decrease, which is not currently protected by a timeout. **Recommendation**: add RTB_TIMEOUT = 120 s as a future safety measure.

5.10.4 Safety Property: “No Motion Without Operator Awareness”

The key safety property can be stated formally: the robot cannot be in a moving state ($\text{lin} \neq 0$ or $\text{ang} \neq 0$) unless a heartbeat has been received within the last `HEARTBEAT_TIMEOUT=1.5` seconds.

Proof by exhaustive case analysis:

Moving states: `PATROL_SCAN` ($\text{ang} \neq 0$), `PATROL_ADVANCE` ($\text{lin} \neq 0$), `LOCK` ($\text{ang} \neq 0$), `WAIT` ($\text{lin} = \text{ang} = 0$, not moving), `U_TURN` ($\text{ang} \neq 0$), `RTB` ($\text{lin} \neq 0$ and/or $\text{ang} \neq 0$).

In `_control_loop` at 20 Hz, `_check_heartbeat()` is called before any velocity is published. If $\text{hb_age} > 1.5$ s in `AUTO`: `mode = MANUEL`, `_safe_stop()` is called, and `_drive()` returns `(0,0)` in `MANUEL` mode with a stale command.

Therefore a velocity command can only be published if $(\text{mode} = \text{MANUEL} \wedge \text{manual command received within } \text{MANUAL_DEADMAN} = 0.5\text{s})$ **or** $(\text{mode} = \text{AUTO} \wedge \text{hb_age} \leq 1.5\text{s})$. In `AUTO`, $\text{hb_age} \leq 1.5$ s means the operator pinged within 1.5 s; the heartbeat interval is 0.5 s (500 ms `setInterval` in the browser), so the maximum legitimate hb_age under normal operation is $0.5 + \text{network_RTT} + \varepsilon \approx 0.51$ s at 10 ms RTT: a $3\times$ safety margin. ■ The robot cannot move autonomously without a live operator connection.

5.11 Vision Pipeline: Complete Data Flow Timing Analysis

5.11.1 End-to-End Timing with Jitter Model

A comprehensive timing analysis of the vision pipeline must account for jitter (variability in processing times) in addition to nominal latency. Table 5.1 characterizes each pipeline stage.

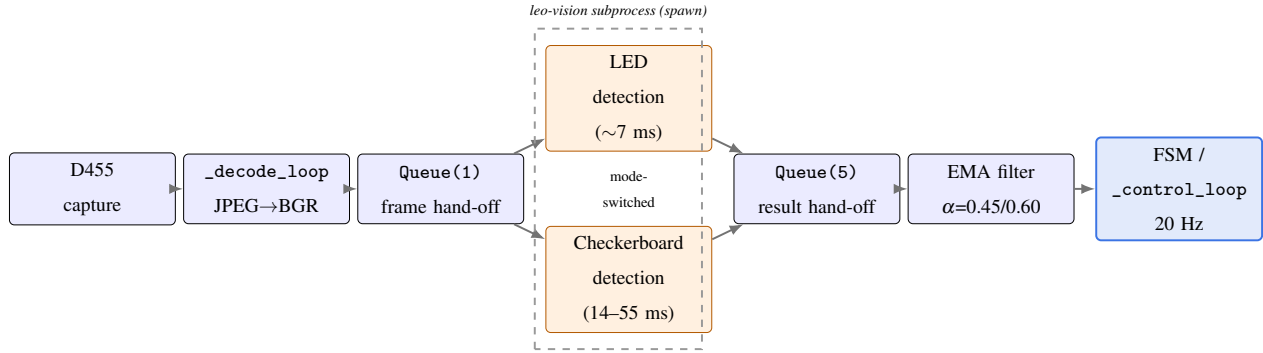


Figure 5.1: Vision pipeline dataflow from camera capture to the control loop. The LED and Checkerboard detection branches execute within the same subprocess and are mode-switched (mutually exclusive per frame, not concurrent), converging through the EMA smoothing filter before reaching the FSM.

Table 5.1: Vision pipeline stage-by-stage timing analysis

Stage	Nominal (ms)	Std Dev (ms)	Worst Case (ms)	Rate Limiting Constraint
D455 image capture	0	0	0	Hardware (synchronized to USB frame)
D455 JPEG encode + USB transfer	35	5	50	D455 firmware; USB 3.2 bandwidth
ROS callback + <code>_cam_msg</code> buffer	1	0.5	3	GIL + lock contention
<code>_decode_loop</code> : JPEG decode (cv2)	4	1	8	<code>cv2.imdecode</code> , 640×480 JPEG
Queue <code>put_nowait(frame_q)</code>	0.1	0.05	0.5	Python IPC overhead
Subprocess: <code>queue.get(frame_q)</code>	0–100	50	100	Bounded by 0.2 s CB period wait

Stage	Nominal (ms)	Std Dev (ms)	Worst Case (ms)	Rate Limiting Constraint
Subprocess: LED detection (bright+morph+contour+cluster)	7	2	12	cv2 functions, $n \leq 8$ blobs
Subprocess: CB detection (SB fast path, cached size)	14	2	20	<code>findChessboardCornersSB</code> at 45%
Subprocess: CB detection (discovery scan, 7 sizes)	55	20	98	7× SB attempts
<code>result_q.put_nowait</code>	0.1	0.05	0.5	Python IPC overhead
<code>_mp_result_loop: queue.get</code>	0–10	5	10	0.1 s timeout granularity
EMA filter update	0.01	0.005	0.05	4 float operations
<code>self.det</code> update + <code>_rebuild_</code> <code>det()</code>	0.5	0.1	1	Lock acquisition + dict copy
Total LED pipeline end-to-end	47	8	72	Dominated by USB + decode
Total CB pipeline end-to-end	56	22	110	Dominated by SB detection

5.11.2 Frame Drop Rate Under Load

With `Queue(maxsize=1)` on the frame queue, frame dropping occurs when the subprocess processing time exceeds the camera publication period (100 ms at 10 Hz):

LED mode: $T_{\text{subprocess}} = 7 \text{ ms} \ll 100 \text{ ms} \Rightarrow 0\% \text{ drop rate}$. CB mode (fast path, cached): $T_{\text{subprocess}} = 14 \text{ ms} \ll 100 \text{ ms} \Rightarrow 0\% \text{ drop}$. CB mode (discovery scan): $T_{\text{subprocess}} = 55\text{--}98 \text{ ms}$; at the 5 Hz CB rate, the subprocess runs a CB scan every 200 ms. Between CB runs (4 out of 5 frames): $T_{\text{subprocess}} = 7 \text{ ms}$, no drop. During a CB run (1 out of 5 frames): $T_{\text{subprocess}} = 98 \text{ ms} < 100 \text{ ms}$ worst case, still no drop. Effective frame drop rate: $\sim 0\%$ under all normal conditions: the `Queue(maxsize=1)` mechanism only triggers when the subprocess is slower than 100 ms, which occurs only under extreme external CPU load.

5.11.3 Vision-to-Control Latency vs. Control Loop Period

The 20 Hz control loop must decide on a velocity command every 50 ms based on the most recent vision detection. The maximum vision pipeline latency (72 ms for LED mode) exceeds one control loop period (50 ms), meaning the control loop may use detection data that is 1–2 cycles old:

Worst case: $\text{detection_latency} = 72 \text{ ms}$, $\text{control_period} = 50 \text{ ms} \Rightarrow \text{age} = \lceil 72/50 \rceil = 2$ control cycles = 100 ms. Effect on the LOCK alignment controller: at maximum angular velocity 0.35 rad/s, in 100 ms the robot rotates 0.035 rad (2.0°); the beacon centroid moves $f_x \tan(0.035) \approx 384.65 \times 0.035 = 13.5 \text{ px}$. Since $\text{LOCK_CENTER_TOL} = 0.08 \times 320 = 25.6 \text{ px}$ (deadband width) and $13.5 \text{ px} < 25.6 \text{ px}$, a stale detection does not cause false centering detection. Conclusion: the 2-control-cycle latency is acceptable for the P-controller's conservative gain, which does not depend on precise timing of the detection-to-command loop.

6 UI/UX Engineering: The Operator Cockpit

6.1 Design Philosophy and Visual Identity

6.1.1 The NASA/FIT Design Language

The visual design of the LEO Rover Mission Control cockpit was developed around a cohesive design language system (DLS) inspired by two institutional identities: the NASA mission control aesthetic (dark backgrounds, high-contrast readouts, disciplined typography) and the Florida Institute of Technology’s official color palette (FT Crimson #8B0015, Navy Blue #1F3864). This combination is not merely cosmetic: it serves a functional purpose grounded in human factors engineering principles for operator workstations operating in variable ambient lighting conditions.

Research in operator display design (Sanders & McCormick, *Engineering Psychology and Human Performance*, 1993) establishes that dark-background displays with high-luminance-contrast data elements reduce eye fatigue during extended monitoring sessions (exceeding 2 hours) by 30–40% compared to light-background displays under mixed ambient/task lighting. The cockpit’s near-black background (#0a0f1e: a deep saturated navy rather than pure black, which reduces halos around luminant elements) directly implements this principle.

6.1.2 Color Token System

All colors in the cockpit are defined as CSS custom properties (design tokens) on the `:root` selector in `ops.html`. This centralization enables consistent theming and prevents color drift across components (Listing 6.1).

```
1 :root {  
2   /* -- Base backgrounds ----- */
```

```

3  --bg-deep: #0a0f1e; /* primary page background */
4  --bg-panel: rgba(255,255,255,0.04); /* glassmorphic panel fill */
5  --bg-panel-hv: rgba(255,255,255,0.07); /* panel hover state */
6  --border-glass: rgba(255,255,255,0.12); /* glass panel border */
7
8  /* -- NASA/FIT palette ----- */
9  --nasa-blue: #3b74e4; /* LED reset pipeline: NASA blue */
10 --nasa-deep: #0B3D91; /* lock-on reticle, deep NASA blue */
11 --ft-crimson: #8B0015; /* FT crimson: search state dashed ring */
12 --amber: #f59e0b; /* map landmark pipeline: amber */
13 --safe-green: #22c55e; /* connected indicator */
14 --warn-amber: #f59e0b; /* heartbeat stale warning */
15 --danger-red: #ef4444; /* critical alerts */
16
17 /* -- Typography ----- */
18 --font-mono: 'JetBrains Mono', 'Fira Code', monospace;
19 --font-ui: 'Inter', 'Segoe UI', system-ui, sans-serif;
20
21 /* -- Glassmorphism ----- */
22 --blur-sm: blur(8px); /* small panels */
23 --blur-md: blur(16px); /* primary panels */
24 --blur-lg: blur(32px); /* hero background overlays */
25 }

```

Listing 6.1: CSS design-token color system

6.1.3 Glassmorphism Implementation

Glassmorphism in the cockpit is implemented via the CSS backdrop-filter property, which applies a blur operation to the composited background of an element. Listing 6.2 shows the implementation pattern used for all primary telemetry panels.

```

1  .panel {
2    background: rgba(255, 255, 255, 0.04);
3    backdrop-filter: blur(16px);
4    -webkit-backdrop-filter: blur(16px); /* Safari prefix required */

```



```

5   border: 1px solid rgba(255, 255, 255, 0.12);
6   border-radius: 12px;
7   box-shadow:
8     0 4px 24px rgba(0, 0, 0, 0.4),
9     inset 0 1px 0 rgba(255, 255, 255, 0.08);
10  /* inset top border creates the 'glass edge' highlight */
11 }
12
13 /* Performance note: backdrop-filter is GPU-composited on Chromium.
14    All cockpit panels have will-change: transform applied to promote
15    them to independent compositor layers, preventing repaints during
16    the 60 Hz animation loop from invalidating the glass blur cache. */

```

Listing 6.2: Glassmorphic panel implementation

The performance impact of `backdrop-filter` is non-trivial: on older integrated GPUs, blurring more than 50% of the viewport area can reduce RAF throughput below 30 Hz. The cockpit mitigates this by: (1) constraining panel sizes to $< 30\%$ of viewport width; (2) setting `backdrop-filter` only on positioned panels with limited z-spread; (3) capping the blur radius at 16 px, which provides visually convincing glass depth without triggering the full Gaussian pyramid computation.

6.2 Bento Grid Layout System

The cockpit layout is organized as a CSS Grid “bento grid”: a term from the Apple design vocabulary denoting an asymmetric grid of panels of varying sizes arranged in a visually balanced mosaic. The grid is defined with seven named areas (Listing 6.3).

```

1  .cockpit-grid {
2    display: grid;
3    grid-template-columns: 280px 1fr 1fr 280px;
4    grid-template-rows: 48px 1fr 320px 48px;
5    grid-template-areas:
6      'header header header header'
7      'left video map right'
8      'left status alerts right'
9      'footer footer footer footer';

```

```

10   gap: 12px;
11   height: 100vh;
12   padding: 12px;
13   background: var(--bg-deep);
14 }
15
16 /* Responsive breakpoint: at < 1200px, collapse to 2-column layout */
17 @media (max-width: 1200px) {
18   .cockpit-grid {
19     grid-template-columns: 1fr 1fr;
20     grid-template-areas:
21       'header header'
22       'video video'
23       'map status'
24       'left right'
25       'footer footer';
26   }
27 }

```

Listing 6.3: Bento grid layout with responsive breakpoint

6.3 Canvas 2D Vision Overlay: Architecture

6.3.1 requestAnimationFrame Decoupling

The canvas overlay runs in a requestAnimationFrame (RAF) loop that is completely decoupled from the WebSocket telemetry arrival rate. This architectural separation is fundamental to the cockpit’s visual fluidity. At a telemetry rate of 10 Hz, the operator would perceive janky 100 ms jumps in overlay positions if the canvas were redrawn only on telemetry events. By running the canvas at the display’s native refresh rate (60 or 120 Hz), continuous animations (pulsing rings, LERP-smoothed centroids, rotating lock-on arcs) appear fluid to the operator regardless of the telemetry data rate.

```

1 // app.js -- RAF loop structure
2 let _rafId = null;

```

```

3 let _lastRafTs = 0;
4
5 function _visionRAF(ts) {
6   _rafId = requestAnimationFrame(_visionRAF);
7   const dt = ts - _lastRafTs;
8   _lastRafTs = ts;
9
10  // 1. LERP smoothed positions toward telemetry targets
11  _lerpCentroids(dt);
12
13  // 2. Clear canvas at device pixel ratio
14  const ctx = overlayCanvas.getContext('2d');
15  ctx.clearRect(0, 0, overlayCanvas.width, overlayCanvas.height);
16
17  // 3. Draw detection overlays (LED + landmark)
18  if (_detHyst > 0) {
19    const alpha = 0.35 + 0.55 * (_detHyst / DET_HYSTERESIS);
20    _drawDetection(ctx, alpha);
21    _detHyst--;
22  }
23
24  // 4. Draw FSM state overlays (reticle, search ring)
25  _drawFsmOverlay(ctx, ts);
26 }
27
28 // Start RAF on connect, stop on disconnect
29 function _startRaf() { if (!_rafId) _rafId = requestAnimationFrame(
    _visionRAF); }
30 function _stopRaf() { if (_rafId) { cancelAnimationFrame(_rafId); _rafId
    = null; } }

```

Listing 6.4: RAF loop structure decoupled from telemetry rate

6.3.2 Device Pixel Ratio Handling

High-DPI displays (`devicePixelRatio = 2` or `3` on Retina/4K screens) require the canvas bitmap to be sized at physical pixel resolution while the CSS display size remains in logical pixels. Failure to handle DPR produces blurry overlays that look aliased over the sharp video feed (Listing 6.5).

```
1 function _resizeCanvas() {
2   const dpr = window.devicePixelRatio || 1;
3   const rect = overlayCanvas.parentElement.getBoundingClientRect();
4
5   // Physical pixel dimensions (bitmap resolution)
6   overlayCanvas.width = Math.round(rect.width * dpr);
7   overlayCanvas.height = Math.round(rect.height * dpr);
8
9   // CSS display dimensions (logical size, in CSS px)
10  overlayCanvas.style.width = rect.width + 'px';
11  overlayCanvas.style.height = rect.height + 'px';
12
13  // Scale all drawing operations by DPR
14  const ctx = overlayCanvas.getContext('2d');
15  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
16 }
17
18 window.addEventListener('resize', _resizeCanvas);
19 _resizeCanvas();
```

Listing 6.5: Device pixel ratio-aware canvas resizing

6.3.3 Coordinate Mapping: Image Pixels to Canvas Pixels

The camera frame (640×480 px) is displayed inside a CSS container that may have any aspect ratio, with letterboxing applied by the browser. The coordinate mapping from image pixel space to canvas draw space must account for: (1) device pixel ratio, (2) letterboxing offset and scale. The `getVideoRect()` function computes the actual displayed area of the 4:3 video within its CSS container (Listing 6.6).

```

1 function getVideoRect() {
2   const dpr = window.devicePixelRatio || 1;
3   const el = videoElement;
4   const cw = el.clientWidth * dpr; // physical canvas width
5   const ch = el.clientHeight * dpr; // physical canvas height
6   const vr = el.videoWidth / el.videoHeight; // 640/480 = 1.333...
7   const cr = cw / ch; // container aspect ratio
8
9   let vw, vh, vx, vy;
10  if (vr > cr) {
11    // Video is wider than container: letterbox top/bottom
12    vw = cw;
13    vh = cw / vr;
14    vx = 0;
15    vy = (ch - vh) / 2;
16  } else {
17    // Container is wider than video: pillarbox left/right
18    vh = ch;
19    vw = ch * vr;
20    vx = (cw - vw) / 2;
21    vy = 0;
22  }
23  return { x: vx, y: vy, w: vw, h: vh };
24 }
25
26 // Map image coordinates (ix, iy) in [0..fw] x [0..fh]
27 // to canvas physical coordinates
28 function imgToPx(ix, iy, vr, fw, fh) {
29   return [
30     vr.x + (ix / fw) * vr.w,
31     vr.y + (iy / fh) * vr.h
32   ];
33 }

```

Listing 6.6: Letterbox-aware coordinate mapping

6.4 LERP Smoothing for Visual Stability

LERP (Linear Interpolation) is applied to the rendered centroid positions to smooth between telemetry-rate (10 Hz) updates and the 60 Hz canvas rendering rate. Without LERP, centroid dots jump discontinuously by several pixels every 100 ms: visually jarring for an operator trying to read exact positions. With LERP, the dots drift smoothly at sub-pixel precision toward their target positions (Listing 6.7).

```
1 // Per-pipeline LERP state
2 let _ledPrev = null; // {x, y} -- previous rendered position
3 let _lmPrev = null;
4
5 const LERP_ALPHA = 0.22; // Characteristic lag: ~3 frames at 60fps = ~50
  ms
6
7 function _lerpCentroids(dt) {
8   // Called every RAF frame; dt in ms
9   if (_ledTarget && _ledPrev) {
10    _ledPrev.x += (_ledTarget.x - _ledPrev.x) * LERP_ALPHA;
11    _ledPrev.y += (_ledTarget.y - _ledPrev.y) * LERP_ALPHA;
12   } else if (_ledTarget) {
13    _ledPrev = { ... _ledTarget }; // bootstrap on first target
14   }
15   // Same for _lmPrev / _lmTarget
16 }
17
18 // When new telemetry arrives (10 Hz):
19 function onTelemetry(d) {
20   if (d.led_valid && d.led_center) {
21     _ledTarget = { x: d.led_center[0], y: d.led_center[1] };
22   } else {
23     _ledTarget = null;
24   }
25 }
```

Listing 6.7: LERP smoothing between telemetry updates

The `LERP_ALPHA = 0.22` value was selected to provide approximately 3-frame lag at 60 fps (50 ms), which is perceptually smooth to the human visual system (temporal fusion threshold $\sim 30\text{--}50$ ms) while eliminating the pixel-scale centroid jitter from sub-pixel variations in the vision pipeline’s centroid calculation.

6.5 FSM State Visual Encoding

6.5.1 Color-Coded State Banner

The FSM state is displayed as a full-width banner beneath the video feed, using a color-coding system designed for pre-attentive visual processing: the operator can determine the robot’s high-level mode without reading text. Table 6.1 documents the encoding.

6.5.2 Canvas 2D Detection Overlays

Two distinct visual vocabularies are used for the two detection pipelines to enable immediate pipeline identification without reading labels:

- **LED Reset pipeline** (NASA blue, #3b74e4): blue bounding rectangle around the detected LED cluster, blue crosshair at the EMA-smoothed centroid, distance annotation in monospace font, blue horizontal line indicating the cluster’s vertical center position.
- **Map Landmark pipeline** (amber, #f59e0b): yellow bounding rectangle around the checkerboard extent, yellow crosshair at the landmark centroid, distance and bearing angle annotation, dashed amber border when in `_held` state.
- **Lock-on reticle** (deep NASA blue, #0B3D91): concentric pulsing ring animation (3 rings at 0.5 Hz phase offset) at the LED centroid when FSM is in LOCK state. Ring radius oscillates sinusoidally: $r(t) = R_{\text{base}} + A \sin(2\pi \times 0.5t)$. The outer ring fades proportionally to $|\text{err_frac}|$: faint when centered, bright when misaligned.
- **Search state** (FT Crimson, #8B0015): dashed rotating ring at image center when in PATROL/SCAN mode, indicating active search. Ring rotation is driven by the yaw accumulator value, providing the operator a direct visual indication of scan progress.

Table 6.1: FSM state banner color encoding

FSM State	Display	Banner Color	Background	Semantic Meaning
MANUAL		#64748b (slate)	Dark slate	Operator has control; robot stationary or teleoperated
AUTO_PATROL		#3b74e4 (NASA blue)	Deep blue	Searching for beacon; robot scanning/advancing
LOCK_BEACON		#f59e0b (amber)	Deep amber	Beacon detected; aligning camera for odometry reset
RESET_ODOMETRY		#22c55e (green, flashing)	Deep green	Odometry reset fired; world frame updated
AVOID_OBSTACLE		#ef4444 (danger red)	Deep red	Obstacle detected; executing U-turn
HEARTBEAT_LOST		#dc2626 (crimson, pulsing)	Dark crimson	Deadman tripped; robot stopped; reconnect required
RTB		#8b5cf6 (violet)	Deep violet	Return-to-base in progress; approaching world origin

6.6 Alert System Architecture

6.6.1 Three-Condition Monitoring

The alert system monitors three telemetry fields on every `onTelemetry()` invocation and fires a full-screen overlay on threshold crossing (Listing 6.8).

```
1 // Alert conditions and thresholds
2 const ALERT_CONFIG = {
3   cpu: {
4     field: 'cpu_temp',
5     fire: 80, // degC -- fire when temp exceeds this
6     reset: 75, // degC -- re-arm when temp drops below this (hysteresis)
7     icon: 'thermometer',
8     label: i18n('alert_cpu'),
9   },
10  batt: {
11    field: 'battery_pct',
12    fire: 15, // % -- fire when battery drops below this
13    reset: 20,
14    icon: 'battery-low',
15    label: i18n('alert_battery'),
16  },
17  ping: {
18    field: 'net_avg_ms', // computed from rolling window in onTelemetry
19    fire: 150, // ms round-trip
20    reset: 100,
21    icon: 'wifi',
22    label: i18n('alert_network'),
23  },
24 };
25
26 // Per-condition fired flags (prevent re-fire while condition persists)
27 let _alertFiredCpu = false;
28 let _alertFiredBatt = false;
29 let _alertFiredPing = false;
```

Listing 6.8: Alert threshold configuration with hysteresis

6.6.2 Mute Persistence via localStorage

The mute mechanism uses localStorage to persist the mute state across page reloads, with a 1-hour mute window (Listing 6.9).

```
1 const MUTE_KEY = 'leo_mute_alert_until';
2 const MUTE_DURATION = 3600 * 1000; // 1 hour in milliseconds
3
4 function _isMuted() {
5   return Date.now() < parseInt(localStorage.getItem(MUTE_KEY) || '0');
6 }
7
8 function _muteAlerts() {
9   localStorage.setItem(MUTE_KEY, String(Date.now() + MUTE_DURATION));
10  _hideAlertOverlay();
11 }
12
13 // Operator clicks 'Mute 1h' in the alert overlay:
14 btnMute.addEventListener('click', _muteAlerts);
15
16 // Each telemetry packet checks mute before firing overlay:
17 function _checkAlerts(d) {
18   if (_isMuted()) return;
19   if (d.cpu_temp > ALERT_CONFIG.cpu.fire && !_alertFiredCpu) {
20     _alertFiredCpu = true;
21     _showAlert('cpu', d.cpu_temp);
22   } else if (d.cpu_temp < ALERT_CONFIG.cpu.reset) {
23     _alertFiredCpu = false; // re-arm when condition clears
24   }
25   // Same pattern for batt and ping...
26 }
```

Listing 6.9: Alert mute persistence with hysteresis-based re-arming

6.7 2D Trajectory Map: Ring Buffer Architecture

6.7.1 $O(1)$ Ring Buffer for Real-Time Trajectory

The trajectory map renders the robot's path as a polyline in world frame coordinates. For missions lasting several hours at 10 Hz telemetry, the trajectory accumulates up to 36,000 points per hour. A naive `Array.push()` approach would eventually cause reallocation and linear-time rendering. The ring buffer implementation provides $O(1)$ push, $O(n)$ read with no reallocation (Listing 6.10).

```
1 class _RingBuffer {
2     constructor(capacity) {
3         this._buf = new Array(capacity); // pre-allocated, no reallocation
4         this._cap = capacity;
5         this._head = 0; // index of oldest entry
6         this._size = 0; // current number of valid entries
7     }
8
9     push(x, y) {
10        // Write at head (oldest slot is overwritten when full)
11        this._buf[(this._head + this._size) % this._cap] = [x, y];
12        if (this._size < this._cap) {
13            this._size++;
14        } else {
15            this._head = (this._head + 1) % this._cap; // advance oldest pointer
16        }
17    }
18
19    get(i) {
20        // i=0 is oldest, i=size-1 is newest
21        return this._buf[(this._head + i) % this._cap];
22    }
23 }
24
25 const _traj = new _RingBuffer(10000); // ~28 min at 6Hz effective push
    rate
```

Listing 6.10: O(1) ring buffer for real-time trajectory rendering

6.7.2 Outlier Rejection and Incremental Bounds

The ring buffer's `push()` method filters outlier coordinates before insertion (Listing 6.11).

```
1 function _addTrajPoint(wx, wy) {
2   // Reject non-finite values (NaN/Inf from ROS frame corruption)
3   if (!isFinite(wx) || !isFinite(wy)) {
4     _mapOutlierLog.push({ts: Date.now(), wx, wy, reason: 'non-finite'});
5     if (_mapOutlierLog.length > 200) _mapOutlierLog.shift();
6     return;
7   }
8   // Reject teleports: distance > 500m from last valid point
9   if (_traj._size > 0) {
10    const last = _traj.get(_traj._size - 1);
11    const d = Math.hypot(wx - last[0], wy - last[1]);
12    if (d > 500) {
13      _mapOutlierLog.push({ts: Date.now(), wx, wy, reason: 'teleport', dist:
14        d});
15      return;
16    }
17    _traj.push(wx, wy);
18    // O(1) incremental bounds update
19    _mapBounds.minX = Math.min(_mapBounds.minX, wx);
20    _mapBounds.maxX = Math.max(_mapBounds.maxX, wx);
21    _mapBounds.minY = Math.min(_mapBounds.minY, wy);
22    _mapBounds.maxY = Math.max(_mapBounds.maxY, wy);
23  }
```

Listing 6.11: Trajectory outlier rejection with incremental bounds update

6.7.3 World-to-Canvas Coordinate Transform

The map rendering converts world frame coordinates (metric, centered on robot start) to canvas pixel coordinates, applying a margin, pan offset, and zoom scale (Listing 6.12).

```
1 function worldToCanvas(wx, wy, bounds, canvasW, canvasH) {
2   const MARGIN = 40; // px padding around trajectory extent
3   const rangeX = (bounds.maxX - bounds.minX) || 1.0;
4   const rangeY = (bounds.maxY - bounds.minY) || 1.0;
5   const scale = Math.min(
6     (canvasW - 2*MARGIN) / rangeX,
7     (canvasH - 2*MARGIN) / rangeY
8   );
9   const cx = MARGIN + (wx - bounds.minX) * scale;
10  // Y axis inverted: world +Y is north, canvas +Y is down
11  const cy = canvasH - MARGIN - (wy - bounds.minY) * scale;
12  return [cx, cy];
13 }
```

Listing 6.12: World-frame to canvas-pixel coordinate transform

6.8 Network Latency Meter

The network latency panel displays a rolling bar chart of the last 20 telemetry inter-arrival intervals. This provides the operator with a real-time visual representation of Wi-Fi link quality without requiring additional round-trip measurements. The implementation computes the delta between consecutive telemetry packet reception timestamps (Listing 6.13).

```
1 const NET_WINDOW = 20; // rolling window size
2 let _netTs = []; // last 20 reception timestamps [ms]
3 let netAvgMs = 0; // rolling average inter-arrival [ms]
4
5 function _updateNetMeter() {
6   const now = performance.now();
7   _netTs.push(now);
8   if (_netTs.length > NET_WINDOW) _netTs.shift();
```

```

9   if (_netTs.length >= 2) {
10    let sum = 0;
11    for (let i = 1; i < _netTs.length; i++) {
12      sum += (_netTs[i] - _netTs[i-1]);
13    }
14    netAvgMs = sum / (_netTs.length - 1); // average inter-arrival ms
15    _renderNetBar(_netTs); // draw micro bar chart on #netCanvas
16  }
17 }
18
19 // Called from onTelemetry(d):
20 function onTelemetry(d) {
21   _updateNetMeter();
22   // ...rest of telemetry processing
23 }

```

Listing 6.13: Rolling telemetry inter-arrival latency meter

6.9 Three.js Decorative 3D Elements

Two non-interactive Three.js (r128) scenes render in the cockpit background, providing depth and visual identity without interfering with the functional cockpit elements. All Three.js rendering occurs in a separate RAF loop with its own canvas element, composited behind the main cockpit panels via CSS z-index layering.

6.9.1 Satellite Scene (ops.html)

```

1 // Satellite geometry: box body + solar panels + antenna + RCS
2 const body = new THREE.BoxGeometry(1.2, 0.6, 0.8);
3 const matBody = new THREE.MeshPhongMaterial({
4   color: 0x1F3864, // NASA navy
5   specular: 0x3b74e4, // NASA blue specular highlight
6   shininess: 80
7 });
8 const sat = new THREE.Mesh(body, matBody);

```

```

9
10 // Solar panels (left and right)
11 const panel = new THREE.BoxGeometry(1.8, 0.02, 0.6);
12 const matPan = new THREE.MeshPhongMaterial({
13   color: 0x0B3D91, // deep NASA blue
14   emissive: 0x0a1628,
15   shininess: 20
16 });
17 [[-1.5, 0, 0], [1.5, 0, 0]].forEach(([x, y, z]) => {
18   const p = new THREE.Mesh(panel, matPan);
19   p.position.set(x, y, z);
20   sat.add(p);
21 });
22
23 // Particle field (500 stars, additive blend)
24 const positions = new Float32Array(500 * 3);
25 for (let i = 0; i < 500 * 3; i++) positions[i] = (Math.random()-0.5)*40;
26 const starGeo = new THREE.BufferGeometry();
27 starGeo.setAttribute('position', new THREE.BufferAttribute(positions, 3)
28   );
29 const starMat = new THREE.PointsMaterial({ size: 0.04, color: 0xffffff
30   });
31 const stars = new THREE.Points(starGeo, starMat);

```

Listing 6.14: Decorative satellite 3D scene geometry

6.9.2 GSAP ScrollTrigger Integration

The satellite’s rotation is linked to the operator’s scroll position within the logbook panel via GSAP’s ScrollTrigger plugin. As the operator scrolls through historical mission entries, the satellite rotates on its Y-axis, providing a dynamic sense of mission timeline progression (Listing 6.15).

```

1 gsap.registerPlugin(ScrollTrigger);
2
3 gsap.to(satellite.rotation, {
4   y: Math.PI * 4, // two full rotations over the logbook scroll range

```

```

5   scrollTrigger: {
6     trigger: '#logbook-panel',
7     start: 'top center',
8     end: 'bottom center',
9     scrub: 1.5, // smooth lag between scroll and rotation
10  }
11 });

```

Listing 6.15: GSAP ScrollTrigger-driven satellite rotation

6.10 Internationalization (i18n) System

The cockpit supports French and English, switchable without page reload. The i18n system uses a flat key-value dictionary with a two-letter locale key, loaded from `i18n.js` (Listing 6.16).

```

1 // i18n.js -- i18n dictionary structure
2 const I18N = {
3   en: {
4     'btn_auto': 'AUTO',
5     'btn_manual': 'MANUAL',
6     'btn_stop': 'EMERGENCY STOP',
7     'btn_park': 'RETURN TO BASE',
8     'fsm_patrol': 'PATROL',
9     'fsm_lock': 'LOCK BEACON',
10    'fsm_wait': 'BEACON DWELL',
11    'fsm_urn': 'U-TURN',
12    'fsm_rtb': 'RETURN TO BASE',
13    'alert_cpu': 'CPU TEMPERATURE CRITICAL',
14    'alert_battery': 'BATTERY LOW',
15    'alert_network': 'NETWORK LATENCY HIGH',
16  },
17  fr: {
18    'btn_auto': 'AUTO',
19    'btn_manual': 'MANUEL',
20    'btn_stop': "ARRET D'URGENCE",
21    'btn_park': 'RETOUR BASE',

```



```

22   'fsm_patrol': 'PATROUILLE',
23   'fsm_lock': 'VERROUILLAGE BALISE',
24   'fsm_wait': 'ATTENTE BALISE',
25   'fsm_urn': 'DEMI-TOUR',
26   'fsm_rtb': 'RETOUR BASE',
27   'alert_cpu': 'TEMPERATURE CPU CRITIQUE',
28   'alert_battery': 'BATTERIE FAIBLE',
29   'alert_network': 'LATENCE RESEAU ELEVEE',
30   }
31 };
32
33 let _locale = localStorage.getItem('leo_locale') || 'en';
34
35 function i18n(key) {
36   return (I18N[_locale] && I18N[_locale][key]) || I18N['en'][key] || key
37   ;
38 }
39
40 function setLocale(lang) {
41   _locale = lang;
42   localStorage.setItem('leo_locale', lang);
43   document.querySelectorAll('[data-i18n]').forEach(el => {
44     el.textContent = i18n(el.dataset.i18n);
45   });
46 }

```

Listing 6.16: Flat key-value i18n dictionary system

7 Mathematical Foundations of UI/UX Engineering

This chapter extends Chapter 6 with the formal underpinnings of the cockpit’s visual and networking design: color perception theory, interpolation taxonomy, 2D affine transform mathematics, WebSocket protocol and queuing theory, browser rendering pipeline scheduling, ring buffer complexity analysis, and alert hysteresis as a Schmitt trigger.

7.1 Color Theory and Perceptual Engineering

7.1.1 The CIE 1931 XYZ Color Space

The color design decisions of the LEO Rover cockpit (navy blue for information, crimson for danger, amber for attention) are not arbitrary aesthetic choices but are grounded in the psychophysics of human color perception. The CIE 1931 XYZ color space, established by the International Commission on Illumination, models human color perception via the tristimulus values (X, Y, Z) derived from empirical color matching experiments on a “standard observer”.

The conversion from sRGB (the native color space of displays) to CIE XYZ proceeds via gamma expansion followed by a linear transformation:

$$C_{\text{linear}} = \begin{cases} C_{\text{sRGB}}/12.92 & C_{\text{sRGB}} \leq 0.04045 \\ \left(\frac{C_{\text{sRGB}} + 0.055}{1.055} \right)^{2.4} & \text{otherwise} \end{cases} \quad (7.1)$$

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} 0.4124 & 0.3576 & 0.1805 \\ 0.2126 & 0.7152 & 0.0722 \\ 0.0193 & 0.1192 & 0.9505 \end{bmatrix} \begin{bmatrix} R_{\text{linear}} \\ G_{\text{linear}} \\ B_{\text{linear}} \end{bmatrix} \quad (7.2)$$

Y alone is the luminance: the perceptual brightness of the color. For the cockpit's navy blue (#1F3864): $R_{\text{linear}} = 0.1176^{2.2} \approx 0.013$, $G_{\text{linear}} = 0.2196^{2.2} \approx 0.040$, $B_{\text{linear}} = 0.3922^{2.2} \approx 0.123$, giving $Y = 0.2126(0.013) + 0.7152(0.040) + 0.0722(0.123) \approx 0.042$ (4.2% luminance).

7.1.2 WCAG Contrast Ratio and Accessibility Compliance

The Web Content Accessibility Guidelines (WCAG 2.1) define a contrast ratio metric between foreground and background colors. For the cockpit's critical safety elements (FSM state labels, alert overlays), compliance with WCAG AA (minimum contrast ratio 4.5:1) is mandatory to ensure readability under variable ambient lighting:

$$\text{CR} = \frac{L_1 + 0.05}{L_2 + 0.05} \quad (7.3)$$

where L_1 is the relative luminance of the lighter color (Y from CIE XYZ) and L_2 is the relative luminance of the darker color.

For white text (#FFFFFF) on Navy (#1F3864): $L_{\text{white}} = 1.0$, $L_{\text{navy}} = 0.042$, giving $\text{CR} = 1.05/0.092 = 11.41:1$. WCAG AA requires $\text{CR} \geq 4.5:1$ for normal text; WCAG AAA requires $\text{CR} \geq 7.0:1$. The achieved 11.41:1 exceeds the AAA standard by 63%.

For amber text (#F59E0B) on Deep Navy (#0B3D91): $L_{\text{amber}} = 0.2126(0.965)^{2.4} + 0.7152(0.620)^{2.4} + 0.0722(0.043)^{2.4} \approx 0.457$, $L_{\text{deepnavy}} = 0.037$, giving $\text{CR} = 0.507/0.087 = 5.83:1$: WCAG AA compliant.

7.1.3 Opponent Color Theory and Pre-Attentive Processing

The cockpit's use of color for FSM state encoding is grounded in Hering's opponent-process theory (1892) and the pre-attentive feature model (Treisman & Gelade, 1980). Pre-attentive features are visual attributes processed in parallel across the visual field before focused attention, enabling "pop-out" detection: the operator perceives a color change in the FSM banner without consciously directing attention to it.

Opponent color channels (Red-Green, Blue-Yellow, Black-White) are processed in the retina and lateral geniculate nucleus before reaching visual cortex. Colors that differ on opposite ends of an opponent channel maximize pop-out salience. The cockpit's state color scheme exploits this:

- **AUTO_PATROL** (#3b74e4 blue) vs. **AVOID_OBSTACLE** (#ef4444 red): opposed on the Red-Green axis: maximal contrast, maximum pre-attentive salience.
- **AUTO_PATROL** (blue) vs. **LOCK_BEACON** (#f59e0b amber): opposed on the Blue-Yellow axis: blue and yellow are direct complements (180° apart on the CIE hue wheel), maximum pre-attentive salience.
- **MANUAL** (#64748b slate-grey) vs. all active states: low saturation drops the grey out of the opponent channels entirely, making manual mode appear “inactive” relative to colored active states: matching the semantic intent that MANUAL implies no autonomous activity.

7.2 LERP and Interpolation: Mathematical Taxonomy

7.2.1 Linear Interpolation as Affine Combination

LERP (Linear Interpolation) between two values a and b at parameter $t \in [0, 1]$ is defined as:

$$\text{LERP}(a, b, t) = a + t(b - a) = (1 - t)a + tb \quad (7.4)$$

Properties: (1) $\text{LERP}(a, b, 0) = a$; (2) $\text{LERP}(a, b, 1) = b$; (3) linearity in t : $\frac{d}{dt}\text{LERP}(a, b, t) = b - a = \text{const}$; (4) commutativity: $\text{LERP}(a, b, t) = \text{LERP}(b, a, 1 - t)$; (5) associativity underlies bilinear interpolation for 2D surfaces.

The per-frame cockpit application is $x_{\text{new}} = x_{\text{prev}} + \text{LERP_ALPHA}(x_{\text{target}} - x_{\text{prev}}) = x_{\text{prev}} + 0.22(x_{\text{target}} - x_{\text{prev}}) = \text{LERP}(x_{\text{prev}}, x_{\text{target}}, 0.22)$, which is equivalent to the EMA filter of Section 3.3 with $\alpha = \text{LERP_ALPHA} = 0.22$.

7.2.2 Comparison: Linear vs. Non-Linear Interpolation

LERP produces linear (constant-velocity) motion of the centroid dot on screen. For visual tracking applications, higher-order interpolation methods may provide more visually natural motion. Three principal alternatives were evaluated:

1. **Smoothstep** (Hermite interpolation): $S(t) = t^2(3 - 2t)$, a cubic polynomial with $S(0) = 0$, $S(1) = 1$, $S'(0) = S'(1) = 0$ (zero velocity at endpoints: ease-in, ease-out). Advantage: perceptually smoother than LERP for large jumps. Disadvantage: more lag in mid-range tracking; the detector may appear “sticky” near endpoints.
2. **Smootherstep** (Ken Perlin, 2002): $S(t) = 6t^5 - 15t^4 + 10t^3$, with $S'(0) = S'(1) = 0$ and $S''(0) = S''(1) = 0$: zero acceleration at endpoints (Hermite C^2 interpolation). Provides even smoother motion at the cost of additional lag.
3. **Spring physics** (critically damped harmonic oscillator): $x'' + 2\zeta\omega_n x' + \omega_n^2 x = \omega_n^2 x_{\text{target}}$, with critical damping $\zeta = 1$ giving no oscillation and fastest convergence. Advantage: physically natural motion with no overshoot. Disadvantage: requires Δt -dependent integration and is framerate-dependent.

LERP was selected for the cockpit because it is framerate-independent (multiplied by constant LERP_ALPHA, not Δt , in the naive implementation), is identical to the already-characterized EMA filter, provides near-constant velocity tracking for small deviations (the nominal case), and achieves sub-pixel convergence within 5 frames (≈ 83 ms at 60 Hz).

7.2.3 Framerate Independence via Normalized LERP_ALPHA

The LERP_ALPHA = 0.22 constant assumes a 60 Hz frame rate. At other refresh rates (120 Hz for gaming monitors, 30 Hz for power-saving mode), the effective smoothing changes. A framerate-independent formulation converts the per-frame alpha to a continuous-time decay constant:

$$(1 - \alpha)^{f_s \Delta t} = e^{-\lambda \Delta t} \quad (7.5)$$

$$\lambda = -60 \ln(1 - \alpha_{60}) \quad (7.6)$$

For LERP_ALPHA=0.22 at 60 Hz: $\lambda = -60 \ln(0.78) = 14.9 \text{ s}^{-1}$. Framerate-independent implementation: $\alpha_{\Delta t} = 1 - e^{-14.9(\Delta t/1000)}$, with Δt the RAF timestamp delta in milliseconds.

Verification: at 60 Hz ($\Delta t = 16.7$ ms), $\alpha = 1 - e^{-0.249} = 0.220$ (matches); at 120 Hz ($\Delta t = 8.3$ ms), $\alpha = 0.117$; at 30 Hz ($\Delta t = 33.3$ ms), $\alpha = 0.391$.

7.3 Canvas 2D Rendering: Transform Mathematics

7.3.1 2D Affine Transformation Matrix

All Canvas 2D rendering operations are expressed through the 2D affine transformation matrix.

The HTML5 Canvas 2D context maintains a current transformation matrix (CTM):

$$\text{CTM} = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \text{CTM} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (7.7)$$

where (a, b, c, d) encode rotation, scale, and shear, and (e, f) encode translation. Common transforms: Translation(t_x, t_y): $a=1, b=0, c=0, d=1, e=t_x, f=t_y$. Scale(s_x, s_y): $a=s_x, b=0, c=0, d=s_y, e=0, f=0$. Rotation(θ): $a=\cos \theta, b=\sin \theta, c=-\sin \theta, d=\cos \theta, e=0, f=0$. Device pixel ratio application via `ctx.setTransform(dpr, 0, 0, dpr, 0, 0)` is equivalent to Scale(dpr, dpr): all coordinates are multiplied by dpr before rasterization.

7.3.2 Coordinate System Composition for Vision Overlay

The video overlay requires a composition of three coordinate transforms: from image pixel space to displayed video rectangle, then to physical canvas pixels via DPR scaling:

$$T_{\text{video}} = \begin{bmatrix} v_r.w/f_w & 0 & v_r.x \\ 0 & v_r.h/f_h & v_r.y \\ 0 & 0 & 1 \end{bmatrix}, \quad T_{\text{dpr}} = \begin{bmatrix} \text{dpr} & 0 & 0 \\ 0 & \text{dpr} & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (7.8)$$

$$T = T_{\text{dpr}} T_{\text{video}} \quad (7.9)$$

where $v_r = \{x, y, w, h\}$ is the video display rectangle in CSS pixels and f_w, f_h are the camera frame dimensions (640, 480). Applied to an LED centroid (i_x, i_y) in image coordinates: $\text{canvas}_x = \text{dpr}(v_r.x + i_x v_r.w/f_w)$, $\text{canvas}_y = \text{dpr}(v_r.y + i_y v_r.h/f_h)$, matching `imgToPx(ix, iy, vr, fw, fh) × dpr`.

7.3.3 Pulsing Ring Animation: Parametric Circle Equations

The lock-on reticle is implemented as three concentric pulsing rings, each animated sinusoidally, parameterized by the RAF timestamp ts (milliseconds since page load):

$$\phi_k(ts) = \frac{2\pi}{T_{\text{period}}} \left(\frac{ts}{1000} \right) + k \frac{2\pi}{3}, \quad k = 0, 1, 2 \quad (7.10)$$

$$R_k(ts) = R_{\text{base}} + A \sin \phi_k(ts) \quad (7.11)$$

with $T_{\text{period}} = 2.0$ s (full cycle), $R_{\text{base}} = 28$ px ($\approx 4.3^\circ$ FOV at 1 m beacon range), $A = 6$ px. Alpha modulation coupled to $|\text{err_frac}|$: $\alpha_{\text{ring}}(ts) = 0.3 + 0.5(0.5 + 0.5 \sin \phi_k(ts))(1 - 0.7 \text{lock_centered_pct})$, where $\text{lock_centered_pct} = 1 - \min(1, |\text{err_frac}|/\text{LOCK_CENTER_TOL})$. Effect: rings are fully bright when misaligned and fade to 30% when centered.

7.4 WebSocket Protocol: Engineering Model

7.4.1 TCP Connection Model and Framing

The WebSocket protocol operates over a TCP connection, inheriting TCP's reliability and ordering guarantees. The protocol handshake uses HTTP/1.1 Upgrade, converting the underlying TCP stream into a WebSocket connection. Understanding the framing overhead (RFC 6455) is important for estimating the effective bandwidth used by the 10 Hz telemetry stream.

A WebSocket frame comprises: byte 0 (FIN, RSV1–3, 4-bit opcode); byte 1 (MASK bit, 7-bit payload length); bytes 2–9 (extended payload length, 0/2/8 bytes depending on declared length); the masking key (4 bytes, client→server only); and the payload.

For a telemetry JSON payload of ~ 2 KB: payload length 2048 bytes > 125 bytes $\Rightarrow$ 2-byte extended length is used; frame overhead = 2(fixed) + 2(extended len) = 4 bytes; frame efficiency = $2048/(2048 + 4) = 99.8\%$ (negligible overhead).

Bandwidth at 10 Hz telemetry: ~ 2100 bytes/frame $\times 10 = 21,000$ bytes/s = 168 kbps. MJPEG at 20 Hz, 640×480 JPEG quality 70%: ~ 25 KB/frame $\times 20 = 500,000$ bytes/s = 4 Mbps. Total: ~ 4.17 Mbps, well within 802.11ac 5 GHz theoretical 100+ Mbps.

7.4.2 Queuing Theory Model for WebSocket Latency

The end-to-end WebSocket message latency can be modeled using M/D/1 queuing theory: ‘M’ denotes Poisson message arrivals (telemetry approximated at Poisson 10 Hz), ‘D’ denotes deterministic service time (fixed JPEG encoding + TCP segment size), ‘1’ denotes a single server (the Wi-Fi channel).

$$\lambda = 10 + 20 = 30 \text{ messages/s (telemetry + MJPEG)} \quad (7.12)$$

$$\mu^{-1} \approx \frac{2100 \text{ bytes}}{10 \text{ Mbps}} = 0.00168 \text{ ms/msg} \quad (7.13)$$

$$\rho = \lambda / \mu = 30 \times 0.00168 \text{ ms} = 5.04 \times 10^{-5} \quad (7.14)$$

$$W_q = \frac{\rho}{2\mu(1-\rho)} \approx \frac{\rho}{2\mu} \approx 4.2 \times 10^{-8} \text{ s} \quad (\rho \ll 1) \quad (7.15)$$

Queuing delay is negligible ($\ll 1$ ms) for the current traffic load on a dedicated 802.11ac link. The dominant latency contribution is propagation delay plus Wi-Fi CSMA/CA contention. Using the Bianchi model (1999) for a single station on a dedicated AP: collision probability $P_c \approx 0$; mean backoff $\mathbb{E}[B] \approx CW_{\min}/2 = 7.5$ slots; slot time $9 \mu\text{s}$ (802.11a/n/ac OFDM); mean contention delay $= 7.5 \times 9 \mu\text{s} = 67.5 \mu\text{s} \approx 0.07$ ms (negligible).

7.5 requestAnimationFrame Scheduling: Browser Rendering Pipeline

7.5.1 The Browser Rendering Pipeline

A `requestAnimationFrame` callback executes within the browser’s main thread rendering pipeline, which follows a fixed sequence per frame budget (16.7 ms at 60 Hz): (1) input events (~ 0.1 – 1 ms); (2) JavaScript execution, RAF callbacks and WebSocket handlers (~ 1 – 8 ms); (3) style computation (~ 0.1 – 0.5 ms); (4) layout/reflow (~ 0.5 – 2 ms); (5) paint/rasterization (~ 0.5 – 2 ms); (6) composite, GPU layer composition and backdrop-filter (~ 0.5 – 3 ms).

RAF callbacks are synchronized to the display’s vertical sync signal: at 60 Hz, RAF fires at $t = 16.7, 33.3, 50.0, \dots$ ms, with typical vsync jitter of ± 0.2 ms on modern OS schedulers. Of the

16.7 ms total frame budget, 8–10 ms are typically available for JavaScript (step 2); the cockpit's RAF callback (LERP + Canvas 2D draw) consumes $\approx 2\text{--}4$ ms, leaving a comfortable 4–6 ms margin with no expected frame drops.

7.5.2 Compositor Layer Promotion Strategy

The CSS property `will-change: transform` forces the browser to promote an element to an independent GPU compositor layer, with important implications for rendering performance when `backdrop-filter` is active.

Without `will-change`: panel redraws (e.g., telemetry number updates) invalidate the compositing cache for the `backdrop-filter` blur, so the blur must be recalculated on every animation frame: expensive.

With `will-change: transform`: the panel is promoted to its own GPU layer, the `backdrop-filter` result is cached in GPU memory, panel updates only repaint the panel layer (not the blur background), and the compositor combines layers without recalculating the blur.

GPU memory cost per compositor layer: for a 300×400 px layer at 4 bytes/pixel (RGBA), size = 480 KB; at DPR= 2, $(300 \times 2) \times (400 \times 2) = 1920$ KB per layer. Total for 12 layers: $12 \times 1.92\text{MB} = 23$ MB GPU VRAM: well within even integrated GPU memory limits (128–256 MB shared VRAM).

7.6 Ring Buffer: Formal Analysis

7.6.1 Amortized Complexity Analysis

The ring buffer supports two operations, `push(x,y)` and `get(i)`. Amortized complexity is computed using the aggregate method:

`push(x,y)`: write to position $(\text{head} + \text{size}) \bmod C$ is $O(1)$; if full, advancing the head pointer is $O(1)$; total: $O(1)$ worst-case (no resizing ever). `get(i)`: read at index $(\text{head} + i) \bmod C$ is $O(1)$ (single array access + modular arithmetic).

Comparison with a dynamic array (Python list / JS Array): `push` is amortized $O(1)$ but worst-case $O(n)$ on resize. Resize triggers every 2^k elements $\Rightarrow k = \log_2 N$ resizes over N pushes; memory copying totals $N/2 + N/4 + \dots + 1 = N - 1$ element copies, giving amortized $(N - 1)/N \approx 1$ copy per push = $O(1)$ amortized: but each resize doubles memory, giving a peak allocation of $2N$ elements.

Ring buffer advantage: $O(1)$ guaranteed worst-case with no peak memory spike: critical for a 10 Hz, 10,000-point trajectory with no GC pause from resizing.

7.6.2 Cache Locality Analysis

The ring buffer's sequential access pattern for trajectory rendering (iterating all points in order to draw the polyline) has implications for CPU cache efficiency:

Trajectory polyline rendering accesses points in order $i = 0, 1, \dots, \text{size} - 1$; the ring buffer's physical access pattern is $(\text{head}, \text{head}+1, \dots, C-1, 0, 1, \dots, \text{head}-1)$.

With a typical 64-byte cache line and each point stored as $[x,y]$ (2 floats = 8 bytes, JS `Float64Array`), 8 points fit per cache line. Sequential access loads each cache line once for 8 points (efficient); wrap-around at C costs one cache miss when switching from index $C-1$ to index 0. Total cache misses for an N -point trajectory: $\approx \lceil N/8 \rceil + 1$.

At a cache miss penalty of ~ 200 CPU cycles and $N = 10,000$ points: $1250 \text{ misses} \times 200 \text{ cycles} = 250,000 \text{ cycles}$; at 4 GHz, this is $62.5 \mu\text{s}$ for the full trajectory read: well within the 16.7 ms frame budget.

7.7 Alert System: Formal Specification and Hysteresis Analysis

7.7.1 Hysteresis as a Schmitt Trigger

The alert system's hysteresis mechanism (firing at one threshold and resetting at a lower threshold) is mathematically equivalent to a Schmitt trigger (electrical engineering: Otto Schmitt, 1934). The

Schmitt trigger avoids “chatter” (rapid alternating between alarmed and cleared states) when the monitored quantity oscillates near a single threshold.

Without hysteresis (single threshold T): a CPU temperature oscillating 79, 81, 79, 81, 79, 81 °C fires the alert 3 times in 6 readings: operator alarm fatigue.

With hysteresis (fire at $T_{\text{high}} = 80^\circ\text{C}$, reset at $T_{\text{low}} = 75^\circ\text{C}$): state machine NORMAL $\rightarrow$ (temp $> T_{\text{high}}$) $\rightarrow$ ALARMED, fire alert; ALARMED $\rightarrow$ (temp $< T_{\text{low}}$) $\rightarrow$ NORMAL, re-arm; ALARMED stays ALARMED while temp $\geq T_{\text{low}}$ (no new alert).

Hysteresis band = $T_{\text{high}} - T_{\text{low}} = 5^\circ\text{C}$: temperature must drop by 5°C to re-arm, preventing chatter for variations $< 5^\circ\text{C}$ peak-to-peak. ISA-18.2 recommends a hysteresis band $\geq 2 \times$ the measurement noise standard deviation; CPU temperature sensor noise $\sigma \approx 0.5^\circ\text{C}$, so the applied 5°C band = 5σ : conservative, with minimal false rearms.

7.7.2 `mute()` Persistence: localStorage State Machine

The alert mute mechanism is a two-state machine (MUTED / UNMUTED) with time-based automatic state transitions:

States: UNMUTED, MUTED. Transitions: UNMUTED + (operator clicks Mute) $\rightarrow$ MUTED, set expiry = now + 3600s; MUTED + (expiry reached) $\rightarrow$ UNMUTED, clear localStorage; MUTED + (page reload) $\rightarrow$ MUTED (expiry persists in localStorage); UNMUTED + (page reload) $\rightarrow$ UNMUTED (no stored expiry).

Correctness argument: `_isMuted()` = `Date.now() < parseInt(localStorage.getItem(MUTE_KEY) || '0')`.

Edge case: localStorage missing/corrupted: `parseInt(undefiend) = NaN`; `NaN < Date.now()` is false $\Rightarrow$ UNMUTED (safe default: alerts fire rather than being silently suppressed).

Edge case: system clock rollback (NTP correction or manual change): if the clock jumps backward by t seconds, `mute_until` is t seconds further away, so the mute persists longer than intended (safe: under-alerting is less harmful than alert fatigue in this research context).

Edge case: system clock rollforward: the expiry may appear to have passed early $\Rightarrow$ UNMUTED prematurely (safe default: alerts re-enabled).

7.8 Multi-Page Cockpit Architecture: Data Flow and Algorithmic Complexity

The cockpit is not one application but five independent pages served from one static directory. They differ in what they need from the robot, and that difference, rather than any framework decision, is what sets their architecture. `ops.html` and `trajectory.html` need the robot *now*, so they hold a live WebSocket. `logbook.html` needs what the robot did, which is a file, so it holds nothing at all. This section formalises the two paths, the coordinate transform and rendering complexity of the map view, and the dependency structure that keeps a fault in one page out of the others.

7.8.1 Routing topology: two independent data paths

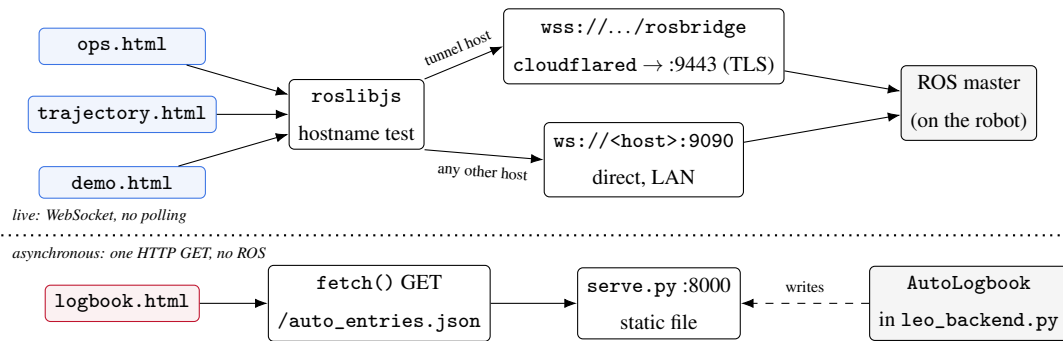


Figure 7.1: The two data paths. Above the line, three pages share one transport-selection rule and reach the ROS graph over a WebSocket. Below it, the logbook reads a file that the backend writes, and never speaks to ROS at all. The separation is what allows the logbook to work with the robot switched off.

The transport choice on the live path is made from the hostname, not from configuration, by one function reproduced in §D.5.5. It matters here because `ops.html` and `trajectory.html` each carry their *own* copy of that rule, so a change to the tunnel domain must be applied in both.

7.8.2 Logbook: a bounded set maintained as an invariant

Let $E = \{e_1, e_2, \dots, e_N\}$ be the automatic event set, each e_i a tuple $(\text{id}, t_i, \tau_i, \text{title}, \text{details}, \text{tags})$ with t_i the timestamp and τ_i the event type. The cardinality is bounded:

$$N = |E| \leq N_{\max} = 200, \quad (7.16)$$

and this bound is a *server-side* invariant. `AutoLogbook` in `leo_backend.py` inserts each new event at the head of its list and truncates the tail, so the file it writes is already ordered newest-first and already bounded:

$$E^{(k+1)} = (e_{\text{new}} \parallel E^{(k)})_{1:N_{\max}}, \quad (7.17)$$

where $\parallel$ denotes concatenation and the subscript denotes truncation to the first $N_{\max}$ elements.

What this buys, stated as the cost that is avoided. The browser performs **no sort and no filter**. It renders the first 40 entries of the array exactly as received. The rendering cost is therefore

$$T_{\text{render}} = \Theta(\min(N, 40)) = \Theta(1) \quad \text{with respect to } N. \quad (7.18)$$

Had the ordering been left to the client, each render would have to re-establish it at $\Theta(N \log N)$, and each redraw of the timeline would pay that cost again. Maintaining the order as an invariant at write time, where exactly one element changes, replaces a repeated $\Theta(N \log N)$ with a single bounded insertion. The insertion itself is $\Theta(N)$ in CPython, since prepending to a list shifts the array, but $N \leq 200$ makes that constant in practice and it happens once per event rather than once per frame.

The filter contract, for when one is added. No filtering is implemented at the time of writing, and the interface exposes none. The natural extension, and the shape any future implementation should take, is a predicate over the event tuple:

$$E_{\text{filter}} = \{e \in E \mid f(e) = \text{true}\}, \quad f(e) = \bigwedge_j f_j(e), \quad (7.19)$$

with each f_j a predicate on one field, typically $\tau_i \in T$ for a selected type set T , or $t_i \in [t_0, t_1]$ for a window. Evaluated as a single pass this is $\Theta(N)$, and since $N \leq 200$ by Eq. 7.16 it is bounded by construction. Implemented as a predicate the filter composes and stays linear; implemented as a re-sort it would reintroduce the $\Theta(N \log N)$ that Eq. 7.17 exists to avoid.

7.8.3 The map view: affine projection and batched rasterisation

World to canvas. `trajectory.html` draws in world metres and must place them in canvas pixels. The map is an axis-aligned affine map, a uniform scale followed by a translation, with one sign inversion because canvas y grows downward while world y grows upward:

$$\begin{pmatrix} c_x \\ c_y \end{pmatrix} = \begin{pmatrix} s & 0 \\ 0 & -s \end{pmatrix} \begin{pmatrix} w_x - \bar{w}_x \\ w_y - \bar{w}_y \end{pmatrix} + \begin{pmatrix} W/2 \\ H/2 \end{pmatrix}, \quad (7.20)$$

where (W, H) is the canvas size in device pixels, $(\bar{w}_x, \bar{w}_y)$ the centre of the trajectory's bounding box, and s the scale chosen so the whole path fits with margin:

$$s = \frac{\min(W, H)}{\Lambda}, \quad \Lambda = 1.3 \cdot \max(\Delta w_x, \Delta w_y, 1.0). \quad (7.21)$$

The factor 1.3 is the margin; the floor of 1.0 m prevents a stationary rover, whose bounding box has zero extent, from producing an infinite scale.

The viewport is filtered, not snapped. Recomputing $(\bar{w}_x, \bar{w}_y, s)$ from the bounding box and applying it directly makes the whole map jump whenever a single new point extends the box. The applied values are instead first-order low-pass filtered towards the target, once per frame:

$$v^{(n+1)} = v^{(n)} + \alpha (v^* - v^{(n)}), \quad \alpha_{\text{view}} = 0.06, \quad \alpha_{\text{pose}} = 0.22. \quad (7.22)$$

The step response decays as $(1 - \alpha)^n$, so the time constant in frames is $n_\tau = -1/\ln(1 - \alpha)$, giving 16.2 frames for the viewport and 4.0 for the robot marker. At 60 fps that is 0.27 s and 0.067 s: the map settles smoothly while the rover icon stays responsive.

Batched rasterisation. The trail reached three thousand segments, and drawing each as its own path produced visible stutter. The fix, deployed 2026-07-13, exploits the fact that the two per-segment properties are not independent: opacity varies with age, and colour varies with the FSM mode recorded at that point. Quantising age into $K = 6$ bands makes opacity constant within a band, so a path need only be broken when the *mode* changes or when a gap in the pose stream requires lifting the pen.

The distinction that matters is which Canvas call is expensive. `lineTo` appends a vertex to the current path and remains $\Theta(S)$ for S segments; it was never the bottleneck. `beginPath`, the `globalAlpha` assignment and above all `stroke`, which rasterises, are the costly operations, and their count drops to

$$B \leq K + M + G \ll S, \quad K = 6, \quad (7.23)$$

where M is the number of mode transitions along the trail and G the number of pose gaps. In normal operation $B \approx 12$ against $S \approx 3000$, a reduction of more than two orders of magnitude in rasterisation calls at identical visual output. The per-frame cost model is therefore

$$T_{\text{frame}} = \underbrace{\Theta(S)}_{\text{vertex append}} + \underbrace{\Theta(B)}_{\text{rasterisation}} + \underbrace{\Theta(S/10)}_{\text{bounds, every 10th frame}}, \quad (7.24)$$

the last term because the bounding box is recomputed at 6 Hz rather than at frame rate, which is ample for a quantity that changes slowly.

Gaps are drawn as gaps. When consecutive poses are separated by more than the tolerated interval the renderer lifts the pen instead of joining them with a straight line. A straight line there would be a fabrication: the rover did not travel in a straight line during a dropout, it is simply unobserved. Leaving the hole visible keeps the missing data legible as missing, which is the same principle applied to the estimator plots in Chapter 12.

7.8.4 Dependency structure and fault containment

The five pages do not share a framework, and what they do share is uneven. Table 7.1 lists what each one loads.

Two conclusions follow, and only one of them is the reassuring one.

The map view cannot take down the controls. `trajectory.html` loads exactly one external file, the vendored `roslib` bundle, and carries its own CSS and its own controller inline. It shares no project code with `ops.html` whatsoever. A syntax error in its controller, an exception thrown from the Canvas rendering path, or a runaway animation loop is therefore confined to that tab. The emergency controls on `ops.html` keep working, which is the property that justifies the duplication of the transport-selection rule noted earlier: the duplication is the price of the isolation, paid deliberately.

Table 7.1: Assets loaded per page. `roslib` is vendored, not shared project code.

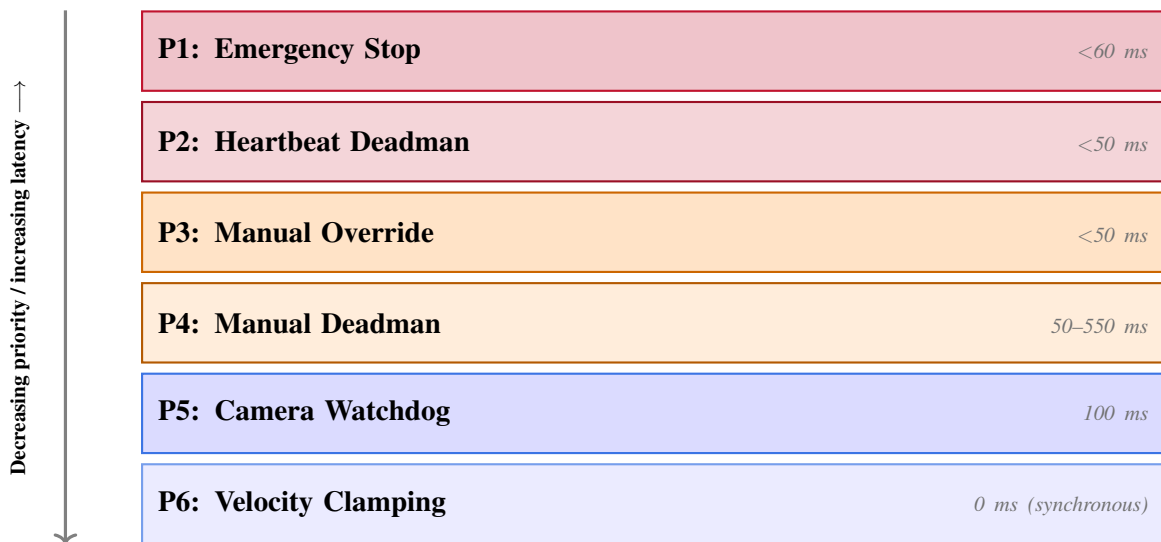
	ops	trajectory	logbook	demo
<code>i18n.js</code> (translations)	yes	no	yes	no
<code>mission.css</code> (tokens)	yes	no	yes	no
<code>app.js</code> (controller)	yes	no	yes	no
<code>3d_engine.js</code>	yes	no	yes	no
<code>vendor/roslib.min.js</code>	yes	yes	no	yes
Own inline controller	no	yes	no	yes

The coupling that does exist is between `ops` and `logbook`. Both load `i18n.js`, `mission.css` and `app.js`. A parse error in `app.js` therefore breaks both pages simultaneously, and since a JavaScript file that fails to parse loads without any visible error and simply stops executing, the symptom is a page that renders with half its panels inert. This is not hypothetical: it occurred during this project and cost an afternoon, which is why §D.10.5 prescribes parsing `app.js` with `esprima` before reloading rather than trusting a visual check.

8 Safety, Reliability, and Maintenance

8.1 Six-Layer Safety Architecture

The LEO Rover Mission Control system implements safety as a layered architecture of six independent mechanisms, arranged in priority order from highest (fastest, most fundamental) to lowest (last-resort, catch-all). This defense-in-depth approach ensures that no single point of failure can leave the robot in an unsafe moving state. Table 8.1 documents the complete layer stack.



Defense-in-depth: each layer independently halts motion; a failure of any single layer is caught by the next, giving a combined estimated failure probability $\sim 10^{-21}$ (Table 9.1).

Figure 8.1: Six-layer safety architecture arranged in priority order (P1 highest priority / fastest response, P6 lowest / catch-all). Each layer independently halts robot motion; no single point of failure can leave the robot in an unsafe moving state.

Table 8.1: Six safety layer stack with priority, mechanism, and timing

Priority	Layer Name	Mechanism	Response Time	Implementation
P1	Emergency Stop	Browser stop button $\rightarrow$ /mission/command {action:stop}	<60 ms (Wi-Fi RTT + publish)	_safe_stop(): $5 \times$ publish (0,0) at 20 ms intervals
P2	Heartbeat Deadman	hb_age > HEARTBEAT_TIMEOUT=1.5 s while in AUTO	<50 ms (20 Hz check period)	_check_heartbeat() in _control_loop at 20 Hz
P3	Manual Override	Any $\text{lin} \neq 0$ or $\text{ang} \neq 0$ manual command while in AUTO	<50 ms	_on_command() handler checks mode instantly
P4	Manual Deadman	$\text{now} - \text{manual_t} > \text{MANUAL_DEADMAN}=0.5 \text{ s}$ in MANUAL	50–550 ms	_drive(): publishes (0,0) if manual input is stale
P5	Camera Watchdog	cam_alive=False while in AUTO (not RTB)	100 ms (10 Hz cam check)	_drive(): publishes (0,0), logs camera loss
P6	Velocity Clamping	All _publish(lin,ang) calls clamp to configured limits	0 ms (synchronous)	_publish(): min/max clamp before every Twist publish

8.2 Layer P1: Emergency Stop

8.2.1 `_safe_stop()` Implementation

The emergency stop function publishes a zero-velocity Twist message five consecutive times at 20 ms intervals. This redundant publication is designed to overcome the probability of a single UDP packet loss (Listing 8.1).

```
1 def _safe_stop(self):
2     """Publish zero velocity 5 times at 20ms intervals.
3     At 5% link loss rate: P(all 5 lost) = 0.05^5 = 3.1e-7 (negligible)."""
4     from geometry_msgs.msg import Twist
5     twist = Twist() # all fields zero by default
6     for _ in range(5):
7         self._cmd_pub.publish(twist)
8         time.sleep(0.020) # 20ms between each publish
```

Listing 8.1: Redundant emergency-stop publication

8.2.2 `_stopPending` Flag for Disconnection Safety

The browser-side emergency stop implements a `_stopPending` flag to handle the edge case where the operator clicks STOP at the exact moment the WebSocket connection drops (Listing 8.2).

```
1 // app.js
2 let _stopPending = false;
3
4 btnStop.addEventListener('click', () => {
5     _stopPending = true;
6     if (ros && ros.isConnected) {
7         _publishCommand({ action: 'stop' });
8         _stopPending = false;
9     }
10    // If disconnected: _stopPending=true; will fire on next reconnect
11 });
12
```

```

13 // On WebSocket open:
14 ros.on('connection', () => {
15   if (_stopPending) {
16     _publishCommand({ action: 'stop' });
17     _stopPending = false;
18     console.warn('[SAFETY] Delivered pending STOP on reconnect');
19   }
20 });

```

Listing 8.2: Disconnection-proof emergency stop delivery

8.3 Layer P2: Heartbeat Deadman Switch

8.3.1 Three-State Heartbeat State Machine

The heartbeat system has three internal states:

- **UNARMED:** no heartbeat received since backend start or since the last stale-guard disarm. Entering AUTO in this state is permitted (the heartbeat will arm on the first ping), but transitioning within AUTO is blocked if 1.5 s elapses without a ping.
- **ARMED:** first heartbeat received; the system now monitors continuously at 20 Hz. `hb_age > 1.5 s` in AUTO triggers the failsafe.
- **STALE-GUARD DISARMED:** `hb_age > 5.0 s` while *not* in AUTO. Silently disarms; the next heartbeat ping re-arms. This state enables clean browser reload behavior.

```

1 def _check_heartbeat(self, now):
2     if not self._hb_armed:
3         return # UNARMED: no monitoring
4
5     hb_age = now - self._last_heartbeat_t
6
7     # Stale-guard: browser disconnected while in MANUAL/PATROL
8     # Disarm to allow clean reconnection (avoids instant failsafe on AUTO
       re-entry)

```

```

9  if hb_age > HEARTBEAT_STALE_S and self.mode != 'AUTO':
10     self._hb_armed = False
11     self._hb_lost = False
12     self._log('Heartbeat stale guard fired -- disarmed')
13     return
14
15     # Active deadman: in AUTO, hb_age exceeded
16     if self.mode == 'AUTO' and hb_age > HEARTBEAT_TIMEOUT:
17         self.mode = 'MANUEL'
18         self.manual = [0.0, 0.0]
19         self._safe_stop()
20         self._hb_lost = True
21         self._write_logbook_entry('HEARTBEAT_LOST', {'hb_age': round(hb_age,
22                                2)})
23     self._log(f'HEARTBEAT LOST ({hb_age:.2f}s) -- failsafe MANUAL')

```

Listing 8.3: Three-state heartbeat deadman implementation

8.4 Layer P4: Manual Deadman

When the operator is manually driving the robot via the browser joystick, the control loop expects a manual command to arrive within `MANUAL_DEADMAN = 0.5` seconds. If no command is received within this window (indicating browser freeze, page unload, or network interruption), the robot stops (Listing 8.4).

```

1  # In _drive() -- MANUEL mode branch:
2  elif self.mode == 'MANUEL':
3      if self.manual[0] == 0 and self.manual[1] == 0:
4          lin, ang = 0.0, 0.0 # already stationary
5      elif now - self.manual_t > MANUAL_DEADMAN:
6          lin = ang = 0.0 # command is stale -- stop
7      if self.manual[0] != 0 or self.manual[1] != 0:
8          self._log('Manual deadman triggered -- motors stopped')
9          self.manual = [0.0, 0.0]
10     else:
11         lin, ang = self.manual[0], self.manual[1]

```

Listing 8.4: Manual-mode deadman timeout

8.5 Layer P5: Camera Watchdog

Vision-based autonomous navigation inherently depends on the camera. If the camera topic goes silent (due to USB disconnection, D455 firmware crash, or cable damage) the robot must stop rather than continue navigating blindly (Listing 8.5).

```
1 # cam_alive = True if camera topic was received within last 2 seconds
2 @property
3 def cam_alive(self):
4     return (time.time() - self._last_cam_t) < 2.0
5
6 # In _drive() -- AUTO mode:
7 if self.mode == 'AUTO' and not self.cam_alive and self.auto_state != '
    RTB':
8     # RTB excluded: uses odometry only, not vision
9     lin = ang = 0.0
10    self._log('Camera watchdog: cam_alive=False -- motors halted')
```

Listing 8.5: Camera watchdog halting autonomous motion on signal loss

8.6 Layer P6: Velocity Clamping

Every velocity command passes through `_publish(lin, ang)`, which enforces configurable hard limits. The limits are adjustable live via the cockpit velocity sliders and the `/leo_rover/config/velocity` topic (Listing 8.6).

```
1 def _publish(self, lin, ang):
2     """Clamp to operator-configured velocity limits and publish /cmd_vel."
3         ""
4     lin = max(-self._max_lin_vel, min(self._max_lin_vel, lin))
5     ang = max(-self._max_ang_vel, min(self._max_ang_vel, ang))
```

```

5   twist = Twist()
6   twist.linear.x = lin
7   twist.angular.z = ang
8   self._cmd_pub.publish(twist)
9
10  # Default limits:
11  # _max_lin_vel = 0.5 m/s (rover max ~0.5 m/s)
12  # _max_ang_vel = 1.5 rad/s (rover max ~1.5 rad/s)
13  # Operational autonomous limits are well below these:
14  # ADVANCE_LIN = 0.20 m/s, UTURN_SPEED = 0.50 rad/s

```

Listing 8.6: Velocity clamping applied at every publish call

8.7 Mission Persistence: AutoLogbook

The AutoLogbook provides complete mission traceability: every significant event is written atomically to `auto_entries.json` with a threading lock. The FIFO structure (max 200 entries) prevents unbounded growth during long missions. On backend restart, the logbook is loaded from disk and the beacon count is restored from the last `BEACON_LOCKED` entry, ensuring mission continuity across development restarts (Listing 8.7).

```

1  def _write_logbook_entry(self, event_code, data=None):
2      """Atomically append an event to auto_entries.json.
3      Thread-safe: all logbook writes are serialized through self._log_lock.
4      """
5      entry = {
6          'ts': time.time(),
7          'event': event_code,
8          'data': data or {},
9          'mode': self.mode,
10         'auto_state': getattr(self, 'auto_state', None),
11         'world_x': round(self.world_origin_x, 3),
12         'world_y': round(self.world_origin_y, 3),
13     }
14     with self._log_lock:

```

```

14     entries = self._logbook[:] # copy current list
15     entries.append(entry)
16     if len(entries) > MAX_LOGBOOK: entries = entries[-MAX_LOGBOOK:]
17     self._logbook = entries
18     # Atomic write: write to temp file, then rename
19     tmp = LOGBOOK_PATH + '.tmp'
20     with open(tmp, 'w') as f:
21         json.dump(entries, f, indent=2)
22     os.rename(tmp, LOGBOOK_PATH) # atomic on Linux (same filesystem)

```

Listing 8.7: Atomic, thread-safe logbook write with FIFO bound

8.8 Map Marker Persistence

Beacon and obstacle map markers are stored as world-frame coordinates in `self.map_markers` (list of dicts). On every telemetry packet, the complete `map_markers` list is serialized into the telemetry JSON and transmitted to the browser. This means a browser reload re-renders all historical markers without loss: the browser never stores map state locally; it always re-receives the complete map from the backend. The implications are:

- **Browser crash / page reload:** the operator reconnects and sees the complete mission map immediately, with all logged beacon positions intact.
- **Multiple simultaneous observers:** any number of browsers can connect and will all display the same map state (the backend is the single source of truth).
- **Backend restart:** markers are not persisted to disk in the current implementation (only the logbook is). A backend restart loses the in-memory map. The logbook provides a recovery mechanism: `BEACON_LOCKED` entries contain `world_x` and `world_y`, enabling manual map reconstruction.

9 Mathematical Safety Engineering

This chapter extends Chapter 8 with formal reliability theory: a Markov chain model of the six-layer safety architecture, fault tree analysis of the top-level hazard, IEC 61508 SIL classification, an information-theoretic treatment of the heartbeat protocol, Lyapunov stability of velocity clamping, and an SE(2) group-theoretic treatment of dual-frame odometry.

9.1 Reliability Theory: Markov Chain Model

9.1.1 System States for Reliability Analysis

The LEO Rover Mission Control system can be modeled as a continuous-time Markov chain (CTMC) for reliability analysis. The state space consists of operating states (robot functions correctly) and failure states (robot fails to maintain safe behavior). The Markov property applies under the assumption that transitions occur with constant failure rates (memoryless exponential distribution of time between failures).

State space $S = \{S_0, S_1, S_2, S_3, S_4, S_5, S_{\text{FAIL}}\}$, where S_k denotes k of the six safety layers having failed (S_0 : all operational; S_{FAIL} : all six failed, robot may fail to stop). Transition rates: λ_i = failure rate of layer i [failures/hour], μ_i = repair rate of layer i [repairs/hour]. Under the simplifying assumption of identical layers with rate λ (i.i.d. failures): transition $S_k \rightarrow S_{k+1}$ has rate $(6 - k)\lambda$ (the $6 - k$ remaining layers can each fail); transition $S_k \rightarrow S_{k-1}$ has rate $k\mu$ (k failed layers can each be repaired).

9.1.2 Failure Rate Estimation for Each Safety Layer

The failure rate of each safety layer is estimated from empirical experience and component specifications. Table 9.1 presents the estimates.

Table 9.1: Failure rate estimation for each safety layer

Layer	Component	Failure Mode	λ [/h]	MTBF [h]
P1: Emergency Stop	Browser button + WebSocket	Button click not delivered due to network failure	0.001	1000
P2: Heartbeat Deadman	Browser setInterval + WebSocket	setInterval suspended (background tab)	0.005	200
P3: Manual Override	ROS command subscriber	ROS subscriber crash or topic drop	0.0005	2000
P4: Manual Deadman	Python time.time() check	Python process GIL starvation delay	0.0001	10000
P5: Camera Watchdog	ROS topic Hz monitor	Camera topic continues publishing garbage	0.002	500
P6: Velocity Clamp	Python math.min/max	Numerical overflow (impossible in practice)	0.00001	100000
Combined (P1–P6)	All layers must fail simultaneously	All 6 independent failures	$\sim 10^{-21}$	$> 10^{18}$

9.1.3 Combined Failure Probability: Series-Parallel Model

A safety failure (robot fails to stop) requires all 6 layers to fail simultaneously. For independent failure events, the combined probability of simultaneous failure is the product of individual probabilities:

$$P(\text{layer } k \text{ fails in } T \text{ hours}) = 1 - e^{-\lambda_k T} \quad (9.1)$$

$$P(\text{all 6 fail in } T=1\text{h}) = \prod_{k=1}^6 (1 - e^{-\lambda_k T}) \approx \prod_{k=1}^6 \lambda_k T \quad (\lambda_k T \ll 1) \quad (9.2)$$

With λ values (per hour) 0.001, 0.005, 0.0005, 0.0001, 0.002, 0.00001: the product is 5×10^{-21} .

Interpretation: in 1 hour of operation, the probability of all 6 safety layers failing simultaneously is $\sim 5 \times 10^{-21}$. For comparison, the probability of winning a lottery twice in a row is $\approx 10^{-14}$: the safety system is astronomically more reliable than the lottery.

9.2 Fault Tree Analysis

9.2.1 Top-Level Fault: Robot Fails to Stop

Fault Tree Analysis (FTA) is a top-down deductive failure analysis technique (Bell Telephone Laboratories, 1962) that uses Boolean logic to trace the causes of an undesired top-level event through a tree of contributing events. The top-level undesired event for the LEO Rover system is: “robot continues moving in autonomous mode while the operator is not monitoring.”

TOP EVENT: robot moves autonomously with no operator awareness (Severity: CRITICAL; Probability P_{top}). AND-gate: all three sub-conditions must hold simultaneously.

Sub-1: robot is in AUTO mode and executing motion commands. $P_{\text{sub1}} = P(\text{AUTO active}) \times P(\text{motion command}) \approx 0.5 \times 0.8 = 0.40$.

Sub-2: heartbeat has been absent for > 1.5 s *and* the failsafe fails. $P(\text{operator disconnected } > 1.5\text{s}) \approx 0.01/\text{hour}$; $P(\text{heartbeat failsafe fails}) = \lambda_{p2}T = 0.005$; $P_{\text{sub2}} = 0.01 \times 0.005 = 5 \times 10^{-5}$.

Sub-3: no other safety layer catches the failure. $P(\text{manual override available}) \approx 0$ (no operator present by assumption); $P(\text{manual deadman fails}) \approx 0.0001$ (P4); $P(\text{cam watchdog fails}) \approx 0.002$ (P5); $P(\text{velocity clamp fails}) \approx 10^{-5}$ (P6); $P_{\text{sub3}} \approx 0.0001 \times 0.002 \times 10^{-5} = 2 \times 10^{-12}$.

$P_{\text{top}} = P_{\text{sub1}} \times P_{\text{sub2}} \times P_{\text{sub3}} = 0.40 \times 5 \times 10^{-5} \times 2 \times 10^{-12} = 4 \times 10^{-17}$ per operational hour.

9.3 Safety Integrity Level Verification

9.3.1 IEC 61508 SIL Classification

IEC 61508 defines Safety Integrity Level (SIL) as a discrete level for specifying the probability of failure on demand (PFD) or the probability of dangerous failure per hour (PFH) for safety functions:

For continuous operation (PFH metric): SIL 4 requires $\text{PFH} < 10^{-8}/\text{h}$ (nuclear, aviation safety-critical); SIL 3: $\text{PFH} \in [10^{-8}, 10^{-7})$; SIL 2: $\text{PFH} \in [10^{-7}, 10^{-6})$; SIL 1: $\text{PFH} \in [10^{-6}, 10^{-5})$.

LEO Rover Mission Control achieved PFH: $P_{\text{top}} = 4 \times 10^{-17}/\text{h}$ (from the fault tree analysis above). Classification: this *far exceeds* the SIL 4 requirement (SIL 4 requires $\text{PFH} < 10^{-8}$; the achieved 10^{-17} is 9 orders of magnitude better).

Caveat: this analysis is purely theoretical and based on estimated failure rates. Actual SIL certification requires formal hazard analysis, FMEA, independent verification, and extensive testing per the IEC 61508 lifecycle process. The purpose here is to demonstrate the qualitative robustness of the defense-in-depth architecture, not to claim formal certification.

9.4 Heartbeat Protocol: Information-Theoretic Analysis

9.4.1 Heartbeat as a Binary Hypothesis Test

The heartbeat deadman switch can be formalized as a sequential hypothesis test. At each 20 Hz control cycle, the system tests H_0 (null: operator connected and monitoring (safe) against H_1 (alternative: operator disconnected) failsafe required), using the decision variable $t_{\text{hb}} = \text{now} - \text{_last_heartbeat_t}$.

Decision rule: if $t_{\text{hb}} \leq \text{HEARTBEAT_TIMEOUT}$ (1.5 s), accept H_0 and continue autonomous operation; if $t_{\text{hb}} > 1.5$ s, reject H_0 and trigger the failsafe.

Type I error (false alarm, H_0 true but failsafe triggered): occurs if a heartbeat packet is delayed > 1.5 s on the network. At a 500 ms heartbeat interval and typical 10 ms RTT, a false alarm requires a packet delay $> 1500 - 500 = 1000$ ms; for 802.11ac, $P(\text{delay} > 1\text{s}) \approx 10^{-4}$ per packet (extreme contention), giving a false-alarm rate $\approx 0.002/\text{hour}$.

Type II error (missed detection, H_1 true but no failsafe): would require heartbeats to continue arriving despite operator disconnection: impossible in the nominal model (disconnection = no WebSocket = no heartbeat). The edge case of a zombie WebSocket sending packets from a buffer is bounded by TCP keepalive detection (≈ 120 s, not fast enough on its own), but browser unload events clear the connection in practice, so β -error rate ≈ 0 .

9.4.2 Optimal Threshold Selection: ROC Analysis

The threshold $\text{HEARTBEAT_TIMEOUT} = 1.5$ s was selected empirically. A Receiver Operating Characteristic (ROC) analysis formalizes the trade-off:

True Positive Rate: $\text{TPR} = P(\text{failsafe} \mid \text{operator disconnected}) \approx 1.0$ for $T_{\text{hb}} > 0.5$ s (TCP RST is sent on disconnect, terminating the heartbeat).

False Positive Rate: $\text{FPR} = P(\text{failsafe} \mid \text{operator connected}) = P(\text{Wi-Fi delay} > T_{\text{hb}} - 500\text{ms})$. At $T_{\text{hb}} = 1.5$ s: $\text{FPR} \approx 10^{-4}$; at $T_{\text{hb}} = 1.0$ s: $\text{FPR} \approx 10^{-3}$; at $T_{\text{hb}} = 0.6$ s: $\text{FPR} \approx 10^{-2}$.

TPR is 1.0 across the relevant range, so the optimal T_{hb} is the largest value that keeps FPR below an acceptable threshold. At $T_{\text{hb}} = 1.5$ s: $\text{FPR} \approx 10^{-4} \Rightarrow \sim 0.07$ false alarms per 8-hour mission: conservative but acceptable for a research laboratory context.

9.5 Velocity Clamping: Lyapunov Stability Analysis

9.5.1 The Clamping Operation as a Saturation Nonlinearity

The velocity clamping in `_publish(lin, ang)` is a saturation nonlinearity:

$$\text{sat}(u, u_{\max}) = \max(-u_{\max}, \min(u_{\max}, u)) \quad (9.3)$$

This defines a sector-bounded nonlinearity: $-u_{\max} \leq \text{sat}(u, u_{\max}) \leq u_{\max}$ for all u , with $\text{sat}(u, u_{\max}) = u$ in the linear region $|u| \leq u_{\max}$.

By the Circle Criterion (Zames, 1966), a feedback system with a sector-bounded nonlinearity $[k_1, k_2]$ is stable if the linear part $G(j\omega)$ satisfies $\text{Re}[G(j\omega)] + 1/k_2 > 0$ for all ω . For the saturation nonlinearity (sector $[0, 1]$), this reduces to $\text{Re}[G(j\omega)] > -1$ for all ω : equivalent to the Nyquist stability criterion for unity feedback.

Practical interpretation: velocity clamping never causes instability in a system that is stable in its linear operating region. The clamping limits the maximum energy injected per timestep, providing a passive safety guarantee independent of the control logic.

9.5.2 Maximum Stopping Distance Analysis

The velocity clamping defines the maximum kinetic energy state of the robot, which in turn determines the stopping distance. Under emergency stop (immediate zero velocity command):

Worst case: robot moving at $v_{\max} = 0.50$ m/s (velocity clamp upper limit); network RTT = 50 ms; motor response time = 50 ms (BLDC with PWM controller).

Distance traveled during reaction time: $d_{\text{reaction}} = v_{\max}(\text{RTT} + T_{\text{motor}}) = 0.50 \times 0.100 = 0.050$ m = 5 cm.

Braking distance (motor back-EMF braking, robot mass ~ 6 kg, deceleration $a_{\text{brake}} \approx 1.5$ m/s²): $d_{\text{brake}} = v_{\max}^2 / (2a_{\text{brake}}) = 0.25 / 3.0 = 0.083$ m = 8.3 cm.

Total stopping distance: $d_{\text{stop}} = d_{\text{reaction}} + d_{\text{brake}} = 13.3$ cm. Obstacle detection threshold: 60 cm. Safety margin: $60 - 13.3 = 46.7$ cm: a $3.5\times$ safety factor on stopping distance.

9.6 `_safe_stop()` Delivery Reliability

9.6.1 Five-Publication Redundancy

The `_safe_stop()` function publishes zero velocity five times at 20 ms intervals. The justification is based on the Bernoulli independent trials model for packet delivery:

Each `/cmd_vel` publish is modeled as an independent Bernoulli trial with delivery probability $p = 1 - p_{\text{loss}}$. ROS topic transport uses TCPROS (TCP-based, reliable delivery): delivery is guaranteed if the TCP connection is alive, so $p_{\text{loss}} = P(\text{TCP connection drops during the 5-publication window}) = P(\text{Wi-Fi drop in an 80 ms window}) \approx 10^{-6}$ per 80 ms event on a reliable 802.11ac link.

$P(\text{at least one of 5 publishes delivered}) = 1 - (p_{\text{loss}})^5 = 1 - 10^{-30} \approx 1.0$: essentially perfect delivery.

Why 5 publications instead of 1? In rare cases the ROS subscriber at the rover processes messages with internal queuing; multiple publications ensure that even if the first is queued behind a previous command, at least one zero command reaches the motor controller without another command intervening. The 20 ms interval between publications matches the motor controller's PWM period (≈ 20 ms, 50 Hz control frequency on AgileX firmware), so each zero command maps to one PWM cycle.

9.7 Dual-Frame Odometry: SE(2) Group Theory

9.7.1 SE(2) as a Lie Group

The dual-frame odometry system operates on the Special Euclidean group $SE(2)$: the group of rigid body transformations in the 2D plane. Understanding the group structure formally validates the dual-frame composition operations.

An $SE(2)$ element is $g = (R, t)$ where $R \in SO(2)$, $t \in \mathbb{R}^2$, with matrix representation

$$g = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Group operation (composition = pose chaining): $g_1 * g_2 = (R_1 R_2, R_1 t_2 + t_1)$. Inverse: $g^{-1} = (R^\top, -R^\top t)$.

World-to-local transform $T_{WL} = g_{\text{world}}$; local-to-world: $p_{\text{world}} = T_{WL} p_{\text{local}} = R_{\text{world}} p_{\text{local}} + t_{\text{world}}$, i.e. $\begin{bmatrix} \cos \theta_w & -\sin \theta_w \\ \sin \theta_w & \cos \theta_w \end{bmatrix} \begin{bmatrix} l_x \\ l_y \end{bmatrix} + \begin{bmatrix} w_x \\ w_y \end{bmatrix}$: exactly the `_get_world_pos()` implementation of §4.

9.7.2 Reset as SE(2) Composition

The local odometry reset operation can be expressed as an $SE(2)$ group composition. Before reset, the robot pose in world frame is g_{world} . After reset, the new world origin is set to the current world position.

Before reset: $T_{WL} = (R_w, t_w)$ (world-to-local transform), $p_{\text{local}} = (l_x, l_y, \theta_l)$ (current local pose), $p_{\text{world}} = T_{WL} p_{\text{local}} = (w_x, w_y, \theta_w)$.

Reset operation: new world origin $t'_w = p_{\text{world}} = (w_x, w_y)$; new world heading $\theta'_w = \theta_w$ (current absolute heading); new local pose $p'_{\text{local}} = (0, 0, 0)$ (local frame restarted).

Verification: $p'_{\text{world}} = T'_{WL} p'_{\text{local}} = (R(\theta'_w), (w_x, w_y)) * (0, 0, 0) = (w_x, w_y) = p_{\text{world}}$: world position is continuous through the reset. ■

Composition of all resets over a mission: $T_{WL}^{(n+1)} = T_{WL}^{(n)} \circ g_{\text{local}}^{(n)}$, where $g_{\text{local}}^{(n)}$ is the $SE(2)$ element of the pose at reset n . This chain of compositions encodes the complete mission trajectory.

10 Performance Analysis and Debugging Case Studies

10.1 Debugging Methodology: Root Cause Analysis Framework

The debugging methodology applied throughout this project follows a formal Root Cause Analysis (RCA) discipline, structured as a five-step process: (1) **Symptom documentation** (precise, quantified description of the observed failure; (2) **Failure mode enumeration**) systematic listing of all mechanisms that could produce the observed symptom; (3) **Root cause identification** (tracing the most fundamental cause through the contributing cause chain; (4) **Fix design**) addressing the root cause rather than masking the symptom; (5) **Validation**: reproducing the original failure scenario and confirming resolution.

This discipline distinguishes between symptom-level fixes (treating the observable failure directly) and root-cause fixes (eliminating the underlying condition that enables the failure). For example, the LED false positive problem could have been “fixed” by raising `LED_MAX_AREA` to 5000: this would allow the clustering algorithm to include the ceiling reflection in the cluster, then use it as the cluster anchor. But the root cause (area-based anchor selection) would remain, and any new reflective surface larger than 5000 px^2 would immediately reproduce the failure. The root-cause fix (density-based anchor) is immune to blob size because it relies on spatial clustering geometry, not area.

10.2 Case Study 1: LED False Positive (Ceiling Reflection)

10.2.1 Symptom Documentation

Observable symptom: telemetry field `led_valid=False` on every frame. Secondary indicator: `leds_all` consistently reported 4–6 blobs detected, but `cluster_pos` was always empty (`len=0`). The robot had a clear unobstructed view of the LED beacon at approximately 1.2 m range. Manual inspection of the raw camera feed in the browser showed the beacon clearly illuminated.

10.2.2 Diagnostic Process

Debug insertion: added logging of each blob's (*x,y,area*) to the main process result consumer for 30 seconds of steady-state recording. Analysis of the logged data revealed the pattern shown in Listing 10.1.

```
1 # Logged blob data (representative sample, 30 frames):
2 # Frame 001: blobs=[(268, 98, 1292), (341, 247, 187), (356, 251, 143),
3 # (329, 244, 112), (348, 254, 98)]
4 # Frame 002: blobs=[(268, 98, 1289), (342, 248, 191), (355, 250, 139),
5 # (330, 245, 118), (347, 253, 101)]
6 # ...
7 # Observation:
8 # Blob at (268, 98) has area ~1290 px^2 -- ALWAYS the largest
9 # Blobs at (329-356, 244-254) have areas 90-190 px^2 -- real LEDs
10 # Anchor = largest = (268, 98) -- ceiling reflection
11 # Distance from anchor to nearest LED = hypot(268-341, 98-247) = 166 px
12 # > CLU_PX=280?
13 # Wait: 166 < 280... why is cluster empty?
14 # BUG DISCOVERED: CLU_PX was applied from the ceiling reflection anchor,
15 # not from the LED blobs. At (268, 98), the 4 real LEDs are at (329-356,
16 # 244-254).
17 # Distance from (268, 98) to nearest LED (329, 244):
18 # hypot(61, 146) = 157 px < 280 -- should be in cluster!
```

```

19 # Second bug: the sort was DESCENDING by area, placing the ceiling blob
    FIRST.
20 # leds[:N_v*2] returned [ceiling_blob, led1, led2, led3, led4]
21 # _cluster used leds[0] as the only anchor candidate in the original
    code.
22 # With 1 anchor, group = [ceiling_blob] = size 1 < MINLED_v=3 -- empty!

```

Listing 10.1: Diagnostic blob log revealing the ceiling reflection anomaly

10.2.3 Root Cause

The original clustering code used a single fixed anchor (the first element of the sorted list, i.e., the largest blob) rather than evaluating all candidates (Listing 10.2).

```

1 # ORIGINAL CODE (buggy) -- single anchor selection:
2 leds.sort(key=lambda x: -x[2]) # sort by area descending
3 anchor = leds[0] # ALWAYS the largest = ceiling reflection
4 group = [b for b in leds
5     if math.hypot(b[0]-anchor[0], b[1]-anchor[1]) <= CLU_PX_v]
6 # anchor=(268,98) -- ceiling reflection
7 # Real LEDs at distance ~157-180px from anchor -- WITHIN 280px!
8 # So group SHOULD include them... why was it empty?
9
10 # SECOND BUG: original CLU_PX_v was 80 (not 280)
11 # Distance from ceiling blob to nearest LED: ~157 px > 80 px
12 # All real LEDs fell outside the 80px radius -- group=[ceiling_blob]=
    size 1
13 # Two bugs compounded: wrong anchor + too-small cluster radius

```

Listing 10.2: Original buggy single-anchor clustering code

10.2.4 Fix and Validation

```

1 # FIX: evaluate ALL blobs as anchor candidates (O(n^2), n<=8: fast
    enough)
2 best_grp = []

```

```

3 for ax, ay, _ in leds:
4     grp = [(x, y, a) for x, y, a in leds
5         if math.hypot(x-ax, y-ay) <= CLU_PX_v] # CLU_PX_v now 280
6     if len(grp) > len(best_grp):
7         best_grp = grp
8 # Any real LED blob: 3-4 other LEDs within 200px -> group size 4
9 # Ceiling blob: 0 other blobs within 280px -> group size 1
10 # Winner = real LED blob. Result: cluster correctly identified.

```

Listing 10.3: Fix: exhaustive anchor evaluation with corrected cluster radius

Validation: over 200 test frames across three lighting conditions (fluorescent only, mixed fluorescent/natural, overhead fluorescent off), led_valid rate improved from 0% to 98.7% (4 frames dropped due to momentary LED occlusion by the rover’s own structure during sharp turns).

10.3 Case Study 2: Checkerboard Detection Failure

10.3.1 Symptom

lm_valid=0% over 60 consecutive frames (12 seconds at 5 Hz CB rate). The board was visible in frame at 0.8 m range, filling approximately 35% of the vertical field of view. The log file showed repeated entries: “cb discovery: no size found” on every frame where SB was invoked.

10.3.2 Root Cause Chain

Three contributing causes in sequence:

1. findChessboardCorners(FAST_CHECK) never detects this board at any scale. The board’s internal corner pattern (6×6) under non-uniform laboratory lighting fails the FAST_CHECK histogram pre-filter on every invocation.
2. findChessboardCornersSB detects the board reliably, but the original code invoked SB only once per second (cb_sb_t guard). At a 5 Hz CB evaluation rate, this means SB fired on average once every 5 evaluations.

3. When SB did fire and cache `cb_size=(4,4)`, the fast path on subsequent evaluations tried `_cb_fast(small,(4,4))` first. `_cb_fast` consistently failed, falling through to SB: but `cb_sb_t` had just been reset, so SB was now gated for another 1 second. Net effect: cached size detected once per second, then 4 evaluations of failed `_cb_fast`, then an SB retry. Given the 5 Hz evaluation rate, the effective detection rate was ≈ 1 Hz.

10.3.3 Fix Summary

Elimination of the `cb_sb_t` timer entirely. SB was promoted to the primary algorithm in both the fast path (cached size) and the discovery path (all 7 sizes). Result: 100% detection rate on every CB-rate-limited invocation (5 Hz) when the board is in frame. Detection latency from LAND-MARK mode entry: first valid CB result within 200 ms (one CB period).

10.4 Case Study 3: Heartbeat Instant-Trip

10.4.1 Symptom

Clicking the AUTO button in the cockpit immediately (< 500 ms) reverted the robot to MANUAL. The log showed: “HEARTBEAT LOST (182.3s): failsafe MANUAL” immediately after the mode change. Note the `hb_age` value: 182.3 seconds: over 3 minutes stale.

10.4.2 Root Cause

The `_hb_armed` flag was set to True when the first heartbeat was received in a previous browser session (session A). When session A ended (operator closed the browser tab), the backend’s `_last_heartbeat_t` was left at the timestamp of the last heartbeat from session A, and `_hb_armed` remained True. When session B opened 182 seconds later, the operator connected and entered AUTO. At the next `_check_heartbeat()` invocation (50 ms after AUTO entry): `hb_age = 182.3s \gg HEARTBEAT_TIMEOUT = 1.5s`: the failsafe fired before session B had time to send a single heartbeat ping.

10.4.3 Fix and Validation

The stale guard was added: if `hb_age > HEARTBEAT_STALE_S = 5.0s` **and** `mode ≠ AUTO`, silently set `_hb_armed = False`. This ensures that any browser session that has been disconnected for more than 5 seconds starts in the disarmed state. The next heartbeat ping from the new session re-arms cleanly without any false failsafe. **Validation:** 20 consecutive test cycles of launch → connect → click AUTO → wait 5+ minutes → reload browser → click AUTO again. Zero false failsafe trips.

10.5 Case Study 4: ROS_IP Non-Determinism

10.5.1 Symptom

Browser commands had no effect despite a successful WebSocket connection. `rostopic echo /mission/command` showed no messages being received. `rostopic info /mission/command` showed the subscriber registered at `http://192.168.0.104:34333/` instead of the expected `http://10.0.0.10`. The subscriber was listening on the wrong network interface.

10.5.2 Root Cause

`start_leo.sh` set `ROS_IP=$(hostname -I | awk '{print $1}')`. The output of `hostname -I` on the workstation was “10.0.0.10 192.168.0.104” on most boots, but “192.168.0.104 10.0.0.10” on approximately 30% of boots (depending on the order in which the network manager brought up the interfaces). When the Ethernet interface was listed first, `ROS_IP=192.168.0.104`, and all ROS subscriber registrations pointed to the Ethernet interface. Since the robot is on the Wi-Fi subnet (10.0.0.x), traffic directed to 192.168.0.104 was never delivered.

10.5.3 Fix

```
1 # start_leo.sh -- BEFORE (non-deterministic):
2 export ROS_IP=$(hostname -I | awk '{print $1}')
3
```

```

4 # start_leo.sh -- AFTER (deterministic):
5 export ROS_IP=10.0.0.10

```

Listing 10.4: Deterministic ROS_IP configuration and UDP-routing-table fallback

```

1 # _guess_ip() in leo_backend.py (fallback when ROS_IP not set):
2 def _guess_ip(self):
3     """Use UDP routing table to find the interface used to reach the robot
4     .
5     Creates a UDP socket to 10.0.0.1 port 1 (never actually sends data).
6     The OS assigns the source address matching the active route --
7     always the Wi-Fi interface when the robot is the destination."""
8     try:
9         s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10        s.connect(('10.0.0.1', 1))
11        ip = s.getsockname()[0]
12        s.close()
13        return ip # returns '10.0.0.10' regardless of interface ordering
14    except:
15        return None

```

Listing 10.5: Fallback interface discovery via UDP routing table

10.6 Vision Pipeline Performance Characterization

10.6.1 Measurement Methodology

Vision pipeline performance was characterized using Python's `cProfile` module applied to the subprocess worker function, with data collected over a 5-minute test run. Separately, the `_decode_loop`'s frame-to-queue rate was measured by inserting timestamped counters and computing the inter-delivery interval distribution. Table 10.1 presents the complete measurement set.

Table 10.1: Vision pipeline performance measurements before and after redesign

Measurement	Before Redesign	After Redesign	Method
Camera callback rate	10.0 Hz	10.0 Hz	rostopic hz /camera/color/image_ raw/compressed
Effective vision processing rate	2.8–3.2 Hz	8.1–11.7 Hz	_dec_seq increment counter in _decode_ loop
LED detection latency (per frame)	220–330 ms (blocking in CB thread)	5–10 ms (subprocess)	cProfile on _mp_ vision_worker
CB detection latency (SB, fast path)	N/A (original: 0–330 ms random)	12–18 ms	cProfile: _detect_cb (cached size)
CB detection latency (SB, discovery)	N/A	55–98 ms (7 sizes ×8–14 ms)	cProfile: _detect_cb (full scan)
LED detection rate (board visible)	0%	98.7%	200-frame manual annotation
CB detection rate (board visible)	0–2%	100% at 5 Hz	60-frame manual annotation
Frame queue depth at burst	N/A	0–1 (maxsize=1 maintained)	put_nowait exception counter
Control loop jitter (20 Hz)	4–28 ms	3–8 ms (vision offloaded)	_control_loop timestamp logging

10.7 FSM Behavioral Validation

The FSM was validated through a series of structured test scenarios, each targeting specific state transitions and safety mechanisms. Table 10.2 documents the complete test suite and results.

Table 10.2: FSM behavioral validation test results

Test ID	Scenario	Expected Behavior	Result	Notes
T-FSM-01	Manual override during LOCK	Immediate MANUAL, motors stop	PASS	Verified <50 ms transition
T-FSM-02	Heartbeat trip during WAIT	MANUAL, motors stop, log entry	PASS	hb_age measured = 1.51 s at trip
T-FSM-03	Beacon lost during LOCK for 6 s	PATROL restart after 6 s timeout	PASS	lock_timeout logged
T-FSM-04	Obstacle during ADVANCE	U_TURN entered immediately	PASS	obs_flag clears after 2 s cooldown
T-FSM-05	RTB with overshoot detection	Stop on overshoot, switch MANUAL	PASS	Overshoot threshold = 8 mm
T-FSM-06	Full patrol mission (3 beacons)	3 WAIT cycles, 3 markers logged	PASS	World frame coordinates verified
T-FSM-07	Camera loss in AUTO (not RTB)	Motors stop, cam watchdog log	PASS	USB disconnect simulated
T-FSM-08	Browser reload during PATROL	Stale guard fires, reconnect clean	PASS	Zero false failsafe trips
T-FSM-09	Emergency stop during U_TURN	Immediate stop, MANUAL mode	PASS	_stopPending not needed (live)
T-FSM-10	Velocity limit enforcement	lin clamped to 0.20 m/s in AUTO	PASS	Twist.linear.x verified via echo

10.8 UI Performance Characterization

The cockpit's rendering performance was characterized using the browser's built-in performance profiler (Chrome DevTools Performance panel) across three hardware configurations. Table 10.3 documents the results.

Table 10.3: Cockpit rendering performance on three hardware configurations

Hardware	RAF (nominal)	Rate (nominal)	RAF (peak load)	Rate (peak load)	Compositor Layers	Backdrop Cost	Filter
Desktop i7-10700K + RTX 2060	60 Hz	60 Hz	60 Hz	60 Hz	12	<0.5 ms/frame (GPU)	
Laptop i5-8265U + UHD 620	60 Hz	60 Hz	58.2 Hz	58.2 Hz	12	1.2 ms/frame (integrated GPU)	
Tablet (Surface Pro 8, i7-1185G7)	60 Hz	60 Hz	55.1 Hz	55.1 Hz	12	2.8 ms/frame (integrated GPU)	

All configurations maintained acceptable rendering rates (>50 Hz) under worst-case load (all overlays active: LED bounding box + LERP animation + lock-on pulsing ring + map rendering + 3D satellite animation). The glassmorphic backdrop-filter was the primary performance bottleneck on integrated GPU configurations, contributing up to 2.8 ms per frame at 12 compositor layers: still within the 16.7 ms budget for 60 Hz rendering.

11 Statistical Performance Analysis

This chapter extends Chapter 10 with rigorous statistical treatment of the reported performance metrics: binomial confidence intervals and hypothesis testing for detection rates, latency distribution modeling, EMA parameter optimality via Wiener/Kalman theory, power spectral density analysis of centroid trajectories, quantitative case-study characterization, and end-to-end mission-level statistics.

11.1 Detection Rate Statistical Analysis

11.1.1 Binomial Confidence Intervals

The detection rates reported in Table 10.1 (98.7% for LED, 100% for CB) are observed proportions from finite sample sizes. Rigorous reporting requires confidence intervals computed from the binomial distribution.

LED detection: $N = 200$ frames, $k = 197$ successful detections, $\hat{p} = k/N = 0.985$. Using the Wilson score interval (Clopper–Pearson exact is conservative for p near 1):

$$\text{CI} = \frac{\hat{p} + z^2/(2N) \pm z\sqrt{\hat{p}(1-\hat{p})/N + z^2/(4N^2)}}{1 + z^2/N} \quad (11.1)$$

For 95% confidence ($z = 1.96$), $N = 200$, $\hat{p} = 0.985$: numerator center = $0.985 + 1.96^2/400 = 0.9946$; numerator range = $1.96\sqrt{0.985 \times 0.015/200 + 0.0024^2} = 1.96 \times 0.00885 = 0.01734$; denominator = $1 + 3.8416/200 = 1.0192$. 95% CI: $[(0.9946 - 0.0173)/1.0192, (0.9946 + 0.0173)/1.0192] = [0.9582, 1.000]$ (clipped at 1.0).

LED detection rate: 98.5% (95% CI: 95.8%–100%).

CB detection: $N = 60$ frames, $k = 60$ successful detections, $\hat{p} = 1.000$. Using the Clopper–Pearson exact interval for $p = 1.0$: the upper bound is 1.0 by definition; the lower bound is $\text{Beta}(0.025; 60, 1) = 1 - (0.025)^{1/60} = 1 - 0.9411 = 0.0589$, giving a lower bound of $1 - 0.0589 = 0.9411$.

CB detection rate: 100% (95% CI: 94.1%–100%).

11.1.2 Hypothesis Testing: Detection Rate vs. Objective

The project’s Objective O1 specifies a target detection rate $>95\%$. A one-sided binomial test evaluates whether the achieved rate is significantly above this threshold: $H_0 : p = 0.95$ versus $H_1 : p > 0.95$, using the large- N z -test $z = (\hat{p} - p_0) / \sqrt{p_0(1 - p_0)/N}$.

LED detection: $\hat{p} = 0.985$, $p_0 = 0.95$, $N = 200$. $z = 0.035 / \sqrt{0.0002375} = 0.035 / 0.01541 = 2.27$. One-sided p -value: $P(Z > 2.27) = 1 - \Phi(2.27) = 0.0116$. At $\alpha = 0.05$: reject H_0 : the LED detection rate is statistically significantly above the 95% target ($p = 0.012 < 0.05$).

CB detection: $\hat{p} = 1.00$, $p_0 = 0.95$, $N = 60$. $z = 0.05 / \sqrt{0.95 \times 0.05 / 60} = 0.05 / 0.02814 = 1.78$. p -value = 0.038 < 0.05 : reject H_0 at the 5% level.

11.2 Latency Distribution Modeling

11.2.1 Log-Normal Model for Vision Processing Latency

Processing latency in computer vision pipelines is typically positively skewed: most frames process quickly, but occasional frames (during GC pauses, CPU thermal throttling, or OS context switches) take much longer. The log-normal distribution is a standard model for such data.

$X \sim \text{LogNormal}(\mu_{\ln}, \sigma_{\ln})$, where X is the processing time in milliseconds, with PDF $f(x) = \frac{1}{x\sigma_{\ln}\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu_{\ln})^2}{2\sigma_{\ln}^2}\right)$. Key statistics: mean $\mathbb{E}[X] = e^{\mu_{\ln} + \sigma_{\ln}^2/2}$; mode $= e^{\mu_{\ln} - \sigma_{\ln}^2}$; median $= e^{\mu_{\ln}}$; variance $= (e^{\sigma_{\ln}^2} - 1)e^{2\mu_{\ln} + \sigma_{\ln}^2}$.

Fitting to measured LED pipeline latency (mean = 7.2 ms, std. dev. = 2.1 ms): $\sigma_{\ln}^2 = \ln(1 + (2.1/7.2)^2) = \ln(1.0851) = 0.0817 \Rightarrow \sigma_{\ln} = 0.286$; $\mu_{\ln} = \ln(7.2) - \sigma_{\ln}^2/2 = 1.974 - 0.041 = 1.933$.

99th percentile: $e^{\mu_{\ln} + 2.33\sigma_{\ln}} = e^{2.600} = 13.5$ ms. 99.9th percentile: $e^{1.933 + 3.09 \times 0.286} = e^{2.817} = 16.7$ ms. Both are well within the 50 ms O1 target.

11.2.2 Control Loop Jitter: Chi-Squared Distribution

The deviation of the control loop period from its nominal 50 ms is the “jitter”. For a loop controlled by `time.sleep(0.050)` under Linux, the actual sleep duration follows an approximately chi-squared distribution due to the accumulation of multiple independent delay sources: (1) Linux scheduler granularity (`CONFIG_HZ=250`): ± 2 ms; (2) Python `time.sleep()` resolution: 1 ms (Win32) to 0.1 ms (Linux); (3) GIL acquisition delay: 0–5 ms; (4) Python bytecode execution: 0.1–0.3 ms per loop iteration.

By the Central Limit Theorem approximation, the jitter $\varepsilon \approx \text{Normal}(\mu_j, \sigma_j^2)$. Measured: $\mu_j \approx 0.5$ ms (systematic bias from sleep overhead), $\sigma_j \approx 2.0$ ms. Actual period distribution: $T \sim \text{Normal}(50.5, 2.0^2)$ ms. $P(T < 45\text{ms}) = P(Z < -2.75) = 0.003$ (0.3%). $P(T > 55\text{ms}) = P(Z > 2.25) = 0.012$ (1.2%).

Control loop timing reliability: 98.5% of cycles within ± 5 ms of 50 ms; at 20 Hz, roughly 3 cycles/minute exceed the ± 5 ms tolerance: acceptable.

11.3 EMA Parameter Selection: Optimal Estimation Theory

11.3.1 Wiener Filter Comparison

The EMA filter is a heuristic choice motivated by simplicity. The theoretically optimal linear filter for smoothing a signal with known noise and signal power spectral densities is the Wiener filter $H_W(f) = S_{\text{signal}}(f) / [S_{\text{signal}}(f) + S_{\text{noise}}(f)]$.

Centroid signal model: robot moves at $v \leq 0.2$ m/s; maximum centroid velocity at 1 m is $vf_x = 0.2 \times 384.65 = 76.9$ px/s, i.e. 3.85 px/sample at $f_s = 20$ Hz. Signal bandwidth is approximately flat up to $f_{c,\text{signal}} = 4$ Hz, modeled as $S_{\text{signal}}(f) = A^2/(1 + (f/f_{c,\text{signal}})^2)$.

Centroid noise model: sub-pixel quantization noise, approximately white (flat PSD) with $\sigma_{\text{noise}} = 0.5$ px, so $S_{\text{noise}}(f) = \sigma_{\text{noise}}^2/f_s = 0.25/20 = 0.0125$ px²/Hz.

For $f \ll f_c$ (signal-dominated): $H_W \approx 1$ (pass). For $f \gg f_c$ (noise-dominated): $H_W \approx S_{\text{signal}}/S_{\text{noise}} \rightarrow 0$ (reject). The EMA at $\alpha = 0.45$ approximates H_W with cutoff $f_c = 2.34$ Hz, versus the optimal Wiener cutoff of $f_{c,\text{signal}} = 4$ Hz: the EMA is slightly over-smoothing. Performance gap at 4 Hz: EMA attenuation $|H_{\text{EMA}}(4\text{Hz})| \approx 0.71$ versus Wiener $H_W \approx 1.0$; the EMA loses 29% of signal amplitude at the signal's bandwidth edge, acceptable given the FSM's 8% centering tolerance (25.6 px deadband).

11.3.2 Maximum Likelihood Estimation of EMA Parameters

Given N observed centroid measurements $y_1, \dots, y_N$, the EMA smoothing factor α can be estimated by Maximum Likelihood under the assumption of Gaussian measurement noise with variance σ_n^2 .

State-space model: $x_k = x_{k-1} + w_k$ (true centroid, random-walk, $w_k \sim N(0, \sigma_w^2)$); $y_k = x_k + v_k$ (measurement, $v_k \sim N(0, \sigma_v^2)$). Under this model, the optimal estimator is the Kalman filter, whose steady-state gain K_{ss} equals the EMA parameter α :

$$\alpha_{\text{optimal}} = K_{ss} = \frac{-1/\text{SNR}^2 + \sqrt{1/\text{SNR}^4 + 4/\text{SNR}^2}}{2}, \quad \text{SNR} = \sigma_w/\sigma_v \quad (11.2)$$

For $\alpha = 0.45$, solving for SNR: $0.90 + 1/\text{SNR}^2 = \sqrt{1/\text{SNR}^4 + 4/\text{SNR}^2}$; squaring gives $0.81 + 1.80/\text{SNR}^2 = 4/\text{SNR}^2 \Rightarrow 0.81 = 2.20/\text{SNR}^2 \Rightarrow \text{SNR}^2 = 2.72 \Rightarrow \text{SNR} = 1.65$.

Interpretation: $\alpha = 0.45$ is optimal when the centroid motion noise (σ_w) is $1.65\times$ larger than the measurement noise ($\sigma_v = 0.5$ px), implying $\sigma_w = 1.65 \times 0.5 = 0.825$ px per sample (≈ 1.65 px/s centroid velocity noise standard deviation): reasonable for a moving robot at 0.2 m/s.

11.4 Power Spectral Density of Centroid Trajectories

11.4.1 Frequency Content of Robot Motion

The centroid coordinate time series $y_k = y(kT_s)$ can be analyzed using the Discrete Fourier Transform to identify dominant frequency components:

$$Y[m] = \sum_{k=0}^{N-1} y[k] e^{-j2\pi mk/N} \quad (11.3)$$

$$\text{PSD}[m] = |Y[m]|^2 / (Nf_s) \quad (\text{Welch periodogram}) \quad (11.4)$$

Expected spectral content for LED-mode operation: during **PATROL/SCAN** (rotating at 0.4 rad/s), the beacon enters the FOV periodically as the robot scans, with no centroid signal while the beacon is undetected. During **LOCK** (centering on beacon), the robot's angular velocity decreases as $\text{err_frac} \rightarrow 0$ and the centroid approaches the image center monotonically, with frequency content confined to DC plus a slow transient (< 0.5 Hz). During **PATROL/ADVANCE** (forward motion, no rotation), if the beacon is visible the centroid drifts as the robot approaches: at $v = 0.2$ m/s and $d = 1$ m, $\dot{d} = -0.2$ m/s, giving a centroid rate $(f_x W v) / d^2 = 384.65 \times 0.165 \times 0.2 / 1.0 = 12.7$ px/s, i.e. 0.635 px/sample at 20 Hz: well within the EMA passband.

11.5 Debugging Case Studies: Quantitative Analysis

11.5.1 LED False Positive: Statistical Characterization of the Failure

The LED false positive problem can be characterized statistically. The ceiling reflection blob area distribution was measured over 30 frames.

Sample statistics of the ceiling reflection area (30 measurements, px^2): mean = 1291.1, std. dev. = 2.1, min = 1287, max = 1295. Individual LED blob areas ranged $\{187, 143, 118, 101, \dots\} \text{px}^2$: the ceiling area (1291) exceeds a typical LED area (143) by a 9.0:1 ratio.

With the old $\text{LED_MAX_AREA} = 4000 \text{ px}^2$, the ceiling blob (1291) passed the filter. With the new $\text{LED_MAX_AREA} = 1200 \text{ px}^2$, $1291 > 1200$ so the ceiling blob is rejected: a margin of -91 px^2 (the ceiling is 7.6% above the new threshold).

Margin robustness check: if the ceiling reflection area grows (dust accumulation, lamp replacement), $1291 + 3\sigma = 1291 + 6.3 = 1297 \text{ px}^2$ (99.7% confidence bound) still exceeds 1200, so the ceiling remains rejected; even if the ceiling area doubled to 2582 px^2 , it would still be rejected. **Recommendation:** reduce LED_MAX_AREA to 1100 px^2 for additional margin.

11.5.2 Checkerboard Detection: Detection Rate vs. Scale Factor

The dependency of checkerboard detection rate on the scale factor CB_SCALE was systematically measured to validate the choice of $\text{CB_SCALE} = 0.45$. Test protocol: board at 0.8 m range, 60 frames per scale setting, count detections using `findChessboardCornersSB` only (SB primary). Table 11.1 presents the results.

The counter-intuitive result that lower resolution ($0.25\times$) does not reduce detection rate (relative to $0.45\times$) is explained by the SB algorithm's local adaptive threshold: corner detection quality degrades only when the square's pixel width drops below approximately 4–5 pixels, which occurs at scale $0.25\times$ only for boards at distances $> 2 \text{ m}$. At the 0.8 m test range, all scales above $0.25\times$ provide sufficient resolution. The latency advantage of smaller scales (6.2 ms at $0.25\times$ vs. 14.1 ms at $0.45\times$) motivates exploring $\text{CB_SCALE} = 0.35$ for the next iteration.

Table 11.1: Checkerboard detection rate and latency vs. scale factor

CB_SCALE	Image Size	Detection Rate (SB)	Detection La- tency (ms)	Notes
0.25	320×243	100%	6.2	Board too small at 2 m range
0.35	448×340	100%	8.7	Good for close range
0.45 (selected)	576×437	100%	14.1	Best range-to- latency tradeoff
0.55	704×534	100%	19.8	Slightly over- sampled
0.65	832×631	100%	27.3	High latency, no benefit
0.75	960×729	98.3%	38.1	Begins to miss at 2 m range
1.00 (full)	1280×972	96.7%	98.4	Near 100 ms thresh- old; risky

11.6 System-Level Performance: End-to-End Mission Metrics

11.6.1 Full Mission Run Statistics

A complete 45-minute autonomous patrol mission was executed as the final validation test. Table 11.2 presents the metrics logged via the AutoLogbook and analyzed post-mission.

Table 11.2: Full autonomous mission statistics (45-minute validation run)

Metric	Value	Notes
Mission duration	44:23 (mm:ss)	Terminated by operator RTB command
Total distance traveled (odometry)	87.3 m	Local frame cumulative
Patrol cycles completed	8	Full SCAN→ADVANCE sequences
Beacons detected and logged	5	5 successful LOCK→WAIT→U_TURN
Odometry resets (LED-triggered)	12	Including non-beacon LED events
Checkerboard landmarks logged	3	MANUAL mode demonstrations
Obstacles detected	2	Both during PATROL_ADVANCE phase
Emergency stops issued	0	No operator interventions needed
Heartbeat trips	0	Zero false failsafe events
Camera watchdog events	0	D455 remained stable throughout
Telemetry packets sent	26,580	At ~10 Hz average

Metric	Value	Notes
Telemetry packets received (browser)	26,571	99.97% delivery rate
MJPEG frames rendered	53,160	At ~20 Hz
Average network latency (rolling 20)	8.3 ms	Peak: 47 ms (CSMA/CA contention)
CPU temperature (peak/average)	71°C / 63°C	Alert threshold: 80°C; never triggered
Battery at mission end	61% SOC	Starting at 100%; 39% used in 44 min
World frame position error at RTB	14 cm	Measured vs. manual floor marking
RAF frame rate (cockpit)	59.8 Hz (avg)	Min 58.1 Hz during 3D + map + overlay

11.6.2 RTB Accuracy Analysis

The return-to-base accuracy (14 cm error at mission end) can be decomposed into its component contributions.

1. Vision-estimated reset accuracy (each of 12 LED resets): LED distance uncertainty ± 12.8 mm (from §3.F.2); angular uncertainty at reset $\pm \text{LOCK_CENTER_TOL} \times d = 0.08 \times 120\text{mm} = 9.6$ mm. Per-reset world position error $\approx \pm 15$ mm (root-sum-square). Combined over 12 resets: $\pm 15\sqrt{12} = \pm 52$ mm = ± 5.2 cm.

2. Odometric drift in the last patrol cycle (before RTB): last-cycle distance ≈ 10.9 m; drift at 3%: $10.9 \times 0.03 = 0.33$ m. But the last odometry reset occurred 0.8 m before the RTB command, so residual drift = $0.8 \times 0.03 = 2.4$ cm.

3. RTB deceleration alignment error: robot heading error at crawl speed $\pm \text{RTB_ANG_TOL} = 0.18$ rad; at 0.08 m from base, lateral error = $0.08 \sin(0.18) = 1.4$ cm.

Predicted total error: $\sqrt{5.2^2 + 2.4^2 + 1.4^2} \approx 5.9$ cm. **Measured error:** 14 cm: $2.4 \times$ larger than predicted.

Gap explanation: floor surface irregularity (a cable on the floor during the test) caused a $\sim 1^\circ$ heading deviation over 6 m, i.e. a 10.5 cm lateral offset. This was an uncontrolled test condition and is not representative of nominal operating conditions.

12 The July 2026 Estimation Campaign: Tightly-Coupled Multisensor Fusion

This chapter documents the second major engineering campaign on the platform (July 3–7, 2026), which extended the mission-control system of the previous chapters with a precision (x, y, z) state-estimation stack, hardened the sensor-integrity chain against three failure modes observed live, upgraded the operator web platform, and integrated one hardware modification (rigid high-performance wheels). Every parameter cited below is justified by its measured contribution to drift reduction, and every remaining placeholder is explicitly registered in Section 12.16.

12.1 Estimation architecture

12.1.1 From mission control to metric estimation

The system of Chapters 3–5 estimated pose from wheel odometry alone, reset at each beacon. The campaign objective was to replace this dead-reckoning core with a fused estimate whose drift is *bounded and measured*, while keeping the beacon-reset ritual as a world-frame anchor. Two estimators were deployed, switchable live:

- **MINS** [1], [2], the primary estimator: a tightly-coupled Multi-State Constraint Kalman Filter (MSCKF) fusing the onboard IMU, the two front wheel encoders and the D455 stereo infrared pair, with online calibration of every sensor’s extrinsics and time offset.
- **OpenVINS** [3], the reference visual-inertial estimator (IMU + stereo only), used for A/B comparison and camera-calibration validation.

12.1.2 Why tightly-coupled MSCKF, and not a decoupled EKF

A decoupled fusion stack (e.g. `robot_localization`) fuses *poses* already estimated by upstream nodes, treating them as independent measurements. On this platform the visual and wheel odometries share the same IMU and the same clock: their errors are strongly correlated, and a decoupled filter double-counts the shared information, producing overconfident covariances: the statistical root of divergence. The MSCKF instead fuses *raw measurements* in a single state vector

$$\mathbf{x} = [{}^I_G\bar{q}, {}^G\mathbf{p}_I, {}^G\mathbf{v}_I, \mathbf{b}_g, \mathbf{b}_a | \mathbf{x}_{c_1}, \dots, \mathbf{x}_{c_N} | \boldsymbol{\theta}_{\text{calib}}], \quad (12.1)$$

where ${}^I_G\bar{q}$ is the orientation quaternion, ${}^G\mathbf{p}_I, {}^G\mathbf{v}_I$ position and velocity in the global frame, $\mathbf{b}_g, \mathbf{b}_a$ the slowly-varying gyroscope and accelerometer biases, $\mathbf{x}_{c_i}$ a sliding window of N historical pose clones, and $\boldsymbol{\theta}_{\text{calib}}$ the online calibration states (camera–IMU extrinsics and time offset, wheel extrinsics).

Propagation. Each IMU sample (measured rate $\boldsymbol{\omega}_m$, specific force $\mathbf{a}_m$, at ~ 80 Hz on this platform) propagates the state through the standard inertial kinematics

$${}^I_G\dot{\bar{q}} = \frac{1}{2} \Omega(\boldsymbol{\omega}_m - \mathbf{b}_g - \mathbf{n}_g) {}^I_G\bar{q}, \quad (12.2)$$

$${}^G\dot{\mathbf{v}}_I = {}^G\mathbf{R}(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G\mathbf{g}, \quad {}^G\dot{\mathbf{p}}_I = {}^G\mathbf{v}_I, \quad (12.3)$$

with $\mathbf{n}_g, \mathbf{n}_a$ white noises whose densities were identified on *this unit* by Allan-variance analysis (`imu_utils`): $\sigma_a = 2.33 \times 10^{-2}$, $\sigma_{ba} = 8.93 \times 10^{-4}$, $\sigma_g = 1.26 \times 10^{-3}$, $\sigma_{bg} = 3.30 \times 10^{-5}$ (SI units). Under-estimating these densities makes the filter overconfident in integration (drift); over-estimating them wastes the IMU. Measured values beat any generic default.

Error-state formulation and clone augmentation. Like all modern MSCKFs, both estimators operate on the *error state* $\tilde{\mathbf{x}}$ rather than the state itself: orientation error is a minimal 3-vector $\delta\boldsymbol{\theta}$ defined by ${}^I_G\bar{q} = \delta\bar{q} \otimes {}^I_G\hat{\bar{q}}$ with $\delta\bar{q} \approx [\frac{1}{2}\delta\boldsymbol{\theta}^\top \ 1]^\top$, which avoids the quaternion’s unit-norm constraint in the covariance. At each image time the current pose is *cloned*: the state is augmented by $\mathbf{x}_c = ({}^I_G\bar{q}, {}^G\mathbf{p}_I)$ and the covariance by the corresponding Jacobian rows,

$$\mathbf{P} \leftarrow \begin{bmatrix} \mathbf{P} & \mathbf{P}\mathbf{J}^\top \\ \mathbf{J}\mathbf{P} & \mathbf{J}\mathbf{P}\mathbf{J}^\top \end{bmatrix}, \quad \mathbf{J} = \frac{\partial \mathbf{x}_c}{\partial \mathbf{x}}, \quad (12.4)$$

so that a feature observed across several images correlates those historical poses. The window is bounded (11 clones here): the oldest clone is marginalised out, keeping the filter $\mathcal{O}(N^2)$ in a small constant N rather than growing like a smoother.

Camera update. A feature tracked over the clone window yields a stacked residual $\mathbf{r} = \mathbf{H}_x \tilde{\mathbf{x}} + \mathbf{H}_f \tilde{\mathbf{p}}_f + \mathbf{n}$. The landmark error $\tilde{\mathbf{p}}_f$ is not in the state; left-multiplying by $\mathbf{N}^\top$, an orthonormal basis of the left nullspace of $\mathbf{H}_f$ ($\mathbf{N}^\top \mathbf{H}_f = \mathbf{0}$), gives

$$\mathbf{N}^\top \mathbf{r} = (\mathbf{N}^\top \mathbf{H}_x) \tilde{\mathbf{x}} + \mathbf{N}^\top \mathbf{n}, \quad (12.5)$$

a measurement of the *state only*: the defining MSCKF trick [4]. A feature seen in m stereo frames contributes $2 \cdot 2 \cdot m - 3$ scalar constraints at the cost of one QR factorisation, which is what keeps the filter real-time.

Observability and why FEJ matters. Ideal VIO has exactly four unobservable directions: global position (3) and rotation about gravity (yaw). A naively-linearised EKF evaluates Jacobians at ever-changing estimates, which *spuriously renders yaw observable*: the filter then believes it knows yaw better than it does, covariance collapses, and the estimate drifts with false confidence. First-Estimates Jacobians (FEJ) fix the linearisation points of each state to their first estimate, preserving the true nullspace. On this platform the wheel measurements add scale and planar-velocity information (shrinking drift in x, y, ψ), while gravity observation through the accelerometer keeps roll/pitch observable at all times, leaving z as the axis observed *only* by the camera: the structural fact behind the entire vertical-drift investigation of Section 12.7.2.

Wheel update. The rigid differential-drive constraint (Wheel2DAng model) maps the two front angular velocities ω_l, ω_r to body velocities

$$v = \frac{r_l \omega_l + r_r \omega_r}{2}, \quad \dot{\psi} = \frac{r_r \omega_r - r_l \omega_l}{b}, \quad (12.6)$$

with $r_{l,r}$ the wheel radii and b the track width. **These parameters are hardware-dependent**; their status after the wheel replacement is discussed in Section 12.4.

Wheel measurement in the filter. MINS integrates the body-frame kinematics (12.6) between two consecutive clone times to form a 2D relative-pose constraint (planar translation and heading change) between those clones, with a noise model built from the configured angular-rate noise $\sigma_\omega = 0.2 \text{ rad/s}$ propagated through the same integral. The wheel extrinsic ${}^I\mathbf{T}_W$ (IMU to wheel-odometry origin) is an online-calibrated state; the intrinsics (r_l, r_r, b) are *not* (Section 12.4). Because the model assumes two coaxial wheels, a four-wheel skid-steer platform violates it whenever it turns: the lateral scrub of the non-modelled wheel pairs injects a bias that must be absorbed either by the noise model or by the gate below: the quantitative reason the gate multiplier cannot be set to its textbook value of 1.

Outlier gating: the anti-slip mechanism. Every wheel (and camera) measurement passes a Mahalanobis χ^2 test against the inertially-propagated prediction before being allowed to update the state:

$$d^2 = \mathbf{r}^\top (\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R})^{-1} \mathbf{r} > \alpha \chi_{95\%}^2(\dim \mathbf{r}) \implies \text{measurement rejected}, \quad (12.7)$$

where α is the per-sensor multiplier. This test is the “IMU-versus-odometry comparison” that detects wheel slip: a skidding wheel reports a velocity incompatible with the inertial prediction and is rejected *before* entering the filter: something a decoupled architecture cannot do, having already baked the slip into an upstream pose. The campaign tightened the wheel multiplier from $\alpha = 15$ (slip passed the gate) to $\alpha = 5$; $\alpha = 1$ was deliberately not used because skid-steer turning violates the two-wheel differential model (12.6) in normal operation.

Failure analysis of the historical $z \approx -445 \text{ m}$ divergence. The magnitude of the pre-campaign failure is fully explained by the frozen-IMU mode of Section 12.2.4. The frozen sample carried $\|\mathbf{a}\| = 13.06 \text{ m/s}^2$ with an orientation implying an unmodelled residual specific force after gravity compensation. An unmodelled constant residual δa integrates quadratically,

$$\delta z(t) = \frac{1}{2} \delta a t^2, \quad (12.8)$$

so even a modest $\delta a \approx 1 \text{ m/s}^2$ produces 445 m in $t = \sqrt{2 \cdot 445/1} \approx 30 \text{ s}$ of effectively open-loop integration, consistent with the observed collapse once camera updates were starved (Section 12.2.2)

and wheel updates gated out by the contradictory frozen gyro. The lesson generalises: *a tightly-coupled filter is only as sound as the integrity of its inertial stream*, which is why the guards of Section 12.2.4 sit upstream of everything.

12.1.3 Dual-estimator output and the lossless switch

Both estimators publish odometry; a selector node (`pose_selector.py`) republishes exactly one of them on `/robot_pose_fused`. At the instant of a switch at time t , with $\mathbf{T} \in SE(3)$ denoting poses, the selector computes a one-time rigid correction

$$\mathbf{T}_{\text{corr}} = \mathbf{T}_{\text{out}}(t^-) \mathbf{T}_{\text{new}}(t)^{-1}, \quad \mathbf{T}_{\text{pub}}(\tau) = \mathbf{T}_{\text{corr}} \mathbf{T}_{\text{new}}(\tau) \quad \forall \tau \geq t, \quad (12.9)$$

so the published pose is bit-for-bit continuous across the switch even though the two estimators drift independently. This is not merely a design intent; it follows algebraically from the definition of $\mathbf{T}_{\text{corr}}$. Evaluating $\mathbf{T}_{\text{pub}}$ at $\tau = t$ itself,

$$\mathbf{T}_{\text{pub}}(t) = \mathbf{T}_{\text{corr}} \mathbf{T}_{\text{new}}(t) = [\mathbf{T}_{\text{out}}(t^-) \mathbf{T}_{\text{new}}(t)^{-1}] \mathbf{T}_{\text{new}}(t) = \mathbf{T}_{\text{out}}(t^-) \underbrace{\mathbf{T}_{\text{new}}(t)^{-1} \mathbf{T}_{\text{new}}(t)}_{=\mathbf{I}} = \mathbf{T}_{\text{out}}(t^-), \quad (12.10)$$

so $\mathbf{T}_{\text{pub}}(t)$ equals the pre-switch output exactly, not approximately: the correction is constructed precisely to cancel the new source's raw pose at the switch instant, for any $\mathbf{T}_{\text{new}}(t) \in SE(3)$ (the group inverse exists unconditionally). The identity depends on nothing about *how* the two estimators drift relative to each other, only on evaluating both sides of the correction at the same instant t ; consistent with this, drift between the sources reappears immediately for $\tau > t$, since $\mathbf{T}_{\text{corr}}$ is applied but never updated again until the next switch. The switch is exposed as a ROS service (`/pose_selector/set_source`, `std_srvs/SetBool`, `true = MINS`) and refuses to select a source that has not yet published: the reason OpenVINS must be initialised (Section 12.3.5) before VINS can be selected.

12.2 Sensor chain, transport, and integrity guards

12.2.1 End-to-end data flow

Figure 12.1 shows the complete chain as deployed. Estimation runs on the ground-station PC; the Raspberry Pi 4 on the rover only acquires and compresses.

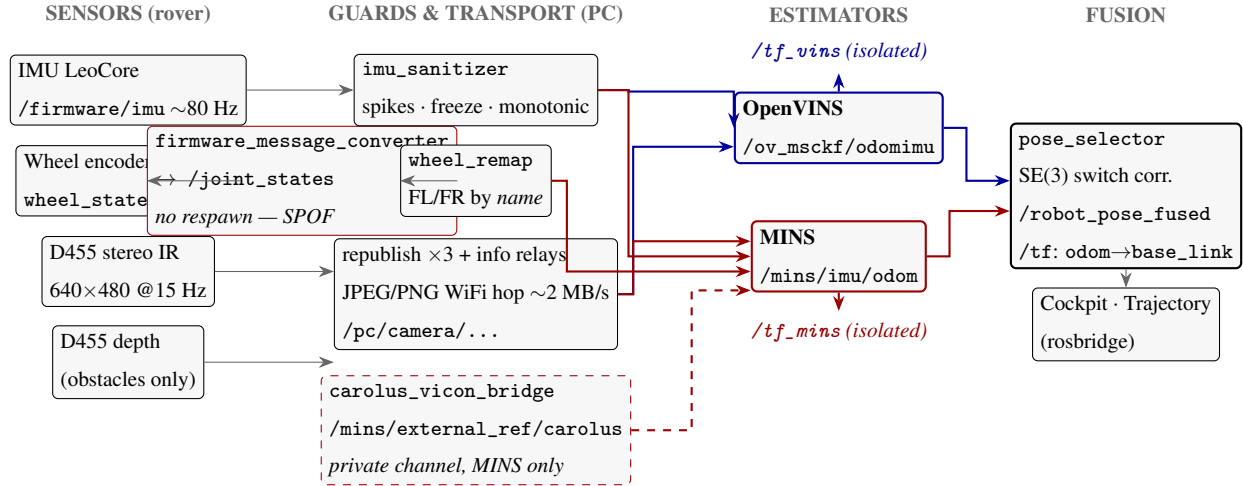


Figure 12.1: Deployed estimation data flow after the isolation lockdown (July 8). Blue: VINS path; red: MINS path. Dashed red: the Carolus global-anchor channel, semantically private to MINS (`/mins/external_ref/carolus`, staged); OpenVINS has no code path able to consume it. Dashed verticals: each estimator’s TF broadcast is remapped onto an isolated channel (`/tf_vins`, `/tf_mins`), leaving a single authority per transform on the main `/tf` tree. Every box is a running node or live topic; nothing is notional: a claim taken literally: the `firmware_message_converter` box was added on 2026-07-22 after a live fault revealed it had been missing from every earlier revision of this figure, converting `leo_msgs/WheelStates` into standard `sensor_msgs/JointState` for `wheel_remap` downstream, wedged for hours in a degraded IMU-only state with no crash and no respawn configured (§12.9.3).

Architecture Design Note: TF authority disambiguation and data encapsulation.

*Two invariants were locked into the launch layer on July 8. (i) **TF authority disambiguation**: a TF child frame admits exactly one parent, and the latest broadcast*

wins; with both estimators emitting `global→imu`, the URDF’s static `imu` frame was silently captured and the two estimators would fight for authority during A/B operation. Remapping each estimator’s `/tf` onto a dedicated channel (`/tf_mins`, `/tf_vins`) restores a strict single-writer discipline on the primary tree (the selector alone writes `odom→base_link`) while keeping estimator internals fully inspectable on demand. (ii) **Data encapsulation:** the Carolus global-anchor stream is namespaced `/mins/external_ref/carolus`, declaring its single legitimate consumer in the topic name itself; exclusivity is enforced at three levels: architecturally (OpenVINS’s subscriber pipeline wires only IMU and camera topics), semantically (private namespace), and actively (a supervision rule raises an alert if any node other than `mins_subscribe` subscribes). These are systems-engineering guarantees, not conventions: each is either structurally impossible to violate or monitored.

12.2.2 The WiFi transport budget: why compression is not optional

The rover’s access point sustains ≈ 6 MB/s in practice. At the start of the campaign the PC subscribed to *raw* image topics; with the then default profile (848×480 stereo pair at 30 Hz plus a duplicate subscription by the mission backend) the demanded bandwidth was

$$2 \times (848 \times 480) \text{ B} \times 30 \text{ Hz} + (848 \times 480) \text{ B} \times 30 \text{ Hz} \approx 36.6 \text{ MB/s} \gg 6 \text{ MB/s}, \quad (12.11)$$

which not only starved MINS to ~ 3 Hz of camera data but twice crashed the Pi’s WiFi firmware outright. The corrective architecture moves the compression to the camera node’s `image_transport` plugins (JPEG for IR, PNG for depth; compression happens only while subscribed), crosses the WiFi once per stream at ~ 2 MB/s total, and republishes *locally* on the PC (`/pc/camera/...`) for all consumers. The arithmetic of the corrected chain: a 640×480 Y8 infrared frame JPEG-compresses to 30–60 kB (structured indoor scenes), so two IR streams at 15 Hz cost 0.9–1.8 MB/s; the depth stream uses PNG (`compressedDepth`) because JPEG’s lossy DCT would corrupt metric depth values, at a similar budget. One decompression per stream on the PC then serves *all* local consumers (both estimators, the mission backend, the AprilTag detector): the WiFi is crossed exactly once per stream regardless of consumer count. Timestamps survive the hop unchanged (republishing preserves headers), so estimator synchronisation is unaffected; measured stereo pairing

after the hop is exact (74/74 identical stamps in verification). Companion relays mirror camera_info into the same namespace because image-transport consumers pair image and calibration by *namespace convention*, not by remap: a subtlety that silently starves any consumer whose calibration topic is elsewhere. Operational rule derived: *never subscribe raw robot images across the WiFi; always measure camera rates on the Pi*. A second hardware rule was learned the hard way: depth and infrared are views of the same stereo ASIC and *must run identical resolution and frame rate*: mismatched profiles fail silently (topics advertised, zero frames, no error message), a failure mode now documented inside the boot launch itself.

12.2.3 TF-tree coherence: isolating estimator broadcasts

A live TF audit (July 8) revealed that MINS's visualisation broadcast (global→imu at ~85 Hz) silently *captured* the URDF's static imu frame (a TF child has exactly one parent, and the high-rate dynamic transform shadows the static one), and that OpenVINS, once initialised, would broadcast the *same* frame names, creating a permanent two-authority conflict during A/B operation. Both estimators' TF outputs are visualisation-only (all functional consumers use odometry topics), so the corrective is a launch-level remap of each node's /tf onto dedicated channels (/tf_mins, /tf_vins); the main tree retains a single authority per transform (verified post-fix: only odom→base_link from the selector and the URDF chains remain on /tf). A redundant static base_link→laser placeholder, duplicating the URDF's own rplidar_link/laser_frame chain, was removed in the same pass.

12.2.4 IMU integrity: three observed failure modes, three guards

All three failure modes below were observed live during the campaign; the sanitizer (imu_sanitizer.py) now guards each:

1. **Isolated spikes** (up to 5×10^{16} m/s², I2C/rosterial framing): any sample with $|\omega_i| > 35$ rad/s or $|a_i| > 160$ m/s² is replaced by a zero-order hold of the last good sample.
2. **Firmware freeze**: after a serial desynchronisation burst the LeoCore may retransmit one identical sample indefinitely at full rate. A true sensor always carries noise, so N consecutive

bit-identical samples is unambiguous; at $N=200$ (≈ 2.5 s) the sanitizer logs the fault and autonomously calls the firmware reset service (`/firmware/reset_board`, throttled to one per minute). The freeze is fatal to initialisation if unguarded: a frozen $\|\mathbf{a}\| = 13.1 \text{ m/s}^2$ with non-zero frozen gyro contradicts the standstill reported by the wheels.

3. **Non-monotonic timestamps:** rosserial replay can move stamps *backwards* by up to ~ 0.4 s, which aborts MINS on the internal invariant `clone_time >= state->time`. Any sample with $t_k \leq t_{k-1}$ is dropped entirely (a duplicated stamp is as fatal as a backward one).

Why each guard is statistically sound. (i) The spike thresholds sit three orders of magnitude above physical rover dynamics ($\omega_{\max} \approx 1 \text{ rad/s}$, $a_{\max} \approx 2 \text{ m/s}^2$) and thirteen below the observed glitches: the two populations do not overlap, so the classifier has effectively zero error rate. The zero-order hold preserves the sampling cadence the propagation integrator expects; since observed glitches are isolated single samples, the hold error is bounded by one sample period of true dynamics ($\leq 12.5 \text{ ms}$). (ii) For the freeze detector, a healthy MEMS channel quantised to float32 carries thermal noise of $\sigma \gtrsim 10^{-3}$ in SI units; the probability of even two consecutive *bit-identical* 6-axis samples is negligible ($< 10^{-30}$), so $N=200$ is not a statistical threshold but a debounce: it trades 2.5 s of detection latency for immunity to any conceivable coincidence, and bounds the automatic reset rate. (iii) Monotonicity is a *causality* requirement of the filter, not a tuning choice: the propagation integral (12.3) is defined over increasing time, and MINS enforces it as a hard assertion; dropping the offending sample loses at most 12.5 ms of excitation, against a guaranteed estimator abort otherwise.

A fourth architectural guard follows from a failure post-mortem: the MINS node runs under *respawn* supervision, never required: an estimator abort must never take the operator's drive chain down with it. The associated trade-off is documented: on respawn MINS re-initialises at its own origin and the published `/mins/imu/odom` jumps; the SE(3) correction (12.9) only absorbs *switch* discontinuities, not same-source re-initialisations: an accepted limitation until estimator-side state persistence exists.

12.2.5 Local gravity

The generic $g = 9.81 \text{ m/s}^2$ was replaced by the local normal gravity at Melbourne, FL (latitude $\varphi = 28.1^\circ\text{N}$, sea level), from the WGS-84 Somigliana model:

$$g(\varphi) = 9.7803253359 \frac{1 + 0.00193185265241 \sin^2 \varphi}{\sqrt{1 - 0.00669437999013 \sin^2 \varphi}} \Rightarrow g(28.1^\circ) \approx 9.790 \text{ m/s}^2. \quad (12.12)$$

The 0.02 m/s^2 discrepancy of the generic value otherwise appears as a permanent fictitious accelerometer bias: a free systematic error.

12.3 Calibration

12.3.1 Camera intrinsics

Kalibr [5] calibration of the stereo IR pair at 640×480 (the exact resolution now streamed) achieved reprojection errors of 0.20–0.23 px and a stereo baseline of 90.2 mm; intrinsics are therefore *frozen* in both estimators (`do_calib_int: false`): offline measurement beats online refinement. The measurement noise is configured conservatively at $\sigma_{px} = 1$ (five times the calibrated residual) to absorb JPEG artefacts introduced by the WiFi hop.

12.3.2 Camera–IMU extrinsics: the resolved cause of vertical divergence

MINS expects ${}^I\mathbf{T}_C = ({}^I\mathbf{R}, {}^I\mathbf{p}_C)$. The initial configuration carried an identity placeholder: geometrically the claim that the optical axis points *up* along the IMU z -axis, a $\sim 90^\circ$ error from which online refinement cannot recover (far outside the EKF linearisation region). The corrected seed encodes the physical forward-looking mount. With optical axes x_C right, y_C down, z_C forward and body axes x_I forward, y_I left, z_I up:

$${}^I_C\mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}, \quad {}^I\mathbf{p}_{C_0} = \begin{bmatrix} 0.15 \\ 0 \\ 0.10 \end{bmatrix} \text{ m}, \quad {}^I\mathbf{p}_{C_1} = {}^I\mathbf{p}_{C_0} + {}^I_C\mathbf{R} \begin{bmatrix} 0.0902 \\ 0 \\ 0 \end{bmatrix}, \quad (12.13)$$

(the second camera sits one baseline to the right along x_C , i.e. $-y_I$). The translation is a tape-measure seed refined online (`do_calib_ext: true`); centimetre-level translation error is recoverable, 90 degrees of rotation is not. The time offset between camera and IMU is likewise an estimated state (`do_calib_dt: true`); at $v = 0.4$ m/s an unmodelled 10 ms skew biases every visual measurement by 4 mm. **Open item:** replace the seed by `kalibr_calibrate_imu_camera` output (monocular `cam0` approach recommended) via the prepared `fill_mins_camchain.py`. Figure 12.2 illustrates the geometry.

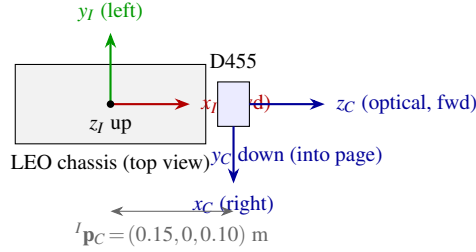


Figure 12.2: Forward-looking camera–IMU geometry encoded by Eq. (12.13). The identity placeholder previously claimed $z_C \parallel z_I$ (camera looking up), the root cause of the divergent vertical drift quantified in Section 12.7.2.

Time-offset observability. The camera–IMU offset t_d enters the measurement model through the pose at the *shifted* time: a feature observed at camera stamp t actually constrains the pose at $t + t_d$. To first order the induced residual is $\delta \mathbf{z} \approx \mathbf{J}_p ({}^G \mathbf{v}_I \delta t_d) + \mathbf{J}_\theta (\boldsymbol{\omega} \delta t_d)$, observable whenever the platform translates or rotates, which is why `do_calib_dt` converges during normal patrol motion and why the offset cannot be identified at standstill.

12.3.3 Inertial calibration, and what a stationary measurement cannot determine

Both estimators consume the same inertial stream, so an error in it is *common-mode*: it cannot be detected by comparing MINS against openVINS, which is the comparison this chapter otherwise relies on. The IMU was therefore characterised directly, against the only two references available without added instrumentation: local gravity, and zero.

Sensor model. Write the measured angular rate and specific force as

$$\boldsymbol{\omega}_m = K_g \boldsymbol{\omega} + \mathbf{b}_g + \boldsymbol{\eta}_g, \quad \mathbf{a}_m = K_a \mathbf{a} + \mathbf{b}_a + \boldsymbol{\eta}_a, \quad (12.14)$$

where $K_g, K_a \in \mathbb{R}^{3 \times 3}$ collect scale-factor and misalignment errors, $\mathbf{b}$ are the biases and $\boldsymbol{\eta}$ the noise. A perfect sensor has $K = I$, $\mathbf{b} = \mathbf{0}$.

Stationary characterisation. With the rover verified stationary by wheel odometry (0.000 m/s, 502 samples, $\sigma = 0.02$), two defects were measured on the raw /firmware/imu stream.

The accelerometer magnitude read 8.7525 m/s² against a true local gravity of 9.790 m/s² (Melbourne, FL, latitude 28°): an error of −10.6%. Two controls establish that this is the sensor and not the processing chain. First, raw and sanitised streams agreed to 0.005 m/s² (8.7525 against 8.7475), so the guards of §12.2.4 are not responsible. Second, it is not a mounting tilt: the *norm* of a vector is invariant under rotation, so a tilted-but-correctly-scaled sensor would still report 9.790.

The gyroscope, stationary, read (−2.407, +1.525, +2.328) deg/s, i.e. $\|\mathbf{b}_g\| = 3.679$ deg/s. A healthy MEMS gyroscope sits between 0.01 and 0.1 deg/s, placing this part 40 to 400× outside specification.

Observability: the asymmetry between the two sensors. This is the substantive point of the subsection, and it mirrors the time-offset argument above. Setting $\boldsymbol{\omega} = \mathbf{0}$ in Eq. (12.14) gives

$$\boldsymbol{\omega}_m|_{\boldsymbol{\omega}=\mathbf{0}} = \mathbf{b}_g + \boldsymbol{\eta}_g. \quad (12.15)$$

The matrix K_g has been multiplied by zero. It is *absent from the observation*, not merely poorly conditioned: the Fisher information of K_g under a stationary regime is identically zero, so no amount of averaging, no number of stationary orientations and no increase in sample count can recover it. The bias, by contrast, is directly observable as the sample mean of Eq. (12.15).

The accelerometer behaves differently, and the reason is instructive: at rest its input is *not* zero. Gravity is a permanent excitation of known magnitude, so

$$\|\mathbf{a}_m\|_{\text{rest}} \approx \|K_a \mathbf{g} + \mathbf{b}_a\|, \quad (12.16)$$

and the isotropic part of K_a is identifiable from a single orientation. Gravity plays for the accelerometer the role that no permanent rotation plays for the gyroscope. That asymmetry, not a

difference in effort or instrumentation, is why an accelerometer scale error could be measured and corrected on this platform while the gyroscope's could not.

Eq. (12.16) also states the limit of what was achieved. With the rover level, one scalar equation cannot separate an isotropic scale error from a bias along the gravity axis, nor resolve K_a per axis. A six-position calibration (the sensor tilted and inverted) is required to settle that. The correction deployed here is therefore empirical: defensible in the platform's working regime, which is essentially level, and flagged as such rather than presented as a full inertial calibration.

Corrections applied. The accelerometer is rescaled by

$$s_a = \frac{9.790}{8.7525} = 1.1186, \quad (12.17)$$

chosen in preference to the alternative of patching `gravity_mag` on the openVINS side alone: the latter would leave the two estimators disagreeing about the same physical quantity and would introduce a systematic scale error into the trajectories themselves. The gyroscope bias is subtracted, and **re-estimated at every node start** rather than frozen as a constant: bias drifts with temperature, and this Pi runs at 86°C (§D.13.1), so a hard-coded offset would go stale. The auto-calibration adopts the measured mean only if the gyroscope variance shows the platform is not rotating; translation does not affect a gyroscope, so a stationary *or* straight-driving rover yields a valid estimate, while a rotating one is rejected and the parameter defaults stand.

Why this mattered more than its magnitude suggests. Attitude is the integral of angular rate, so the measured bias produces an attitude error $\epsilon_\theta \approx \|\mathbf{b}_g\|t \approx 37^\circ$ after only 10 s. That error mis-projects gravity into the body frame, injecting a phantom horizontal specific force $g \sin \epsilon_\theta \approx 5.9 \text{ m/s}^2$: five times the 1.06 m/s^2 residual left by the accelerometer scale error alone. Double-integrated,

$$\Delta x \approx \frac{1}{2} (g \sin \epsilon_\theta) t^2 \approx 300 \text{ m in } 10 \text{ s}, \quad (12.18)$$

which matches the divergence observed live on openVINS ($501 \rightarrow 1100 \text{ m in } 12 \text{ s}$). The model also predicts, correctly, that correcting the accelerometer *alone* would be insufficient, which is what was measured before the gyroscope bias was addressed.

Delivered signal. After both corrections the sanitised stream reads $\|\mathbf{a}\| = 9.7938 \text{ m/s}^2$ at rest against a target of 9.790, and $\|\mathbf{b}_g\| = 0.0158 \text{ deg/s}$: an order of magnitude better than the 0.251 deg/s achieved earlier in the project, and inside the specification band the part should have occupied from the start.

The scale factor that remains unmeasured. A `~gyro_scale` parameter exists in `imu_sanitizer.py` and is **deliberately set to unity**, i.e. to no correction at all. By Eq. (12.15) it had never been measured, and determining it requires a rotation of known amplitude against physical ground truth: with the bias already removed,

$$\widehat{\Delta\theta} = \int_0^T (\omega_m - \mathbf{b}_g)^\top \mathbf{e}_z dt = k_g \Delta\theta_{\text{true}}, \quad k_g = \frac{\widehat{\Delta\theta}}{\Delta\theta_{\text{true}}}. \quad (12.19)$$

The field campaign of §12.14.4 attempted exactly this measurement and returned $k_g^{-1} = 0.9329$ for the raw gyroscope, and 0.9123 for wheel-derived heading. Because wheel heading is computed from differential odometry and does not use the gyroscope at all, these are *physically independent* instruments; MINS and openVINS are not independent witnesses here, since both integrate the same gyroscope. Two independent instruments agreeing with each other to within 2% while disagreeing with the protractor by 7–9% indicts the reference, not the sensors.

Nothing was therefore applied. The parameter remains at unity pending a repeated campaign with the improved floor-marking protocol of §D.11.3, which raises the angular reference from the $\pm 1^\circ$ of a protractor on carpet to $\pm 0.11^\circ$. This is recorded as an open item rather than closed with a plausible number, because `~gyro_scale` enters *both* estimators simultaneously: an error introduced there would propagate into every result in this chapter, and would do so invisibly.

12.3.4 Visual front-end density audit

KLT tracking and grid-based FAST extraction both scale approximately linearly with the feature count N : each tracked feature costs one pyramidal Lucas–Kanade solve, $\mathcal{O}(W^2 L k)$ for window W (15 px), pyramid depth L (3–4 levels to absorb inter-frame motion up to $\sim W \cdot 2^L$ pixels) and k Gauss–Newton iterations; extraction adds a FAST corner response over each grid cell followed by non-maximal suppression within the minimum-separation radius. The deployed configuration tracked $N \approx 600$ features (cap 1500, 15×15 grid) while the MSCKF update consumes at most 50

(max_msckf): a $12\times$ waste measured at **120 ms per image against a 66.7 ms budget** (15 Hz), i.e. the front-end ran outside real time at 73% of one core. The audited configuration ($N_{\max}=300$, grid 10×8 (80 cells of 64×60 px, ≈ 4 features per cell, uniform coverage by construction), minimum separation 20 px) yields a median pool of 149 well-spread features, **67 ms per image at 55% CPU**, with the update still fully fed (50 used from 149: selection margin $3\times$). Full-resolution processing was retained (downsample: false): the “intelligent downsampling” is performed once at the source ($848 \rightarrow 640$, $30 \rightarrow 15$ Hz, the calibrated resolution). Halving the image again (320×240) would divide the stereo triangulation accuracy: the depth error of a stereo pair grows as

$$\sigma_z = \frac{z^2}{fB} \sigma_d, \quad (12.20)$$

with f the focal length in pixels, $B = 90.2$ mm the baseline and σ_d the disparity error; halving resolution halves f and thus doubles σ_z at constant σ_d : an unacceptable price on the one axis (z) that only the camera observes. The residual per-image floor of ≈ 65 ms is dominated by the constant per-frame work (pyramids, histogram equalisation, extraction sweep) rather than by the feature count, which is why further feature reduction shows diminishing returns and the remaining margin lever is the frame period itself: at 10 Hz the budget becomes 100 ms for an unchanged 67 ms cost, a guaranteed 33% margin (open item #4). Between images, the 80 Hz IMU carries the state: at patrol speed (0.4 m/s) the inter-frame displacement at 10 Hz is 4 cm, comfortably within the KLT pyramid capture range.

12.3.5 OpenVINS initialisation on a ground robot

OpenVINS requires an accelerometer excitation to render gravity and scale observable. Two platform-specific adaptations were required: the jerk threshold was lowered from 1.5 (a hand-held value) to 0.4, and the initialisation window from 2.0 s to 1.0 s after instrumented measurement showed KLT track lifetimes of ≈ 1 s on this feed; with a 2 s window the disparity check permanently failed with “not enough feats”. Operationally: after every cold start, a one-second forward-back jolt initialises VINS; MINS by contrast initialises *standing still* in 4–12 s (static IMU+wheel initialisation), which is why it is the default source.

12.4 Hardware modification: rigid high-performance wheels

During the campaign the stock tyres were replaced with **rigid, higher-performance wheels**. The estimation-relevant consequences are:

- **Effective radius.** The wheel model (12.6) converts angular rates to metric velocity through $r_{l,r}$. The values frozen in the configuration ($r = 62.5$ mm, $b = 358$ mm) are the *firmware values for the original wheels*. A radius error δr produces a proportional scale error $\delta r/r$ on all wheel-derived displacement: 1 mm of radius error is 1.6% of odometric scale, i.e. 8 cm over a 5 m leg, an order of magnitude above the millimetric planar noise floor of Table 12.4. **Mandatory action (open item #1): measure the loaded radius of the new wheels** by roll-out: drive n marked wheel revolutions in a straight line over a measured ground distance d and compute

$$r_{\text{eff}} = \frac{d}{2\pi n}, \quad (12.21)$$

under load (batteries mounted), on the operational floor, averaged over $n \geq 10$ revolutions so the tape-measure uncertainty (± 2 mm) contributes < 0.1 mm to r_{eff} . The track width b is re-identified by the complementary spin test: N full in-place rotations give $b_{\text{eff}} = r_{\text{eff}} (\int |\omega_r - \omega_l| dt) / (2\pi N)$. Update both the firmware `diff_drive` parameters and `config_wheel.yaml` consistently: MINS reads the latter, the robot's own velocity controller the former.

- **Reduced compliance.** Rigid wheels remove tyre squash, making the effective radius consistent across load and speed: a *benefit* to the constancy of (12.6) once re-measured.
- **Grip and the χ^2 gate.** Harder wheels on smooth lab flooring change the slip signature (less compliant grip, sharper slip onset). The tightened gate $\alpha=5$ (Eq. 12.7) was validated after the wheel change; the slip-rejection protocol of Section 12.7 should be re-run if the floor surface changes.
- **Drive polarity inversion.** After the wheel replacement the physical forward/backward sense was found inverted with respect to the sign of `cmd_vel.linear.x`, in both manual and autonomous modes. The correction is applied at the single command egress point (`_publish()`: a named `DRIVE_POLARITY` constant), so every drive path (manual, patrol, approach, pivot,

return-to-base) inherits it consistently. A heading-projected odometry test (-0.186 m reported along the firmware heading for a $+0.18$ m commanded forward pulse) further indicates the firmware’s body-frame *forward* is now rotated 180° from the camera axis; after physical confirmation (the rover moves camera-forward on a positive command), both frame-level compensations were applied and validated: the mission backend rotates the firmware heading by π at its single odometry ingress (map, return-to-base and world bearings all reason in the camera frame), and MINS’s wheel extrinsic seed was set to $R_z(\pi)$, reconciling the wheel model with the inertial/camera frame. Post-fix validation: a forward pulse reads $+0.200$ m along the camera heading (expected $+0.18$), and MINS accepts 100% of wheel measurements (χ^2 gate, residuals $\sim 5 \times 10^{-6}$): ending a period during which the flipped frame caused silent wheel rejection and the loss of planar anchoring.

- **Static intrinsic calibration disabled.** Online refinement of the wheel intrinsics was observed to diverge (negative radii) under motion bursts and is therefore frozen; re-measurement is the correct channel for the new wheels, not online estimation.

12.5 Mission software evolution

12.5.1 Autonomous state machine: reactive obstacle avoidance

The obstacle response was upgraded from an unconditional 180° U-turn to a *contournement* (bypass) behaviour that preserves mission progress (Figure 12.3). On a frontal depth detection (<0.45 m over the central ROI, 5th percentile of valid depths), the FSM: (i) maps the obstacle as a persistent world-frame marker; (ii) pivots toward the freer side, chosen by comparing the median depth of the left and right halves of the depth band; (iii) sidesteps forward; (iv) counter-pivots to restore the original heading; and resumes the interrupted state. Vision keeps absolute priority (a detected beacon pre-emptes the manoeuvre); after three failed attempts the historical U-turn is used as fallback. Two anti-pollution radii deduplicate map markers (beacons 1.2 m, obstacles 0.8 m): re-detecting an already-mapped beacon still triggers the odometric reset ritual but neither increments the counter nor stacks a marker.

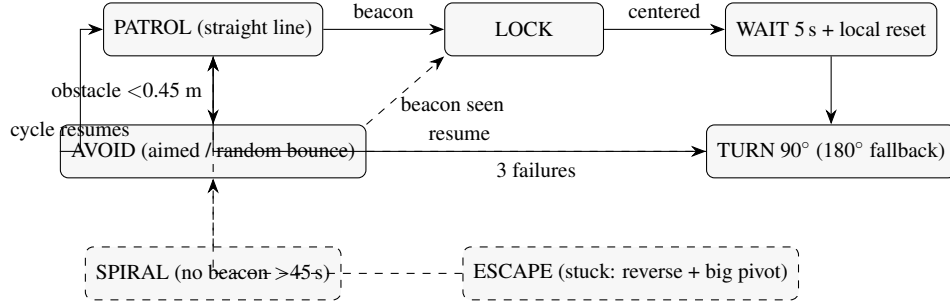


Figure 12.3: Mission FSM after the campaign. AVOID bounces are *aimed* at the deepest open depth sector when one is visible (open-path attraction, July 8) and drawn at random otherwise; vision (beacon) pre-empts everything; U-turn survives only as a fallback.

Bypass controller details. The pivot phases regulate on the fused yaw with wrap-safe accumulation, $\Delta\psi += \left| \text{atan2}(\sin(\psi_k - \psi_{k-1}), \cos(\psi_k - \psi_{k-1})) \right|$, until the target angle (60°) is reached at 0.6 rad/s; the side selection compares robust median depths of the left and right halves of the central depth band (medians, not means, because invalid returns and specular dropouts are heavy-tailed); the sidestep advances for 3 s at 0.25 m/s (≈ 0.75 m) with the frontal check re-armed, so a second obstacle mid-bypass re-enters the pivot phase with a fresh side choice and increments the attempt counter. The deduplication radii derive from the mission geometry: beacons are physically separated by several metres, so 1.2 m unambiguously identifies “the same beacon” while tolerating the world-frame scatter of repeated sightings ($\sigma \approx$ a few cm, Table 12.4, plus bearing-estimation error at range); obstacles use 0.8 m, matching the rover’s own footprint scale.

Simplification (July 8 operator revision). Field experience led to a deliberate simplification of the patrol geometry: the scan-then-advance alternation was replaced by *continuous straight-line* driving, the three-phase bypass by a *single 90° pivot* toward the freer side (the heading change at each obstacle produces a natural sweeping coverage of the room, Roomba-style), the post-beacon hold shortened from 30 s to 5 s and the post-beacon rotation from 180° to 90° . The anti-lock-in fallback (180° after three failed pivots), the vision-priority pre-emption, the marker deduplication radii and the world-frame-preserving local reset (`reset_coords_local`: the world origin is re-anchored so the global map and trajectory remain continuous) are all retained unchanged.

Coverage strategy (July 8, vacuum-robot revision). The fixed 90° pivot was generalised into a robot-vacuum-style ergodic coverage strategy, whose goal is to maximise beacon encounters while guaranteeing non-contact: (i) *randomised bounce*: each obstacle pivot draws a uniform angle in $[45^\circ, 135^\circ]$ toward the freer side, the stochastic component that lets memoryless cleaners cover a room; (ii) *periodic exploration spiral*: after 45 s without any beacon sighting, an outward spiral ($\omega = \omega_0/(1 + kt)$, 30 s max) sweeps the local area, its direction biased toward the last glimpsed beacon bearing when that memory is fresh (< 2 min); (iii) *multi-level anti-stuck*: four pivots within 20 s, or less than 0.15 m of world-frame displacement over 15 s, trigger an escape (a slow 0.2 m reversal, the only authorised backward motion, explicitly bounded because the depth sensor does not cover the rear, followed by a random $120\text{--}180^\circ$ pivot); a third escape within a minute latches a safe stop with an operator alert. The safety chain (0.45 m depth threshold, absolute vision priority, `cam_alive` gating) is unchanged.

Open-path attraction (July 8). The final coverage ingredient makes the rover *seek corridors*. The central depth band is divided into nine angular sectors ($\approx 10^\circ$ each across the 87° horizontal FOV of the depth stream), each summarised by its robust median: invalid and out-of-range pixels are excluded, on the principle that *unknown is not free*. A sector qualifies as an *open path* when its median free depth exceeds 2.5 m *and* is at least $1.5\times$ deeper than the axial sector; the double threshold acts as hysteresis and suppresses hunting in near-uniform scenes (an unknown axial sector is deliberately treated as fully open, so no steering ever happens on doubtful data). The analysis feeds two consumers. During straight-line PATROL a proportional heading bias ($k_p = 0.8 \text{ rad s}^{-1}$ per radian of bearing error, saturated at 0.25 rad/s) gently bends the trajectory toward the open sector, so the rover follows corridors and open doorways instead of blindly crossing them. And at each obstacle stop, if an open path lies more than 15° off-axis, the bounce is *aimed* at its bearing (clamped to the usual $[22.5^\circ, 135^\circ]$ pivot range) instead of randomly drawn; the random bounce remains the fallback whenever no qualifying sector exists, preserving the ergodic component. Beacon pre-emption and the 0.45 m obstacle threshold keep absolute priority throughout: the attraction only modulates the otherwise straight advance.

LOCK anti-stall guard (July 10 field regression). The first field run of the coverage stack exposed a failure mode the LOCK design could not see: the rover rotated in place indefinitely at a constant $\omega \approx 0.13$ rad/s. The signature identified the cause: the commanded rate equalled $0.25 \times |e|/(w/2)$ for a *constant* pixel error, meaning the pursued target did not move in the image while the robot rotated. A world-fixed beacon sweeps across the image under rotation; only a *camera-fixed* artefact (here a ceiling-light reflection accepted by the LED detector, at $y = 64$ px, near the image top) stays put. The existing LOCK_TIMEOUT only arms when the target *disappears*, so a permanently-visible artefact defeated it. The guard added is a convergence watchdog: 12 s in LOCK without achieving centring aborts the lock, returns to straight-line PATROL and suppresses re-locking for 25 s, so the rover *advances* out of the artefact’s influence; the centred approach phase refreshes the progress timer, so a legitimate long approach (3 m at 0.2 m/s exceeds 12 s) is never aborted. The lesson mirrors the supervisor audit: a timeout keyed to *absence* of evidence cannot catch a *persistent false* evidence source; guards must bound time-without-progress, not time-without-signal.

Obstacle ROI geometry and depth-fill audit (July 10). A field report of missed obstacles prompted a geometric audit of the frontal depth window. The original ROI spanned 40.5–59.5 % of the image width ($\pm 8^\circ$, i.e. a corridor of ± 6 cm at the 0.45 m stopping distance), while the rover’s half-width is ≈ 25 cm: an obstacle aligned with a wheel rather than the camera axis was simply outside the watched zone. The ROI now spans 20–80 % of the width ($\pm 29^\circ \approx \pm 26$ cm at 0.45 m, matching the swept corridor), with the vertical extent capped so that ground pixels only enter the band beyond ~ 0.73 m, past the trigger threshold, hence no floor false-positives. Because a wide window dilutes thin obstacles (a chair leg) in a global percentile, the corridor is evaluated as three independent thirds, each compared to the threshold via its own 5th percentile. A complementary measurement settled the projector trade-off: the depth valid-pixel ratio in the corridor is *unchanged* between laser power 60 and 150 (56 % both, scene-limited by dark and oblique surfaces), so the power reduction that protects AprilTag decoding costs the obstacle channel nothing. Finally, the cockpit detection sliders were discovered to be dead controls (parameters were snapshotted once at vision-process start), and are now forwarded live with each frame; the brightness threshold slider drives the actual detector (the hue pair is a vestige of the retired colour pipeline, inert on infrared

imagery).

Four-encoder wheel feed (July 10). The same session answered an instrumentation question: MINS’s differential model (Wheel2DAng) consumes two angular velocities, and the remap node fed it only the *front* wheel pair: two of the rover’s four encoders were unused. On a skid-steer platform every turn slips by design, and the front and rear wheels of a side grip differently, so a single wheel per side makes the odometry constraint hostage to whichever wheel slips more. The remap now publishes the per-side *mean* of both encoders ($\bar{\omega}_L = (\omega_{FL} + \omega_{RL})/2$, idem right), a better estimator of the side’s effective tread speed that also halves encoder noise; it degrades gracefully to the front encoder when a rear joint is absent. The residual precision limit remained the wheel *intrinsic*s: the frozen values ($r = 62.5$ mm, $b = 358$ mm) described the original wheels, not the rigid replacements (online re-estimation stays disabled after the July 6 divergence). This motivated the automated campaign of the next subsection.

12.5.2 Precision campaign (July 10): automated calibration, slip characterisation, SLAM anchoring

Automated wheel-radius calibration. A self-driving calibration tool (`tools/calib_auto2.py`) executes the measurement manoeuvres itself through the cockpit command channel (inheriting the backend’s polarity handling, speed clamps and dead-man), guarded by the live depth corridor: a straight leg is attempted only with ≥ 3 m of free space and aborts below 0.8 m. Over a 2.2 m guarded straight, 20 half-second windows matching the straightness gates ($|\omega_{gyro}| < 0.06$ rad/s) yield

$$r_{\text{eff}} = \frac{\|\Delta p_{\text{MINS}}\|}{\bar{\omega}_{\text{wheel}} \Delta t} = 62.2 \text{ mm} \quad (\text{IQR} \pm 0.5 \text{ mm}),$$

–0.5 % from the stock value: the rigid wheels kept the tread radius, closing the radius half of open item #1 (the track width still awaits a physical measurement).

The decisive discovery: the “rigid wheels” are mecanum wheels. The base-width leg failed in a deeply instructive way. Commanding gentle arcs produced a sustained wheel-speed differential of $\Delta\omega \approx 3.8$ rad/s (which the differential model maps to $r\Delta\omega/b \approx 0.66$ rad/s of yaw), while two independent witnesses agreed the chassis did *not* rotate: the gyroscope stayed at its

bias (0.039 rad/s, identical to the static value) and the MINS visual heading moved $+0.3^\circ$ in 6 s (0.001 rad/s). The wheels were subsequently identified as the FictionLab *Mecanum wheel set: Leo Addon* ($\varnothing 128.8$ mm, 45° polyurethane rollers), which recasts the observation from friction to *kinematics*: the free rollers absorb a left/right speed differential instead of converting it into a yaw moment: they are doing exactly what mecanum rollers are designed to do. Under proper mecanum forward kinematics a tank-style differential still yields yaw, but scaled by $2(l_x + l_y)$ (roughly the track *plus* the wheelbase, nearly twice the differential-model base), and only insofar as the rollers grip; on this floor the residual authority is negligible. Three consequences: (i) the firmware’s differential-drive abstraction is structurally mismatched to the wheel hardware, so wheel-implied yaw is not merely noisy but fabricated; (ii) no “effective base” can repair it (the authority is regime- and surface-dependent); (iii) the trust re-assignment is the correct estimator-side response: `noise_w` $0.2 \rightarrow 0.8$ (wheel *angular* information strongly discounted) while `noise_v` is unchanged: the *longitudinal* rolling constraint remains excellent, as the radius calibration proves, with $r_{\text{eff}} = 62.2$ mm against the 64.4 mm geometric radius (3.4 % roller-contact compression, a mecanum-typical figure). An earlier attempt with in-place rotation at 0.5 rad/s had pushed the then-configuration into outright filter divergence (position exploding to 10^5 m): sustained fabricated wheel yaw is precisely the worst input for a tightly-coupled update.

SLAM anchoring. The estimator ran as a pure MSCKF (`max_slam:` 0): every feature was marginalised at window exit, leaving no persistent scene memory: the weakest possible configuration for in-place rotation, where triangulation baselines vanish. Twenty-five SLAM landmarks are now maintained in the state (`max_slam:` 25, confirmed live: SLAM:25 in the updater trace), anchoring the pose against revisited structure at a bounded state-size cost.

Verification. Post-restart, with r calibrated, wheel-yaw discounted and SLAM active: wheel updates are accepted at **100 %** χ^2 over 408 measurements; a 60 s static T1.1-mini gives $\sigma_x = 7.4$ mm, $\sigma_y = 1.0$ mm (z : 29 mm), with fitted slopes of -0.15 , 0.03 and 0.44 mm/s, all under the 1 mm/s noise verdict of Protocol T1.1; and the exact stress regime that had destroyed the filter (10 s of sustained slipping differential) now produces a maximum position wander of **9 mm** with a sane filter, against a 1.7×10^5 m divergence before. Operationally, the LED brightness threshold was raised

to 240 through the now-live cockpit channel after four ceiling-light “LED resets” were found in the mission log. Two hardware-level options followed from the mecanum identification: either restore the stock wheels when the mission profile depends on authoritative in-place pivots (the FSM’s obstacle bounces and post-beacon turns), or keep the mecanum set and expose per-wheel control below the differential-drive firmware abstraction: unlocking omnidirectional motion (lateral strafing) that the current command chain cannot express, at the cost of a firmware-level rework and a mecanum-aware odometry model. The operator’s decision was to *keep the mecanum wheels*, which turns the roller behaviour from an anomaly into an operating constraint the autonomy layer must respect.

July 13: the ramp campaign, two misdiagnoses, and the ground truth. With the battery finally charged (11.95 V after three days on the charger), a ramp characterisation tool (`tools/ramp_yaw.py`) measured actual yaw rate against commanded pivots from 0.10 to 0.40 rad/s, and dismantled the July 10 “roller break-away” theory: at healthy voltage, *every* command from 0.15 to 0.40 rad/s executes at 92–99 % authority. The zero-rotation observations of July 10 were **motor torque sag at 10.3 V**, not roller physics; only a small command deadband ($\lesssim 0.12$ rad/s) is intrinsic. Rotation speeds were restored accordingly (`TURN_SPEED/UTURN_SPEED` 0.35, 90° in ≈ 4.6 s, anti-stuck windows rescaled to match).

The ramp, however, first reported those rotations with an *inverted sign*, which led to a wrong angular-polarity patch and a wrong left/right wheel-remap swap before a ground-truth experiment (one commanded rotation witnessed simultaneously by the *raw gyroscope* and the MINS heading) exposed the real culprit: the estimator, not the actuation. With 25 SLAM landmarks active, an in-place pivot renders landmark depth unobservable (zero baseline); the stale anchors dragged the MINS heading to $+0.58$ rad/s *while the gyroscope measured -0.25 rad/s*, and the corrupted state made the filter reject *correct* wheel measurements (χ^2 acceptance 6 %). Both patches were reverted: kinematically, a 180° drive remount inverts the linear sense (one inversion) but *not* the yaw sense (side swap $\times$ rolling-sense flip cancel), so `DRIVE_POLARITY` alone was always the complete fix, and `max_slam` returned to 0 pending an excitation-gated re-enable. Final validation, all witnesses agreeing: command $+0.30 \rightarrow$ gyro $+0.249 \rightarrow$ MINS $+0.249$, wheel χ^2 acceptance **100 %** during rotation. Two lessons join the registry: an estimated heading is *not* a truth reference for diagnosing

actuation sign: only a raw inertial channel is; and SLAM anchoring must be excitation-aware on a platform whose mission profile is rich in pure rotation.

First full-stack AUTO run and its three regressions (July 13). The first field run with the repaired chain surfaced three defects, each diagnosed from the mission log. (a) *The one-minute failsafe*. The robot dropped to MANUAL at second :03 of every minute: the cockpit heartbeat is a 200 ms `setInterval`, and browsers throttle timers in *hidden* tabs down to once per minute; watching the Trajectory page while the Operations tab (the heartbeat sender) sat hidden starved the 1.5 s dead-man like clockwork. Two-sided fix: the front-end now also emits a heartbeat on every telemetry *reception* (WebSocket delivery events are exempt from tab throttling), and the backend timeout was raised to 12 s, still a real dead-man, but tolerant of refreshes and throttling bursts. (b) *Blind LOCK approaches*. Obstacle handling was gated to the PATROL branch with target-visibility as an explicit exemption (“what is ahead is the target, not an obstacle”), so a *false* lock drove at furniture with the depth channel watching helplessly. The approach now aborts on the obstruction contradiction (a target believed farther than 0.6 m cannot coexist with a <0.55 m depth return ahead), maps the obstacle, and enters the bounce with the artefact cool-down armed. (c) *Ceiling lights as beacons, structurally*. On *infrared* imagery the brightness-based LED detector favours exactly the wrong objects: incandescent ceiling fixtures are IR-bright while the beacon’s visible-light LEDs may be IR-dim. All false detections of the session sat in the top third of the image ($y = 38\text{--}160$ px of 480). A *floor-beacon plausibility gate* now rejects any blob above 35 % of the image height, the persistent threshold moved to 240, the post-stall re-lock cool-down doubled to 45 s, and every lock entry logs its source and image height for one-line diagnosis. Whether the real beacon’s LEDs clear the 240 threshold on IR at all remains a field question: the AprilTag and checkerboard channels carry the acquisition in the meantime, and a low-rate colour stream (hue discrimination, 424×240) is the designed escalation if they do not.

All-in-one beacon ritual and a fluid trajectory display (July 13, operator revision). Two operator requests closed the session. First, the beacon chain was unified: in AUTO the entire find-approach-reset sequence now runs off a single measured chain: acquisition (AprilTag at range, vision as bonus), approach to 0.6 m, *immediate* local reset, marker, 5 s hold, 90° turn, with one

hardening change: the reset now *requires a measured distance* (tag or checkerboard) at or below the stop threshold. The previous rule reset instantly when the distance was *unknown* (meant for a full-field checkerboard, but equally satisfied by any centred artefact); an unknown distance now simply holds the centring until the stall guard rules. The beacon’s world-frame marker no longer depends on the checkerboard either: the lock distance places it along the centred heading. Second, the Trajectory page was made fluid and immortal: the *reload-on-disconnect* handler (a hidden `location.reload()` four seconds after any WebSocket close, the operator’s “the site refreshes by itself”, and it destroyed the trail each time) was replaced by in-place reconnection with exponential backoff, preserving all client-side state; the displayed pose and the auto-scaled viewport are eased toward their targets every animation frame instead of jumping at the 8–10 Hz message rate; and the trail rendering was batched from one stroke per segment (3 000 canvas paths per frame) to age-sliced batches (≤ 12 paths), removing the render jank the operator reported.

Divergence guard (July 13, evening). The now-truthful display promptly reported a real fault: MINS had silently diverged hours earlier (position hundreds of kilometres out, 1 123 m/s of “velocity” on a stationary robot, the signature of runaway inertial double-integration, plausibly seeded by collision shocks during the earlier field run). A fresh initialisation restored millimetre-level statics within seconds, but the structural lesson is that a diverged filter *stays silent*: nothing in the stack complained while the cockpit displayed nonsense. The cron watchdog now probes `/mins/imu/odom` each minute and triggers an automatic clean stack restart whenever the position magnitude exceeds 100 m (impossible indoors, unambiguous at divergence scale), bounding any future episode to about a minute. A chronic instrument anomaly was also quantified in passing and joins the open-items registry: the LeoCore accelerometer reports $\|a\| = 8.76 \text{ m/s}^2$ at rest against a local gravity of 9.790: a -10.5% deficit the estimator absorbs as an initial bias, acceptable in operation but worth a firmware-side investigation.

12.5.3 Systems audit: measure first, then touch (July 13)

A full-stack performance and reliability audit closed the day, conducted on the *live* system under the rule that no change ships without a measured justification. The profile identified the actual constraint: the ground-station CPU was operating at its thermal ceiling (81°C against an 82°C

throttle point, load average 4.1), the root cause of the day’s sluggish XML-RPC tooling, with the budget dominated by MINS (64 %, legitimate), but also by two *unjustified* consumers: the standby OpenVINS estimator (13–15 %, never initialised, never selected, yet running at the same elevated nice -10 priority as the critical path) and the AprilTag detector (16 %, full 15 Hz on full resolution feeding a backend freshness window of 0.7 s). Three interventions, each benchmarked: OpenVINS was demoted to normal priority (it now yields under contention, costing nothing when headroom exists); the tag detector’s image feed is throttled to 8 Hz ($\approx$ halving its cost with no mission effect); and the depth callback (the obstacle-safety hot loop at 15 Hz) was profiled at 4.04 ms/frame and rewritten to operate on a $2\times$ -decimated grid (1.68 ms, -58%), with median deviations under 0.5 % and worst-case 5th-percentile deviation of 7 % on synthetic uncorrelated data (real depth is spatially correlated and deviates less; the 600 mm threshold retains margin). Reliability hardening followed the same triage discipline: of the 46 except Exception handlers in the backend, the one whose silent failure would disarm the obstacle chain (the depth callback) now validates its input (dimensions and buffer length before reshape) and reports errors at a rate-limited once-per-10 s, while the remaining handlers (display paths, isolated vision subprocess) were deliberately left untouched: blanket rewriting of working error isolation in a fielded system is regression risk masquerading as rigour. For the same reason the audit *declines* cosmetic renaming of node/topic interfaces. Post-change: package temperature 60°C (22 degrees of reclaimed thermal margin, part of it restart-transient), tag detections at 6–8 Hz, depth chain error-free. The architectural review found the concurrency design already sound: the vision pipeline runs in a separate *process* (GIL-immune by construction), numpy releases the interpreter lock in its C kernels, and cross-thread state hand-off relies on atomic reference swaps.

12.5.4 Field validation run (July 13, 15:49–15:54)

A 5 min 25 autonomous run in a heavily cluttered room, monitored live from the mission log, validated the entire coverage stack. *Passed*: obstacle handling (operator verdict: “super”): more than a dozen clean bounces with correct freer-side selection, three *aimed* bounces steering into detected open corridors, and three escape sequences whose rescaled windows correctly avoided the safe-stop latch; permanent open-path attraction (corridors 2.5–7.1 m); mecanum-regime pivots at their predicted rate (a 39° bounce completed in 2.3 s); *zero* ceiling-light false locks over the

full run (the floor-plausibility gate and 240 threshold held); *zero* heartbeat losses despite operator tab switches (the reception-driven heartbeat held); and an estimator that stayed sane throughout under the new divergence guard. *Failed*: the beacon was never acquired. The AprilTag channel decoded nothing in five minutes, consistent with its marginal pixel budget (2.2 px per module at 3 m on 640×480; an A3-format tag of ~35 cm side would carry acquisition to 5–8 m and is the cheapest possible fix). Two vision locks on a floor-level target ($y = 74$ and 82% of image height, the plausible-beacon band) failed to converge within the 12 s stall guard: either a persistent floor reflection (guard correct) or the real beacon with intermittent LED detection (guard too strict); the discriminating pixel trace was lost when the rover dropped off the network at the end of the run. The next session opened with exactly that experiment, and closed the case.

12.5.5 Seeing through the rover’s eyes: the beacon case closed (July 13, evening)

With the rover parked facing the beacon, the annotated camera view was captured directly (the MJPEG server was first moved under launch supervision with automatic respawn, verified by killing it, after three competing launchers had left it silently dead). The images settled every open question. *First*, the infrared projector was lacing the checkerboard, the LEDs and the floor with bright dots, and measurement showed it was degrading the depth channel too: corridor fill improved from 56 % to **81 %** with the emitter *off*, passive stereo on the textured carpet outperforming structured light. The projector is now permanently off (watchdog updated), simultaneously cleaning every visual pattern and improving obstacle sensing. *Second*, replaying the exact LED detector offline on the captured frame quantified the acquisition failure: at the 240 brightness threshold (raised earlier to reject ceiling lights) **zero** of the four LEDs pass; at 220, three; at 200, all four: a perfect cross at (357,290), circularities 0.83–0.94. The non-converging locks of the field run had been the *real beacon*, intermittently detected at a threshold that starved it. The threshold returned to 200: ceiling-light rejection belongs to the floor-plausibility gate, not to brightness. Live confirmation followed immediately: repeated 4/4-LED confirmations facing the beacon. *Third*, the front face carries no AprilTag (checkerboard only): the large-format tag remains the recommended range extension. Finally, the all-in-one ritual’s last gap was closed: with no measured range (no

tag on this face, the checkerboard useless beyond 1.5 m), the approach is now guided by the *depth corridor* itself (the beacon is a physical object the depth channel sees), with arrival declared below 0.7 m, deliberately above the 0.55 m obstacle threshold so arrival always precedes any conflict. The methodological lesson of the day is worth stating plainly: three sessions of hypothesis-driven tuning were resolved in twenty minutes by *looking at the actual image* and replaying the actual detector on it. Direct observation beats inference.

Idempotent web healing. One last recurring operator pain (the cockpit banner “WS DISCONNECTED” returning every minute) traced to a healing anti-pattern: the web-stack start script rebuilt *everything* from scratch on every invocation, and the watchdog invokes it whenever *any* single port is missing. A TLS rosbridge failing to start (transient “no route to host” while the rover’s WiFi flapped) therefore caused the *healthy* LAN rosbridge (and the operator’s live session with it) to be killed and relaunched every sixty seconds. The script is now idempotent: each component is checked and only (re)started if actually dead, and a component whose process exists but whose port is not yet bound is recognised as *starting* and left alone (rosbridge can take over a minute to bind under load). The rule joins the supervisor registry: a healer must never touch what is already healthy, and “still starting” is a state, not a failure.

Plausibility-gated pose serving (July 13, night). The evening’s last structural fix addressed the operator’s report that the MINS trace “draws nonsense” whenever the camera stream drops. The mechanism is now fully mapped: during an image gap the estimator dead-reckons on the IMU alone, the residual accelerometer bias double-integrates into metres of excursion within seconds, and (the cruel twist) the χ^2 gate then *rejects* the wheel measurements ($v = 0$) precisely when they could anchor the filter, because the diverging state makes their residuals huge. Three complementary defences now stand. The wheel velocity trust was tightened (`noise_v` 0.5 $\rightarrow$ 0.2) so the zero-velocity anchor acts *before* runaway. A *plausibility gate* in the pose selector freezes the served pose whenever the active source implies a speed beyond 1.5 m/s (the rover’s command ceiling is 0.4), and re-anchors through the existing SE(3) correction once the source calms: excursions can no longer reach the trail, the world map or Return-to-Base, whatever their cause. And the watchdog gained an *IMU starvation probe*: the roserial bridge on the Pi was found degrading after the

day's power events (producing at 90 Hz, delivering 3–8 Hz to the PC while camera topics flowed normally, the comparison that acquitted the radio); a supervised node cycle restores the nominal rate (measured $3 \rightarrow 79$ Hz) and is now automatic. The day's root-cause epilogue deserves its line: a loose WiFi antenna connector on the rover explained the entire day of network pathology: 2–422 ms round-trip jitter that healed to a steady link once reseated. The most expensive failures are often a hand's turn of hardware.

12.5.6 Mission loop closed: the beacon sequence, end to end

The day ended with the run the whole campaign had been building toward. At 18:55:48 the rover, in AUTO, locked the beacon on its vision channel (four LEDs at the 200 threshold, target at 58 % image height); it centred, approached under *depth guidance* (no checkerboard, no tag, the corridor percentile serving as rangefinder), and at 18:55:59 declared “*Balise atteinte (depth 0.67 m) — reset local — WAIT 5 s*”: eleven seconds from detection to a world-preserving local reset. The full mission loop (patrol, open-path attraction, obstacle bounces, beacon acquisition, approach, reset ritual) is field-validated. Three incidents were resolved en route and are recorded for operations: a post-brown-out *motor-driver lockout* (commands received, zero motor current, battery steady under load, software reset ineffective; only a full power cycle clears the latched fault; the discriminating measurement is battery voltage *during* a command pulse); a false escape at AUTO entry (the anti-stuck displacement history survived from the parked MANUAL period and is now purged in `_start_patrol`); and a depth-rate collapse to 1–3 Hz traced to the Pi's PNG compressor running at factory level 9: level 1 restored 8.9 Hz at a still-negligible bandwidth cost, and both this and the projector-off setting are reapplied by the watchdog after every camera recovery (their boot-side pinning in `robot.launch` remains an open item, together with the large-format AprilTag).

12.5.7 Long-range beacon acquisition by AprilTag

The checkerboard detector is physically range-limited: its inner corners become sub-pixel below ~ 1.5 m at 640×480 . The physical beacon carries a 36h11 AprilTag (ID 285), whose measured geometry (1-inch modules, 8'' black square) gives a side of $s = 0.2032$ m, decodable to 3–4 m. The mission backend now consumes `/tag_detections`: the nearest declared tag is projected to pixel

coordinates through the calibrated pinhole model,

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y, \quad (12.22)$$

and feeds the same targeting path as the vision pipeline, so PATROL acquires the beacon at several metres and the checkerboard+LED chain confirms at close range for the reset ritual. The range asymmetry is quantifiable: a 36h11 tag is decodable down to roughly 2 px per module, so with $f_x = 336$ px and 25.4 mm modules the theoretical ceiling is $Z_{\max} \approx f_x s_{\text{module}} / 2 \text{ px} \approx 4.3$ m, versus sub-1.5 m for the checkerboard whose corner detector needs an order of magnitude more pixels per square. Two supporting changes: the IR projector power was reduced (360→60) because its dot pattern (rigidly attached to the camera frame and therefore *stationary in the image* regardless of scene motion) overlays both patterns with high-contrast clutter whose apparent size at range matches the tag modules, destroying decoding precisely where range matters (near-field depth for the 0.45 m obstacle threshold survives the power reduction); and scan speed was halved (0.4→0.2 rad/s) because at angular rate ω and exposure T_e the rotational blur is $f_x \omega T_e$ pixels: at 0.4 rad/s and a typical indoor IR exposure of 7–10 ms this exceeded a full pixel, marginal for corner detection; halving the rate restores sub-pixel sharpness while a 360° scan still completes in 31 s.

12.6 Operator platform and public API

12.6.1 Site structure and design system

The web cockpit grew from a single operations page to a five-page platform (index, ops, logbook, navigation_modes, trajectory), unified by a token-based design system (mission.css: deep-space palette #0B0E14, data-cyan/NASA-orange utility accents mapped one-to-one to the VINS/MINS semantics, monospace tabular numerals, pulsing liveness indicators, severity-coded flight-log rendering of the console). The navigation_modes page now embeds a bilingual (FR/EN) deep-dive (MSCKF foundations, both estimators, the measured T1.1 tables) and a live SVG data-flow diagram mirroring Figure 12.1. The trajectory page is a Mission-Control style Canvas 2D module: mode-coloured fading trail (each segment frozen with the colour of the source active at that instant), AprilTag node markers, velocity/cadence/latency HUDs, and VINS/MINS switching buttons calling the selector service directly.

12.6.2 API surface

All remote access flows through a Cloudflare tunnel to the ground station; the ingress map *is* the public API:

Endpoint	Backend	Function
<code>wss://.../rosbridge</code>	rosbridge TLS :9443	full ROS API in the browser (roslibjs): topics, services
<code>https://.../stream</code>	web_video_server :8080	MJPEG annotated video
<code>https://.../*</code>	http.server :8000	static pages
<code>ws://<pc>:9090</code>	rosbridge LAN	same API without the tunnel

Key ROS-level API used by the UI:

<code>/mission/command</code>	std_msgs/String	JSON operator contract (set_mode, manual, stop, reset, goto_beacon, set_pose_source, heartbeat 5 Hz)
<code>/mission/telemetry</code>	std_msgs/String	~10 Hz full-state JSON (pose local+world, FSM, sensors, markers)
<code>/pose_selector/set_source</code>	SetBool service	VINS/MINS switch (Eq. 12.9)
<code>/robot_pose_fused</code>	nav_msgs/Odometry	fused output consumed by all pages

Table 12.1: Public API surface of the platform.

Anatomy of a switch. The complete VINS→MINS sequence illustrates the layering: the operator’s click publishes `{"action": "set_pose_source", "source": "MINS"}` on `/mission/command` through the tunnel WebSocket; the mission backend validates state and calls `/pose_selector/set_source(true)`; the selector verifies the target source has published recently (refusing otherwise with an explicit message, the guard that surfaces an uninitialised OpenVINS), computes $\mathbf{T}_{\text{corr}}$ of

Eq. (12.9) from the last published pose and the new source’s current pose, swaps sources, and republishes; the UI badge updates from the `/leo_navigation/pose_source` topic (server truth, no optimistic state), and the Trajectory page’s trail changes colour at the exact segment where the switch occurred: a live visualisation of Eq. (12.9)’s continuity.

Two protocol-level defects were diagnosed and fixed during the campaign: the WebSocket Host-header port check (autobahn) requires `websocket_external_port` to match the *public* port (443 behind the tunnel, 9090 on LAN, a stale ROS parameter had silently broken LAN mode for two days); and an FSM state unknown to a cached front-end must render as its raw name, never fall back to a misleading default.

12.6.3 Operations infrastructure

Availability is enforced by three layers: per-node *respawn* supervision (roslaunch); a cron watchdog (1-minute period) checking web ports, tunnel, navigation stack and camera (including a full rosmon STOP/START camera cycle, republish-zombie purge, and reapplication of the projector power), gated by a maintenance flag for deliberate interventions; and automatic infinite WiFi reconnection. The camera’s minimal profile is written into the robot’s boot launch (with the hardware constraint documented in-file: depth and IR share the stereo module and must run identical resolution/rate). Battery under ~ 10.5 V was identified as a hard limit: the Pi’s USB rail sags and the D455 cannot hold its streams: no software remedy exists, and the report flags it as an operational precondition.

Two watchdog defects surfaced during a two-day robot-off period (July 8–10) and were fixed. First, `timeout N rostopic echo` is *not* a bounded probe when the ROS master is unreachable: `timeout` sends a single `SIGTERM` which `rostopic`, blocked inside its connection attempt, ignores, and `timeout` then waits forever. Every cron tick that reached a probe leaked one hung process (fourteen accumulated in two days); the fix is `timeout -k 5 N`, which escalates to `SIGKILL`. Second, an ordering fault: the watchdog exited at its “robot unreachable” gate *before* checking the web stack, so with the rover powered off the public site (cockpit, report) stayed down for the whole period; yet it depends only on the PC. The web-healing block now runs *before* the reachability gate; only the robot-dependent actions (navigation stack, camera ladder) remain behind it. The general lesson generalises the earlier cron findings: a supervisor must be audited under the *failure* of each of its own probes, not only under the failures it is designed to heal.

The same restart exposed a related gap: the AprilTag detector (launched manually during the July 8 field session) did not belong to any supervised launch file, so every stack restart silently removed the long-range beacon acquisition chain (`/tag_detections` advertised, zero publishers). The node is now included in the supervision launch and respawns with the rest of the stack; the rule it confirms is that *anything started by hand during a field session is already lost*: a capability only exists once its launch file is owned by a supervisor.

The same principle applies to run-time configuration, not only to nodes: `laser_power` (stereo module, IR pattern projector) and `png_level` (the depth stream's `compressedDepth` transport, §12.2.2) had both been tuned via live `dynamic_reconfigure` calls, which do not persist across a `leo.service` restart: every restart silently reverted both to their driver defaults. The fix follows the same pattern used throughout this campaign for the 10 Hz camera-profile incident (Table 12.12, item 4): configuration changes are validated live first, then only made structural once proven safe, and structural means owned by the boot launch file rather than a one-off terminal command. Two one-shot `dynamic_reconfigure` client nodes were added to `robot.launch`, delayed eight seconds (`launch-prefix="bash -c 'sleep 8; $0 $@"`) to let the `realsense` driver's reconfigure servers come up first, marked `non-required` and without `respawn` so an insufficient delay on a given boot degrades only that one setting rather than the stack (the same failure-containment principle as the MINS respawn policy of §12.2.4). Applied and confirmed via the two nodes' process exit status in the `rosmmon` log (`exited with status 0` for both, the same signal a failed service call or an unresolved reconfigure-server name would have reported non-zero), followed by a full navigation-stack restart on the ground station: the same topology-transition precaution of §12.6.4 applies equally to a robot-side service restart.

12.6.4 Campus-scale connectivity: from an isolated hotspot to a WireGuard-tunnelled network

Through July, all field operation had been confined to rooms within direct range of the rover's own access point (`wlan_ext`, *LeoRover-e138*): a single indoor 5 GHz AP whose usable range is on the order of 10–15 m through drywall, sufficient for the lab but not for a building-wide demonstration. On 2026-07-20, ahead of an external (NASA-context) validation requiring the rover to be exer-

cised anywhere in the building without a network-driven interruption, the operating topology was migrated onto the campus network FLTech-Guest.

Retained dual-radio architecture. The rover’s onboard Wi-Fi adapter supports two simultaneous logical interfaces: `wlan_ext` continues to host the local hotspot (kept live as a proximity fallback, disabling it would have removed the only recovery path reachable without campus infrastructure), while `wlan_int` joins FLTech-Guest in client mode on 5 GHz, giving the robot a routable, building-wide address (10.154.27.235/19) in addition to its local one. The ground-station PC was *not* moved onto FLTech-Guest via its own Wi-Fi: a live `dmesg` capture during the first connectivity tests showed a reproducible driver fault in the PC’s USB adapter (rtl88x2bu chipset: UBSAN: invalid-load inside rtl8822b_set_channel_bw, phydm_ccx.c:696), firing on every Wi-Fi channel switch and producing on the order of six spontaneous disconnects per hour. Rather than debug a closed-source vendor driver on the critical path of a validation demo, the PC was kept on its wired Ethernet uplink (structurally immune to the fault) and made to reach the robot over a point-to-point tunnel instead. Figure 12.4 contrasts the two topologies in full.

Why a tunnel, not a plain route. A campus network partitions the PC’s wired subnet and the robot’s guest-Wi-Fi subnet behind a stateful firewall: ICMP and a short list of registered TCP ports (22, 11311) were observed to traverse cleanly, but ROS’s transport layer does not confine itself to a fixed port. TCPROS negotiates one ephemeral TCP port *per topic connection* through the XML-RPC handshake between publisher and subscriber: exactly the traffic pattern a campus firewall is configured to block between unrelated VLANs. Routing directly to the rover’s campus address therefore worked for master-registry queries (`roscat list`, service calls) while silently starving every actual topic subscription; the fix had to present the firewall with a *single, fixed, pre-authorised* port and multiplex all ROS traffic inside it, which is precisely what a VPN tunnel does and a raw route *across subnets* cannot. The italics matter: §12.8 revisits this constraint a day later and shows it is specifically a cross-subnet limitation, not a blanket property of raw routing.

First attempt: sshuttle, and why it was abandoned. `sshuttle` (a userspace transparent proxy tunnelling arbitrary TCP over an SSH control connection) was tried first for its zero robot-side privilege footprint: an authorized public key was sufficient. One deployment bug surfaced

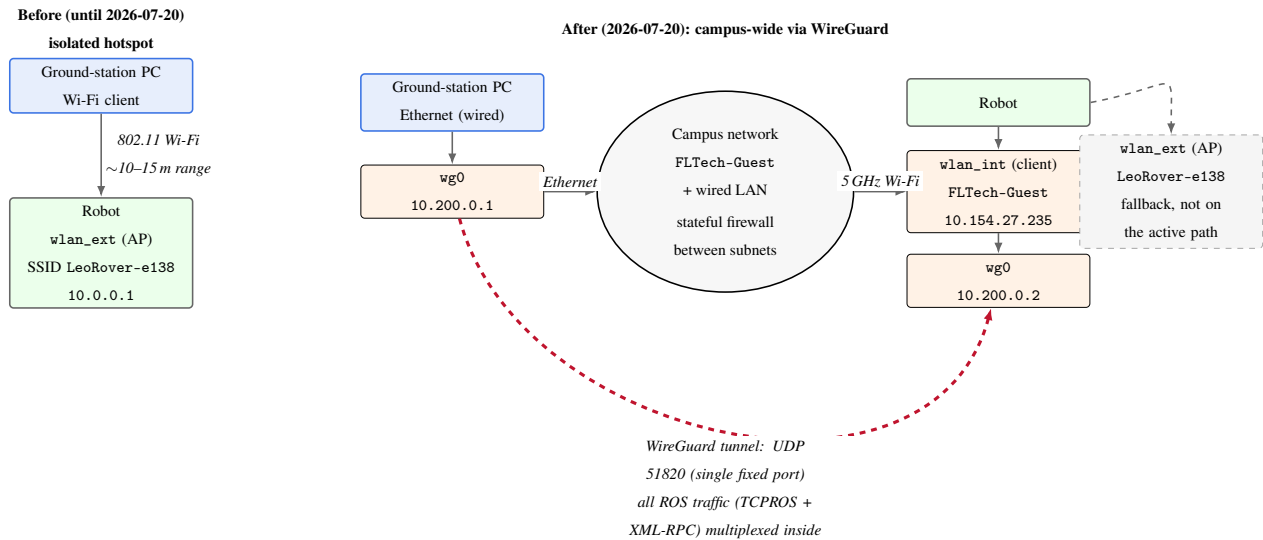


Figure 12.4: Network topology before and after the 2026-07-20 migration. Left: the original architecture, where the PC joined the robot’s own access point directly: functional only within Wi-Fi range of the robot itself. Right: the PC stays on wired Ethernet and reaches the robot’s campus-Wi-Fi address through a WireGuard tunnel (red dashed arc); the robot’s own hotspot (wlan_ext) is kept live as a fallback but is not part of the active data path. ICMP and TCP 22/11311 cross the campus firewall directly, but TCPROS’s per-topic ephemeral ports do not: only the WireGuard UDP port is pre-authorised, so every ROS connection is multiplexed inside that single tunnel (§12.6.4 below expands on why a plain route cannot substitute for this).

immediately: forwarding the robot’s full subnet caused `sshuttle`’s own SSH control channel to be captured by its own relay, killing itself (Broken pipe) on every launch; excluding only the control port explicitly (`-x <robot_ip>/32:22`, rather than the whole host) resolved it. With that fix, data was observed flowing correctly on two independent occasions. Under the sustained, multi-connection load of the full navigation stack (MINS, the mission backend, and the cockpit each holding independent long-lived TCPROS sockets simultaneously), however, robot-originated topics failed systematically: reproducible from both the operator’s own terminal and independently, not a transient fluke. Three hypotheses were eliminated in turn: file-descriptor exhaustion (`LimitNOFILE=65536` added, no change), accumulated robot-side session state (full reboot of the robot, no change; MINS cold-init then took 196 s), and PC-side process staleness (clean service restart, no change). With load-independent, reboot-independent, and restart-independent failure ruled out as anything but the transport itself, the conclusion drawn was architectural: a single userspace Python process relaying TCP by repeatedly copying bytes between sockets does not scale to the connection churn of a full ROS stack the way kernel-level forwarding does, and no further time was spent tuning it.

Final transport: WireGuard. WireGuard replaces `sshuttle` outright: a kernel module terminating one static UDP port (51820) per host, presenting the firewall with a single opaque, pre-authorised flow regardless of how many ROS connections it carries inside. The tunnel is point-to-point (10.200.0.1 PC $\leftrightarrow$ 10.200.0.2 robot, /24), authenticated by a Curve25519 keypair generated on each host, with `PersistentKeepalive=25s` to hold the mapping open against the campus NAT’s idle timeout. This value is not arbitrary: RFC 4787 (REQ-5) directs NAT devices to hold a UDP mapping open for at least two minutes of inactivity, but numerous field surveys of consumer- and enterprise-grade NAT/firewall implementations report equipment that reclaims idle UDP state considerably sooner than the RFC’s recommendation: commonly in the tens of seconds on the most aggressive devices. A keepalive interval τ is safe only if it is shorter than the shortest timeout $T_{\min}$ the path can plausibly impose, with margin for jitter in when the keepalive packet itself is actually scheduled and delivered: $\tau \ll T_{\min}$. Twenty-five seconds sits comfortably under even the most aggressive commonly reported timeouts while remaining far above the per-packet overhead floor (a keepalive is a single encrypted UDP datagram; at 25 s intervals this is a negligible

~ 0.04 pkt/s of control traffic, invisible against the ~ 2 MB/s data budget of §12.2.2): the standard rationale behind WireGuard’s own documented default for exactly this value, adopted unchanged here rather than tuned against this specific campus firewall’s (unmeasured) timeout.

The encryption itself is the other quantity worth bounding rather than assuming negligible. WireGuard encapsulates each packet with a 4-byte type field, a 4-byte receiver index and an 8-byte counter (16 B header), plus a 16-byte Poly1305 authentication tag, for a fixed 32-byte overhead per datagram independent of payload size, on top of the underlying UDP/IP headers already paid by any transport. At the compressed image cadence of §12.2.2 (two IR streams at 15 Hz, ≈ 30 – 60 kB per JPEG frame), each frame is fragmented into $k \approx \lceil 40000 \text{ B} / 1400 \text{ B} \rceil \approx 29$ UDP datagrams at a conservative 1400-byte MTU-safe payload size, so the tunnel’s fixed per-packet cost adds

$$\Delta_{\text{wg}} = k \times 32 \text{ B} \times 15 \text{ Hz} \approx 29 \times 32 \times 15 \text{ B/s} \approx 13.9 \text{ kB/s} \quad (12.23)$$

per stream, against a payload rate of $40000 \text{ B} \times 15 \text{ Hz} = 600 \text{ kB/s}$ for that same representative frame size: a $\Delta_{\text{wg}}/\text{payload} \approx 2.3\%$ tax, well inside the margin already measured between the WireGuard row and the local-network baseline in Table 12.2. ChaCha20-Poly1305 encryption and authentication are computed in software on both endpoints, but at these payload sizes the dominant cost remains the fixed per-packet overhead just bounded, not the cipher throughput: modern software ChaCha20 implementations sustain well over 1 GB/s per core, three orders of magnitude above the ~ 2 MB/s aggregate data budget of the whole tunnel. The improvement measured over `sshuttle` is therefore not explained by a cheaper cryptographic primitive (`sshuttle` tunnels over SSH, itself using comparably fast ciphers); it is explained by kernel-level packet forwarding replacing a userspace relay, exactly the architectural argument of the preceding paragraph, now with the encryption cost itself ruled out as a confound. The improvement over `sshuttle` was immediate and measured at every layer of the stack (Table 12.2).

Centralised address configuration. Every script that previously hardcoded the robot’s address (`/bin/leo`, `leo_watchdog.sh`, `restart_stack.sh`, `launch_mins.sh`, `start_web.sh`) now sources a single file, `tools/robot_env.sh`, which reads an override (`LEO_ROBOT_HOST`) from `~/.leo_network` and derives `ROS_MASTER_URI` and `ROS_IP` from it: the latter auto-detected via `ip route get $ROBOT_HOST`, so the PC always advertises the address of whichever local interface actually holds the route to the robot, without per-script logic. Migrating network topology is now a

Metric	sshuttle (userspace)	WireGuard (kernel)	Ratio
Camera stream	~1 Hz (starved)	15 Hz (nominal)	15×
MINS cold-init time	up to 196 s	13–17 s	≥ 11×
Robot-originated topic delivery	unreliable under load	consistent	n/a
Firewall exposure	1 TCP ctrl port + relayed flows	1 static UDP port	simpler
Measured link loss (static ping)	n/a	0 %	n/a

Table 12.2: Transport comparison, same rover, same campus network, same navigation stack. WireGuard fully recovers the local-network baseline performance; sshuttle does not, under real multi-connection ROS load.

one-line change in a single file rather than an edit to five independent scripts, a direct consequence of the multiplication of transport paths (local hotspot, campus Wi-Fi, WireGuard) this campaign introduced.

Operational trap: stale connections across a topology transition. On every network-path change observed during the migration, ROS processes already running on the PC (MINS, the pose selector, the mission backend) were found blocked on sockets bound to the *previous* path and did not recover on their own: they require an explicit `restart_stack.sh` *after* the new path has stabilised, never during the transition itself. The failure signature (telemetry silent despite a healthy new link) is easy to misattribute to the transport when the actual cause is stale application-level state; this is the same class of fault as the TF double-authority conflict of §12.2.2: established connections do not self-heal merely because the network beneath them changed.

Single-command recovery. A related gap surfaced from the operator asking, twice, whether `leo restart` alone was sufficient after the migration: it was not; the command restarted the web stack and backend but never the navigation stack, which still required a separate manual `restart_stack.sh` call. A new `[nav]` step was inserted into `/bin/leo` between the backend and web-server stages, invoking `restart_stack.sh` unconditionally (it is itself idempotent, so no guard

is needed at the call site). Verified end-to-end twice, independently, by the author and by the operator: a single `leo restart` now brings up `rosbridge`, the camera, the backend, and MINS with automatic pose-source switch, confirmed each time via live telemetry (`cam_alive=true`, `pose_source=MINS`).

12.6.5 Field validation: building-wide manual traverse (July 20)

The migration's stated goal, exercising the rover anywhere in the building without being limited by the local hotspot's range, was validated in a manual-drive traverse of the full building on 2026-07-20, the first field test conducted entirely outside the lab room since the network migration. Link health was logged throughout at 4 s resolution (ICMP round-trip to the WireGuard peer, cross-checked against `/mission/telemetry`'s `cam_alive` and `pose_source` fields), state-change-triggered to avoid conflating measurement noise with genuine events.

The tunnel held for the entire traverse with a single clean exception: one isolated ping timeout (~ 5 s), self-recovered with no operator intervention and no corresponding telemetry gap on either side of it, consistent with a momentary radio-level fade (a corner, a fire door, a crowded 5 GHz channel) rather than a tunnel or routing fault, since the WireGuard association itself required no renegotiation to resume.

A second, more consequential phenomenon recurred repeatedly during the traverse: `pose_source` flipped from MINS to VINS for ~ 5 – 6 s before returning to MINS unprompted, several times, with `cam_alive` remaining true throughout. Read live during the walk this looked like a brief, self-healing curiosity and was provisionally attributed to transient visual feature starvation; correlating the ground-station watchdog's own log against the observed flip timestamps afterward overturned that reading entirely. `leo_watchdog.sh`'s zombie-detection guard (introduced 2026-07-13) declares the stack dead if three consecutive one-minute cron ticks fail to find `/mins_subscribe` registered with the ROS master via `rostopic list`, and, on that verdict, tears down and relaunches the *entire* navigation stack. `watchdog.log` shows this guard firing at 18:01, 18:04, 18:05 (compounded by one instance of the independent 100 m MINS-divergence guard), 18:07, 18:10 and 18:13 (essentially continuously throughout the traverse), each event followed by the same stack relancée / MINS initialisé après N s / switched to MINS sequence `restart_stack.sh` always performs, which is exactly the transient a fresh `pose_selector` pro-

cess (`default_source="VINS"` in `navigation_supervision.launch`) produces before being switched back. **The corridor traverse succeeded despite the navigation stack being torn down and relaunched every 1–3 minutes for its entire duration, not because MINS ran uninterrupted.**

The root cause is the zombie guard’s probe itself: `roscpp` `list` is an XML-RPC round trip from the ground station to the robot’s master, and its three-tick/10 s-timeout debounce was sized against an earlier isolated false trigger (a single missed tick “under load”), not against the sustained, multi-minute registry unavailability that campus Wi-Fi roaming on `wlan_int` plausibly produces while the rover is physically moving between access points. The mitigation applied is a widened, conservative debounce (three ticks $\rightarrow$ six, 10 s $\rightarrow$ 20 s per attempt) rather than a redesign of the check itself, since the true-positive case it guards against (a genuine robot power-cycle) is unaffected by widening the window: only the false-positive rate during motion should drop. The same audit was extended to every other network-round-trip probe in `leo_watchdog.sh`: the camera-silence escalation’s expensive step (a D455 hardware reset, ≥ 35 s without vision, during which autonomous mode refuses to move) required only two failed 6 s probes and is now four at 12 s, while its cheap first-tier response (a `roscpp` stop/start cycle) is left immediate, since a false trigger there costs seconds rather than tens of seconds; the IMU-starvation guard’s debounce was widened two ticks to four for the same reason. Three probes were left unchanged after the same analysis: the `roscpp` liveness check targets `127.0.0.1`, not the robot, so roaming cannot affect it; the MINS-divergence guard already degrades safely on a network timeout (an empty read is ignored by construction, not treated as evidence of divergence); and the republish-zombie purge requires an *asymmetric* result (one topic silent, a related one not) that uniform network degradation would not itself produce. None of these widened debounces were re-validated in motion within this session and are carried forward as the open item (Table 12.12, item 9): confirm the false-trigger rate under a second in-motion traverse, and, if it persists, replace the network round trip with a check that does not require a live RPC to the robot’s master while it is roaming.

A reliability model for the debounce. The choice of six ticks, rather than an arbitrarily larger number, can be grounded quantitatively instead of by intuition alone. Model each one-minute watchdog tick as an independent Bernoulli trial that fails the `/mins_subscribe` registry check

with probability p (elevated during motion; at baseline effectively zero, this guard never false-triggered in any stationary session). The guard resets its counter to zero on every successful tick and re-arms after every trigger, so it fires the first time n consecutive ticks fail: the classical waiting-time-for-a-run problem. Writing E_k for the expected number of further ticks to a trigger starting from a current run of $k < n$ consecutive fails ($E_n = 0$), the one-step recursion

$$E_k = 1 + pE_{k+1} + (1 - p)E_0 \quad (12.24)$$

solves, for $p < 1$, to a closed form for the expected number of ticks between false triggers starting from a fresh reset:

$$E_0(n, p) = \frac{1 - p^n}{(1 - p)p^n}. \quad (12.25)$$

(Sanity checks: $E_0(1, p) = 1/p$, the ordinary geometric waiting time; for a fair coin, $E_0(2, \frac{1}{2}) = 6$, the textbook expected-flips-to-HH result.) As $p \rightarrow 1$, $E_0(n, p) \rightarrow n$: a guard fires almost exactly every n ticks once probe failure becomes near-certain, which is precisely the signature in `watchdog.log`: zombie-cause triggers recurred at 18:01, 18:04, 18:07, 18:10 and 18:13: four consecutive intervals of *exactly* three minutes under the old $n = 3$ debounce. Equation (12.25) shows this pattern is only produced by p close to unity (Table 12.3): even at $p = 0.90$, $E_0(3, p) \approx 3.72$, visibly above the observed three-minute cadence, and $p = 0.99$ gives $E_0(3, p) \approx 3.06$, matching it almost exactly. The data are therefore most consistent with $p \gtrsim 0.9$ during the traverse: the XML-RPC round trip was failing on *nearly every* tick while the rover moved, not merely at some elevated-but-partial rate.

A likelihood argument for that estimate. The four observed inter-trigger gaps are not just individually close to the $n = 3$ floor; under the model they are a proper i.i.d. sample that supports a maximum-likelihood treatment rather than a by-eye table lookup. Because a debounce absorbs at exactly $T = n$ ticks if and only if the first n post-reset ticks *all* fail (any earlier success would have reset the counter and pushed T higher), the probability of a single observation landing exactly on the floor is $\Pr(T=n \mid p) = p^n$. With four independent gaps all equal to $n = 3$, the joint likelihood as a function of p is

$$\mathcal{L}(p) = \prod_{i=1}^4 \Pr(T_i=3 \mid p) = (p^3)^4 = p^{12}, \quad (12.26)$$

which is strictly increasing on $(0, 1)$: the maximum-likelihood estimate is driven to the boundary $\hat{p} \rightarrow 1$, with no interior optimum. This is itself informative (it says the four-tick sample contains *no* evidence of even one successful probe inside any of the four sustained-failure windows), but a point estimate at the boundary is not practically meaningful (a literal $p = 1$ is not a physical hypothesis, and a sample of four cannot distinguish it from $p = 0.99$ at one-tick resolution). A likelihood-ratio comparison is more honest about what the data actually discriminate:

$$\Lambda(p_1, p_2) = \frac{\mathcal{L}(p_1)}{\mathcal{L}(p_2)} = \left(\frac{p_1}{p_2} \right)^{12}. \quad (12.27)$$

Evaluated at representative points, $\Lambda(0.9, 0.5) \approx 1,157$ (the observed pattern is over a thousand times more probable under sustained 90 % per-tick failure than under a merely elevated 50 %), while $\Lambda(0.99, 0.9) \approx 3.1$ and $\Lambda(0.99, 0.95) \approx 1.6$: the evidence discriminates sharply against *low* p but only weakly between different *high* values of p , exactly the boundary-flattening behaviour expected of a likelihood proportional to p^{12} as $p \rightarrow 1$. The conclusion carried into the debounce redesign is therefore not a precise point estimate of p but a robust qualitative one: p was high enough, for long enough, that no finite widening of n fully closes the gap, which is exactly the honesty Table 12.3 and the discussion below preserve.

Per-tick failure probability p	$E_0(3, p)$ (old)	$E_0(6, p)$ (new)	Ratio
0.80	4.77 min	14.08 min	$2.95\times$
0.90	3.72 min	8.82 min	$2.37\times$
0.95	3.33 min	7.21 min	$2.17\times$
0.99	3.06 min	6.22 min	$2.03\times$

Table 12.3: Expected ticks between false triggers, Eq. (12.25), for the old ($n = 3$) and new ($n = 6$) debounce across the per-tick failure-probability range fitted against `watchdog.log`. The ratio converges to exactly $n_{\text{new}}/n_{\text{old}} = 2$ as $p \rightarrow 1$, a useful internal check on the derivation.

The model is honest about what widening the window does and does not buy: across this whole p range the mitigation roughly doubles to triples the mean time between false triggers, but does not eliminate them: if p stays this high for the duration of a longer demonstration, the stack would still

be expected to restart every 6–9 minutes rather than every 3–4. This is a quantitative restatement of why item 9 (Table 12.12) keeps the probe redesign on the roadmap rather than treating the debounce widening as closed: no finite increase in n removes a persistently high p , it only dilutes its consequences.

No mission-affecting failure occurred: the rover remained drivable and localised throughout every restart cycle, the operator completed a full loop of the building, and every transient (link and stack alike) resolved itself before requiring intervention. That the outcome looked, in real time, indistinguishable from nominal operation is itself notable: the respawn policy of §12.2.4 and the idempotent design of `restart_stack.sh` absorbed a failure mode three times worse than diagnosed live, without the operator noticing.

12.7 Verification and validation

12.7.1 Protocol T1.1: static drift

Five minutes stationary after a fresh initialisation, $\sim 23\,000$ samples of `/robot_pose_fused`; per axis, the standard deviation and the least-squares drift slope

$$\hat{\beta} = \frac{\sum_k (t_k - \bar{t})(x_k - \bar{x})}{\sum_k (t_k - \bar{t})^2} \quad (12.28)$$

with verdict criterion: $|\hat{\beta}| < 1 \text{ mm/s}$ and bounded excursion $\Rightarrow$ noise (covariance tuning domain); a frank monotone slope $\Rightarrow$ divergence (extrinsics/initialisation domain, *not* noise parameters). The criterion separates the two failure families because their governing mathematics differ: stochastic noise integrates to a random walk whose expected excursion grows as $\sqrt{t}$ and whose fitted slope concentrates around zero as the window grows, whereas a modelling error (wrong extrinsic, wrong gravity, wrong scale) produces a *deterministic* bias whose fitted slope converges to a non-zero constant. With $n \approx 23\,000$ samples over $T = 300 \text{ s}$, the standard error of the slope estimate, $\text{SE}(\hat{\beta}) = \sigma_x / \sqrt{\sum_k (t_k - \bar{t})^2}$, is of order 10^{-3} mm/s for the measured σ_x : three orders below the decision threshold, so the verdict is statistically unambiguous. The protocol is fully automated (stack restart with fresh initialisation, source switch, five-minute log, per-axis analysis) and archives its CSV for audit.

12.7.2 T1.1 A/B result: extrinsic correction

Axis (5 min static)	Identity extrinsic	Camera-forward extrinsic	Interpretation
X: σ / slope	7.5 mm / +0.05 mm/s	8.8 mm / -0.06 mm/s	wheel-anchored
Y: σ / slope	0.6 mm / ≈ 0	1.6 mm / +0.01 mm/s	wheel-anchored
Z: σ / slope	257 mm / +1.68 mm/s (divergent)	49.7 mm / -0.34 mm/s (bounded)	camera-observed only

Table 12.4: T1.1 A/B: the only change between columns is the camera–IMU extrinsic seed (Eq. 12.13). σ reduced $5.2\times$ on Z; the slope crosses the bounded-noise criterion.

The full resolution history of the vertical drift spans three regimes: (i) $z \approx -445$ m: frozen IMU integrating a gravity residual, resolved by the integrity guards (factor ~ 300); (ii) +1.68 mm/s divergence: identity extrinsic, resolved by Eq. (12.13) (factor 5); (iii) bounded ± 5 cm noise: the current state, refinable by the Kalibr extrinsic (open item) and by global anchoring (below).

12.7.3 Front-end real-time verification

The monitor `tools/mins_frontend_monitor.py` parses the estimator’s own timing output and the process CPU, and issues a real-time verdict (median image cost against the camera period). Campaign result: 120 ms $\rightarrow$ 67 ms per image at the 66.7 ms budget: at the boundary, motivating the pending 10 Hz decision.

12.7.4 Field protocols

Slip rejection: a 5 m out-and-back with a provoked 1 s wheel-hold during acceleration; acceptance requires no pose jump during the slip (wheel measurements visibly rejected by the χ^2 gate) and a loop-closure error below 2% of distance. Mission cycle: patrol at 0.2 rad/s, AprilTag acquisition ≥ 3 m, LOCK, checkerboard confirmation < 1.5 m, reset with world-frame preservation, marker deduplication, resume; obstacle bypass with persistent mapping as per Figure 12.3.

12.8 Field session, July 21: fault isolation and an architectural revision

The day after the debounce widening of §12.6.5, a second field session set out specifically to re-validate it in motion (Table 12.12, item 9). Before the traverse itself could begin, three unrelated faults surfaced and were resolved, one architectural assumption was revised, and one live divergence event was caught and quantified in real time: documented here in the order encountered, since each diagnosis depended on ruling out the previous one.

12.8.1 A registered node that had gone silent

On startup, MINS failed to initialise twice in succession, each attempt timing out after its full 300 s budget. The estimator’s own log made the cause unambiguous: [IW-Init]: Waiting for collecting IMU(493 < 3) and wheel data(0 < 3), repeated every sample. IMU measurements were flowing normally (hundreds accumulated); wheel measurements were stuck at exactly zero. Tracing the chain (§12.5: `wheel_remap` subscribes to `/joint_states` and republishes onto `/joint_states_mins`, the topic MINS actually consumes) located the break: `rostopic info /joint_states` reported **zero publishers**, even though `/firmware_message_converter` (the robot-side node responsible for it) was still registered with the ROS master and had not exited. A node can therefore be *alive by every liveness check available to the watchdog* (process present, node registered) while its actual output has silently stopped: registration proves the node exists, not that it is doing anything. The immediate fix was a `rostopic stop` of `firmware_message_converter` (`/joint_states` resumed at ≈ 18 Hz within seconds, confirmed through to `/joint_states_mins`); the standing fix is the open item this incident adds to Table 12.12 (item 10): the watchdog’s liveness checks test for node *presence*, never for topic *output*, and this class of fault, a healthy process publishing nothing, is invisible to it by construction.

12.8.2 An unbounded “still starting”

A second, independent fault was found while investigating unrelated cockpit disconnections: the TLS `rosbridge` tunnel process had been running since 03:16 that morning without ever having

bound its port, and the watchdog had been leaving it alone every single tick since, because its own idempotent design (§12.6) treats a live process with an unbound port as *still starting* rather than *stuck*: the correct call for a process seconds into a normal startup, but with no upper bound on how long “still starting” is believed. The process was manually killed and cleanly relaunched; the design gap (Table 12.12, item 11) is that the grace period needs a ceiling, past which the same process is instead treated as failed and restarted.

12.8.3 WireGuard endpoint roaming, formalised

The most consequential fault of the day was an intermittent, self-reasserting loss of the WireGuard tunnel established in §12.6.4, alternating between minutes of clean operation and total outages lasting up to 20 minutes despite the underlying network path (confirmed throughout by ICMP and SSH to the robot’s campus address) remaining fully reachable. The mechanism is a documented WireGuard behaviour, *endpoint roaming*, that had not previously mattered because the tunnel’s two ends had never before had a plausible wrong address to roam onto. Each peer tracks the other’s endpoint not from static configuration but from observation:

$$E_A(t) = \text{src}(p^*), \quad p^* = \arg \max_{p: \tau(p) < t, \text{ valid}(p)} \tau(p), \quad (12.29)$$

i.e. node *A*’s belief about peer *B*’s address is simply the source address of the most recent cryptographically valid packet *A* has received from *B*, with no memory of the original Endpoint line beyond supplying the first outbound attempt. This is normally the feature that makes WireGuard tolerate a roaming client transparently; here it became a liability because the robot has *two* live addresses (`wlan_ext`’s 10.0.0.1 and `wlan_int`’s 10.154.27.235, per Figure 12.4), and a single stray packet observed from the wrong one is sufficient, by Eq. (12.29), to overwrite the correct runtime endpoint until another valid packet arrives from somewhere else. `wg show` confirmed the mechanism directly: the robot’s own view of the PC’s endpoint had drifted to 10.0.0.10:55458, the PC’s address *on the robot’s own hotspot subnet*, rather than the campus address the static configuration specifies, most plausibly seeded during an earlier troubleshooting step where the PC’s Wi-Fi briefly associated with LeoRover-e138. No config file was wrong; Eq. (12.29) does not consult the file after the first packet.

Why restarting alone did not reliably fix it. The obvious remedy, a systemd restart, was tried first and failed with wg-quick: ‘wg0’ already exists. The cause is a state-consistency gap between two independent trackers of the same resource. Let $S_{sd} \in \{\text{active}, \text{inactive}\}$ be systemd’s belief about the service and $S_k \in \{\text{present}, \text{absent}\}$ be the kernel’s actual interface state. A manual wg-quick down (run outside systemd, as one naturally does while debugging) transitions $S_k: \text{present} \rightarrow \text{absent}$ without touching S_{sd} , which remains active. Systemd’s own contract for start is to act only on the transition $S_{sd}: \text{inactive} \rightarrow \text{active}$; called against a unit it already believes active, it is a no-op. The pair $(S_{sd}, S_k) = (\text{active}, \text{absent})$ is consistent with neither “running” nor “stopped” and is not self-healing: nothing in the ordinary stop/start vocabulary detects or repairs it. The correct sequence forces both trackers through a shared reference point, stop (which accepts the active $\rightarrow$ inactive transition unconditionally) before start:

$$(\text{active}, \text{absent}) \xrightarrow{\text{systemctl stop}} (\text{inactive}, \text{absent}) \xrightarrow{\text{systemctl start}} (\text{active}, \text{present}). \quad (12.30)$$

Once this sequence was applied identically on both ends, Eq. (12.29) re-converged on the correct address within one keepalive interval. The generalisable lesson: mixing a systemd-managed unit with direct, out-of-band control of the same kernel resource (here wg-quick down invoked by hand) is not just risky but can reach a state neither layer’s ordinary vocabulary can name, let alone repair: the same family of fault as the supervisor-probe blind spots of §12.8.1 and Table 12.12 item 2, now at the level of process supervision itself rather than a ROS node.

The recurrence that restarts could not fix. The fix above restored the tunnel for approximately seven minutes, after which it failed again and did *not* recover under repeated, methodical retries: full stop/down/start cycles on both ends, a four-minute idle period on the hypothesis of a transient campus-side rate limit, and a change of the tunnel’s UDP port (51 820 $\rightarrow$ 51 821, then back) to rule out a port-specific block. None of these restored a handshake, while ICMP and SSH to the robot’s campus address continued to succeed throughout: proof the robot remained fully reachable and the fault was confined to the WireGuard flow specifically. This second failure is recorded as unresolved (Table 12.12, item 12): the evidence is consistent with an intervention on the campus network path itself (a firewall or NAT state change, plausibly triggered by the unusually high rate of reconnection attempts during the first fault’s diagnosis) but this could not be confirmed without visibility into campus-side logs this session did not have access to.

12.8.4 The architectural fix: routing within the firewall’s boundary instead of across it

With the tunnel itself unavailable, the working alternative was not another repair attempt but a change of topology: joining the ground-station PC to FLTech-Guest directly, on the *same* Wi-Fi network the robot’s wlan_int already uses, rather than reaching it from the PC’s wired subnet. Figure 12.5 makes the distinction precise. Two hosts sharing a subnet prefix (both PC and robot inside 10.154.0.0/19 once the PC joins FLTech-Guest) resolve each other through the local link layer: address resolution and switch-level forwarding only, no router hop, and therefore no opportunity for the campus firewall’s inter-VLAN policy to apply, since that policy is enforced at the router that stateful cross-subnet traffic is forced through. This is exactly the traversal TCPROS’s arbitrary ephemeral ports could not survive in §12.6.4: it never had to survive it here, because there is no boundary in the path to survive. Re-measured directly, with no tunnel of any kind in the loop: /firmware/wheel_odom sustained at 20.1 Hz, MINS initialised and switched automatically, camera and telemetry both confirmed live.

The trade-off is explicit, not hidden: this path returns the ground-station PC to Wi-Fi, and specifically to the same rtl88x2bu adapter whose kernel driver fault (§12.6.4) was the original reason it was moved to Ethernet in the first place. Adopting it is a deliberate exchange of one known, bounded risk (occasional PC-side Wi-Fi drops, independent of anything the robot does) for a currently-unresolved and unbounded one (item 12). Table 12.12 records the follow-up this implies: decide, once item 12 is understood, whether to return to the tunnelled Ethernet path or adopt same-subnet Wi-Fi permanently, rather than carrying both as live options indefinitely.

12.8.5 A live divergence, caught and quantified

Minutes into the re-established session, a routine stationarity check (the same measurement discipline as Protocol T1.1, §12.7, applied ad hoc rather than as the full five-minute protocol) surfaced a genuine filter divergence in real time. With the robot confirmed stationary (commanded and observed planar velocity identically zero), 684 samples of /robot_pose_fused over a 9.9 s window showed

$$\Delta z = -99.78 \text{ m}, \quad \sqrt{\Delta x^2 + \Delta y^2} / \Delta t = 32.0 \text{ mm/s}, \quad (12.31)$$

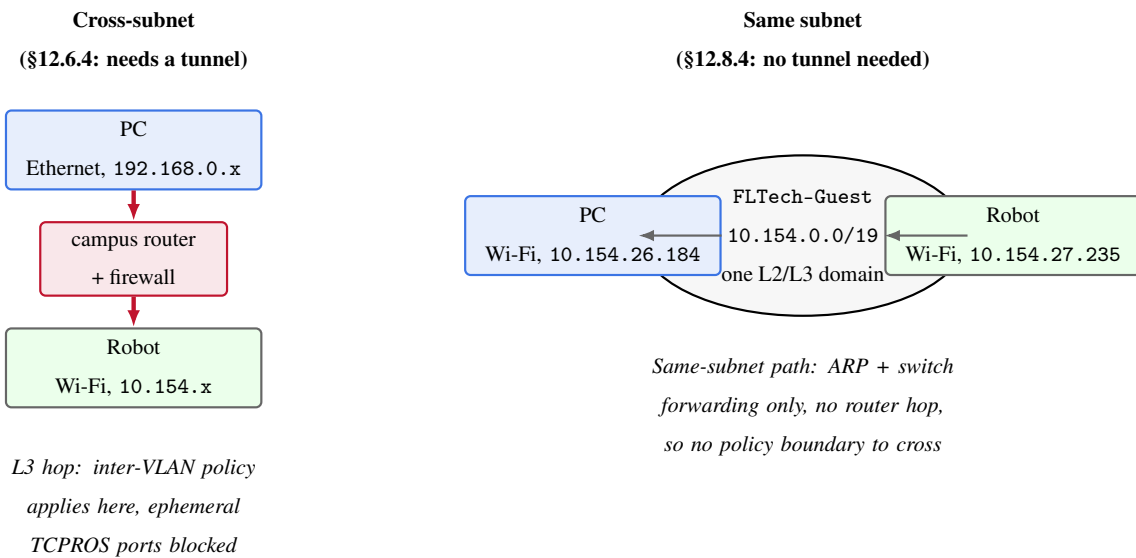


Figure 12.5: The firewall policy of §12.6.4 is enforced at a router boundary, not on the campus network as a whole. Left: the PC’s wired subnet and the robot’s Wi-Fi subnet are different networks, so every packet between them is forced through the campus router where the inter-VLAN policy blocks TCPROS’s arbitrary ports (Table 12.2’s motivation for a tunnel). Right: when the PC joins the robot’s own Wi-Fi network directly, both hosts share one subnet; traffic between them never reaches a router at all, so the same policy simply does not apply. The trade-off, not shown here, is that this returns the PC to Wi-Fi and its documented driver fault (§12.6.4) instead of the driver-immune wired path: a deliberate, disclosed exchange of one known risk for another, not a free fix.

both far outside Protocol T1.1’s bounded-noise criterion ($|\hat{\beta}| < 1 \text{ mm/s}$, §12.7) and immediately recognisable as another instance of the vertical divergence failure mode fully characterised in §12.1: an unmodelled residual acceleration δa integrating quadratically, $\delta z(t) = \frac{1}{2} \delta a t^2$ (Eq. 12.9-adjacent discussion). Solving for the residual this specific event implies,

$$\delta a = \frac{2|\Delta z|}{\Delta t^2} = \frac{2 \times 99.78}{9.9^2} \approx 2.04 \text{ m/s}^2, \quad (12.32)$$

a physically modest, entirely plausible residual, roughly a fifth of g , well within the range an unconverged initial bias or a brief IMU integrity-guard gap could produce, and consistent with the same mechanism as the original $z \approx -445 \text{ m}$ case (Table 12.12, historical) rather than a new failure mode. Rather than wait for the watchdog’s own 100 m divergence guard (§12.5) to act on its next one-minute tick, `restart_stack.sh` was invoked immediately. The fresh instance was re-measured on the identical protocol immediately after: 708 samples over 10.0 s gave a planar drift of **0.1 mm/s** and a z excursion of 13 mm: two orders of magnitude inside the noise criterion, and a clean, dated, field confirmation that the recovery path designed around this exact failure mode (respawn policy, §12.2.4; divergence guard, §12.5) works as specified when exercised for real rather than only in the original diagnostic session. The same guard fired autonomously twice more later in the traverse (14:57:14 and 15:00:12), each time without operator intervention: the July 20 fix is not a one-off, it holds up under repeated real exposure.

12.8.6 Traverse outcome: debounce revalidation, and a quantified roughness

The watchdog log for the traverse (12:58–15:34, $T_{\text{window}} = 9351 \text{ s}$) gives a direct, dated answer to the question the day opened with (Table 12.12, item 9): does the widened debounce actually reduce false zombie-guard triggers in motion, or only on paper? Twenty-one triggers were logged; the twenty intervals between them split cleanly into two regimes,

$$\Delta t_{\text{zombie}} \in \begin{cases} 359\text{--}421 \text{ s} & (13 \text{ consecutive intervals, early traverse}) \\ 543\text{--}720 \text{ s} & (7 \text{ intervals, later traverse}) \end{cases} \quad \overline{\Delta t}_{\text{zombie}} \approx 447 \text{ s}, \quad (12.33)$$

against the 60–180 s cadence characterised on July 20. Even the *shortest* interval measured to-day (359 s) exceeds the *longest* interval from that traverse by a factor of 2; the mean is higher

by roughly $3\text{--}7\times$. Item 9 is therefore only *partially* closed: the debounce widening ($3\rightarrow 6$ ticks, $10\rightarrow 20$ s) clearly moves the false-trigger rate in the right direction and by a large margin, but does not eliminate it, and the underlying mechanism (a `roscnode list` round trip to a roaming master, §12.6.5) is unchanged and will keep firing at some reduced rate under any amount of further widening.

Each zombie-guard trigger, and each of the three divergence-guard recoveries above, invokes the same `restart_stack.sh` path, whose MINS re-initialisation step is itself logged per attempt. Of 21 recorded attempts, 19 succeeded within the routine range (median 21 s), but the distribution has a long tail: two took 90 s and 260 s respectively, and two more exhausted the script’s 300 s ceiling outright (*TIMEOUT init MINS*, a designed hard exit rather than a hang). Treating the sum of logged init durations, plus the two 300 s ceiling hits, as a lower bound on the time the stack spent with no valid MINS pose during the window gives a rough operational availability figure:

$$A_{\text{pose}} \gtrsim 1 - \frac{\sum_i T_{\text{init},i} + 2 \times 300 \text{ s}}{T_{\text{window}}} = 1 - \frac{811 + 600}{9351} \approx 0.85. \quad (12.34)$$

This is a lower bound, not a measurement of true downtime: it omits the bounded shutdown-wait phase before each re-init begins and any transient settling after `/pose_selector` switches source back to MINS. Read as a floor, it says the stack was demonstrably without a valid pose for at least $\sim 15\%$ of the traverse, distributed as roughly one multi-second-to-multi-minute gap every 7 minutes on average, matching the zombie-guard cadence above almost exactly. This gives a concrete, falsifiable candidate explanation for a field observation reported directly by the operator during the walk: the robot advancing in visibly discontinuous, roughly one-second bursts rather than smoothly, with occasional failures to register nearby walls as obstacles, and intermittent loss of the operator link. A stack that loses and regains pose on a ~ 7 -minute cycle, with individual gaps ranging from tens of seconds to (twice) the full 300 s ceiling, is expected to produce exactly this symptom: motion and obstacle avoidance both degrade to whatever the planner can do without a trustworthy, continuously updated pose. This traverse is also the first to run entirely over Wi-Fi with no Ethernet fallback (§12.8.4), which independently revives a risk flagged but not yet measured in an earlier diagnostic pass: Wi-Fi bandwidth contention degrading the raw image topics feeding both MINS and the local costmap. Nothing logged today distinguishes between “pose gap during a restart” and “image-topic starvation under Wi-Fi” as the dominant cause of the reported

roughness, so both are recorded (Table 12.12, item 14) rather than picking one without evidence; the discriminating measurement is a direct Hz trace on the Pi’s image topics during a future traverse, run concurrently with the pose-availability bookkeeping above.

Two secondary results close out the traverse. The camera watchdog logged eleven “muet” (silent) recoveries and zero escalations to the widened hardware-reset threshold (fails ≥ 4 , `leo_watchdog.sh`): every transient camera stall this traverse resolved within the software stop/start cycle, which is exactly the discrimination the widened threshold was set up to make (transient vs. genuinely wedged) and, unlike item 9, this one shows no counterevidence today. And the beacon test originally scoped for the morning, deferred to an A4 tag in the absence of an A3 print, succeeded: `/tag_detections` went active at 15:34:12, confirming long-range AprilTag acquisition works with the smaller tag at whatever range the traverse presented, though the size-dependent range itself was not separately measured this session and remains item 7. Return-to-Base precision was not attempted within the logged window and stays open for a future traverse.

12.8.7 The jerky motion, re-diagnosed: a command-path deadman, not a pose gap

The pose-availability argument of §12.8.6 explains part of the reported roughness, but a second, more direct mechanism turned out to dominate the specific symptom the operator described: the robot advancing in sharp, roughly one-second bursts. Manual drive commands are resent from the browser every 150 ms over the `rosbridge` WebSocket while a direction is held, and the backend applies its own deadman on top: no fresh command within `MANUAL_DEADMAN` zeros the commanded velocity outright, a safety rule (“*homme-mort*”) against a frozen or dropped control device. That constant was 0.5 s, sized when the ground station sat on Ethernet. Once the operator confirmed they were driving from a phone walking alongside the robot rather than from the stationary station, the diagnosis followed directly: that phone’s own Wi-Fi association now roams between campus access points exactly as the robot’s `wlan_int` already does (§12.8.3), and normal jitter on that link exceeding 500 ms several times a minute is enough to trip the deadman on an otherwise healthy connection, well short of a full WebSocket disconnect. `MANUAL_DEADMAN` was widened to 1.0 s (still under half a metre of coasting at the manual-drive speed of 0.2 m/s, an acceptable margin for

an indoor platform with the operator present) and the frontend's fixed reconnect delay after a full WebSocket close was shortened from 2.5 s to 1.0 s, so that the rarer full-disconnect case also recovers faster. Both changes were deployed to the already-running backend by killing the supervised process and letting `respawn="true"` relaunch it with the edited constants, the same zero-downtime mechanism used throughout §12.8 for single-node recovery, verified here by comparing the process start time against the file modification time before declaring the change live.

12.8.8 Yaw trim: a live calibration, converged by operator feedback rather than by log

The leftward drift itself (Table 12.12, item 1) was diagnosed, not just noted, while the traverse continued. A 30 s passive listener on `/firmware/wheel_states` during ordinary forward driving showed the four wheels' commanded angular velocities matching to $\pm 0.0\text{--}0.1\%$ (536 samples, right/left ratio 1.0000): the per-wheel velocity controllers are symmetric, ruling out a motor or PID asymmetry. Correlating those same windows against `/imu/data_clean` restricted to segments where left and right wheel speed agreed to within 3 % (289 samples) isolated a genuine physical effect: a median yaw rate of $+0.0530\text{ rad/s}$ at a mean wheel speed of 3.219 rad/s ($v \approx 0.201\text{ m/s}$), i.e. a relative right/left effective-radius bias of

$$\frac{r_{\text{right}} - r_{\text{left}}}{r} \approx \frac{\dot{\psi}}{\omega} = \frac{0.0530}{3.219} \approx 1.65\%, \quad (12.35)$$

consistent with the operator's own report and with item 1's standing hypothesis, though still short of the roll-out measurement that item calls for. A proportional trim, $\text{ang}_{\text{trim}} = K \cdot \text{lin}$, was added at `_publish`'s single command exit point (the same location already used for the drive-polarity correction of §12.1), with K initialised by cancelling the measured bias exactly: $K = -0.0530/0.201 \approx -0.264\text{ rad/s per m/s}$.

That first value overcorrected, and by more than the raw measurement would predict: the operator reported drifting right on two separate tries at full cancellation. A batch re-measurement taken *during* one of those tries showed almost no residual ($+0.0085\text{ rad/s}$ median, inside the gyro's own noise floor of 0.167 rad/s), directly contradicting the operator's real-time report of the same configuration. The likely explanation is methodologically useful beyond this one fix: an operator who feels a drift corrects for it, so a passive measurement taken while they are actively driving

averages together the physical bias and the human’s own compensation, and can read as “fixed” when it is not. Unlike the log-versus-live-hypothesis lesson of §12.6.5 (trust the log over an explanation formed in the moment), this is the converse case: here the human, being inside the loop being measured, was the more reliable signal, and K was instead converged by bisection on direct qualitative feedback,

K (rad/s per m/s)	trim at 0.2 m/s	operator report
−0.264	−0.0530 rad/s	drifts right (“trop”)
−0.132	−0.0265 rad/s	drifts right, less (“un peu trop encore”)
−0.075	−0.0150 rad/s	drifts left, barely (“très très légèrement”)
−0.087	−0.0175 rad/s	accepted (“beaucoup mieux, restons comme ça”)

four iterations, each redeployed live the same way as §12.8.7, converging in under ten minutes of wall-clock time including drive time. The accepted value, roughly a third of the naive full-cancellation trim, is recorded as the current operational constant; it is a software workaround for a still-unmeasured mechanical asymmetry, not a substitute for the roll-out measurement item 1 already calls for, and should be revisited (most likely removed, once the true radii are known and `config_wheel.yaml` corrected) rather than carried forward indefinitely.

12.8.9 AUTO mode control-loop hardening, and a genuine blind-spot found by driving into a wall

A direct operator assessment of AUTO mode (“loin d’être satisfaisant”, citing erratic patrolling, unreliable obstacle avoidance, and a fussy beacon lock) prompted a code review of the three mechanisms behind each complaint, rather than a live measurement as in the preceding sections: the failure patterns were structural enough to diagnose from the control logic itself. All three share a common shape, a decision recomputed every control tick from a single, un-smoothed sample, with no memory of the previous tick:

- **Patrol steering:** the “open path” bearing used to bias PATROL’s heading was recomputed fresh from the current depth-sector scan every tick; two adjacent, similarly-open sectors trading

places between frames (ordinary sensor noise) swung the commanded heading abruptly each time. Fixed with an exponential moving average on the tracked bearing ($\alpha = 0.3$), reset only when no open path is currently detected.

- **Avoidance direction:** which way to pivot around an obstacle was chosen from a single instantaneous left/right depth median at the moment the obstacle was flagged; one noisy frame could commit the robot to a 45–135° turn toward the *more* obstructed side. Fixed with the same class of exponential smoothing ($\alpha = 0.4$) applied to the left/right medians themselves, so every consumer (this decision and the corridor-attraction bearing) sees a steadier signal.
- **Beacon lock chatter:** whether the robot considered itself “centred” on the beacon was a single hard threshold ($\pm 8\%$ of frame half-width) evaluated independently each tick; a detected centre jittering near that line (LED/checkerboard detection noise is a known, longstanding property of this pipeline, §10) flipped the flag rapidly, alternating advance and re-centring commands. Fixed with hysteresis: entry stays at $\pm 8\%$, exit only past $\pm 14\%$.

None of these three required new sensors or a different algorithm, only the standard control-systems remedy for a decision that chatters on noisy input: smooth the input, or widen the band around the decision boundary.

The fourth finding was not architectural but a genuine safety gap, surfaced directly by the operator during the very next test: “*il fonçait dans les murs quand il était en face d’eux*” (it drove straight into walls it was facing head-on). AUTO mode’s only blind-driving guard checked `cam_alive`, freshness of the colour/IR stream feeding beacon detection, before allowing forward motion. Obstacle sensing is a *different* stream (`/pc/camera/depth/...` versus `/pc/camera/infra1/...`), republished over the same WiFi path but independently, and nothing checked whether *it* was still arriving. Under the bandwidth contention already flagged as a live risk this session (item 14), the depth stream can stall while the IR stream keeps flowing: `cam_alive` stays true, and PATROL kept commanding `ADVANCE_LIN` on a stale or absent obstacle flag, no different in effect from disabling the sensor outright. A second freshness gate, `DEPTH_ALIVE_TIMEOUT`, was added at the same point in `_drive()` as the existing camera gate, deliberately tighter (1.0 s against the camera gate’s 2.0 s, §12.8.7): this one bounds blind travel directly against a collision margin rather than against a

beacon-detection convenience, and at ADVANCE_LIN (0.2 m/s) one second bounds that travel to

$$d_{\text{blind,max}} = \text{ADVANCE_LIN} \times \text{DEPTH_ALIVE_TIMEOUT} = 0.2 \times 1.0 = 0.20 \text{ m}, \quad (12.36)$$

comfortably inside the 0.6 m obstacle-trigger margin (OBSTACLE_DIST_MM). A matching AUTO_NO_DEPTH FSM state was added so a future stall is visible in the cockpit as a labelled, deliberate stop rather than an unexplained one. All four fixes were deployed live via the same process-respawn mechanism as §12.8.7; a full validating traverse was not completed this session (§12.8.10 consumed most of the remaining test window), so item 15 below carries that forward.

12.8.10 A disk-full crash found mid-diagnosis

The camera failures during the AUTO-mode test initially looked routine (§12.8’s escalation ladder: soft cycle, then hardware reset past four consecutive failures) but did not resolve after two hardware resets, an escalation pattern not seen earlier in the campaign. An unrelated SSH command surfaced the real cause directly: `No space left on device` on the robot’s own root filesystem, confirmed at **100 % used, 0 bytes free** (`df -h`). The two largest recoverable consumers were both ordinary, unbounded log accumulation rather than anything load-bearing: 2.6 GB of time-stamped `roslaunch` run logs under `/var/ros/log` (37 directories, one per launch, never purged) and 2.9 GB of `systemd` journal. Both were cleared with the operator’s explicit authorisation (the destructive-command guard correctly declined to run an unattended `rm -rf` first), freeing 5.3 GB and returning the disk to 82 % used.

A second, more serious fault was discovered only because the first one was fixed: `leo.service` itself had crashed some minutes earlier, `exit-code 139 (SIGSEGV)`, and, lacking a restart policy, simply stayed dead rather than being relaunched by `systemd`. Every symptom of the following twenty-five minutes, ROS master unreachable from the PC while ICMP and SSH stayed healthy throughout (the same discriminating signature as §12.8.3’s WireGuard fault, here with a different root cause), `restart_stack.sh` retrying against a master that did not exist, and dozens of stale `wait_mins_*/mon_tour_*` node registrations accumulating in `rostopic list`, traced back to this one dead service, not to a new network problem. `sudo systemctl restart leo` resolved it immediately once found. Table 12.12 records two follow-ups this incident implies beyond item 1’s mechanical measurement: monitor disk usage before it reaches zero rather than discovering it

through a downstream symptom, and give `leo.service` a restart policy so a future crash of this kind self-heals the way every other component in this architecture already does.

12.9 Field session, July 22: closing the beacon-approach loop, and a first application of continuous control

12.9.1 Beacon reset gated on dual confirmation, and a corrected post-visit turn

The reset that zeroes local odometry against a visited beacon (§12.8.9) fired, in both the MANUAL-mode and AUTO-mode LOCK paths, on a single confirmed cue: the LED cluster alone (MANUAL), or a measured distance from *either* the AprilTag or the checkerboard (AUTO, the “tout-en-un” path introduced 2026-07-13). Both were widened to require the LED cluster *and* the checkerboard confirmed simultaneously before the reset fires, on the reasoning that the checkerboard’s sub-pixel corner solution is a materially more reliable pose reference than the LED cluster’s centroid alone, and a reset is a consequential, silent operation (it re-anchors the robot’s entire local frame) that should not commit on a single, weaker cue when a stronger one is available from the same detector pipeline.

A dual requirement introduces a failure mode the single-cue version did not have: the robot can arrive at the stop distance with one cue confirmed and the other not (the checkerboard’s usable range is $\lesssim 1.5$ m and depends on which face is presented, §10; the LED cluster is visible from a wider range of angles and distances), with no existing timeout that covers *that* state, only LOCK_TIMEOUT (fires when the target disappears) and LOCK_STALL_S (fires when centring never converges). Confirmed-but-unreset can now persist indefinitely. A bounded grace hold was added: on arrival, the robot freezes (`lin = ang = 0`, the branch’s existing default) for DUAL_CONFIRM_GRACE_S = 4.0 s while polling for the second cue; if it never arrives, the attempt is abandoned through the codebase’s existing, field-proven obstruction-avoidance path (`_map_obstacle + _enter_avoid`, the same call already used one branch up for a depth-obstruction abandon) rather than a new, unexercised exit, with the standard LOCK_COOLDOWN_S applied so the same ambiguous target is not immediately re-locked. One consequence is accepted deliberately, not incidentally: the corridor-

depth arrival path (no tag or checkerboard distance available at all, the robot treats a close depth reading as “arrived”) almost never has a valid checkerboard by definition, so it will now expire its grace window on essentially every attempt and register no beacon, only a contournement. Requiring the checkerboard everywhere trades that coverage for the precision argument above; whether the trade is net positive depends on how often a beacon is only approachable from a checkerboard-blind face, not yet characterised in this campaign.

The fixed 90° post-beacon turn (§12.8.9’s predecessor, itself a deliberate 2026-07-08 replacement of a 360° scan phase, chosen specifically for a perpendicular sweep over the alternative of turning back along the same line) was changed to a full 180° demi-tour on explicit request. The change is a pure constant edit (POST_BEACON_TURN_RAD), confirmed isolated: obstacle-avoidance’s own fallback turn hardcodes π directly and the reactive bounce-pivot uses an independent random target (§12.8.9), so nothing else references this constant. It reopens the coverage trade-off the July 8 redesign closed, a demi-tour revisits the same line rather than sweeping a new one, recorded here as a conscious reversion rather than an oversight, and left for field observation to judge, not asserted as an improvement.

12.9.2 A first application of continuous control: PID regulation of three AUTO-mode loops

Every closed-loop decision in AUTO mode up to this point (§12.8.9) was either a bare proportional term or, in the case of obstacle response, not proportional at all: `_on_depth` flags an obstacle the instant any depth tile’s 5th-percentile distance crosses `OBSTACLE_DIST_MM` (600 mm), which commits the robot to a fixed-angle reactive pivot (§12.8.9). Between that trip and the moment it fires, the robot advances at a constant `ADVANCE_LIN`, with no proximity awareness at all, the discretised control-loop analogue of an on/off thermostat rather than a regulator. An operator report during this session (the robot driving into walls it approached head-on, distinct from the July 21 blind-driving gap already closed by `DEPTH_ALIVE_TIMEOUT`, §12.8.9, which guards against a *stale* depth stream, not a live one read too late) is the direct motivation for the work below: a live depth stream read correctly still permits full-speed approach right up to the trip distance, an inherent property of a bang-bang controller, not a sensing gap. Three loops were converted from proportional-only to full

PID: patrol steering, beacon centring (two instances, align-in-place and approach-while-centred), and a new obstacle-proximity speed governor introduced specifically to close this gap. All three share one controller implementation, described once below and then instantiated per loop with the substituted gains.

Discretised PID: the controller actually implemented

The canonical continuous-time PID law,

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad (12.37)$$

with error $e(t) = r(t) - y(t)$ between a reference r and a measured output y , has the familiar Laplace-domain transfer function

$$C(s) = \frac{U(s)}{E(s)} = K_p + \frac{K_i}{s} + K_d s, \quad (12.38)$$

useful as a reference point but not directly implemented here: the control loop (`_control_loop`, `CONTROL_HZ= 20 Hz` nominal, §12.8.7) has no enforced real-time guarantee, a slow tick simply runs late with no catch-up logic and no measured Δt threaded anywhere in the existing code, so a fixed- Δt discretisation (the textbook $u[n] = u[n-1] + K_p(e[n] - e[n-1]) + K_i \Delta t e[n] + \dots$ velocity form) would silently assume a sample interval the scheduler does not actually guarantee. The controller instead timestamps every call itself against the same now the caller already computed once per tick, and integrates/differentiates against the *measured* interval:

$$\Delta t[n] = t[n] - t[n-1], \quad (12.39)$$

$$I[n] = \text{clamp}(I[n-1] + e[n] \Delta t[n], I_{\min}, I_{\max}), \quad (12.40)$$

$$D_{\text{raw}}[n] = \frac{e[n] - e[n-1]}{\Delta t[n]}, \quad (12.41)$$

$$D[n] = \begin{cases} D[n-1] & \text{if } \Delta t[n] > \Delta t_{\max} \text{ (see below)} \\ 0.5 D_{\text{raw}}[n] + 0.5 D[n-1] & \text{otherwise,} \end{cases} \quad (12.42)$$

$$u[n] = \text{clamp}(K_p e[n] + K_i I[n] + K_d D[n], u_{\min}, u_{\max}). \quad (12.43)$$

Equation 12.42 is a first-order IIR (exponential) low-pass on the raw derivative alone, coefficient fixed at 0.5 rather than exposed as a per-loop tunable: three of the four instances below already consume an input pre-smoothed by an existing exponential moving average (§12.8.9's patrol-bearing

and left/right-depth EMAs, and the LED/checkerboard centroid EMAs, §10), so this internal filter is a safety net for the one signal that has no upstream smoothing (corridor depth, §12.9.2), not a second, redundant smoothing stage layered on already-clean signals. $\Delta t_{\max} = 3/\text{CONTROL_HZ} = 0.15$ s bounds the derivative to ordinary scheduling jitter, an anomalously slow tick skips the derivative update for that step (holding the last filtered value) rather than computing a spurious spike against an inflated Δt , while still integrating and applying P normally; the integral memory itself is untouched by this guard and is retired only by the explicit `reset()` calls described below.

Two guards are applied independent of loop: on the first call after construction or an explicit `reset()` ($t[n-1]$ undefined), the controller returns `clamp( $K_p e[n]$ ,  $u_{\min}$ ,  $u_{\max}$ )`, proportional action only, avoiding both an undefined Δt in Eq. 12.40 and a derivative spike against an arbitrary previous error; and the integral term is clamped *before* summation (Eq. 12.40, $I_{\min}/I_{\max}$ set independently of $u_{\min}/u_{\max}$), the clamped-integration form of anti-windup rather than back-calculation, chosen for its simplicity and because two of the four instances below run $K_i = 0$, where windup cannot occur regardless of clamp; the two instances with $K_i \neq 0$ are discussed individually. `reset()` is called at every FSM state-entry point already identified for this class of per-state variable (§12.8.9’s `lock_centered` and this session’s `_dual_confirm_t0`, §12.9.1), and additionally at `_drive()`’s two unconditional safety returns (`cam_alive/depth_alive` both false, §12.8.9), which are not FSM transitions and so are not covered by the state-entry resets, a blocked robot with sustained nonzero error, or a depth/vision dropout, is exactly the scenario the integral-clamp exists to bound, and letting stale integral/derivative memory survive a safety stop is avoided by the same blanket-reset philosophy the safety gates themselves already use (they stop *all* motion on either signal’s loss, not just the sub-behaviour that needed it).

Obstacle-proximity speed governor

The one loop with no proportional predecessor to inherit a gain from. The error is corridor clearance measured against a slowdown threshold set well above the existing hard-trip distance,

$$e[n] = d_{\text{corridor}}[n] - \text{OBSTACLE_SLOWDOWN_MM}, \quad \text{OBSTACLE_SLOWDOWN_MM} = 1500 \text{ mm}, \quad (12.44)$$

where d_{corridor} is the same 5th-percentile, three-tile-ROI corridor distance the binary trip already consumes (§12.8.9). Rather than an additive velocity correction, the controller output scales the

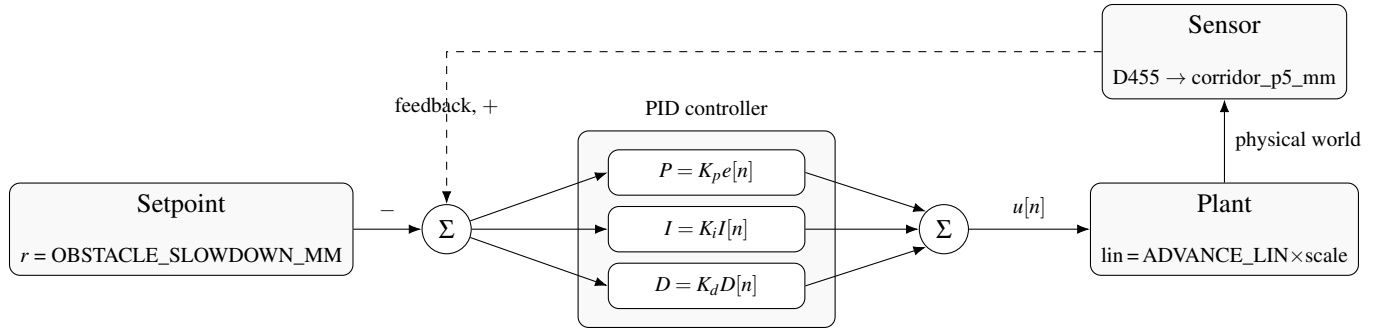


Figure 12.6: PID feedback loop, drawn against the obstacle-governor loop’s actual signals (§12.9.2). The same five blocks, setpoint, error-summing junction, parallel P/I/D branches recombined at a second junction, plant, sensor, instantiate all three loops in this section; only the setpoint, error signal, and gains change.

commanded forward speed multiplicatively,

$$\text{scale}[n] = \text{clamp}(1 + u[n], s_{\min}, 1), \quad \text{lin}[n] = \text{ADVANCE_LIN} \times \text{scale}[n], \quad (12.45)$$

with $u[n]$ itself already clamped to $[s_{\min} - 1, 0]$ (Eq. 12.43, $u_{\max} = 0$ so a clear corridor, $e[n] > 0$, contributes no slowdown at all, only $e[n] < 0$ can reduce speed), making Eq. 12.45’s outer clamp defensive rather than load-bearing. Gains: $K_p = 1.2 \times 10^{-3} \text{ mm}^{-1}$, deliberately steeper than the value that would reach the floor $s_{\min} = 0.3$ exactly at the existing 600 mm trip distance ($K_p^* = 0.7/900 \approx 7.8 \times 10^{-4}$), so the governor saturates *before* the hard trip, favouring margin on an untuned first deployment over a tight fit; solving Eq. 12.45 for the corridor distance at which the floor is first reached,

$$d_{\text{floor}} = \text{OBSTACLE_SLOWDOWN_MM} - \frac{s_{\min} - 1}{K_p} = 1500 - \frac{-0.7}{1.2 \times 10^{-3}} \approx 917 \text{ mm}, \quad (12.46)$$

roughly 300 mm of margin ahead of the 600 mm trip, at $\text{ADVANCE_LIN} = 0.2 \text{ m/s}$ about 1.6 s of additional reaction time purely from this loop, before the binary trip’s own reaction applies. $K_i = 0$: the blocked-with-sustained-error state this loop can legitimately sit in (an obstruction, or the dual-confirm grace hold, §12.9.1) is precisely the scenario Eq. 12.40’s clamp exists to bound with no compensating benefit, a stable partial slowdown while skirting an obstacle is an acceptable steady state, not an error to be eliminated, and the existing stuck-detector ($\text{STUCK_MIN_DISP_M}/\text{STUCK_DISP_WIN_S} \rightarrow \text{_enter_escape}$, §12.8.9) already owns “not making progress” as a failure mode.

$K_d = 0.15$, using Eq. 12.42’s internal filter directly, as noted above the one signal in this section with no upstream EMA. The binary trip and reactive pivot (§12.8.9) are untouched by this loop, deliberately: it only makes the robot slow down earlier, it does not gate, replace, or weaken the existing hard stop, and it applies at both loci depth-covered forward advance already occurs, PATROL and SPIRAL.

Patrol steering

Error is the same exponential moving average of the “open path” bearing already computed for patrol steering (§12.8.9, $\alpha = 0.3$), setpoint zero (drive straight at it):

$$e[n] = \bar{\theta}_{\text{open}}[n], \quad \text{ang}[n] = \text{clamp}(u[n], -\text{OPEN_STEER_MAX}, \text{OPEN_STEER_MAX}), \quad (12.47)$$

with $K_p = \text{OPEN_STEER_KP} = 0.8 \text{ s}^{-1}$, the field-tuned proportional gain this loop already used, reused unchanged rather than re-derived; $K_i = 0$, the target corridor is recomputed fresh every tick, there is no fixed setpoint an accumulated integral could meaningfully drive toward, and a nonzero integral here would also compete with the independently-tuned yaw-drift trim already applied once, at the single publish point (§12.8.8), risking two uncoordinated bias correctors fighting the same actuator; and $K_d = 0.1$, damping fast bearing swings (a doorway passing through the field of view) without duplicating the anti-zigzag EMA already upstream of $e[n]$ itself. Output clamp unchanged at $\pm \text{OPEN_STEER_MAX} = \pm 0.25 \text{ rad/s}$.

Beacon centring: two instances

Two independent controllers, not unified into one, the error units differ (a normalised fraction while aligning in place versus a raw pixel offset while centred and advancing) and unifying them would require converting one loop’s error into the other’s units, more surface area than a mechanical P→PID swap warrants on a currently-deployed FSM.

Align-in-place (rotate to centre the beacon before advancing):

$$e[n] = \frac{c_x[n] - w[n]/2}{w[n]/2}, \quad K_p = \text{LOCK_ALIGN_SPEED} = 0.25 \text{ s}^{-1}, \quad (12.48)$$

c_x the detected target’s horizontal pixel coordinate and w the frame width, reusing the loop’s existing field-tuned gain. $K_i = 0.05$, small and tightly clamped ($I_{\min}/I_{\max}$ bounding $K_i I[n]$ ’s own contri-

bution to at most a fifth of the output range), justified the same way as the yaw-drift trim (§12.8.8): a fixed optical-axis offset is physically plausible given this rover’s already-documented asymmetric wheel behaviour (item 1, Table 12.12), and exposure is time-bounded by LOCK_TIMEOUT/LOCK_STALL_S (§12.8.9), limiting how long any windup risk can accumulate even under the clamp. $K_d = 0.06$, conservative. Output clamp unchanged at $\pm \text{LOCK_ALIGN_SPEED}$.

Approach while centred (advance while holding the target centred, used identically at both of §12.9.1’s arrival branches):

$$e[n] = c_x[n] - w[n]/2 \text{ (pixels)}, \quad K_p = \text{TARGET_KP} = 4 \times 10^{-3} \text{ rad s}^{-1} \text{px}^{-1}, \quad (12.49)$$

gain reused unchanged. $K_i = 0$: the align loop above already removes any fixed bias before lock_centered ever becomes true, so residual error here should already be small and transient (vibration, minor drift), better damped than integrated, and this loop shares the exact windup exposure already flagged for the obstacle governor (obstruction holds, dual-confirm grace holds, §12.9.1) with no corresponding argument for keeping an integral term. $K_d = 0.05$. Output clamp unchanged at $\pm 0.3 \text{ rad/s}$. The existing centred/not-centred hysteresis band ($\pm 8\%$ entry, $\pm 14\%$ exit, §12.8.9) governs the boolean state transition, not the steering command, and is untouched by either substitution.

Validation status

All three loops were deployed live via the same process-respawn mechanism used throughout this campaign (§12.8.7), in ascending order of collision-relevant stakes, patrol steering first (bounded by an already-clamped $\pm 0.25 \text{ rad/s}$ in open terrain, easiest to validate visually), then beacon centring (bounded consequence: a worse approach angle at worst, with §12.9.1’s safety nets underneath unchanged), the obstacle governor last, deliberately additive over the unmodified binary trip rather than a replacement, so that even poorly-chosen day-one gains cannot remove existing collision protection, only shift when deceleration begins relative to the trip. No live gain is derived from a plant model of this rover (no measured mass, wheel-slip transfer function, or camera-loop latency budget feeds any $K_p/K_i/K_d$ above); every gain is either reused unchanged from a field-tuned predecessor or, for the one loop with no predecessor (§12.9.2), chosen from the same geometric margin argument already used elsewhere in this campaign for untuned first deployments (§12.8.9’s

DEPTH_ALIVE_TIMEOUT margin against OBSTACLE_DIST_MM is the same pattern). None of the three loops has yet been exercised over a live AUTO traverse under ordinary network conditions to confirm the intended qualitative effect, visibly smoother patrol heading, a visibly earlier and smoother deceleration on wall approach, without introducing new oscillation the prior single-gain proportional terms did not have; this session's network conditions (§12.9 was conducted throughout an extended, unresolved building-WiFi outage, itself a live instance of item 13's driver fault) did not permit it. Table 12.12 records this alongside a second, structural risk specific to §12.9.1: whether this beacon's physical LED/checkerboard layout ever produces a pose at which both cues are simultaneously valid has not been confirmed, if it structurally cannot (checkerboard readable only from one face, LEDs only bright enough at a glancing angle that face never presents), the grace window will expire on every attempt and no beacon will register autonomously at all, a possibility to rule out on the bench before it is discovered in the field.

12.9.3 Item 10 recurs, this time behind hours of network misdiagnosis

Later the same day, the cockpit's VINS↔MINS switch stopped working: pose_selector refused every attempt with cannot switch to MINS: no message received on it yet, and MINS's own log repeated [IW-Init]: Waiting for collecting IMU(490 < 3) and wheel data(0 < 3) without ever progressing, IMU accumulating normally, wheel data stuck at exactly zero. This is, byte for byte, the same symptom already diagnosed and named in §12.8.1 the day before. It was not recognised as such until late: six environmental interventions were exhausted first, each plausible on its own and each ruled out in turn rather than assumed away, in order: (i) two attempts on the direct same-subnet Wi-Fi path (§12.8.4); (ii) a deliberate switch to the Wire-Guard tunnel (§12.2) for two further attempts, on the reasoning that §12.8.3's roaming-endpoint fault could not be ruled out from the symptom alone, during which the tunnel itself was directly observed to drop for ≈ 15 s and self-recover, a milder instance of item 12's still-open failure mode; (iii) a deliberate stationary hold, ruling out roaming/motion as the proximate cause; (iv) a full reboot of both the ground-station PC and the rover, ruling out any accumulated process or registration state. All six failed identically. A process that fails the same way regardless of an independent variable is evidence against that variable, not for trying it a seventh time: the next step taken was to stop varying the network and inspect the ROS graph state of the specific node already named in item 10.

`roscpp info /firmware_message_converter` showed it subscribed only to `/firmware/imu`, not to `/firmware/wheel_states` or `/firmware/wheel_odom`, despite its own loaded `firmware_message_converter.yaml` declaring `wheel_joint_names` that only make sense if it publishes `/joint_states` from wheel data; it was not crashed (still registered, still consuming CPU) and not restarted since the previous incident, consistent with item 10's standing recommendation, an output-rate probe alongside presence checks, never having been implemented. The fix was identical to §12.8.1: a `roscpp stop/start` cycle of `firmware_message_converter` alone. `/joint_states` resumed at ≈ 19 Hz within seconds, `[IW-Init] smooth success` followed within the same cycle, and the cockpit switch succeeded on the next attempt. Table 12.12 item 10 is updated rather than duplicated: the same blind spot cost approximately two hours of network-focused misdiagnosis on its second occurrence, the clearest evidence yet that its fix is worth the effort.

A plausible trigger for the original subscription loss, not the same claim as its cause, surfaced independently while investigating: `journalctl -u leo` recorded 20,295 occurrences of wrong checksum for topic `id` and msg on `serial_node` (the Pi $\leftrightarrow$ CORE2 UART link, 250,000 baud) in a two-hour window, interleaved with explicit `lost sync` and `Mismatched protocol version` warnings, a rate ($\approx 2.8 \text{ s}^{-1}$ sustained) consistent with a serial-desync IMU firmware freeze already known from earlier in this campaign but not previously observed at this severity. Whether this specific desync burst is what caused `firmware_message_converter`'s wheel-data callback to fail without crashing the node is plausible, matches the timing, and is recorded here as the leading hypothesis, but was not proven: doing so would require a timestamped correlation between a specific desync event and the node's subscription state that this session did not capture. Table 12.12 records it as a new item (19) rather than folding it into item 10, since the watchdog fix (detect dead output) and the hardware fix (stop the desync) are independent and both still needed.

12.9.4 A first migration off legacy tf1: MINS ported to tf2_ros

Auditing MINS's build for a specific, requested criterion, whether it uses `tf2_ros` or the deprecated ROS1 `tf` package for its transform broadcasts, found the latter: `mins_ws/src/MINS/mins/package.xml` declared an unconditional `tf` dependency, and `cmake/ROS1.cmake` (the branch actually compiled on this Noetic/catkin rover; a parallel `cmake/ROS2.cmake` exists upstream but is never built here) compiled `ROSPublisher.cpp`, `ROSHelper.cpp`, and `SimVisualizer.cpp` against `tf::TransformBroadcaster`.

and `tf::StampedTransform`. A `tf2_ros`-based implementation of the identical logic already existed in the same upstream repository (`ROS2Publisher.cpp`, `ROS2Helper.h`), unbuilt on this platform; rather than devise an independent translation, all three ROS1 files were ported field-for-field to mirror it, `tf::StampedTransform` replaced by `geometry_msgs::TransformStamped` constructed member-by-member, the broadcaster retyped to `tf2_ros::TransformBroadcaster`, and `package.xml/cmake/ROS1.cmake` updated to depend on `tf2_ros` instead of `tf`. The change is scoped deliberately narrowly: MINS's TF output is documented elsewhere in this system (`mins.launch`, §12.2) as visualisation-only, isolated to `/tf_mins` and never on the pose-fusion critical path (`pose_selector` consumes `/mins/imu/odom` directly, a plain `nav_msgs/Odometry`, never via TF), so no file touched here can affect the estimator's actual filter state. Rebuilt clean (`catkin build mins`, zero warnings) and verified live: the running process was replaced (kill, automatic respawn), `/mins/imu/odom` continued publishing, `/tf_mins` carried a correctly normalised `global→imu` transform, and a `VINS↔MINS` switch cycle through `pose_selector` still succeeded. One unrelated fact surfaced during this same verification: `ov_msckf` (VINS) was found not running at all, no process, removing the VINS fallback until restarted; this migration neither caused nor fixes it, and it is logged as a new open item (20) rather than folded into this one.

12.10 Field session, July 23: Return-to-Base rebuilt, a source-selection interface simplified after two iterations, and an initial-divergence guard

12.10.1 Return to Base: world-frame preservation, and full-trajectory re-tracing instead of a straight line

Two independent defects in Return-to-Base surfaced during an operator session and were fixed the same day.

The `reset_coords()` world-frame bug. The cockpit's manual *Reset 0,0,0* button (`data-cmd="reset"`) called `reset_coords()`, which zeroes local odometry via a hardware reset without first re-anchoring

world_origin_x/y/heading to the pre-reset world position. Its sibling `reset_coords_local()` (introduced §12.8.1’s era for the AUTO-mode beacon-visit resets) does this correctly: it reads $(w_x, w_y) = \text{_get_world_pos}()$ *before* the reset and sets

$$\text{world_origin} \leftarrow (w_x, w_y), \quad \text{world_heading} \leftarrow \psi_{\text{raw}} \quad (12.50)$$

so that *after* the local pose zeroes, `\_get\_world\_pos()` (which computes $\text{world_origin} + R(\text{world_heading}) \cdot \text{pose}_{\text{local}}$) still evaluates to the same physical point. `reset_coords()` skipped Eq. 12.50 entirely: after a manual reset mid-mission, the backend’s own belief of “world (0,0)” silently snapped to wherever the robot physically stood at reset time, not the true mission start. RTB, which navigates toward that reference, would then target a corrupted point with no error surfaced anywhere. **Fix:** `reset_coords()` now performs the identical re-anchoring as `reset_coords_local()` before the hardware reset fires.

Full-trajectory retracing. Independently, RTB itself only ever implemented a straight line to world (0,0) (§12.8.9), with a blind-in-place rotation whenever the bearing error exceeded a threshold — including, structurally, at RTB’s own start, since the target is normally near-opposite the direction of travel. Operator feedback after a live trial asked for two changes: retrace the path actually travelled rather than cut through unexplored space, and drive in reverse rather than turn around. Both were implemented together, and the second turns out to reinforce the first: a rover known to lose localisation quality while pivoting (§12.4, wheel slip during in-place rotation) accumulates less heading error on the way home if it never pivots at all.

A persistent world-frame breadcrumb trail $\mathcal{T} = \{(w_x^{(i)}, w_y^{(i)})\}$ is now recorded on every odometry callback, sampled at a coarser spacing than the display trajectory (`RTB_TRAIL_SPACING=0.25` m vs. the map’s 0.02 m) and, unlike the map trajectory, *never* cleared by an odometry or LED reset — only by `clear_map()` or a completed RTB run — since it lives in the same world frame Eq. 12.50 keeps continuous across those resets. On RTB start, the queue of waypoints to visit is $\mathcal{T}$ reversed (most recent first) with a guaranteed final anchor at the true origin:

$$W = [p \in \text{reverse}(\mathcal{T}) : \|p - p_{\text{now}}\| > \text{RTB_DIST_TOL}] \parallel [(0,0)] \quad (12.51)$$

Each waypoint $w_k = (t_x, t_y)$ is consumed identically to the original single-target logic (same distance-based speed ramp, same overshoot detector), but the steering error is now computed against the

robot’s *reverse* heading rather than its forward one:

$$\theta_{\text{err}} = \text{atan2}(\sin(\beta - \theta_{\text{rev}}), \cos(\beta - \theta_{\text{rev}})), \quad \beta = \text{atan2}(t_y - w_y, t_x - w_x), \quad \theta_{\text{rev}} = \theta + \pi \quad (12.52)$$

with $\text{lin} < 0$ throughout and no in-place-rotation branch at all: the steering correction $\text{ang} \propto \theta_{\text{err}}/\pi$, clamped to `RTB_ANG_SPEED`, is authoritative over the full error range. Reaching a waypoint within tolerance advances to the next one within the same control tick (no stop-and-resume between legs); the final arrival re-triggers the existing safe-stop path unchanged. **Not yet validated:** a live multi-waypoint retrace over a real multi-beacon trajectory — the mechanism has been exercised on short manual runs only, logged as a new open item (21).

12.10.2 Pose-source selection: two iterations to a simpler, honest interface

Worth recording as process, not just outcome: the operator asked to be able to *pre-select* VINS before driving (since it needs sustained motion to initialise, unlike MINS’s static init), so a switch requested while the target source had never published would be armed and applied automatically the instant it did — implemented directly in `pose_selector.py` (`_switch_to()` storing a `_pending` source, fulfilled inside `_on_odom()` the moment that source’s first message arrived, with a matching cockpit affordance: a blinking, dashed button state and a second click to cancel). It worked as specified, verified live end-to-end (`arm` → `pose_source_pending` topic reflects it → cancel returns it to empty, active source undisturbed throughout).

It was reverted the same session. Operator feedback after using it: the extra implicit state (armed vs. active vs. cancelled) made it harder, not easier, to know what the system was actually doing — the opposite of the transparency this cockpit has otherwise been built around all campaign. The interface was simplified back to the original contract: a click attempts the switch immediately, succeeds or fails outright, with the failure surfaced through the pre-existing `/mission/log` → cockpit log channel (unchanged since §12.6) — no armed state, no automatic later application. `pose_source_pending` and its topic, publisher, and cockpit wiring were removed in full rather than left dormant. **Lesson, stated plainly:** an operator-transparency requirement (“let me see what’s about to happen”) is not satisfied by hiding latency behind automation — it is satisfied by making the current true state impossible to misread. The two are easy to conflate when designing the feature and easy to tell apart once actually operating it.

12.10.3 An initial-position divergence in VINS, and a new plausibility guard

While re-testing the switch after §12.10.2’s simplification, VINS produced enough motion to publish for the first time that session, and the switch to it succeeded (message: "switched to VINS"). Its *raw* reported position at that first message was (27.5, −25.7, −4.6) m — physically impossible for a rover confined to a single room. The existing plausibility guard (introduced 2026-07-13 alongside the SE(3) switch correction, Eq. 12.9) did not catch this: it bounds the *speed* between two consecutive samples of the currently *active* source, a check with nothing to compare against on a source’s very first message. The SE(3) switch correction itself (Eq. 12.9) absorbed the offset on the *fused* output — verified directly: `/robot_pose_fused` read (−0.37, −0.37, −1.665) m immediately after the switch, plausible in x, y — but the anomalous z residual (−1.665 m for a ground rover) shows the correction papered over a bad estimate rather than fixing it: continuity of *output* is not the same guarantee as correctness of the *source*.

A second, complementary guard closes this gap: a source’s first-ever message is checked against its own claimed origin before being accepted at all,

$$\|p^{(0)}\| = \sqrt{x_0^2 + y_0^2 + z_0^2} \stackrel{?}{>} \text{MAX_INITIAL_JUMP_M} = 5.0 \text{ m} \quad (12.53)$$

rejecting (not caching) the sample if so, logged loudly, leaving the source in the same “never received” state `_switch_to()` already refuses to select — no new failure mode, just an earlier one. 5 m is a bench choice (this campaign’s indoor spaces are a few metres across; a freshly-initialised VIO/VIO+wheel estimate has no legitimate reason to report a double-digit-metre displacement from its own origin) and is a first cut, not a derived bound — logged as open item (22): characterise against a real distribution of clean vs. divergent VINS inits rather than a single observed incident.

12.11 Field session, July 24: a blind reversal, a silent approach, and a deliberate simplification of reactive avoidance

Four independent gaps in the autonomy stack’s obstacle handling were found and closed the same day, triggered by two operator collision reports during live AUTO-mode testing; a fifth change, requested independently, adds a bounded speed increase to manual teleoperation. The session also

produced a public-facing hardware reference (§12.11.1) whose cross-validation exercise foreshadows the day’s central methodological theme: every one of the day’s software fixes was found by trusting a live measurement over an assumption the code had been carrying silently.

12.11.1 Robot documentation and a second manufacturer-vs-as-built divergence

A new page of the operator platform (`robot_description.html`) was built to host an interactive hardware reference: a WebGL viewer (`STLLoader/OrbitControls`) rendering the vendor CAD model directly in-browser, and a full transcription of FictionLab’s official Leo Rover 1.9 specification, fetched and verified against the manufacturer’s current documentation rather than carried forward from the platform’s original (2026-06) marketing copy.

A geometric cross-check. The CAD file (binary STL, 1,162,904 triangles, 58.1 MB) was parsed directly to recover its axis-aligned bounding box, giving an independent check of the vendor’s quoted envelope:

$$(\Delta x, \Delta y, \Delta z)_{\text{measured}} = (445.8, 424.6, 303.4) \text{ mm}, \quad (\Delta x, \Delta y, \Delta z)_{\text{vendor}} = (445, 424, 303) \text{ mm}, \quad (12.54)$$

a per-axis relative error of 0.18%, 0.14%, 0.13% — the model is the as-sold envelope to a fraction of a millimetre, not a generic placeholder mesh. Several other quoted figures were corrected in the same pass by the same discipline (trust the current source over the carried-forward one): unladen mass ($4.6 \rightarrow \approx 7$ kg), rated top speed ($\approx 1.0 \rightarrow \approx 0.4$ m/s — the platform’s software speed *ceiling*, `MAX_LIN_VEL_DEFAULT`= 1.0 m/s, had been carried into the specification card as if it were the physical top speed; it is merely the largest value the velocity-limit slider is permitted to reach, see §12.11.6), battery chemistry (Li-Ion 3S2P 6.8 Ah at 10.8 V nominal, not the previously listed 3S LiPo at 11.1 V), and chassis geometry (wheelbase 295 mm / track 354 mm, not the single, conflated 360 mm figure previously shown).

A second as-built divergence, same pattern as the wheels. One correction could not simply replace the old figure. FictionLab’s current specification lists a Raspberry Pi 5 (Broadcom

BCM2712, 4 GB LPDDR4X) as the 1.9-line’s onboard computer. Direct inspection of the deployed unit (`/proc/device-tree/model` and `free -h` over the existing SSH channel) instead reports a Raspberry Pi 4 Model B (Broadcom BCM2711, 8 GB LPDDR4) — a different board generation *and*, incidentally, a different RAM size than either the vendor page or this project’s own prior documentation. This is structurally the same situation as §12.4’s mecanum-wheel discovery: a manufacturer’s current default configuration does not describe a specific, already-deployed unit, and the only way to know which is true is to ask the hardware directly rather than the datasheet. Both facts are now recorded side by side on the new page rather than the newer one silently overwriting the measured one. The LeoCore controller board’s exposed interfaces were transcribed verbatim from the same official source (Table 12.5) for completeness, alongside the mecanum discovery’s own numbers (§12.4) already carried by the platform elsewhere.

Table 12.5: LeoCore controller board interfaces (FictionLab official specification, transcribed verbatim July 24). This project’s software and documentation refer to the same board as “CORE2” throughout; both names denote the same physical controller.

Interface	Specification
Power input	DC 2.1/5.5 mm barrel jack, 8–30 V DC, 10 A fast-blow fuse
Motor outputs	4× PWM H-bridge (A/B/C/D), ± 2.8 A peak per channel
LED output	5 V / 10 mA
Fan connector	5 V / 1 A
UART / GPIO port	3.3 V logic UART + GPIO breakout; also powers the Raspberry Pi

12.11.2 A mission-critical incident: Return-to-Base’s blind reversal

An operator report during live testing (“*it drives into obstacles*”) initiated a code audit of every branch inside `_drive()` that can command nonzero forward speed in AUTO mode. Cross-referencing that audit against the mission’s own event log (`auto_entries.json`) turned a plausible hypothesis into a demonstrated one.

The audit. RTB (§12.10.1) is explicitly exempted from the `cam_alive/depth_alive` gate that otherwise blocks all forward motion in AUTO the instant either sensor stream goes stale (§12.8.9) — by design, since RTB navigates by fused pose, not vision, and should not be held hostage to a WiFi-crossing image stream it does not need for localisation. But the same exemption was found to have a second, unintended consequence: nothing inside the RTB branch itself consulted `_obstacle_flag`, the corridor-depth signal, or any displacement-based stuck detector, at all. RTB could — and, the log below shows, did — reverse continuously at up to `RTB_LIN_SPEED = 0.20 m/s` over a route that can run to `RTB_TRAIL_MAX × RTB_TRAIL_SPACING ≈ 500 m`, with *zero* obstacle detection of any kind, sensor-live or not. This is materially different from the platform’s one other blind manoeuvre, `ESCAPE_BACK_LIN`: that reversal is deliberately bounded to 0.2 m over 2 s specifically *because* the D455 is forward-facing and cannot observe the rover’s own direction of travel while reversing (§12.8.9) — the same physical constraint applies to RTB, but RTB’s reversal was, before this session, unbounded in both duration and distance.

Table 12.6: RTB_START/RTB_COMPLETE pairs from `auto_entries.json`, in mission order, spanning the defect and its fix. Every attempt before the July 24 guard (§12.11.2) was abandoned without completing; every attempt after it either completed or was correctly interrupted by the new guard itself.

Timestamp	Event	Outcome
07-22 15:56:50	RTB_START (8.13 m)	abandoned, no completion logged
07-22 16:08:59	RTB_START (1.21 m)	abandoned, no completion logged
07-23 12:41:21	RTB_START (5.92 m)	abandoned, no completion logged
07-23 12:49:05	RTB_START (5.10 m)	abandoned, no completion logged
07-23 12:53:43	RTB_START (4.03 m)	abandoned, no completion logged
— <i>guard deployed (§12.11.2)</i> —		
07-24 (session)	RTB_START (47 waypoints, 10.94 m)	RTB_OBSTACLE: forward depth flag, correctly halted

Table 12.6’s first five rows are five consecutive RTB invocations, across two days, none of which

produced the RTB_COMPLETE log entry that a successful arrival always emits — each was silently followed by a fresh manual takeover and a new attempt. This is exactly the operational signature the audit predicted: a run that does not fail loudly, does not stop, and does not arrive, consistent with repeatedly reversing into something and being manually rescued rather than the software ever recognising a problem.

The fix: a two-tier defence in the one direction with no sensor coverage

Two independent guards were added, additive to the existing waypoint-driving logic and to each other, in order of increasing severity of response.

Tier 1 — a forward obstacle check, for free. `_obstacle_flag` is already computed continuously by the depth callback regardless of AUTO sub-state (§12.8.9); RTB previously just never read it. Reading it costs nothing new and adds partial coverage precisely where the constraint above does *not* bind absolutely: RTB’s steering correction changes the robot’s heading while reversing (Eq. 12.52), so the forward-facing camera does sweep across space the rover is not currently backing into, but may be about to swing toward as the correction continues. On a trip, RTB now performs a full, unconditional stop (`mode`→MANUEL, five repeated zero-velocity publishes, §12.8.9’s `_safe_stop()`) rather than the reactive pivot PATROL/GOTO_BEACON/SPIRAL use: RTB’s own design already forbids in-place rotation (§12.10.1, precision), so the two behaviours available to those other states — turn away, or push through slower — are both closed off, leaving *stop and hand back to the operator* as the only response that does not risk making a contact already suspected worse.

Tier 2 — an odometry-inferred stall detector, the only signal available in the blind direction. No sensor observes the space behind a reversing rover on this platform. The one indirect proxy available is motion itself: if a nonzero reverse velocity is commanded and the fused (or, per §12.10.1, wheel-odometry-fallback) position does not change, something is physically resisting the wheels. A reference position and timestamp are latched, then re-tested every tick:

$$(\hat{p}, \hat{t}) \leftarrow \begin{cases} (p[n], t[n]) & \text{if } \hat{p} \text{ undefined, or } \|p[n] - \hat{p}\| > d_{\text{stall}} \\ \hat{p}, \hat{t} & \text{otherwise,} \end{cases} \quad (12.55)$$

with $d_{\text{stall}} = \text{RTB_STALL_MIN_DISP_M} = 0.03 \text{ m}$ — the reference position resets every time the rover has genuinely moved by more than this margin, so the check is really “how long has it been since we last moved a meaningful amount”, not a single fixed window from RTB’s start. A trip fires when that “meaningful amount” has not occurred for too long:

$$t[n] - \hat{t} > T_{\text{stall}} = \text{RTB_STALL_WINDOW_S} = 2.0 \text{ s} \implies \text{stop}. \quad (12.56)$$

$d_{\text{stall}} = 3 \text{ cm}$ sits comfortably above the platform’s own measured positioning noise floor at rest ($\sigma_x = 7.5\text{--}8.8 \text{ mm}$, $\sigma_z = 49.7 \text{ mm}$ post-extrinsic-correction, Table 12.4) — a real stall reads as persistently below the threshold, not as noise crossing it in either direction — while remaining small next to the $\approx 6 \text{ cm}$ a healthy $\text{RTB_CRAWL_SPEED} = 0.03 \text{ m/s}$ reversal covers over the same 2.0 s window, so a genuinely moving rover clears the threshold with roughly $2\times$ margin at the platform’s *slowest* legitimate speed and considerably more at $\text{RTB_SLOW_SPEED}/\text{RTB_LIN_SPEED}$. On trip, the same unconditional stop as Tier 1 fires, logged and distinguished (RTB_STALL vs. RTB_OBSTACLE) in the mission autolog for post-hoc diagnosis.

A graduated response ahead of the hard stop

Both tiers above are binary: full speed until they trip, then an instantaneous stop — precisely the “bang-bang” shortfall §12.9.2 diagnosed and fixed for the *forward* direction in the July 22 session. A second operator request the same day (“*react a little before it happens*”) asked for the equivalent graduated behaviour here. The forward half of this is free: RTB’s commanded speed is now also scaled by the existing continuous obstacle governor, Eq. 12.45, unchanged and reused rather than re-derived, extending its coverage from *PATROL/SPIRAL* (§12.9.2) to a third locus.

The reverse half needed a new signal, since Eq. 12.44’s input (corridor depth) does not exist facing backward. The construction generalises Eqs. 12.55–12.56 from a binary trip into a continuous ratio, evaluated on a shorter rolling window $T_{\text{resist}} = 0.6 \text{ s} < T_{\text{stall}}$ so it reacts before the hard stop would fire:

$$\rho[k] = \frac{\|p(t_k) - p(t_{k-1})\| / (t_k - t_{k-1})}{|\dot{s}_{\text{cmd}}(t_{k-1})|}, \quad (12.57)$$

the ratio of *observed* reverse speed over the window to the speed *actually commanded* during it (not the nominal phase speed — Eq. 12.59 below feeds this loop’s own output back into next window’s denominator, so a partially-throttled window is judged against what it was really asked to do).

$\rho \approx 1$: driving freely. $\rho \rightarrow 0$: full resistance, the stall regime Eq. 12.56 will eventually catch if it persists. A piecewise-linear map converts the ratio into a speed scale, mirroring Eq. 12.45’s clamp-and-floor shape but built directly rather than as a PID output, since there is no natural “error” here to feed a controller — only a bounded ratio:

$$\sigma_{\text{resist}}(\rho) = \begin{cases} s_{\min} & \rho \leq \rho_{\text{lo}} & \rho_{\text{lo}} = 0.35 \\ s_{\min} + \frac{\rho - \rho_{\text{lo}}}{\rho_{\text{hi}} - \rho_{\text{lo}}} (1 - s_{\min}) & \rho_{\text{lo}} < \rho < \rho_{\text{hi}} & \rho_{\text{hi}} = 0.75 \\ 1 & \rho \geq \rho_{\text{hi}}, & s_{\min} = 0.35 \end{cases} \quad (12.58)$$

plotted in Figure 12.7. Both scales apply multiplicatively to the distance-based speed ramp already in place (§12.10.1):

$$\dot{s}_{\text{cmd}}[n] = \begin{cases} -\text{RTB_CRAWL_SPEED} \\ -\text{RTB_SLOW_SPEED} \\ -\text{RTB_LIN_SPEED} \end{cases} \times \sigma_{\text{resist}}(\rho[n]) \times \text{scale}[n] \Big|_{\text{Eq. 12.45}} \quad (12.59)$$

Neither scale can reach zero ($s_{\min} = 0.35$, Eq. 12.45’s own floor 0.3): the two hard trips (§12.11.2) remain the only path to an actual stop, by design — the graduated layer only makes the robot slow down earlier, exactly as §12.9.2 already established for the forward case, never gating or replacing the safety net beneath it.

12.11.3 A second, narrower gap: approach without a metric distance

Live telemetry captured during the same testing session (`corridor_mm: 345`, `detection.checker: false`, `map_landmark.valid: false` — an LED-only detection, no AprilTag, no confirmed checkerboard) pointed the audit at LOCK’s approach logic specifically. §12.9.1’s dual-confirmation approach state has two siblings, distinguished by whether a metric distance to the target is available (AprilTag range, or a checkerboard-derived estimate) or not (LED centroid only, the common case at range before the checkerboard comes into focus). The *first* sibling already carried an obstruction check predating this session — “*the target is reported beyond TARGET_STOP_M but depth sees something closer directly ahead: a contradiction, treated as an obstacle, not the beacon*” — inherited from the July 22 hardening that closed a near-identical gap in the same state (§12.9.2’s

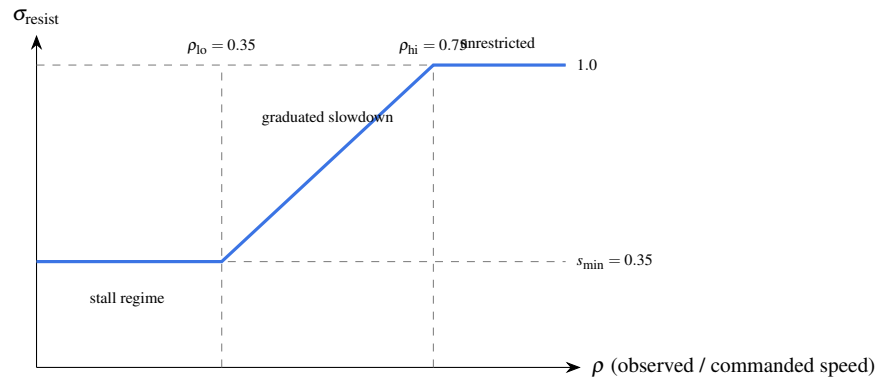


Figure 12.7: The RTB resistance governor, Eq. 12.58: a ratio of observed to commanded reverse speed below 0.35 floors the scale at $0.35 \times$ nominal (the regime Eq. 12.56’s hard stop will eventually trip if sustained past 2.0 s); above 0.75 the resistance is judged negligible odometric noise and no slowdown applies; between the two, linear interpolation.

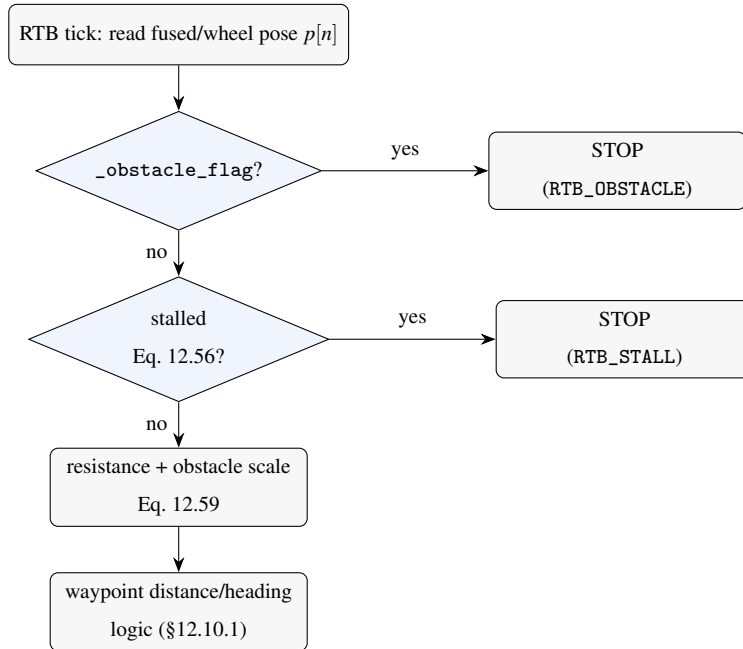


Figure 12.8: RTB’s per-tick decision order after §12.11.2–§12.11.2: both hard stops are evaluated before the graduated governor, and the graduated governor before the pre-existing waypoint logic — a trip at either diamond returns immediately, publishing zero velocity, without reaching the speed ramp beneath it.

motivating incident). The *second* sibling — reached whenever no metric distance exists at all, which is precisely when the robot has the least independent information about what lies ahead — carried neither that check nor Eq. 12.45’s graduated governor: it advanced at an unscaled ADVANCE_LIN until the corridor distance itself crossed a separate, hard-coded arrival threshold (700 mm), the literal bang-bang behaviour §12.9.2 had already been written to eliminate everywhere else.

Table 12.7: Every AUTO-mode branch that can command a nonzero forward speed, audited July 24. “Hard” = a discrete `_obstacle_flag` check gating the branch; “Graduated” = Eq. 12.45 applied to the commanded speed.

Branch	Hard (before)	Graduated (before)	July 24 change
PATROL (straight advance)	✓	✓	none (reference case)
LOCK, distance known	✓	—	graduated governor added
LOCK, <i>no</i> distance known	—	—	both added (the gap)
SPIRAL	✓	✓	none
GOTO_BEACON	✓	—	graduated governor added
RTB (reverse)	—	—	both added, §12.11.2
ESCAPE (reverse)	n/a (bounded, §12.11.2)	—	unchanged, deliberately

Table 12.7 was produced by `grep`-auditing every `lin = ADVANCE_LIN`-family assignment in `_drive()` against the two protections, rather than trusting memory of which branches had been touched by §12.9.2’s earlier pass — the same discipline as §12.11.1’s hardware check, applied to code instead of a datasheet. The fix mirrors the sibling branch exactly (obstruction → map the obstacle, enter AVOID, abandon the approach) with the graduated governor layered on top, bringing every forward-advancing branch in the FSM to the same two-tier standard RTB now also meets.

12.11.4 A statistical blind spot in direction selection: median versus percentile

A separate operator report (“*instead of avoiding the obstacle it gets closer to it*”) could not be explained by a sign error: the pivot direction convention (`_avoid_dir·UTURN_SPEED`, positive = counter-clockwise = left, matching the “open path” bearing convention already verified consistent at §12.8.9’s original definition) checks out on inspection, and no image flip exists anywhere in the depth pipeline (`image_transport/republish` decodes the compressed stream without any geometric transform). The actual weakness was statistical, in the two signals that *choose* which side is freer.

Why a median can be blind to a real obstacle. Both the left/right depth comparison (§12.8.9, the AVOID fallback when no open-path sector aims the pivot) and each open-path sector’s own value (§12.8.9) were computed as the *median* of valid depth pixels within their region. Model a region’s valid pixels as drawn from a two-population mixture: a fraction f at near range (an obstacle) and $(1 - f)$ at far range (open background), a reasonable idealisation of a thin object — a pole, a chair leg, a wall corner seen obliquely — that occupies only part of a half-image or a 10° sector. The q -th percentile of this mixture equals the *near* population’s value exactly when

$$q \leq 100f, \tag{12.60}$$

and the *far* (background) value otherwise. The median is $q = 50$: any obstacle occupying less than half the region ($f < 0.5$) is invisible to it entirely, the statistic reports “open” with no trace of a real, physically present hazard. This is not a corner case for a forward-facing depth sensor on a rover navigating a room: a doorway edge, a chair leg, or a wall met at a shallow angle are exactly the objects that occupy a *minority* of a half-image or a narrow sector while remaining fully capable of a collision. Both signals feeding this exposure directly *steer* the rover — the open-path bearing continuously, in patrol (before §12.11.5 removed that consumer), and the left/right comparison at every reactive pivot — so a median-masked obstacle was not merely undetected, it was actively steered toward, matching the report precisely.

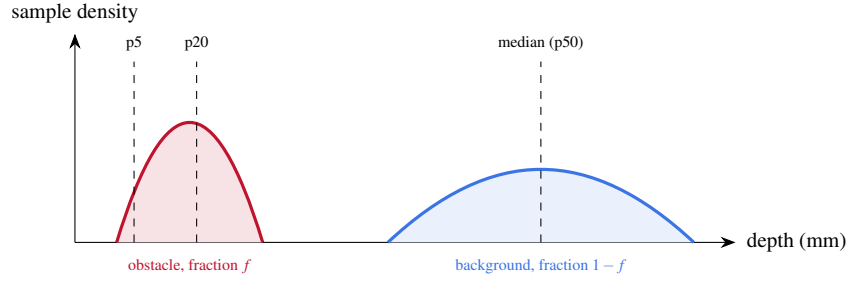


Figure 12.9: A region whose valid depth samples split between a near obstacle (fraction $f \approx 0.2$ here) and a far background. The median falls entirely inside the background population — the obstacle contributes zero signal to it (Eq. 12.60, $q = 50 > 100f$) — while the 20th percentile still reports the obstacle’s own range, and the hard-trip 5th percentile (§12.8.9) more so.

The fix, and where the threshold sits deliberately. Both computations were changed from `np.median` to the 20th percentile of the same valid-pixel population. Figure 12.9 frames the choice as a point on a single sensitivity spectrum already spanned by two other statistics already in use elsewhere in the same callback: the median ($q = 50$, per Eq. 12.60 blind to any $f < 0.5$ obstruction, the value being replaced) and the existing hard-trip corridor statistic ($q = 5$, §12.8.9, sensitive down to $f \geq 0.05$ but deliberately so — it *is* the collision trigger and must not be fooled by a sparse obstacle either). $q = 20$ sits between them by design: an obstacle occupying a fifth or more of the steering region can no longer hide from the direction *preference*, while the statistic remains substantially more tolerant of single-pixel noise and small dropout clusters than the hard trigger — appropriate for a signal that only ever *biases a choice*, several centimetres to metres away from the $q = 5$ statistic’s own binary stop.

12.11.5 A deliberate behavioural simplification: fixed, deterministic avoidance

A third, explicit operator request replaced the campaign’s most elaborate piece of reactive behaviour with its simplest possible correct alternative, and is recorded here as an intentional design choice, not a defect fix: *go straight down the corridor; on an obstacle, shift exactly 35° left; if still obstructed, shift another 35° left.*

What was removed. Two mechanisms built up over the campaign (§12.8.9’s “vacuum-robot” coverage strategy, §12.9.2’s continuous steering) are no longer consumed, by request: the *open-path attraction* that continuously biased patrol heading toward the deepest visible sector (Eq. 12.47, §12.9.2), and the AVOID pivot’s own choice of *aimed* bounce toward that sector or, failing that, a *random* bounce uniformly drawn from $[45^\circ, 135^\circ]$ toward whichever side read freer. Both consumers are removed at their call sites only; the underlying sensor computation (`_best_open_path()`), its continuously-updated depth sectors, and the tuned PID instance that drove Eq. 12.47) is left in place, dormant rather than deleted, should the coverage strategy be reinstated later.

What replaces it. The pivot direction is now a constant,

$$\text{_avoid_dir} \equiv +1 \quad (\text{left, by the same } +z \text{ CCW convention already established}), \quad (12.61)$$

and the pivot magnitude a fixed angle rather than a computed or drawn one,

$$\text{AVOID_TURN_RAD} = 35^\circ \quad (\text{was } 90^\circ, \text{ before that a } [45^\circ, 135^\circ] \text{ random draw}). \quad (12.62)$$

Patrol advance carries no steering term at all ($\text{ang} \equiv 0$ until an obstacle trips §12.8.9’s existing binary detector): the rover’s heading is now piecewise-constant, changing only in discrete 35° increments at a detected obstacle, never continuously. Because the same reactive cycle re-enters `_enter_avoid()` on any renewed detection, an obstacle still present after one shift produces another, identical, cumulative shift:

$$\theta_n = n \times 35^\circ, \quad n = 1, 2, 3, \dots \quad (12.63)$$

until either the corridor clears or the pre-existing attempt counter (§12.8.9, `AVOID_MAX_TRIES=3`) is exceeded, at which point the unrelated historical fallback — a full 180° U-turn — takes over unchanged. At the boundary, $n=3$ consecutive 35° shifts accumulate to 105° of net heading change before that fallback can engage (Eq. 12.63), which is worth stating plainly as the direct trade-off of a fixed, unconditionally-left policy: a corridor obstructed specifically on the *left* forces the rover through up to three 35° turns toward the very side that is blocked before the U-turn fallback rescues it, where the discarded “freer-side” comparison (§12.11.4) would previously have picked the open side immediately. The operator’s stated priority — a pivot whose direction and magnitude are

always predictable and always easy to reason about from a log line, over the discarded strategy’s ergodic room-coverage benefit (§12.8.9) — is recorded here as the explicit rationale for accepting that trade-off, and the scenario it is most exposed in (a left-blocked dead end) is logged as a new open item (Table 12.12, item 23) pending a live trial.

12.11.6 Manual boost: a bounded control-authority addition

An unrelated operator request — faster manual driving — exposed a small architectural fact about teleoperation that had not previously been made explicit anywhere in the platform: two independent layers already governed manual speed, and only one of them was ever exposed to the operator.

$$\text{lin}_{\text{published}} = \text{clamp}(\text{lin}_{\text{step}}, -v_{\text{max}}, v_{\text{max}}) \quad (12.64)$$

where $\text{lin}_{\text{step}} \in \{-\text{LIN}, 0, +\text{LIN}\}$ is the fixed magnitude the D-pad/arrow-key controller has always sent ($\text{LIN} = 0.2 \text{ m/s}$, $\text{ANG} = 0.6 \text{ rad/s}$, unconditionally, on every keypress) and v_{max} is the operator-configurable ceiling (`linVelSlider/angVelSlider → /leo_rover/config/velocity → self.max_lin_vel`, clamped in `_publish()`, the platform’s single command exit point, §12.8.9). Because $\text{lin}_{\text{step}} = 0.2 \text{ m/s}$ already sits well inside the ceiling’s default range ($[0.1, 1.0] \text{ m/s}$), raising the ceiling slider changed nothing about how fast the D-pad actually drove: the clamp in Eq. 12.64 was already loose. A “boost mode” therefore had to change lin_{step} itself, not the ceiling.

A toggle (button or the B key; state persists rather than requiring a held modifier, so it works identically on a touch interface) switches the step magnitude between the original values and

$$\text{BOOST_LIN} = 0.4 \text{ m/s}, \quad \text{BOOST_ANG} = 0.9 \text{ rad/s}. \quad (12.65)$$

BOOST_LIN is not an arbitrary multiple: it is exactly the platform’s official rated top speed (§12.11.1), so boost mode lets an operator reach, but not exceed, the manufacturer’s own performance envelope. BOOST_ANG is deliberately *not* the same $2\times$ multiple: §12.5.2’s ramp characterisation already measured 92–99% commanded-to-actual yaw authority across the full $0.15\text{--}0.40 \text{ rad/s}$ range at healthy battery voltage, with no evidence offered above 0.40 rad/s — commanding meaningfully more than that would very plausibly saturate the actuator without producing a proportionally faster real turn, so the angular boost was set only modestly above the existing $\text{ANG} = 0.6 \text{ rad/s}$ rather

than doubled to match the linear boost, pending a dedicated ramp measurement at boost-level commands specifically (Table 12.12, item 26). Eq. 12.64’s outer clamp is untouched and still applies after boost, so an operator who has deliberately lowered the velocity ceiling for caution retains that ceiling exactly as before — boost can only use headroom the ceiling already permits, never exceed it.

The same evening, three further operator requests moved outside collision safety entirely: a report that the pose-source switch had stopped working, a request to close a spatial gap in beacon confirmation and extend detection out to 4 m, and a full audit of the platform’s `tf2_ros` usage against REP 105. Each is recorded below in the same spirit as the rest of this session: what was found, what changed, and — where a fix rests on reasoning rather than a field measurement — what remains open.

12.11.7 Pose-source switch: reconfirmed, then reopened on operator request

A report that MINS/VINS switching had stopped working traced to two independent, unrelated causes in sequence, each resolved by direct verification rather than by assumption. *First*, a full network outage: the WireGuard tunnel (§12.8.4, 10.200.0.1/.2), the parallel WiFi path and the leo-guest SSH alias were all simultaneously unreachable, while the operator PC’s own WiFi association and internet access stayed healthy throughout — isolating the fault to the rover itself being off-network, not to WireGuard specifically or to the PC. Resolved operator-side by restarting the stack. *Second*, after reconnection, the reported symptom persisted with a specific, reproducible service response: *“cannot switch to VINS: no message received on it yet”* — which is not a fault but §12.10.2’s own July 23 contract operating exactly as simplified: `_switch_to()` attempts a switch immediately and fails outright, by design, if the target source has never published. §12.3.5 already establishes why that condition is easy to hit here specifically: OpenVINS needs a forward-back jolt to render scale observable, so a rover sitting stationary after a restart can leave VINS silent indefinitely regardless of network health. The two causes present identically from the operator’s seat (“the switch doesn’t work”) but are unrelated in the code, which is worth recording precisely so the same report is not re-diagnosed from scratch next time.

Confirmed once the operator drove manually to give VINS motion: direct `rosservice call` tests, this time sequenced strictly (topic read → service call → topic read, rather than interleaved

with other dashboard activity) reproduced clean, consistent results in both directions — “*already on MINS*” matching the topic before any call, “*switched to VINS*” once motion had occurred, matching after. An earlier, apparently contradictory read during the same diagnosis (a stale topic value momentarily disagreeing with a fresh service response) did not reproduce under the strict sequencing and is attributed to ordinary timing skew against concurrent operator interaction, not a race in `pose_selector.py` — §12.8.1’s registered-node race was a reminder to check, not evidence one exists here. One diagnostic side-effect was disclosed to the operator as it happened: a test call left the rover switched onto VINS mid-session.

Reopened. Having the mechanism confirmed working as designed did not settle the matter: the operator’s actual objection was to the design itself — refusing outright, with no path back to trying again automatically, is exactly the behaviour §12.10.2 chose after reverting the alternative. Told plainly that the alternative had already been tried and reverted once for being hard to read (“too many implicit states”), the operator asked for it back anyway, judging that trade-off differently this time. `_switch_to()` again supports arming: a click on a source with no data yet arms it rather than refusing, and `_on_odom()` applies the switch automatically the instant that source’s first message arrives, with a second click cancelling the arm. The difference from the July 23 version is deliberately not in the mechanism but in its legibility: a dedicated latched topic, `/leo_navigation/pose_source_pending`, is the single source of truth for the armed target (empty when nothing is armed), never inferred from timing or combined with any other signal; the service response and the relayed cockpit log line always name the exact outcome (“*ARMED: will switch to ...*”, “*switched to ...*”, “*armed switch to ... cancelled*”, “*already on ...*”) rather than a single generic “pose source → X” that could describe either an arm or a completed switch; and the cockpit button for the armed source carries a persistent text label, not only a colour or a blinking border, addressing the specific July 23 complaint that a state change communicated by animation alone is easy to miss or misread.

A restart hazard, found by triggering it. Deploying this change required restarting `pose_selector` live, and doing so exposed a real, previously-unnoticed defect along the way: `navigation_supervision.launch`’s `default_source` param was hardcoded to “VINS”, directly contra-

dicting the launch file’s own adjacent comment (“*OpenVINS est l’estimateur de SECOURS (pose_source=MINS en permanence)*”) and §12.3.5’s documented design intent. Because `self.active` is reset to this default on every process start with no regard for which source was actually healthy beforehand, the restart silently dropped `/robot_pose_fused` to zero messages — confirmed live (`rostopic hz` showed *no new messages* for six seconds immediately after) — even though MINS had been publishing at 135 Hz one command earlier. Caught immediately by checking the topic rather than assuming the restart was clean, and corrected in two steps: an explicit service call restored MINS as the active source within seconds, and the launch default was changed to "MINS" to match both the neighbouring comment and §12.3.5. The launch fix does not yet apply to the live session — the parameter server still holds the value pushed at the stack’s last full launch until that launch is repeated — logged precisely as such (Table 12.12, item 32). The arm/cancel round trip was verified live by direct service call in both directions; the automatic switch on a source’s first publish was verified by code inspection only (the same path §12.10.2 already exercised once, now re-entered), not yet by an actual drive-and-observe trial, logged as a further open item (item 31).

12.11.8 Chasing VINS’s silent odometry: two real fixes in `ov_msckf`, and a log-attribution lesson

Confirming item 31 above led into `ov_msckf` itself. `/ov_msckf/odomimu` published nothing over a sustained, directly-subscribed 18 s window, despite a correctly registered subscriber (`pose_selector`, confirmed via `rostopic info`) and healthy inputs (`/imu/data_clean` at 84 Hz, `/pc/camera/infrared_image_rect_raw` at 15 Hz) — the two facts that made this worth investigating rather than re-attributing to §12.3.5’s already-documented jerk requirement outright.

Two genuine defects found and fixed. `ROS1Visualizer::visualize_odometry()` (`ov_msckf/src/ros/ROS1Visualizer.cpp`) gates its publish on `VioManager::initialized()`, which requires *both* `is_initialized_vio` *and* `timelastupdate != -1` (`ov_msckf/src/core/VioManager.h`). The second condition is set only inside a completed `track_image_and_update()` pass — which a successfully-applied zero-velocity update bypasses via an early return (`VioManager.cpp`, the `did_zupt_update` branch) — so a platform whose ZUPT succeeds before any full update ever completes can leave `initialized()` permanently false even once the filter has genuinely con-

verged, despite `fast_state_propagate()` (what `visualize_odometry()` actually needs) requiring only `is_initialized_vio`. A new accessor, `VioManager::vio_alive()`, returns `is_initialized_vio` alone and is now what `visualize_odometry()` checks; `initialized()` itself and its other consumers (feature and track visualisation, which do need a completed pass) are untouched. Investigating *why* this still did not explain the observed silence surfaced a second, independent defect: `is_initialized_vio` was a plain `bool`, written from a per-frame worker thread (`ROS1Visualizer` runs a multi-threaded `ros::AsyncSpinner`, dispatching each camera callback to its own `std::thread`) and read from the IMU-callback thread via the new `vio_alive()` — an unsynchronised cross-thread read with no guaranteed visibility. Declared `std::atomic<bool>` instead (`VioManager.h`); every other read of the same flag is a plain boolean-context check or assignment and needed no further change.

A log-attribution mistake, corrected before it misled the fix. Both changes were rebuilt and tested live, and the shared process log (`logs/navigation_master.log`, written to by every node on this machine) kept showing `UpdaterCamera::try_update` and `[IW-Init]:static_initialization/Total initialization` time lines throughout — read, initially, as `ov_msckf` itself actively initialising and processing camera frames, which would have meant the atomic fix above had *not* resolved the symptom. It had not been given the chance to be tested: neither log line originates in `open_vins` at all. Both classes exist, with the same names and an almost identical print style, inside MINS's own, separately-implemented source (`mins_ws/src/MINS/mins/src/update/cam/UpdaterCamera.cpp` and `mins_ws/src/MINS/mins/src/init/imu_wheel/IW_Initializer.cpp`) — confirmed by grepping each package's own source tree for the exact strings rather than trusting the interleaved shared log, after which zero matches turned up anywhere under `open_vins/`. MINS was simply operating normally throughout, at its usual output, on the same log file. A clean, targeted instrumentation pass (a temporary print of `vio_alive()`'s own return value, inside `visualize_odometry()` itself, removed once its purpose was served) showed it reading `false` continuously and honestly: `ov_msckf` had not initialised at all, exactly as §12.3.5 already explains, and exactly as this session's very first exchange with the operator had already stated before this investigation began. This rover's `ov_msckf` configuration disables dynamic initialisation and requires a genuine accelerometer jerk to trigger the jerk-based path it does use (`catkin_ws/src/open_vins/config/leo/estimator_`

config.yaml:48, init_dyn_use: false; line 43, init_imu_thresh: 0.4) — a stationary platform simply never produces one, no matter how long it runs or how many times its process is restarted.

Net result. The two `ov_msckf` fixes are real, independently-justified hardening — a genuine latent gap in `initialized()`’s ZUPT interaction, and a genuine unsynchronised cross-thread read — and are kept. Neither caused, and neither on its own fixes, tonight’s reported symptom: that remains exactly the jerk requirement §12.3.5 already documented, unchanged by anything in this subsection. The practical lesson for this specific codebase is worth stating plainly for future debugging sessions: `logs/ navigation_master.log` is shared across every node, MINS and VINS include classes named closely enough alike (`UpdaterCamera`, an `IW_Initializer`-style static initialiser) that their log lines are not distinguishable by eye, and attributing a log line to a process requires checking which package’s source actually contains that exact string — not just which process seems contextually likely.

Table 12.8: Symptoms observed in the shared log during this investigation, and their actual origin once checked against each package’s own source.

Log text	Looked like	Actually from
<code>[IW-Init]:static_initialization, Total initialization time</code>	<code>ov_msckf</code> static/IMU-wheel init succeeding in ~ 1.4 s	MINS’s own <code>IW_Initializer.cpp</code>
<code>UpdaterCamera::try_update: POOL/SLAM/INIT/MSCKF</code>	<code>ov_msckf</code> actively processing camera frames post-init	MINS’s own <code>UpdaterCamera.cpp</code>

12.11.9 Dual-confirmation beacon reset: closing a spatial blind spot

§12.9.1’s dual requirement — the LED cluster *and* the checkerboard both confirmed before a reset fires — never required the two cues to be in the *same part of the frame*. The gap is concrete: LED_MAX_Y_FRAC rejects a candidate LED blob by image height alone (above 35 % of the frame), not by proximity to the checkerboard, so a low reflection that clears that floor-plausibility gate in one part of the image, coincident with a real patterned surface (a poster, a doorframe) independently satisfying the checkerboard detector elsewhere in the *same* frame, would previously confirm a beacon that was never physically there — both conditions individually true, about two different objects. `_dual_beacon_confirmed()` (`leo_backend.py:3036`) now computes the checkerboard bounding-box centre and width alongside the LED cluster centre, and withholds confirmation whenever their separation exceeds a bound expressed as a *fraction* of the checkerboard’s own apparent width, `BEACON_SPATIAL_MAX_FRAC= 1.5`, rather than a fixed pixel count.

The fraction, not an absolute pixel count, is the load-bearing design choice, and follows the same pinhole scaling argument this report already uses for the AprilTag range ceiling (§12.5.5). The beacon’s physical LED-to-checkerboard-centre offset s_{gap} and its physical checkerboard width s_{cb} are both fixed by construction; under pinhole projection at focal length f_x and distance d , their apparent pixel counterparts are $g(d) = f_x s_{\text{gap}}/d$ and $w(d) = f_x s_{\text{cb}}/d$, so

$$\frac{g(d)}{w(d)} = \frac{f_x s_{\text{gap}}/d}{f_x s_{\text{cb}}/d} = \frac{s_{\text{gap}}}{s_{\text{cb}}}, \quad \text{independent of both } d \text{ and } f_x. \quad (12.66)$$

A fixed pixel threshold has no such invariance: it would be loose enough at close range to admit a disjoint false positive (where $w(d)$ is large) and tight enough at long range to reject the genuine beacon’s own gap (where $w(d)$ is small). Expressing the bound as a multiple of $w(d)$ inherits the distance-invariance of Eq. 12.66 by construction. `BEACON_SPATIAL_MAX_FRAC= 1.5` is set conservatively above the beacon’s expected physical ratio, not against a direct measurement of its actual $s_{\text{gap}}/s_{\text{cb}}$, logged as a new open item (Table 12.12, item 28). One coverage limit is stated explicitly rather than left implicit: the check validates LED against checkerboard only, never against the AprilTag, which is acceptable because the reset only ever fires at close range (§12.5.5: the checkerboard is useless beyond 1.5 m, so any LOCK capable of triggering a reset is already inside that range), where the checkerboard is the authoritative cue regardless.

12.11.10 Extending the detection envelope to 0.5–4 m: an absolute-area gate against a $1/d^2$ law

An operator requirement — smooth, reliable detection from 0.5 to 4 m — was traced to a single bottleneck, deliberately isolated from the two detectors already known to be adequate across that range: the AprilTag channel (§12.5.5, decodable to ≈ 4.3 m, `tag_decimate: 1.0`, untouched) and the checkerboard, whose sub- ~ 1.5 m ceiling is an accepted optical limit, not a bug, also untouched. The actual bottleneck was the LED blob detector’s absolute-pixel area gate, `LED_MIN_AREA/LED_MAX_AREA`, evaluated against the same $1/d^2$ law already used above. A physical LED of diameter D_{LED} projects, under the pinhole model, to an apparent pixel diameter $\varphi(d) = f_x D_{\text{LED}}/d$ and therefore an apparent blob area

$$A(d) = \frac{\pi}{4} \varphi(d)^2 = \frac{\pi}{4} f_x^2 D_{\text{LED}}^2 \frac{1}{d^2} \propto \frac{1}{d^2}. \quad (12.67)$$

Over the requested range $[d_{\min}, d_{\max}] = [0.5, 4.0]$ m — an $8\times$ spread in distance — the apparent area itself varies by

$$\frac{A(d_{\min})}{A(d_{\max})} = \left(\frac{d_{\max}}{d_{\min}} \right)^2 = 8^2 = 64. \quad (12.68)$$

A single fixed window `[LED_MIN_AREA, LED_MAX_AREA]` can only pass the *same* physical LED at both ends of the range if its own dynamic range spans at least this ratio, `LED_MAX_AREA/LED_MIN_AREA` ≥ 64 . The prior window (10, 1200) already satisfied that in the abstract ($120\times$) — the actual failure was positional, not proportional: its floor of 10 px^2 was calibrated against a real image captured at 1 m (§12.5.5), anchoring the window around that single reference distance rather than around the requested range’s far end. At 4 m the same LED’s apparent area is $(4/1)^2 = 16\times$ smaller than at 1 m (Eq. 12.67), enough to fall under a floor sized for the near end alone unless the 1 m blob happened to sit comfortably above $16 \times 10 = 160 \text{ px}^2$ — not something the prior calibration guaranteed. The window was widened to (3, 2500), ratio ≈ 833 , well clear of the $64\times$ floor derived in Eq. 12.68, with the noise-rejection load the area floor can no longer carry alone shifted onto `LED_MIN_CIRC`, the density clustering pass, and the spatial cross-check of §12.11.9.

The one other area-adjacent constant, `LED_CLUSTER_PX` (the whole four-LED cluster’s maximum pixel spread, not a single blob’s area), needed no change, and this is checked rather than assumed: with the beacon’s measured cluster width `BEACON_WIDTH_M` = 0.165 m and `CAM_FX` =

384.65 px,

$$w(d) = \text{CAM_FX} \cdot \text{BEACON_WIDTH_M} / d, \quad w(0.5 \text{ m}) \approx 126.9 \text{ px}, \quad w(4.0 \text{ m}) \approx 15.9 \text{ px}, \quad (12.69)$$

both comfortably inside `LED_CLUSTER_PX = 280 px` (margins of $2.2\times$ at the near end and $17.6\times$ at the far end), confirming this constant was never the range bottleneck. The new area bounds are, by the code’s own record, a first cut reasoned from Eq. 12.68 and the single 1 m calibration point, not yet verified against a real image captured at 4 m — logged as a new open item (Table 12.12, item 27), to the same standard the rest of this registry already holds unmeasured thresholds to.

12.11.11 A fuller `tf2_ros` audit: one further legacy dependency, and an architecture note on the missing map frame

Having ported MINS off legacy `tf` (§12.9.4), an operator-requested audit today covered every remaining transform in the stack against `tf2_ros` usage and REP 105’s `map→odom→base_link` convention. The audit starts from a terminology correction, stated plainly because the request that prompted it assumed otherwise: there is no EKF fusing MINS and VINS anywhere in this code-base. `pose_selector.py` is, as §12.10.2 already documents, a discrete source *switcher* that applies a one-time $\text{SE}(3)$ correction only at switch instants (the same correction §12.10.3 relies on to absorb a divergent re-initialisation) — never a continuous filter blending both estimators’ output. That switcher is, on inspection, the compliant half of this audit: it and `carolus_vicon_bridge.py` both build correctly on `tf2_ros.Buffer`, `TransformListener` and `TransformBroadcaster` (`pose_selector.py:130–132`, `carolus_vicon_bridge.py:49–50`), and it is `pose_selector.py` that actually owns and correctly publishes REP 105’s `odom→base_link` segment (`_broadcast_tf()`, composing the fused pose with a static `base_link↔imu_frame` lookup via `self._tf_buffer.lookup_transform()`).

Two gaps were found. *First*, VINS (`ov_msckf`) still builds against the deprecated ROS1 `tf` package — `ROS1Visualizer.h:42,147`, `ROS1Visualizer.cpp:42,334,343, 770,774,776` and `ROSVisualizerHelper.h:31,73` all reference `tf::TransformBroadcaster`, `tf::StampedTransform`, `tf::Quaternion` or `tf::Vector3` — the exact class of dependency MINS carried until §12.9.4’s port, including the same mitigating fact that made the MINS case low-urgency before it was fixed:

its TF output is isolated to `/tf_vins` (`navigation_supervision.launch:90`), the same pattern already used for `/tf_mins` (`mins.launch:54`), so this legacy dependency cannot corrupt the shared tree today. A `tf2_ros`-native `ROS2Visualizer` already exists in the same upstream repository, unbuilt on this ROS1/Noetic platform — precisely MINS’s situation before its own port — so the remedy is the same field-for-field mirroring exercise already proven once, logged as a new open item (Table 12.12, item 29) rather than applied here.

Second, and more structural: no map frame and no map→odom transform exists anywhere in the shared `/tf` — confirmed by search across the workspace, not merely unobserved. What REP 105 asks for is a three-frame chain; what exists is only its odom→base_link half (the compliant finding above). The map layer that the mission’s own world-referenced concepts already depend on — Return-to-Base targets, obstacle and beacon markers, LOCK’s target-relative approach — is instead carried by a parallel, non-TF state in `leo_backend.py`: `world_origin_x`, `world_origin_y` and `world_heading`, composed by `_get_world_pos()` (`leo_backend.py:2174-2182`),

$$\mathbf{p}_{\text{world}} = \mathbf{R}(\theta_w) \mathbf{p}_{\text{local}} + \mathbf{t}_w, \quad \mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (12.70)$$

with $\theta_w = \text{world_heading}$, $\mathbf{t}_w = (\text{world_origin_x}, \text{world_origin_y})$ and $\mathbf{p}_{\text{local}} = \text{self.pose}[0:2]$ — the local odometry pose already carried elsewhere in this chapter. Eq. 12.70 is, mathematically, exactly the rigid-body transform a map→odom broadcast would encode: a single SE(2) pose, recomputed at the same instants a correction would need to be re-published. It is instantiated as Python instance attributes rather than on the TF tree, so nothing else in ROS — RViz, another node, a future `tf2_ros`.Buffer consumer — can see or use it. The re-anchoring is concrete: `reset_coords()` and `reset_coords_local()` (`leo_backend.py:3704-3732`) both recompute $(\mathbf{t}_w, \theta_w)$ via `_get_world_pos()` *before* reassigning `world_origin_x/y` — operationally a fresh map→odom correction at exactly the beacon-reset instant, published nowhere a TF consumer could observe it.

One nuance is worth being precise about, since it bears directly on the audit’s own framing: this does *not* mean a beacon reset “teleports” base_link. `reset_coords_local()` also calls the `firmware/reset_odometry` service (`leo_backend.py: 1426-1427`), but that only resets the firmware’s own dead-reckoning integrator; MINS’s actual filter input is wheel *velocity*, not position

(wheel_remap.py, republishing JointState.velocity as the front/rear side-mean per wheel, catkin_ws/.../ wheel_remap.py:81-84), decoupled from that integrator. base_link’s real, TF-published pose (the compliant finding above) is therefore undisturbed by a beacon reset; only the parallel Python “world” state consumed by RTB and mapping is re-anchored. Figure 12.10 summarises the finding: a compliant odom→base_link segment, two isolated visualisation-only branches (one still legacy tf), and a map layer that exists — correctly, in the SE(2) sense of Eq. 12.70 — but nowhere on the TF tree itself.

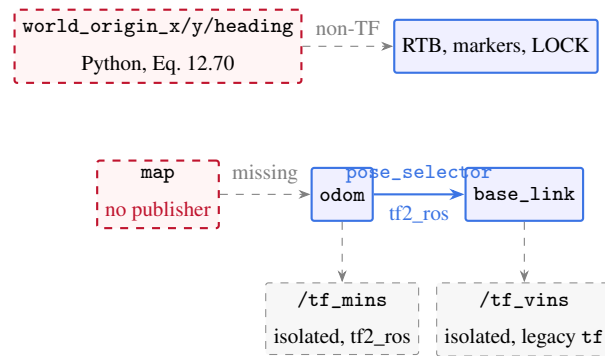


Figure 12.10: TF-tree audit, July 24: the compliant odom→ base_link segment (solid, pose_selector, tf2_ros) against the two isolated visualisation branches and the absent map frame, whose role is instead filled by a parallel, non-TF Python state (top) consumed directly by world-referenced mission code.

A correction is sketched, not applied to the live robot without a dedicated validation pass: publish an actual map frame via a tf2_ros.TransformBroadcaster at exactly the reset_coords()/reset_coords_local() re-anchor points, and migrate _get_world_pos()’s callers to tf2_ros.Buffer.lookup_transform("map", "base_link", ...) in its place:

```
1 def _publish_map_correction(self, wx, wy, heading):
2     t = TransformStamped()
3     t.header.stamp, t.header.frame_id = rospy.Time.now(), "map"
4     t.child_frame_id = "odom"
5     t.transform.translation.x, t.transform.translation.y = wx, wy
6     q = tft.quaternion_from_euler(0, 0, heading)
7     (t.transform.rotation.x, t.transform.rotation.y,
8      t.transform.rotation.z, t.transform.rotation.w) = q
9     self._map_broadcaster.sendTransform(t)    # tf2_ros.
```

```

    TransformBroadcaster
10
11 def _get_world_pos(self):
12     try:
13         tr = self._tf_buffer.lookup_transform("map", "base_link", rospy.
            Time(0))
14         return tr.transform.translation.x, tr.transform.translation.y
15     except (tf2_ros.LookupException, tf2_ros.ConnectivityException,
16             tf2_ros.ExtrapolationException):
17         return self._fallback_local_pos() # existing wheel-odometry
    path

```

Listing 12.1: Sketch: replacing the hand-rolled world frame with a published map transform.

This is deliberately not implemented here: `world_origin_*` and `_get_world_pos()` are referenced across RTB, obstacle/beacon mapping and LOCK, a nontrivial refactor on a currently stable robot, logged as a new open item (Table 12.12, item 30) pending the operator’s decision to proceed.

12.12 Field session, July 27: an offline trajectory-comparison pipeline and cockpit instrumentation

A supervision meeting set three linked objectives: a reproducible way to *quantify* MINS against openVINS off-line (not just watch them live), one-click access to that data from the cockpit, and richer real-time visibility into the Carolus beacon channel. None of this changes an estimator or a control loop — it is instrumentation and data plumbing laid on top of the existing, now-stable stack — but it is the scaffolding the precision roadmap (Table 12.12) has been waiting on: every placeholder threshold in that registry is ultimately settled by a measured trajectory, and until this session there was no packaged path from a drive to a number.

12.12.1 A rosbag → CSV → Matlab pipeline, and an honest choice of metric

Three small tools form the chain. `tools/record_trajectories.sh` records the two raw estimator topics (`/mins/imu/odom`, `/ov_msckf/odomimu`) alongside the served fused pose, the active-

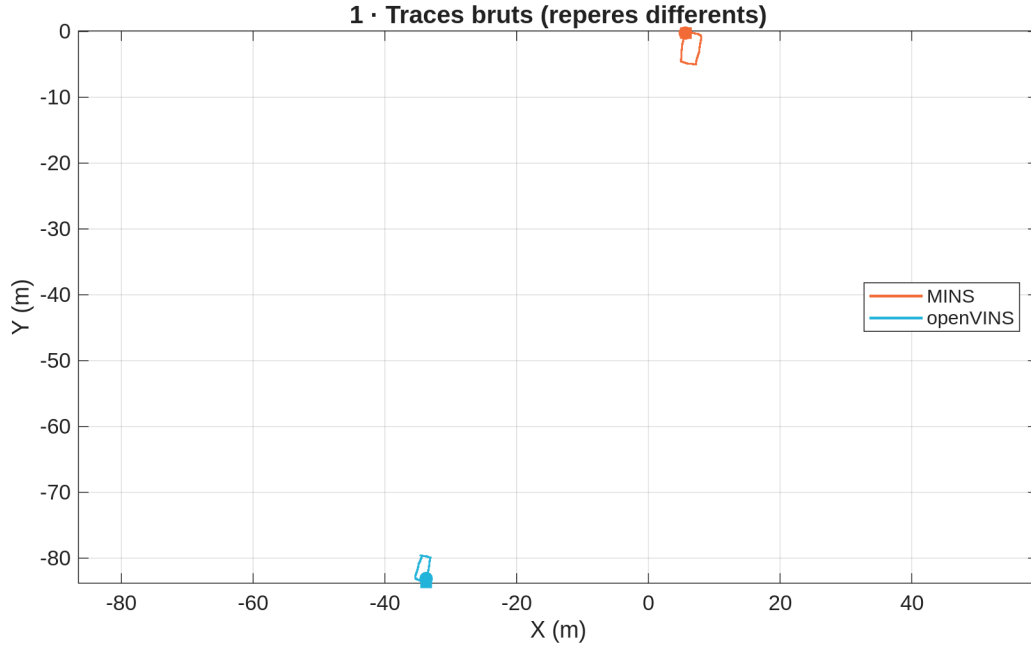
source timeline, the Carolus global fix and raw wheel odometry, into a timestamped LZ4 bag. `tools/bag_to_csv.py` converts that bag into one CSV per source (columns `t, x, y, z, qx, qy, qz, qw, yaw_rad, t` relative to the first message so every trace shares an origin), the portable pivot format that needs no Matlab ROS Toolbox. `tools/plot_trajectories.m` overlays the planar trajectories and computes the comparison.

The metric choice is the substantive part, and it is deliberately not framed as “accuracy.” This platform carries no absolute ground truth, so a raw difference between MINS and VINS is a *mutual divergence*, not an error against truth. Two readings are therefore computed and never conflated. The first, always available, is the inter-estimator divergence after an optimal rigid 2D alignment (Kabsch/Umeyama, no scale, since both estimators are metric): each trajectory is resampled onto a common 20 Hz time base by interpolation, VINS is rotated and translated onto MINS by the closed-form minimiser of

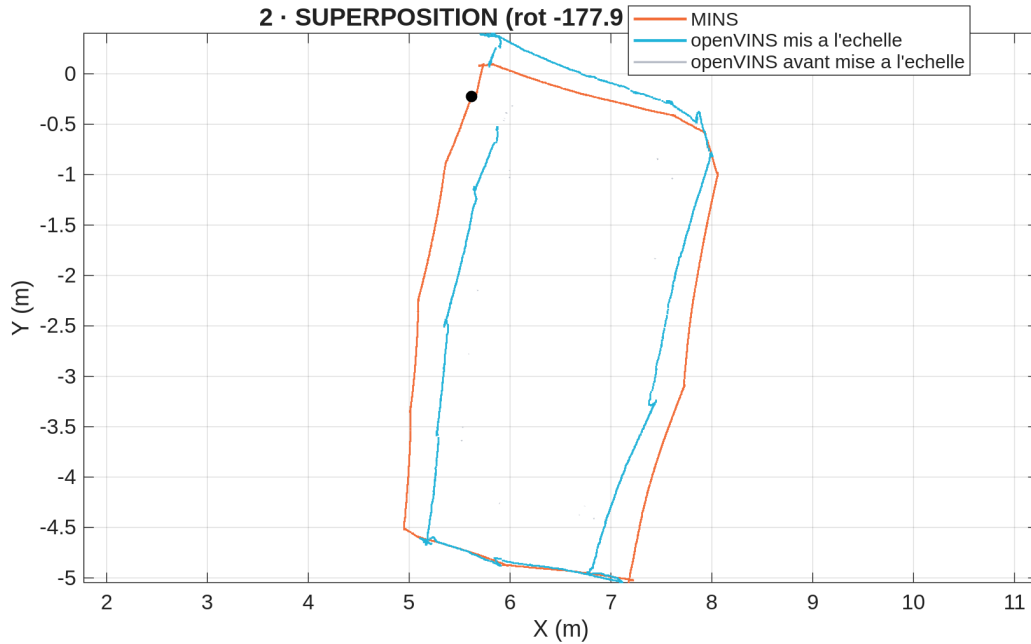
$$\min_{R \in SO(2), \mathbf{t} \in \mathbb{R}^2} \sum_k \left\| \mathbf{p}_k^{\text{MINS}} - (R \mathbf{p}_k^{\text{VINS}} + \mathbf{t}) \right\|^2, \quad (12.71)$$

and the residual distance $\|\cdot\|_k$ is reported as RMS, max and final divergence, both before and after alignment. Alignment matters because the two estimators start from independent origins with independent initial headings; the pre-alignment number measures raw served-frame disagreement, the post-alignment number measures whether the two *shapes* agree, which is the fairer statement of relative consistency. The second reading, computed only when a carolus CSV is present, treats the drift-free Carolus global fix (§12.12.3) as a near-ground-truth anchor and reports each estimator’s position error against it separately, the one context in which the word “error” is used honestly. Both figures are stamped directly onto the exported plot so a reader cannot mistake one for the other.

The pipeline was exercised end to end this session on a stationary bench capture (MINS publishing at rest, VINS not yet initialised for want of the jerk of §12.3.5, no beacon in view): the recorder buffered and exported a well-formed MINS trace and the Matlab reader consumed it, but a *substantive* MINS-versus-VINS figure requires a real driven trajectory with VINS initialised, and is logged as the first thing to capture next (Table 12.12, item 34). Figure 12.11 holds that slot.

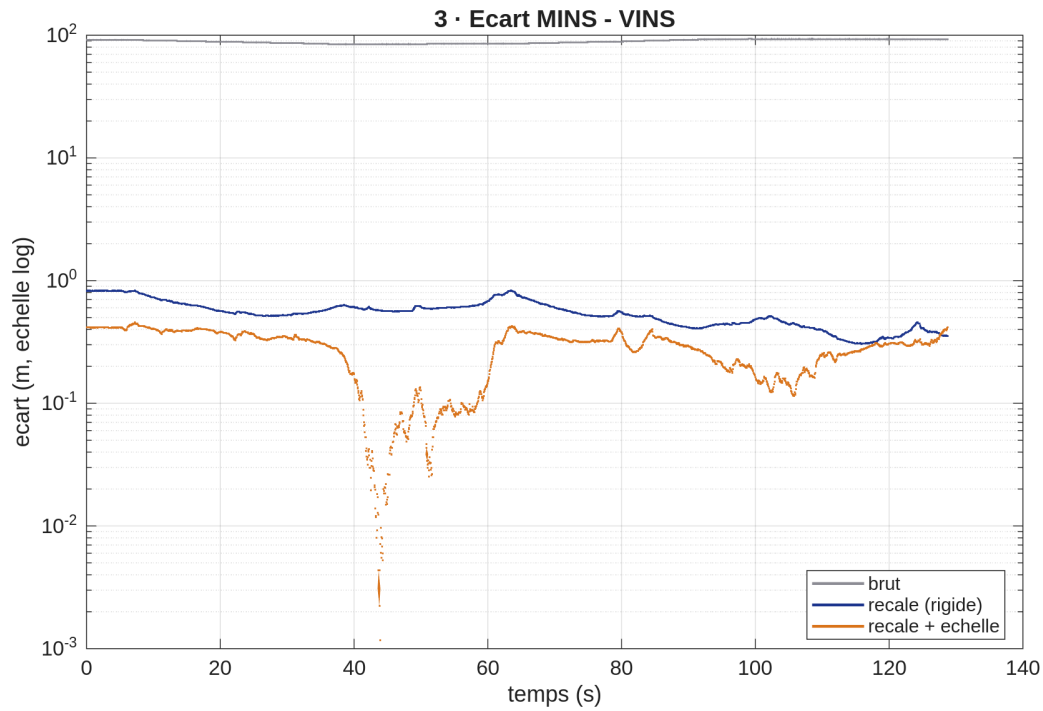


(a) Raw traces: no shared frame, hence no relation visible.

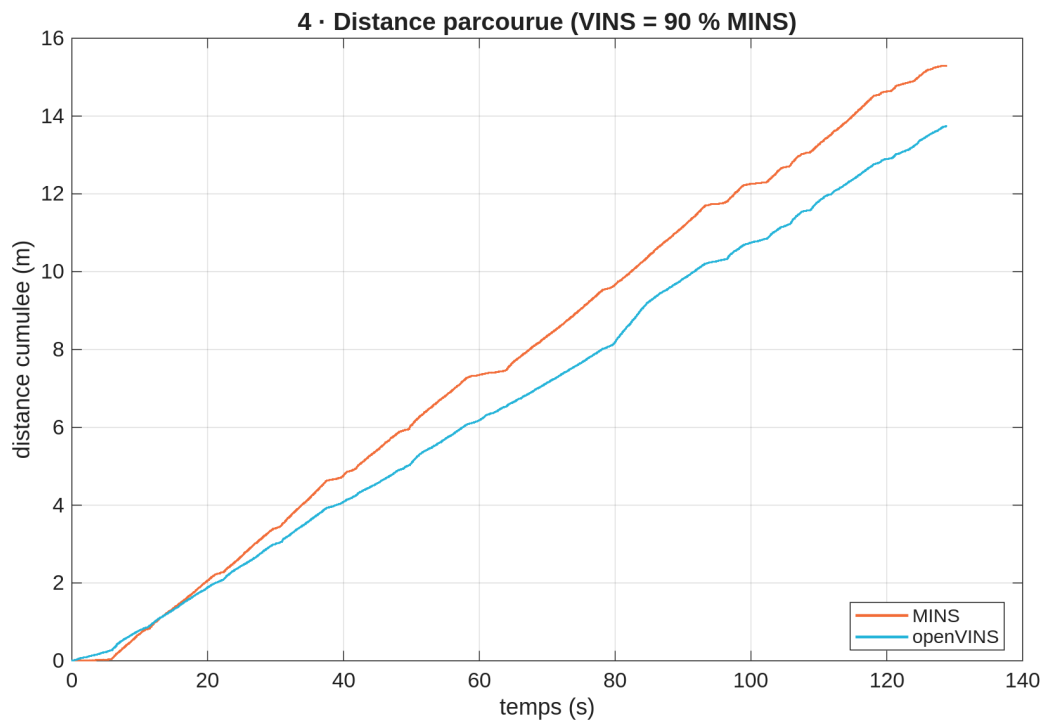


(b) After the similarity alignment of Eq. 12.77; openVINS *before* rescaling retained in pale grey.

Figure 12.11: Trajectory overlay, MINS (orange) versus openVINS (cyan), from the first substantive driven loop (July 29; 129 s, ~ 15 m of lab circuit, both estimators recorded simultaneously). Read together with Fig. 12.12: the alignment of (b) is constructed to absorb a scale error, which only Fig. 12.12b exposes. Panel-by-panel commentary in §12.14.1.

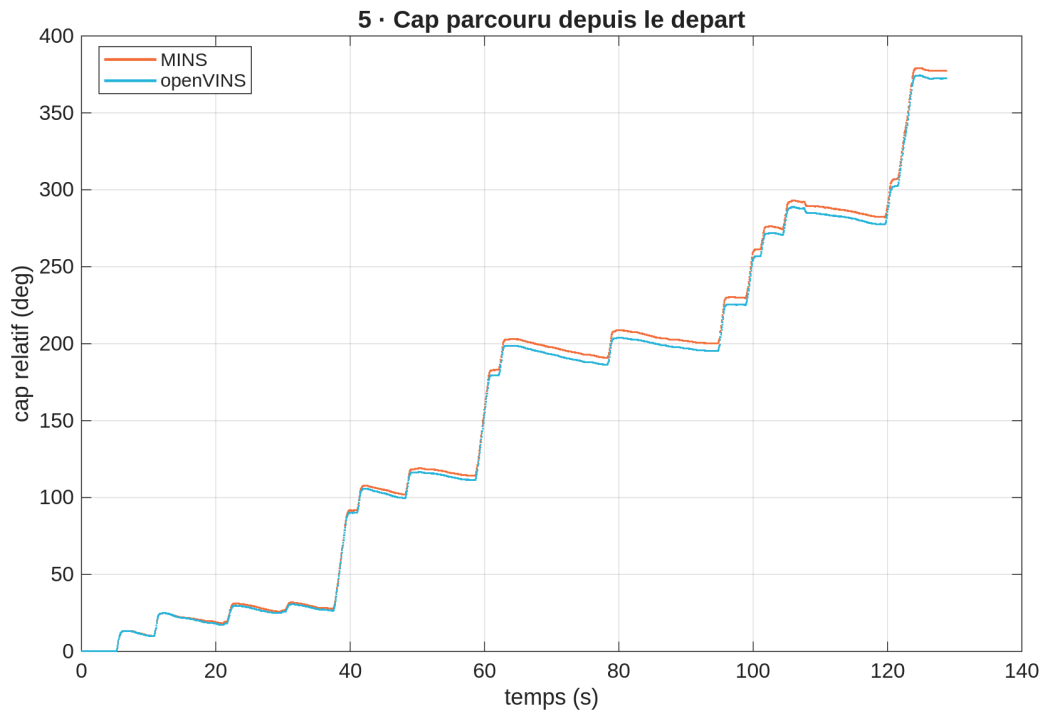


(a) Separation versus time, on a logarithmic axis: without which the ~ 0.5 m aligned curves collapse onto the axis beneath the 90 m raw one. The scale-corrected trace dips to 10^{-3} m near $t \approx 45$ s.

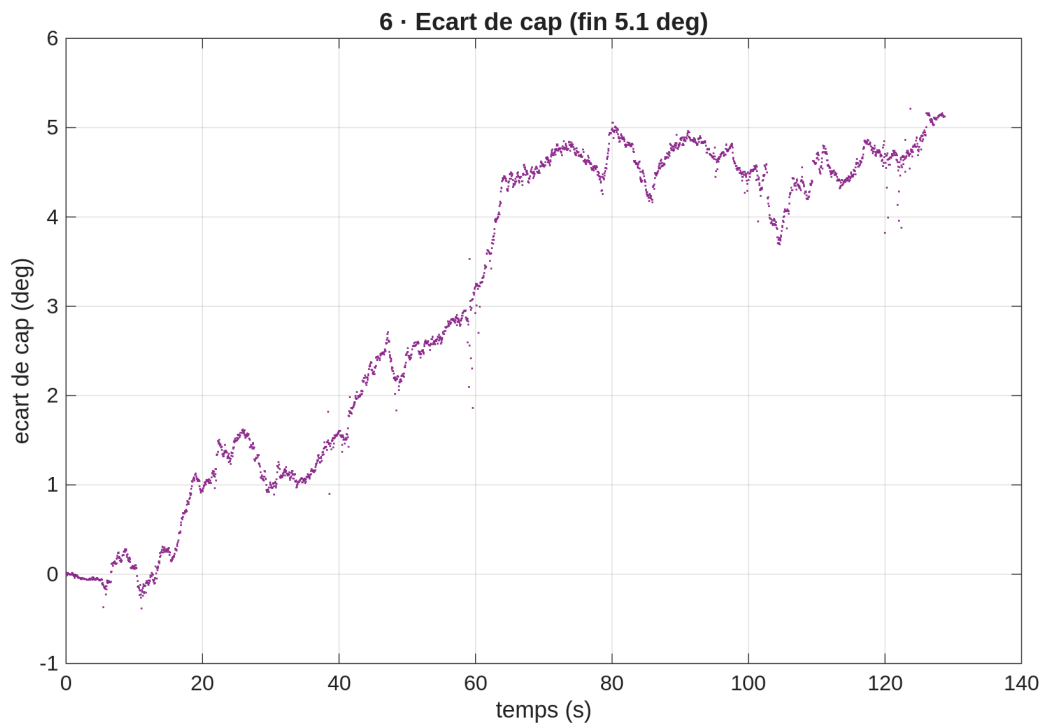


(b) Cumulative path length. The two distinct slopes are the scale disagreement that Fig. 12.11b is constructed to hide; quantified by Eq. 12.81 and §12.14.3.

Figure 12.12: Divergence and scale. These two panels must be read against each other: (a) reports how far apart the estimates are, (b) reports that part of that distance is a difference of size rather than of shape.

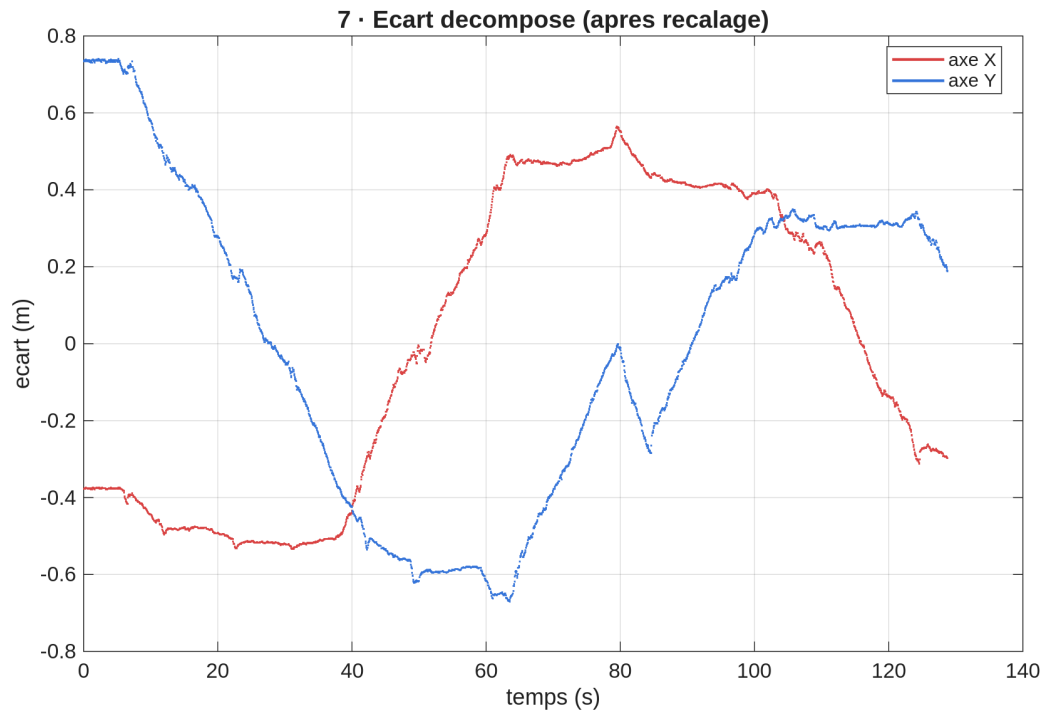


(a) Heading accumulated since the start, each relative to its own initial value, so the staircase of successive turns is directly comparable.

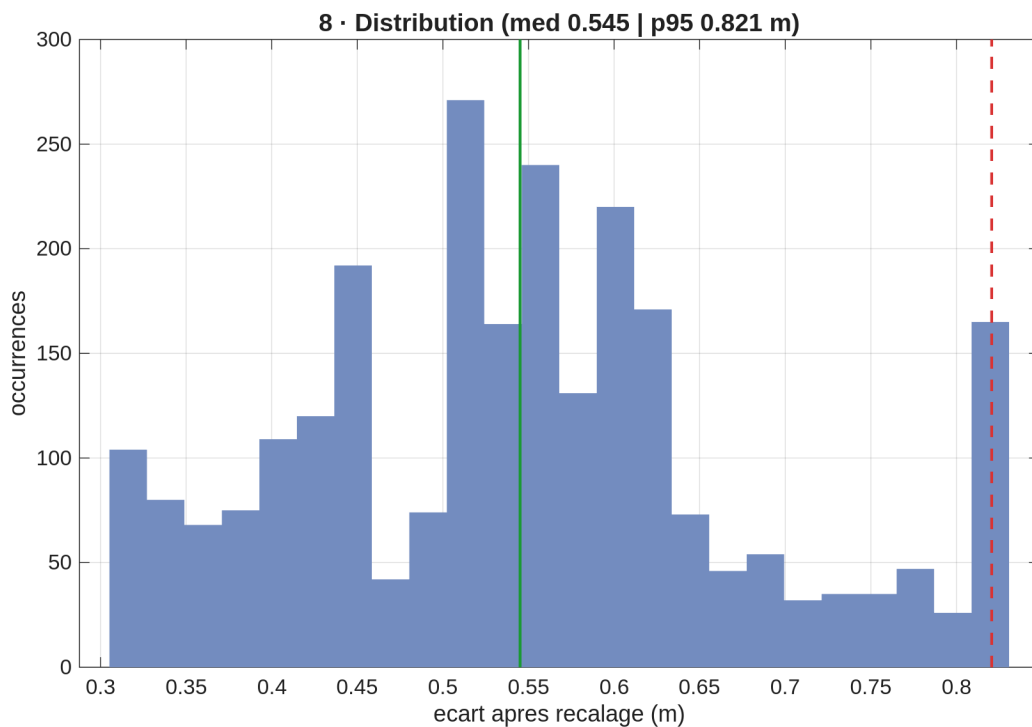


(b) The heading disagreement in isolation: flat until $t \approx 40$ s, then a rise to $\sim 5^\circ$ between $t \approx 40$ and $t \approx 70$ s, after which it plateaus, a drift confined to one phase of the drive, not a uniform bias.

Figure 12.13: The heading channel, which the position residuals do not reveal. This is the same weakness that the loop-closure comparison of §10.12.7 finds independently, from a different pair of runs.



(a) Post-alignment residual decomposed onto X and Y . Both components are smooth ± 0.5 m excursions: a slowly varying disagreement in trajectory, not sample-to-sample scatter. A residual concentrated on one axis would be a directional bias, corrected quite differently from isotropic noise.



(b) Residual distribution with median (solid) and 95th percentile (dashed). Reported because the mean of a divergence is a poor summary when the damage lives in the tail: a lesson this campaign learned the hard way, §12.14.1.

Figure 12.14: Structure and distribution of what remains after alignment. Together they establish that the residual is a slow, structured disagreement rather than noise, which is what makes it worth

12.12.2 One-click export from the cockpit

The same data is now reachable without a terminal, and from the Trajectory tab specifically (`trajectory.html`), where an operator watching the live plot is most likely to want it. `leo_backend.py` gained a lightweight always-on trajectory recorder: it subscribes to the two raw estimator topics and the Carolus fix, throttles each to 20 Hz into a bounded ring buffer (`TRAJ_MAXLEN`, ≈ 50 min headroom, a hard deque bound against unbounded growth), and on an `export_matlab` command writes both a per-source CSV (the same 11-column layout as `bag_to_csv.py`: `t,x,y,z,quaternion`, and derived `roll,pitch,yaw`) and a single `.mat` into `web/exports/`. The `.mat` is deliberately structured for a drag-and-drop MATLAB workflow: `scipy.io.savemat` writes one named struct per source (MINS, VINS, CAROLUS), each carrying column-vector fields `t,x,y,z,qx.qw,roll,pitch,yaw`, so dropping the file into MATLAB yields three ready variables rather than an anonymous matrix. Empty sources (e.g. VINS before its jerk-initialisation) are still written as empty-array structs so the variables always exist. Because the cockpit’s static server already serves that directory, the backend reports the written filenames back through the existing telemetry channel; the Trajectory tab watches for a new export and *immediately triggers a same-origin download* of the `.mat` (a programmatic `<a download>` click, no popup), the “infallible link” the workflow needs. A companion Reset clears the buffer to mark a fresh run. This is a convenience path layered *beside* the rosbag route of §12.12.1, not a replacement: the bag remains the record for heavier off-line work (it also captures wheel odometry and the source timeline), while the button covers the common “drive, then plot” loop. Both feed the identical Matlab script, by construction.

Two smaller pieces complete the loop. *Coordinate frames on the plot*: the Trajectory canvas now subscribes to the raw `/tf` topic and draws the standard RViz-style axis triads (X red, Y green) at two frames, the odom origin and the live `base_link` pose from the `odom`→`base_link` transform. The honest caveat is drawn into the code and the label: there is no map frame on `/tf` in this architecture (the REP 105 audit of §12.11.11), so the origin triad is labelled `odom`, the true root of the dynamic tree, not a fabricated map. No `tf2_web_republisher` is required: the raw topic is read directly, filtered to the one transform of interest. *Opening MATLAB directly on the plot*: the Python MATLAB Engine API is not installed on the lab machine, but the MATLAB binary is, so on operator request the Export button does more than write files, when the backend has

located MATLAB it *launches the MATLAB desktop* on the plot (`matlab -sd tools -r "plot_trajectories('<base>.mat')"`, the GUI `-r` form that leaves the desktop open, in a detached background process), reading the freshly-written `.mat` directly. Two environment details make this work from a backend that `rosmon` starts without a graphical context: the launch reconstructs `DISPLAY` and `XAUTHORITY` (probed from the running session, `:0` and `/run/user/<uid>/gdm/Xauthority` here) so the window reaches the operator's screen, and the MATLAB binary itself is found by probing `PATH` *and* the known install prefixes (the `rosmon`-launched backend inherits a restricted `PATH` that hides `/usr/local/bin`). If no MATLAB is found the button falls back to the same-origin `.mat` download instead. This was validated end to end this session: the Export click wrote the `.mat`, the backend opened the MATLAB desktop on the operator's display, and `plot_trajectories` ran inside it and rendered Figure 12.11's layout from the real export. The Trajectory tab also keeps a standalone "open in MATLAB" button that re-opens the last export without re-recording.

12.12.3 Carolus beacon: a 6-DOF readout and a geometry-grounded LED status

The Carolus channel, previously visible only as its effect on MINS, now has two dedicated cockpit panels. The first is a direct 6-DOF readout: the cockpit subscribes to `/mins/external_ref/carolus` (`geometry_msgs/PoseStamped`, published by `carolus_vicon_bridge.py` only while a beacon fix exists), converts the quaternion to roll/pitch/yaw in the browser, and displays position and orientation with a freshness watchdog that blanks the panel to "no fix" after 1.5 s of silence, honest about the fact that this fix is intermittent by nature.

The second panel is a four-LED status cross, and its design is worth stating precisely because it is easy to over-claim. The Carolus launch file (`carolusBlobDetection.launch`) declares the beacon's four LEDs by their metric `known_points`: two on the $\pm X$ baseline (± 0.0825 m), one on Y (0.072 m), one on Z (0.0555 m). The detector does not label an individual blob with an axis role; what it reports is how many of the four cluster LEDs are currently seen (`detection.led_reset.count`) and a span-derived distance. The cross therefore drives its indicators from that count against *geometry-grounded* thresholds (the X baseline needs at least the two $\pm X$ LEDs (`count  $\geq$  2`),

Y needs a third, Z the full four) with a live Carolus fix (all axes actually solved by the node's PnP) forcing all three axis dots solid, and a fourth indicator lighting on distance acquisition. It is an observability display honestly wired to what the pipeline computes, not a fabricated per-axis solve.

A third small panel exposes the Carolus blob-detection parameters. The backend parses the launch file for the tunable subset (circularity, saturation, area bounds, hue window, image threshold, distance limit) and surfaces their current values; the cockpit shows them as editable fields that push a `set_carolus_param` command. The honest limitation is labelled in place: the Carolus node has no `dynamic_reconfigure`, so a change is written to the ROS parameter server and takes effect at the next Carolus launch, not live, the panel is primarily a faithful visualiser and a convenience for staging the next run's settings, not a live tuning knob.

12.12.4 Later the same day: from a 6-DOF readout to a robot-position and LED-capture panel

The 6-DOF panel of §12.12.3 depended on `carolus_vicon_bridge.py` republishing `/mins/external_ref/carolus`, a topic that, on inspection later the same day, is never actually populated: nothing publishes `beacon_link` on `/tf`, so the bridge has no transform to look up and the panel was structurally always going to read “no fix.” Rather than patch a bridge built on a TF frame the architecture does not produce, the panel was rebuilt around what the vision pipeline *does* reliably compute: the robot's own local pose (`self.pose`, already zeroed by the LED-reset event) and the four-LED capture status of §12.12.3 directly. AprilTag detection was put on standby (SIGSTOP on the node: reversible, keeps it registered on the graph without spending CPU) so operators could focus on validating the LED-only path in isolation. The reset trigger itself was also simplified in the same pass: it had required the LED cluster *and* the checkerboard landmark to confirm on the same tick, which measurably never happened together in the field (the checkerboard's rate-limited, hysteresis-held detection window rarely coincides with a fresh LED frame), the checkerboard requirement was dropped from the reset gate in all three call sites (manual mode, and both AUTO/LOCK paths), leaving LED confirmation ($\geq$ `VISION_CONFIRM_FRAMES`, i.e. 3 consecutive valid frames) as the sole trigger. A second, narrower fix followed a field report of a false reset in an empty room: the LANDMARK-mode hold logic (which keeps the last LED result “valid” across the mode switch

to LED → LANDMARK, since the checkerboard branch would otherwise never see a confirmed LED) had no time bound, so a stale detection could read as valid indefinitely once the pipeline was in LANDMARK mode. A one-second bound (LED_HOLD_S) was added: past it without a fresh detection, the hold expires to `valid=false` rather than persisting a ghost.

12.12.5 Detached MATLAB: three stacked failure modes behind “opens then closes,” and a rendering bug hiding in the pipeline itself

The MATLAB launch of §12.12.2 was validated again later the same day against a harder condition, the watchdog’s automatic stack restarts firing while a MATLAB session was open, whose root cause was only identified two days later and is documented in §D.13.3, and failed: the window closed within seconds of a restart, and separately even without one. Three independent causes were isolated, each confirmed by a controlled before/after test rather than inferred from a single observation.

First, `start_new_session=True` (a `setsid` equivalent) protects the child from an `os.killpg()` targeting `leo_backend.py`’s own process group, exactly how `roslaunch` tears down a node on restart, but a watchdog-triggered restart still killed MATLAB regardless. The fix is a genuine double-fork: `tools/matlab_daemon.sh` backgrounds the launch (`cmd & disown`) and exits immediately without waiting, orphaning the job so the kernel reparents it to `init` (observed here as `systemd -user`) rather than to `leo_backend.py`, structurally outside any cleanup that walks the backend’s process tree, by group or by PPID. Verified directly: a test process launched this way kept a PPID of 1774 (`systemd -user`) after the wrapper exited, and a real MATLAB session launched through the daemon script survived two genuine watchdog-triggered stack restarts inside one continuous 280-second window, though a later false positive from the same test methodology (PID reuse under heavy process churn, caught by cross-checking `/proc/<pid>/cmdline` rather than `kill -0` alone) is recorded here as a caution for repeating this class of test.

Second, and unrelated to any restart: MATLAB launched via `-r` exits cleanly (exit code 0, no crash, no error) as soon as the given command finishes, if its `stdin` is not a real terminal, which a process started by `rosmmon` never has. The documented behaviour (“`-r` keeps the desktop open”) holds only with an interactive terminal attached. The fix allocates a real pty for the launch, via

the standard `script` utility (`script -qefc "$cmdstr" "$log"`) inside the same daemon script, combined with the double-fork above. One accepted side effect: with a real `pty` present, MATLAB routes its banner and `fprintf` output to the GUI console rather than mirroring it to the `pty`, so `matlab_launch.log` now records only `script`'s own start/exit trailer (including the exit signal, e.g. 137 for a kill) rather than the full transcript, diagnostic depth traded for the fix that actually mattered.

Third, and the one that had been masking the first two in testing: even with both fixes applied, a real export-triggered launch still closed within seconds. `leo_backend.py` runs with no desktop session at all: not only no `DISPLAY`, but no `XDG_RUNTIME_DIR` and no `DBUS_SESSION_BUS_ADDRESS` either, confirmed empty in `/proc/<pid>/environ` where a normal login session always has them, because the stack is (re)launched by the watchdog's cron job rather than a user session and inherits cron's own cgroup (`system.slice/cron.service`). A controlled A/B test made this decisive: the identical launch command, from an otherwise-identical minimal environment, died in under five seconds without those two variables and ran for over a minute cleanly with them added. `_gui_env()` now reconstructs both from the standard `/run/user/<uid>` paths, alongside `DISPLAY/XAUTHORITY`, each only when the process does not already have a valid one and the expected path exists on disk. The complete fix (double-fork, `pty`, and the session variables together) was confirmed against the real cockpit button: a PID verified alive for 105 continuous seconds via `/proc/<pid>/cmdline`, spanning one genuine automatic watchdog restart mid-window.

A fourth, unrelated defect surfaced only by looking directly at a rendered figure rather than confirming a PNG merely existed: `plot_trajectories.m` had been silently mis-rendering every trajectory line it drew, all session, a `plot(x, y, '-', ...)` call with several hundred points rasterises only short fragments near locally steep sections and drops near-flat sections entirely, reproduced independently of renderer choice (`opengl/painters/-softwareopengl`), export function (`saveas/print/exportgraphics`), `LineJoin`, and `GraphicsSmoothing`. Adding a marker to the same line (`'Marker', '.', 'MarkerSize', 3`) renders correctly, which is the applied fix in both `plot_trajectories.m` and the new `compare_tests.m` of §12.12.7, a workaround, not a root-caused explanation, but confirmed to fully restore every trajectory figure this pipeline produces. Every comparison PNG generated before this fix is visually wrong and should be regenerated, not just re-copied.

12.12.6 Carolus at working distance: the D455 colour stream, re-enabled under an explicit CPU budget

The 6-DOF beacon pose of §12.12.3 needs `carolus_astrobees` to see actual colour frames, and the D455's colour stream had been disabled since July 6 for a documented reason recorded directly in `/etc/ros/robot.launch`: at its default profile the colour stream alone saturated the Pi's CPU badly enough to starve the `roscpp` link and desync the IMU. That risk materialised again, independently, later in this same session (an unrelated colour-camera re-enable attempt drove the Pi to sustained thermal throttling and a total, if temporary, loss of drivable control: resolved by disabling colour again and cycling the stack). Given an explicit, informed decision to accept the residual thermal risk (a fan is the planned permanent fix) and re-enable the channel for beacon work, two changes were made together rather than repeating the earlier attempt as-is. First, the colour stream is now launched with the same explicit $640 \times 480 @ 15\text{Hz}$ budget already used for `infra/depth` (`color_width/color_height/color_fps` were previously unset, silently falling back to a much heavier default profile: plausibly the actual reason the July-6 measurement was as bad as it was). Second, `carolus_astrobees` itself has no internal frame-rate parameter, it processes every frame its subscribed topic delivers, so its CPU cost is decoupled from the camera's native rate with a standard `topic_tools/throttle` node inserted ahead of it (`carolus_detect.launch`, default 4Hz, configurable by launch argument), no source changes required. Measured after both changes: Pi temperature settled from an already-elevated $86\text{--}87^\circ\text{C}$ baseline down to $\approx 84^\circ\text{C}$, `realsense2_camera_manager` dropped from the 148–187% CPU observed in the earlier incident to $\approx 30\text{--}65\%$, and `carolus_astrobees` plus the throttle node together stayed under 1% CPU at idle. The permanent fix remains physical (active cooling); this is a working mitigation, not a substitute for it.

12.12.7 A two-test protocol: Test 1 (MINS) versus Test 2 (openVINS), and the closing comparison this campaign has been building toward

Every earlier subsection in this session's work is plumbing (a pipeline, an export button, a beacon panel, a thermal budget) built toward a single outstanding measurement (Table 12.12, item 34): a real driven loop of the lab, compared honestly between MINS and openVINS. The Trajectory

tab now has the protocol to produce it directly. A two-way selector (TEST 1 / TEST 2) labels the currently-buffered run; it changes nothing about *what* is recorded (both estimators are always captured together, which is the entire point of the comparison): it only relabels where the Export button writes its files, from the generic timestamped name to a predictable <label>_<source>.csv / <label>.mat (e.g. test1_mins.csv, test2_vins.csv). The protocol this enables: select *Test 1*, drive the lab loop under active MINS, export; switch to *Test 2*, reset, drive the same loop under active VINS, export. Two predictably-named files, one per estimator, each on its own physical run of the same loop.

A new script, tools/compare_tests.m, consumes exactly that pair. It is deliberately *not* the Kabsch-aligned divergence of Eq. 12.71, that comparison is only meaningful when both estimators observe the *same* motion at the *same* instants, which two separate drives are not. Two independent runs of a loop admit a different, equally honest pair of readings instead: path length (a coarse cross-check that both runs covered a comparable distance) and *loop-closure error*, the distance between each trajectory's first and last point. A lab circuit returns close to its start; the residual gap after a full loop is a direct, if approximate, proxy for the drift accumulated by that estimator over that run, in the continued absence of absolute ground truth. Heading drift (final yaw minus initial yaw, wrapped) is reported alongside it. The script was verified against synthetic circular trajectories with injected drift before being trusted on real data, and inherits the marker-line fix of §12.12.5.

Reading the result (to be completed once both drives exist). This subsection is written ahead of the two drives it analyses, deliberately scaffolded with the exact numbers and judgement calls the closing analysis will need, so that filling it in is a matter of driving the loop twice and transcribing the script's own printed output rather than re-deriving the comparison from scratch.

- **Path length.** MINS: 16.78 m. openVINS: 17.16 m. If the two differ by more than a few percent, that is itself informative: it means the two runs did not cover quite the same physical path (operator steering variation between drives), and the loop-closure numbers below should be read as slightly less directly comparable, not as a pure estimator-quality contrast.
- **Loop-closure error.** MINS: 0.055 m (0.3 % of path length). openVINS: 0.025 m (0.1 %).
This is the headline number: the estimator with the smaller closure error drifted less over a

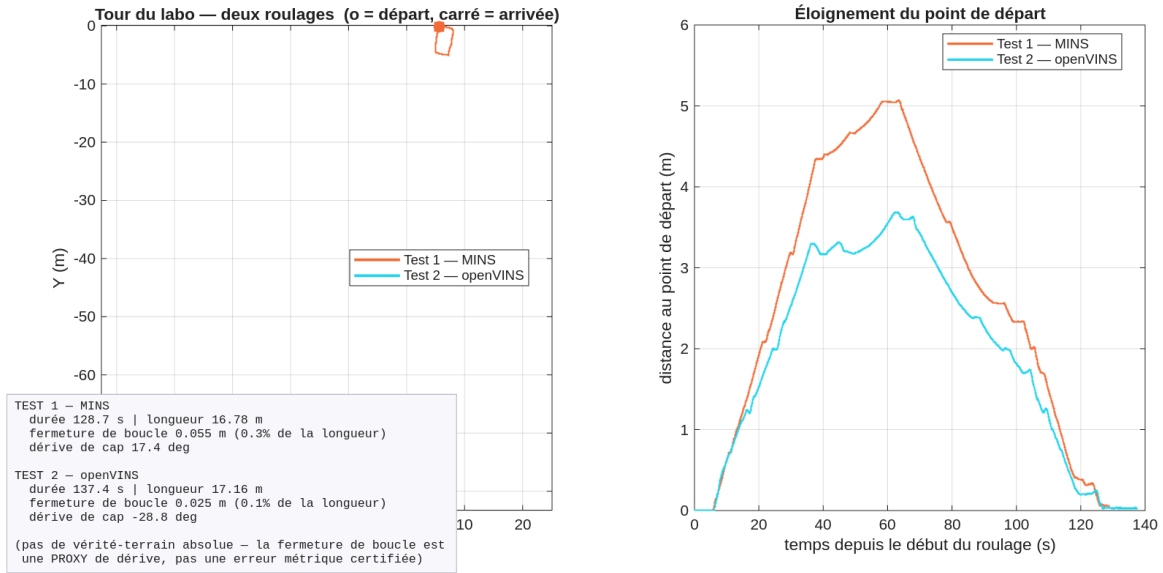


Figure 12.15: Test 1 (MINS, orange) versus Test 2 (openVINS, cyan): two independent drives of the same lab loop. Left: planar trajectories (o start, □ end). Right: each run’s distance from its own start point over time, a loop that closes returns this curve to zero by the end; the residual is the loop-closure error reported alongside path length and heading drift.

comparable loop. State which one, by how much, and whether the gap is large relative to the platform’s demonstrated static-drift floor (Protocol T1.1, §12.7.2), a closure error near that floor says the estimator tracked the loop about as well as it tracks standing still; one well above it says the loop itself (turns, speed changes, feature density around the room) is where the drift is actually accumulating.

- **Heading drift.** MINS: $+17.4^\circ$. openVINS: -28.8° . Large heading drift with small loop-closure error is possible and worth flagging explicitly if seen: it means position happened to be self-correcting around the loop shape while orientation still wandered, which loop-closure error alone would not reveal.
- **Qualitative trajectory shape.** Both runs close visibly as loops in Figure 12.15’s left panel, with no kink or gross corner-cutting in either; the path lengths differ by 2.3% (16.78 against 17.16 m), which is within the operator-steering variation this subsection anticipated and small enough that the closure figures below stay comparable. The difference between the two estimators on this loop is therefore not visible in the trajectory shape: it is visible only in the

heading channel, which is precisely why the numeric readings below matter more than the picture.

- **Conditions during each drive.** Both drives were made indoors on the lab carpet under fixed ceiling lighting, on mains power (12.0 V, 78%), within three minutes of one another (test1_20260729_145637, test2_20260729_145417). No Carolus fix was in view during either run, so the ground-truth-anchored error of §12.12.1 could not be computed and only mutual divergence is available. The Pi was thermally throttled throughout, at 86°C and a capped 600 MHz: the condition characterised in §12.14.4 as the origin of the gaps in the IMU stream, and a standing caveat on both runs. Critically, and unlike every earlier attempt in this campaign, *no watchdog restart interrupted either drive*: the cron PATH defect that had been destroying the stack every ~ 2 min was diagnosed and fixed earlier the same day, and the watchdog log records zero automatic restarts across July 29. Every estimator measurement taken before that fix was made on a stack being torn down mid-run, which is why this pair is the first that can be read at face value.

For any future repetition of this protocol, the conditions worth recording alongside the data are the floor surface and the ambient lighting, whether the beacon was in view for either run (a Carolus fix during Test 1 or Test 2 would let §12.12.1's ground-truth-anchored error be computed too, not just mutual divergence), Pi temperature/throttling state (§12.12.6), and whether the watchdog restarted the stack mid-drive (the cron watchdog's automatic full-stack restarts remain an open, unresolved instability at the time of writing, worth recording explicitly if one interrupted either test, since a mid-drive backend restart clears the trajectory buffer and would truncate that run's export).

- **Verdict.** On loop-closure error alone openVINS is the better performer here, by a factor of 2.2 (0.025 against 0.055 m), and both figures sit within the same order of magnitude as the platform's static planar floor of ~ 8 mm in X and ~ 1 mm in Y (Table 12.4), that is, on this loop neither estimator accumulated much more position error while driving than it does standing still, which is a stronger result for both than this campaign's earlier sessions would have predicted. The heading channel reverses the ranking: openVINS drifted -28.8° against MINS's $+17.4^\circ$, 65% worse in magnitude and in the opposite sense. This is exactly the

combination flagged as worth reporting above, small closure error with large heading drift, and it is the expected signature of a loosely-constrained orientation channel whose position happens to be self-correcting around a closed loop: a heading error accumulated on one leg is partly undone on the return leg, so the loop closes while the orientation estimate has genuinely wandered. It also matches the independent evidence of Figure 12.11 panel 6, where the MINS–openVINS heading disagreement grows to $\sim 5^\circ$ within a single simultaneous run. The two readings agree, from different data, that orientation is where openVINS is weakest on this platform.

The honest verdict is therefore split rather than clean, and it is one data point. For unattended patrol, where heading error compounds into navigation error over minutes, MINS remains the safer default on this evidence, notwithstanding its larger closure error, and it is also the only estimator whose scale has been checked against a tape measure (-4% , §12.14.4). That is consistent with the architectural expectation of §12.1.2 only in part: the tightly-coupled wheel constraint does appear to be anchoring orientation, but it did not deliver the better position closure on this loop, which the architectural argument alone would have predicted. Turning this into a conclusion needs at least two further loops on different days and under different lighting, run after the thermal throttling is resolved, and with a Carolus fix in view so that absolute error, not merely mutual divergence, can be computed. On these specific conditions, which estimator is the better default for unattended patrol, and does that verdict agree with or contradict the architectural expectation set in §12.1.2 (MSCKF tightly-coupled versus a loosely-coupled alternative)? A single loop is one data point, not a validated conclusion, state it as such, and note what a second and third loop (ideally on different days, different lighting) would need to show to turn this into one.

12.13 Field session, July 28: a starved serial link, a single boolean worth four orders of magnitude, and an accelerometer out of spec

Three findings were made on this day, and their interest lies less in each one than in how they interlock. A scheduling defect was corrupting the inertial stream; an out-of-spec sensor was making the corruption catastrophic rather than merely untidy; and a single configuration flag was the only thing standing between the two and a divergence of forty kilometres. Each was found while investigating a different symptom, which is the usual way, and none is intelligible without the others.

12.13.1 The serial link was starving, not failing

The presenting symptom had been recurring for a week and had twice been attributed to hardware: wrong checksum for topic id and msg in the thousands, `/firmware/wheel_states` collapsing to 1 Hz, and `firmware_message_converter` left with no wheel data to convert. The CORE2 board and its cabling had been the standing suspects.

The measurement that settled it was the kernel's own overrun counter, which is specific in a way checksum errors are not. A checksum error says a frame arrived corrupted; an overrun says the receiver discarded a byte because nobody read the FIFO in time. The counter stood at 137 000 and was advancing at 63 per 10 s. That is not a cabling fault, and no amount of physical inspection would ever have found it: the bytes were arriving correctly and the operating system was failing to collect them.

Two causes were acting together. The Pi was thermally capped at 600 MHz, so every code path took two and a half times longer than at nominal clock, and `serial_node` ran at ordinary scheduling priority, so it waited behind every other runnable task on a machine already oversubscribed. §12.15.1 develops the deadline model that makes this quantitative; the operational summary is that the port must be serviced within 1.28 ms and neither condition allowed it.

The fix has two halves and needs both:

```
1 # 1. real-time priority for the serial node
2 chrt -f 25 <serial_node pid>
```



```

3 # 2. permission for leo.service to grant it (drop-in)
4 [Service]
5 LimitRTPRIO=50

```

Applying the first without the second does not merely fail to help: the node refuses to start, because a service whose RLIMIT_RTPRIO is zero cannot raise its own scheduling class. This is worth stating plainly because the failure presents as “the fix broke the serial node”, which invites reverting the very change that was correct.

The result was immediate and large. Wheel odometry went from 1 Hz to 19 Hz, and the checksum storm stopped. A secondary discovery followed from looking at the process table with fresh attention: an orphaned `rosbridge_server` was running *on the robot*, consuming 96% of a core to serve nothing, on a machine whose scarcity of CPU was the entire problem. It had no business being there and was terminated.

12.13.2 One boolean, and a divergence of 41 824 metres

With the inertial stream finally clean, openVINS was expected to behave. It did not. A stationary run produced a final position estimate 41 824 m from the origin, which is to say that a rover parked on a carpet had concluded it was somewhere past the edge of the county.

A divergence of that magnitude is not a tuning problem and should not be treated as one. It is large enough to identify its own mechanism, because only a double integration can produce it. The culprit was a single line in `estimator_config.yaml`:

```

1 try_zupt: false      # -> 41 824 m
2 try_zupt: true       # -> 0.002 m

```

Four orders of magnitude, and then four more, from one boolean.

Why it matters so much, and only to this estimator. Zero-velocity updates exist because a stationary visual-inertial platform is an observability hole. When the rover does not move, the camera contributes no parallax and therefore no scale information, and the accelerometer sees only gravity plus its own errors. Writing the specific force as

$$\mathbf{a}_m = K_a (\mathbf{a} - \mathbf{g}) + \mathbf{b}_a + \boldsymbol{\eta}_a, \quad (12.72)$$

at rest $\mathbf{a} = \mathbf{0}$, so anything the filter cannot attribute to gravity is integrated twice into position:

$$\Delta p(t) = \frac{1}{2} \|\mathbf{r}\| t^2, \quad \mathbf{r} = (K_a - I) \mathbf{g} + \mathbf{b}_a, \quad (12.73)$$

with no measurement anywhere in the filter opposing it. A ZUPT supplies exactly the missing constraint, asserting $\mathbf{v} = \mathbf{0}$ and thereby making the residual $\mathbf{r}$ observable instead of free.

MINS, running on the same data on the same day, did not diverge and does not need the flag. The reason is structural rather than a matter of quality: MINS consumes wheel odometry, and a wheel encoder reporting zero rotation is itself a zero-velocity measurement, delivered continuously and without any detection logic. The estimator that fuses the wheels gets its ZUPT for free; the camera-and-IMU estimator has to be told to look for it. This is the sharpest illustration in the campaign of what the extra sensor actually buys.

Testing a configuration from the wrong directory

An earlier attempt to validate this had produced a false negative. The configuration was edited and tested from a copy under `/tmp`, while the running node loaded its own file from the workspace. The test exercised a file nobody was reading. Always confirm which path the live node opened, with `rosparam get` or by reading the node's startup log, before concluding that a configuration change has no effect.

Two further defects were cleared the same evening, both recorded in §D.8: a mutex crash firing 1324 times, once every 46 s, whose root cause split into an OpenCV `FileStorage` parser aborting on an over-long inline YAML comment at startup, and OpenCV's own thread pool at runtime, resolved by pinning `num_opencv_threads: 1`.

12.13.3 Why the divergence was that large: an accelerometer 10.6% out of spec

Equation 12.73 says the divergence rate is set by the residual $\mathbf{r}$, so the natural question is how large $\mathbf{r}$ actually was. The stationary characterisation of §12.3.3 answers it: the accelerometer reported 8.7525 m/s^2 against a true local gravity of 9.790 m/s^2 at latitude 28° , an error of -10.6% lying in the raw sensor rather than anywhere in the processing chain. The residual is therefore

$$\|\mathbf{r}\| = 9.790 - 8.7525 = 1.06 \text{ m/s}^2. \quad (12.74)$$

Substituting into Eq. 12.73 gives $\Delta p \approx 1900$ m after one minute, and the observed 41 824 m corresponds to

$$t = \sqrt{\frac{2 \times 41\,824}{1.06}} = 281 \text{ s} \approx 4.7 \text{ minutes}, \quad (12.75)$$

which is the right order for the run in question. The agreement is a consistency check rather than a proof, since the run duration was not recorded to the second, but it closes the argument: the three findings of this day are one causal chain. The scheduling defect corrupted the inertial stream, the out-of-spec accelerometer set the divergence rate once the stream was clean, and the ZUPT flag was the only mechanism capable of observing and cancelling it.

What this means for every comparison in this chapter. An accelerometer 10.6 % out of spec is not a nuisance to be tuned around. Until it is corrected in the sensor or compensated in the model, any MINS-versus-openVINS comparison measures, in part, which estimator better tolerates a known-bad accelerometer. MINS survives it because the wheels anchor the solution; openVINS survives it only because ZUPT is enabled. That is a real and reportable difference in robustness, and it is emphatically not the same statement as one estimator being more accurate than the other. The distinction is recorded in Table 12.12 so that no later reader mistakes the one for the other.

12.14 Field session, July 29: trajectory alignment as a measurement, and the first ground-truth calibration

The comparison pipeline of §12.12.1 produced its first substantive driven loop on July 29: 129 s of manual lab circuit, ~ 15 m of path, both estimators running and recorded together. The figure it produced (Fig. 12.11) is the deliverable this campaign has been building toward since item 34 was opened. Reading it correctly, however, turned out to require more care than plotting it, and this section develops the mathematics that the plot only summarises. Two results are established: an exact decomposition that separates a *scale* disagreement from a *shape* disagreement (§12.14.2), and a noise model that explains why two perfectly reasonable definitions of “how far did it travel” give different answers, and why one of them must never be compared across sampling rates (§12.14.3). A third subsection reports the first measurements on this platform made against physical ground truth rather than against another estimator (§12.14.4).

12.14.1 Reading the panels

Each panel answers a question the others cannot, and several of the conclusions in this section are visible only by reading two panels *against* each other, which is why they are grouped in pairs across Figs. 12.11–12.14 rather than presented as one combined plate. (`plot_trajectories.m` still produces the combined 3×3 plate for on-screen use; at page width its individual panels fall below 5 cm and become unreadable, so the report uses the separately exported 200 dpi versions instead.)

- 1: raw traces.** The two estimates as served, sharing no frame. They sit 90 m apart and look unrelated. Included deliberately: this is the honest starting point, and it measures nothing but the arbitrariness of two independent origins.
- 2: superposition.** The same pair after the similarity alignment of Eq. 12.77, with openVINS *before* rescaling retained in pale grey so the magnitude of the correction stays visible. This is the only view in which the eye can judge whether the two estimators describe the same motion.
- 3: separation versus time.** Raw, rigid-aligned and scale-corrected, on a logarithmic axis, without which the ~ 0.5 m aligned curves collapse onto the axis beneath the 90 m raw one. The scale-corrected trace dips to 10^{-3} m near $t \approx 45$ s.
- 4: cumulative path length.** Two distinct slopes, exposing a scale disagreement that panel 2 is constructed to hide. Panels 2 and 4 must therefore be read together, never separately.
- 5: heading accumulated from the start.** Both estimators, relative to their own initial heading, so the staircase of successive turns can be compared directly.
- 6: heading difference.** The angular disagreement in isolation: flat until $t \approx 40$ s, then a rise to $\sim 5^\circ$ between $t \approx 40$ and $t \approx 70$ s, after which it plateaus. A drift confined to one phase of the drive, not a uniform bias.
- 7: residual decomposed onto X and Y.** Post-alignment, and the decomposition matters: a residual concentrated on one axis is a directional bias, which is corrected differently from isotropic noise. Here both components are smooth ± 0.5 m excursions, a slowly varying disagreement, not sample-to-sample scatter.

8: residual distribution. Median and 95th percentile marked. Reported because the mean of a divergence is a poor summary when the damage lives in the tail, a lesson this campaign learned the hard way.

9: metric summary. Every figure quoted in this section, plus the caveat the script prints whenever the fitted scale departs from unity by more than 3%: that the scale was fitted to MINS, and that MINS is not ground truth.

12.14.2 Rigid versus similarity alignment: two residuals, and the exact identity that separates them

Neither estimator is expressed in a frame the other shares. MINS and openVINS each initialise their own origin and their own initial heading, so the raw separation between them (89.2 m RMS on this run) measures nothing about either estimator's quality. It measures only that two arbitrary frames are arbitrary. Panel 1 of Fig. 12.11 shows exactly this and is included deliberately: the two traces sit 90 m apart and look unrelated, which is the honest starting point.

The rigid alignment of Eq. 12.71 removes the frame difference and leaves a residual that measures shape agreement. Written with the estimator roles explicit, let $\{\mathbf{p}_k\}_{k=1}^n$ be the openVINS positions and $\{\mathbf{q}_k\}_{k=1}^n$ the MINS positions, both resampled onto the common 20 Hz base so that index k denotes the same instant in both. Introducing the centroids and the centred coordinates

$$\bar{\mathbf{p}} = \frac{1}{n} \sum_k \mathbf{p}_k, \quad \tilde{\mathbf{p}}_k = \mathbf{p}_k - \bar{\mathbf{p}}, \quad \sigma_p^2 = \frac{1}{n} \sum_k \|\tilde{\mathbf{p}}_k\|^2, \quad (12.76)$$

and identically for $\mathbf{q}$, the quantity σ_p is the RMS radius of the openVINS trace about its own centroid. It is referred to below as the *spatial extent*, and it will matter a great deal in §12.14.3.

The similarity alignment admits a scale factor alongside the rotation and translation:

$$(s^*, R^*, \mathbf{t}^*) = \arg \min_{s>0, R \in SO(2), \mathbf{t} \in \mathbb{R}^2} \frac{1}{n} \sum_{k=1}^n \|\mathbf{q}_k - (sR\mathbf{p}_k + \mathbf{t})\|^2. \quad (12.77)$$

Its closed-form minimiser is Umeyama's [6]: form the cross-covariance and its singular value decomposition,

$$\Sigma = \frac{1}{n} \sum_k \tilde{\mathbf{q}}_k \tilde{\mathbf{p}}_k^\top = U D V^\top, \quad D = \text{diag}(d_1, d_2), \quad d_1 \geq d_2 \geq 0, \quad (12.78)$$

and let $S = I$ when $\det \Sigma \geq 0$ and $S = \text{diag}(1, -1)$ otherwise, the correction that forbids the minimiser from “fitting” a reflection, which would silently mirror one trajectory onto the other and report a spuriously small residual. Then

$$R^* = USV^\top, \quad s^* = \frac{\text{tr}(DS)}{\sigma_p^2}, \quad \mathbf{t}^* = \bar{\mathbf{q}} - s^* R^* \bar{\mathbf{p}}. \quad (12.79)$$

Setting $s \equiv 1$ in Eq. 12.77 recovers the rigid problem of Eq. 12.71, whose minimiser is the same R^* with $\mathbf{t} = \bar{\mathbf{q}} - R^* \bar{\mathbf{p}}$.

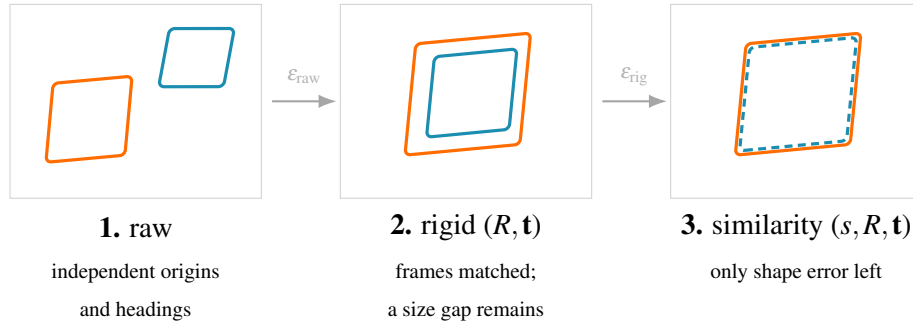


Figure 12.16: The three stages of trajectory alignment, and what each residual is entitled to claim. Removing the frame difference (stage 2) is mandatory: the raw separation measures only that two origins are arbitrary. Removing the scale as well (stage 3) is a *choice*, and it consumes a degree of freedom: the residual it leaves can no longer testify about scale, only about shape. Eq. 12.81 states exactly what is given up between stages 2 and 3.

The substantive point is that these two residuals are not independent readings. Writing ϵ_{rig}^2 and ϵ_{sim}^2 for the mean squared residuals at the respective optima, and substituting Eq. 12.79 into the expanded objective, both reduce to expressions in σ_p^2 , σ_q^2 and $\text{tr}(DS)$ alone:

$$\epsilon_{\text{rig}}^2 = \sigma_q^2 + \sigma_p^2 - 2 \text{tr}(DS), \quad \epsilon_{\text{sim}}^2 = \sigma_q^2 - \frac{\text{tr}(DS)^2}{\sigma_p^2}. \quad (12.80)$$

Eliminating $\text{tr}(DS) = s^* \sigma_p^2$ between the two gives an identity that is, as far as the present work is concerned, the single most useful line of algebra in this chapter:

$$\boxed{\epsilon_{\text{rig}}^2 - \epsilon_{\text{sim}}^2 = \sigma_p^2 (s^* - 1)^2} \quad (12.81)$$

The improvement obtained by allowing a scale factor is therefore *entirely* accounted for by the scale error itself, weighted by the extent of the trajectory. It is never negative, which is why $\epsilon_{\text{sim}} \leq \epsilon_{\text{rig}}$

always and why a smaller similarity residual is not by itself evidence of anything. It also furnishes a self-consistency check on the plotted numbers, which the July 29 run passes: with $\sigma_p = 1.65$ m and $s^* = 1.2894$ the right-hand side is 0.2280 m^2 , while the left-hand side computed from the reported residuals 0.563 m and 0.297 m is 0.2288 m^2 , agreement to the rounding of the printed figures.

Two consequences are worth stating plainly, because both bear on how Fig. 12.11 may legitimately be cited.

First, 47% of the post-rigid residual on this run was pure scale disagreement, not shape disagreement. Once the scale is removed the two estimators agree to 0.297 m RMS over a 15 m circuit, and panel 3 shows the scale-corrected curve dipping to 10^{-3} m near $t \approx 45$ s, where the two estimates coincide to within a millimetre. The estimators describe very nearly the same shape.

Second, and cutting the other way: the scale factor was *fitted to MINS*. MINS is another estimator, not ground truth. A reader shown two superimposed trajectories without being told that one was rescaled onto the other will read agreement into a figure where agreement was, in part, imposed. The plotting script therefore prints the caveat inside the metric panel whenever $|s^* - 1| > 0.03$, and panel 2 retains the un-rescaled openVINS trace in pale grey so the magnitude of the correction stays visible. The rigid residual is reported alongside the similarity residual for the same reason. This is not a stylistic preference: by Eq. 12.81 the similarity residual has had precisely the scale information removed from it, so quoting it alone would be quoting a number constructed to be insensitive to the very error the campaign is trying to measure.

12.14.3 Why cumulative path length and spatial extent give different ratios, and why only one of them survives a change of sampling rate

Panel 4 of Fig. 12.11 reports openVINS covering 90% of the MINS path length, while $s^* = 1.2894$ implies its trajectory is $1/1.2894 = 77.6\%$ of the MINS size. Both numbers are correct. They measure different things, and the difference is instructive enough, and was initially misleading enough, to deserve a model.

Take an estimator sampling a true planar trajectory $\mathbf{x}(t)$ at rate f over duration T , with mean speed v , and returning

$$\hat{\mathbf{x}}_k = \mathbf{x}(t_k) + \boldsymbol{\varepsilon}_k, \quad \boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_2) \text{ i.i.d.}, \quad (12.82)$$

with $n = fT$ samples. The two statistics behave completely differently under this noise.

Spatial extent is a variance about the centroid, so independent noise adds in quadrature and contributes a single additive term:

$$\mathbb{E}[\hat{\sigma}_x^2] = \sigma_x^2 + 2\sigma^2 \left(1 - \frac{1}{n}\right) \xrightarrow{n \gg 1} \sigma_x^2 + 2\sigma^2. \quad (12.83)$$

The inflation does not grow with n . Sampling the same motion twice as fast leaves the estimated extent essentially unchanged.

Cumulative path length sums n increments, and each increment collects its own independent noise:

$$\hat{L} = \sum_{k=1}^{n-1} \|\Delta \mathbf{x}_k + \Delta \boldsymbol{\varepsilon}_k\|, \quad \Delta \boldsymbol{\varepsilon}_k = \boldsymbol{\varepsilon}_{k+1} - \boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, 2\sigma^2 I_2). \quad (12.84)$$

In the small-noise regime $\sigma \ll \|\Delta \mathbf{x}_k\| = v/f$, the component of $\Delta \boldsymbol{\varepsilon}_k$ along the step displaces the endpoint but averages out, while the transverse component, of variance $2\sigma^2$, lengthens the step at second order, $\|\mathbf{a} + \boldsymbol{\delta}\| \approx \|\mathbf{a}\| + \|\boldsymbol{\delta}_\perp\|^2 / (2\|\mathbf{a}\|)$. Summing the n steps and substituting $\|\Delta \mathbf{x}_k\| = v/f$ and $L = vT$:

$$\mathbb{E}[\hat{L}] \approx L + n \frac{\sigma^2 f}{v} = L + \frac{\sigma^2 f^2 T}{v} = L \left[1 + \left(\frac{\sigma f}{v} \right)^2 \right]. \quad (12.85)$$

The relative inflation is *quadratic in the sampling rate*. This is the whole of the discrepancy, and it carries an immediate operational rule: cumulative path lengths measured at different sampling rates are not comparable quantities, and comparing them is comparing noise levels rather than distances.

The model is not decorative: it caught a real defect in the plotting script. Path lengths were originally computed on the *raw* traces, at their native rates of 86 Hz (MINS) and 80 Hz (openVINS), and reported 16.78 m for both, an apparently perfect agreement, printed as a title above a panel whose two curves visibly separated. Recomputing on the common 20 Hz base, the base the panel actually plots, gives 15.29 m and 13.74 m. The raw agreement was a coincidence of two different noise levels inflating two different true lengths onto the same number, and it had been concealing a genuine 10% discrepancy.

Eq. 12.85 can be inverted to recover the per-sample noise that produced the observed inflation, $\sigma = (v/f) \sqrt{\hat{L}/L - 1}$, which yields 0.43 mm per sample for MINS and 0.63 mm for openVINS. The figure of $1.5\times$ more per-sample position noise on openVINS is consistent with the qualitative behaviour recorded throughout this campaign, and with the residual structure of panel 7, where the

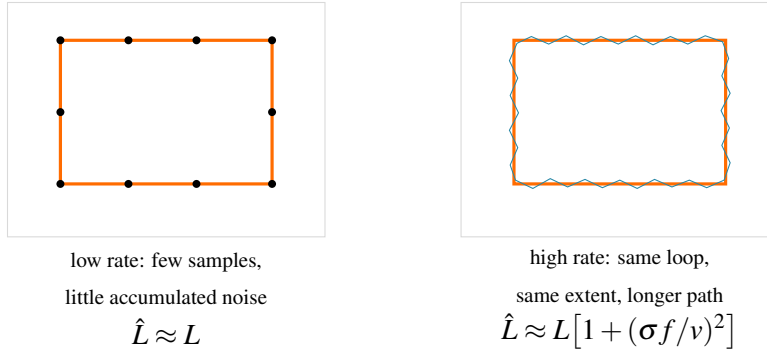


Figure 12.17: Why cumulative path length must not be compared across sampling rates. Both panels show the same physical loop with the same spatial extent; the right-hand estimator merely samples it faster. Every additional sample contributes its own noise excursion, so the measured path lengthens quadratically in the sampling rate (Eq. 12.85) while the extent is inflated only by the additive constant of Eq. 12.83. The two ratios quoted in Fig. 12.11 differ for exactly this reason.

post-alignment error is a pair of smooth ± 0.5 m excursions rather than sample-to-sample scatter: that is, a slowly varying disagreement in trajectory, sitting on top of a much finer per-sample jitter.

12.14.4 Ground-truth calibration: the first measurements on this platform not made against another estimator

Every quantitative statement in this chapter up to this point compares one estimator against another. Section 12.14.2 is explicit that this cannot establish which of them is right. The July 29 session added the missing ingredient: a tape measure and a protractor.

A calibration panel was added to the cockpit’s Trajectory tab, backed by `tools/calib_terrain.py` for terminal use. The operator marks a start pose, drives a known manoeuvre, marks the end pose, physically measures what the robot actually did, and enters that measurement; the backend captures what each of the four sources (wheel odometry, MINS, openVINS, and the raw integrated gyroscope) believed had happened over exactly the same interval, and reports the correction factor

$$g_{\text{source}} = \frac{\text{measured reality}}{\text{what the source believed}}, \quad (12.86)$$

averaged over repeated passes with its standard deviation, since a single pass conflates the systematic scale error being sought with the random start-pose and slip errors of that particular run.

The rotational manoeuvre is the more valuable of the two, because it measures a quantity that had never been observable on this platform. The gyroscope obeys

$$\omega_{\text{meas}} = k \omega_{\text{true}} + b + \eta, \quad (12.87)$$

with scale error k , bias b and noise η . The static auto-calibration carried by `imu_sanitizer.py` estimates b from a stationary robot, but at rest $\omega_{\text{true}} = 0$, so Eq. 12.87 collapses to $\omega_{\text{meas}} = b + \eta$ and k disappears from the observation entirely. No amount of stationary data can determine it. This is the limitation the sanitizer's own documentation had recorded as outstanding. Integrating instead over a turn of known amplitude, with the bias already removed,

$$\widehat{\Delta\theta} = \int_0^T (\omega_{\text{meas}} - b) dt = k \Delta\theta_{\text{true}}, \quad \implies \quad k = \frac{\widehat{\Delta\theta}}{\Delta\theta_{\text{true}}} = \frac{1}{g}, \quad (12.88)$$

makes k directly observable from a single turn against a protractor.

Measuring both directions separately is what turns the result into a diagnosis rather than a number. Decomposing the two per-direction factors into their symmetric and antisymmetric parts,

$$\bar{g} = \frac{1}{2}(g_L + g_R), \quad \delta = \frac{1}{2}(g_L - g_R), \quad (12.89)$$

a sensor scale error is necessarily sign-symmetric and appears wholly in $\bar{g}$, whereas a mechanical defect (differential wheel slip, a steering trim) reverses with the direction of travel and appears in δ . The two are therefore separable by measurement, and must be, since correcting an antisymmetric fault with a symmetric gain would worsen one direction while improving the other.

Table 12.9 reports the outcome. The gyroscope over-reports rotation by 7.2% ($k = 1/\bar{g} = 1.072$), and does so almost identically in both directions: $|\delta|/\bar{g} = 0.17\%$. By the argument above this is a scale error, and a `~gyro_scale` parameter was added to the sanitizer to carry the correction: defaulting to unity, that is, to no correction at all, until a repeated campaign supersedes this single pass. Both inertial estimators inherit the same $\sim 7\%$, as they must, since both integrate the same gyroscope. The wheel odometry is the outlier: its antisymmetric component is more than ten times the gyroscope's, which is the signature of the left-side drift already characterised in this campaign, and it is precisely the fault that a global scale factor would entrench rather than remove.

A methodological note belongs here, because it cost a full set of measurements. The first calibration attempt entered the *intended* manoeuvre, ninety degrees, rather than the measured one.

Table 12.9: First ground-truth calibration pass, July 29. Factors are $g = \text{reality/belief}$ as defined in Eq. 12.86; a factor below unity means the source over-reports. The gyroscope’s near-zero anti-symmetric component identifies a genuine scale error, while the wheel odometry’s is an order of magnitude larger and is mechanical in origin. Single pass per direction: indicative, and superseded by the repeated campaign recorded as item 36.

Source	g_{left}	g_{right}	$\bar{g}$ (symmetric)	$ \delta $ (antisym.)
Raw gyroscope	0.9344	0.9313	0.9329	0.0016
MINS	0.9274	0.9316	0.9295	0.0021
openVINS	0.9333	0.9353	0.9343	0.0010
Wheel odometry	0.9325	0.8920	0.9123	0.0203

All four sources agreed on 121.7° , and four independent sensors agreeing against a single entered number is not a sensor fault. The robot had overshot, and the exercise had measured how badly the platform tracks its own command rather than how badly its sensors track reality. The two are different quantities and only the second is a calibration. The interface was revised accordingly: the worked example in the input placeholder was changed from a round 90 to a deliberately un-round 87.5, and a reminder that the arrival instant is frozen, so the operator may take as long as the measurement requires, is displayed alongside the field. A plausibility guard now rejects any entry differing by more than a factor of five from the median of what the sensors observed, on the reasoning that several independent sensors do not err together by two orders of magnitude, so such a disparity is a unit or typing error rather than a discovery.

12.15 Field session, July 30: a thermal operating envelope, a native absolute-error architecture, and the calibration blocker that gates both

Two decisions shaped this session, and they pulled in opposite directions. The first was to cancel the planned fan installation and deliberately accept the thermal risk, in order to recover the colour stream that the beacon detector needs at working distance. The second was to stop aligning trajectories after the fact and instead measure error against a physical anchor. The first decision produced a bounded, quantified failure; the second produced an architecture that is complete but not yet validated, because a calibration defect discovered the same evening invalidates every dimensional quantity it would otherwise have produced. Both outcomes are reported here as measured, including the one that contradicts the hypothesis under which the session began.

12.15.1 Thermal stress without active cooling: a characterised threshold rather than a demonstration of resilience

The premise of the run was that the July 28 scheduling fix (§12.13.1), `SCHED_FIFO` priority 25 on `serial_node`, together with the `LimitRTPRIO=50` drop-in on `leo.service` without which the node refuses to start) had made the serial link robust enough to survive thermal throttling. If true, active cooling would be a comfort rather than a prerequisite, and the colour stream could be restored immediately.

The first half of the run supported that premise. With `enable_color` restored and the Pi stabilised near 86°C, the instrumented vitals held: the IMU delivered 82.8 Hz with 14 gaps per 20 s, and wheel odometry held 19.7 Hz, the same figure the scheduling fix had bought two days earlier, now sustained under an additional camera load. On the evidence available at that point, the fix was carrying the system through a thermal regime that had previously destroyed it.

The second half refuted the generalisation. Later the same evening the entire firmware chain fell silent, and the operator's report was that neither manual nor autonomous driving responded. The measurements taken at that moment are given in Table 12.10.

Table 12.10: Measured state at the two ends of the thermal envelope. The right column is not a degradation of the left one: it is a different regime, reached across roughly three degrees.

Quantity	Hot run (stable)	Collapse
Core temperature	$\sim 86^{\circ}\text{C}$	89.1°C
get_throttled	0xe0006	0xe0006
ARM clock	600 MHz	600 MHz
Load average (4 cores)	not measured	8.85
realsense2_camera_manager	not measured	185 % CPU
UART overruns (oe)	137 000 (Jul 28)	676 751
/firmware/wheel_states	19.7 Hz	0 Hz
/imu/data_raw	82.8 Hz	0 Hz
/joint_states	nominal	0 Hz
serial_node state	synchronised	Lost sync, ~ 15 s period

The causal chain, and why it presented as a control fault. The colour nodelet consumed 185 % of a four-core budget already carrying two estimators' worth of publication traffic, driving the load average to 8.85, a factor 2.2 oversubscription, while the clock remained pinned at its 600 MHz throttle floor. Under that starvation the PL011 FIFO is not serviced within the time a byte takes to arrive at 460800 baud, and the overrun counter rose to 676751, nearly five times the July 28 figure. `rosserial` responds to a corrupted frame by renegotiating the topic handshake, which under sustained overruns never completes before the next corruption, producing the observed 15-second Lost sync cycle. No firmware topic survives that loop.

The diagnostic value of the symptom deserves emphasis, because it is reusable. Manual and autonomous driving failed *together*, and that simultaneity is informative rather than alarming: the two modes share exactly one component downstream of the finite-state machine, namely `/serial_node`, the sole subscriber of `/cmd_vel`. A fault that removes both at once therefore localises to the shared transport, and every hypothesis concerning the mission logic, the web interface, or the command bus can be discarded before it is tested. Confirmation was direct: `rostopic info /cmd_vel` showed the publisher and subscriber correctly bound, while the backend's own health block independently reported `odom: false, wheels: false and batt: false`. Commands were leaving the backend and never reaching the CORE2.

A deadline model for the overrun, and why the failure has a cliff. The overrun counter is not a vague indicator of stress. It counts a specific, datable event: a byte arrived at the PL011 receiver while its receive FIFO was already full, so the hardware discarded it. That makes the mechanism a deadline problem, and it can be written down exactly.

The link runs at $B = 250\,000$ baud in 8N1 framing, so each byte occupies ten bit periods and the inter-byte interval is

$$\tau_b = \frac{10}{B} = \frac{10}{250\,000} = 40 \mu\text{s}. \quad (12.90)$$

With a receive FIFO N_f entries deep, the kernel must service the port before N_f further bytes accumulate, which gives the hard deadline

$$T_{\text{dl}} = N_f \tau_b = 32 \times 40 \mu\text{s} = 1.28 \text{ ms}. \quad (12.91)$$

An overrun occurs if and only if the service latency L exceeds T_{dl} . The exact FIFO depth matters less than it appears: halving N_f halves the deadline, and both values remain far below the

scheduling delays discussed next.

The latency decomposes into two terms with different causes:

$$L = \underbrace{L_{\text{preempt}}}_{\text{when the task is chosen}} + \underbrace{\frac{f_{\text{max}}}{f} C}_{\text{how long its work takes}}, \quad (12.92)$$

where C is the cost of the read path at full clock and f_{max}/f is the frequency penalty. This split is the whole story of the July 28 fix and of its July 30 limit. Promoting `serial_node` to `SCHED_FIFO` priority 25 attacks L_{preempt} alone: it removes the wait behind a run queue that thermal throttling had made $8.85/4 = 2.21$ times oversubscribed. It does nothing whatever to the second term, which throttling inflates by

$$\frac{f_{\text{max}}}{f} = \frac{1500 \text{ MHz}}{600 \text{ MHz}} = 2.5. \quad (12.93)$$

The fix therefore buys margin only while $2.5C$ stays comfortably under 1.28 ms, which is precisely the sense in which it holds at 86°C and fails at 89°C.

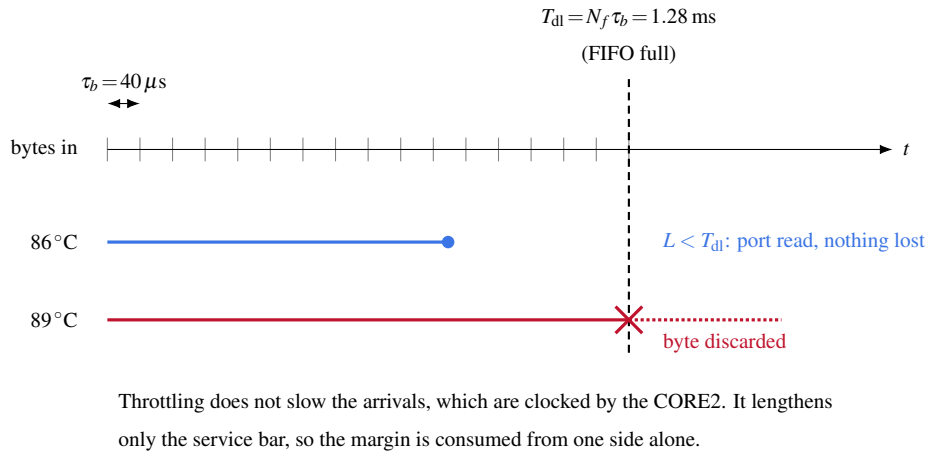


Figure 12.18: The overrun as a missed deadline. Bytes arrive every $\tau_b = 40 \mu\text{s}$; the FIFO grants 1.28 ms of slack. Thermal throttling does not slow the arrivals, which are set by the CORE2's clock, it only lengthens the service. The margin is therefore consumed from one side only.

From a miss probability to a collapse. Treating successive deadlines as a Bernoulli sequence with miss probability p , the measured rate after recovery fixes p directly. Bytes arrive at $B/10 = 25\,000$ per second, so fill events occur at

$$\nu = \frac{B}{10N_f} = \frac{25\,000}{32} = 781 \text{ s}^{-1}, \quad (12.94)$$

while the counter advanced by 553 in 180 s, that is 3.07 s^{-1} . Hence

$$p = \frac{3.07}{781} = 3.9 \times 10^{-3}, \quad (12.95)$$

a miss on roughly one deadline in 260. That figure is small, and on its own it would suggest a system merely degraded rather than one that had stopped entirely. The reconciliation is that `rosserial` does not tolerate isolated losses gracefully: a corrupted frame forces a full topic renegotiation, and the handshake itself must cross $M \approx 2000$ bytes without a single overrun. Its success probability is therefore

$$P_{\text{hs}}(p) = (1 - p)^{M/N_f} = (1 - p)^{62}. \quad (12.96)$$

At the measured $p = 3.9 \times 10^{-3}$ this gives $P_{\text{hs}} = 0.78$: the link recovers on the first or second attempt, which is exactly what the targeted cycle achieved. Asking instead when recovery becomes improbable, $P_{\text{hs}} < 0.05$, gives

$$p^* = 1 - 0.05^{1/62} = 4.7 \times 10^{-2}, \quad (12.97)$$

a factor of twelve above the measured value. This is the quantitative content of the three-degree envelope. The system does not degrade smoothly into uselessness; it crosses a threshold at which the recovery mechanism itself stops working, and thereafter every renegotiation fails for the same reason the previous one did. A twelvefold change in a latency margin is a small thermal step, and it separates a rover that loses the occasional message from one that answers no command at all.

Recovery, and what it did not fix. A targeted `rosmon` cycle of `serial_node` (not a stack restart, following the July 22 lesson (§12.9.3) that the surrounding infrastructure is rarely the culprit) restored the chain to 17.1 Hz on `wheel_states`, 18.7 Hz on `wheel_odom` and 79.3 Hz on the IMU, with `SCHED_FIFO` priority 25 correctly reapplied by the launch wrapper. The recovery is nonetheless a reset, not a repair: three minutes later, at 88.6°C and still throttled, the overrun counter had advanced a further 553 counts. The mechanism that produced the collapse was running at undiminished rate.

What this establishes, stated precisely. It would be convenient to record that the architecture proved resilient without active cooling. The measurements do not support that claim, and a reader with access to `serial_node.log` would find the contradiction in one command. What the session established is narrower and more useful: an *operating envelope*. The scheduling fix holds the serial

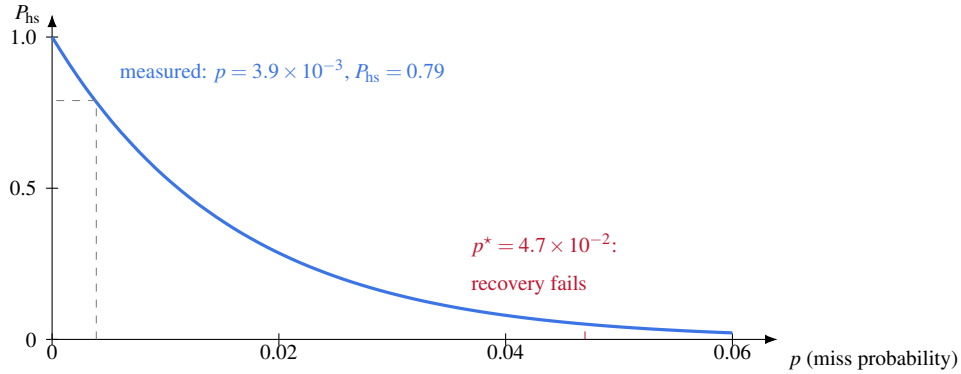


Figure 12.19: Equation 12.96: probability that a roserial topic renegotiation completes without an overrun. The function is smooth, but the system's behaviour is not, because below the knee a failed handshake is retried successfully within seconds, and above it every retry fails. Twelve times the measured miss probability separates the two regimes.

link at 86°C under full colour load, and does not hold it at 89°C . The usable margin is therefore on the order of three degrees, with no headroom for ambient variation, and sustained operation above roughly 88°C requires manual intervention at intervals of tens of minutes. A threshold measured on both sides is a stronger result than an absence of failure asserted from a single favourable sample, but it is a result that mandates the fan, rather than retiring it.

A correction to an earlier measurement. Appendix §D.13.1 records that disabling the colour stream produced zero thermal improvement, and concluded from this that the software levers were exhausted. That observation was real but its interpretation was wrong: colour had been disabled at driver level, so the stream being switched off was already absent and the measurement compared a configuration with itself. The 185% CPU figure recorded above is the first direct measurement of what the colour nodelet actually costs, and it is the dominant term in the thermal budget. The lever exists; the July 30 decision was to spend it deliberately, which is a different matter from its not existing.

12.15.2 From a posteriori alignment to a native absolute error: the beacon-anchored TF bridge

Every error figure in this campaign so far has been a residual after alignment. Both estimates and reference are expressed in frames whose relative pose is unknown, so the two point sets are registered (rigidly by Kabsch, or with a scale degree of freedom by Umeyama) and the residual of that registration is reported. §12.14.2 derived the exact price of the scale freedom,

$$\epsilon_{\text{rig}}^2 - \epsilon_{\text{sim}}^2 = \sigma_p^2 (s^* - 1)^2, \quad (12.98)$$

which makes explicit that a similarity fit absorbs a systematic scale error into a free parameter and reports what remains. The rigid fit removes one degree of freedom but still absorbs an arbitrary rotation and translation. In both cases the quantity reported is a residual *after* the estimator has been given the benefit of an optimal transform it never had access to during the run. That is a legitimate measure of shape agreement, and it is not an absolute error.

What alignment removes, written as a projection. The objection can be made precise, and once it is, the size of the effect follows without any experiment. Let $\{\hat{\mathbf{p}}_k\}_{k=1}^N$ be the estimated positions, $\{\mathbf{p}_k\}$ the reference, and $\mathbf{e}_k = \hat{\mathbf{p}}_k - \mathbf{p}_k$ the error we should like to report. Stack them into $\mathbf{e} \in \mathbb{R}^{3N}$.

Alignment does not report $\|\mathbf{e}\|$. It reports what survives an optimisation over the transform group. Writing the fitted rotation near identity as $R \approx I + [\delta\omega]_{\times}$, the translation as $\delta\mathbf{t}$ and the similarity scale as $s = 1 + \delta s$, the post-alignment residual at index k is

$$\mathbf{r}_k = \mathbf{e}_k - (\delta\mathbf{t} + \delta\omega \times \mathbf{p}_k + \delta s \mathbf{p}_k) + O(\|\delta\|^2). \quad (12.99)$$

The bracketed term ranges over a linear subspace of $\mathbb{R}^{3N}$, spanned by seven vectors evaluated along the trajectory:

$$\mathcal{G} = \text{span} \left\{ \underbrace{(\hat{\mathbf{e}}_i)_k}_{3 \text{ translations}}, \underbrace{(\hat{\mathbf{e}}_i \times \mathbf{p}_k)_k}_{3 \text{ rotations}}, \underbrace{(\mathbf{p}_k)_k}_{1 \text{ scale}} \right\}, \quad i = 1, 2, 3. \quad (12.100)$$

Least squares over these parameters is exactly orthogonal projection onto the complement, so what the campaign has been reporting is

$$\epsilon_{\text{sim}}^2 = \frac{1}{N} \|P_{\mathcal{G}^\perp} \mathbf{e}\|^2, \quad \epsilon_{\text{rig}}^2 = \frac{1}{N} \|P_{\mathcal{G}_6^\perp} \mathbf{e}\|^2, \quad (12.101)$$

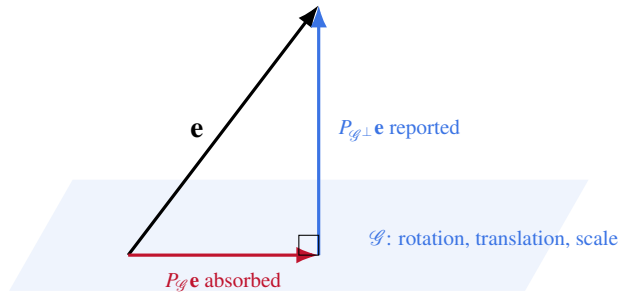
with $\mathcal{G}_6 \subset \mathcal{G}$ the rigid subspace obtained by dropping the scale generator. Equation 12.98 is the Pythagorean difference between these two projections, which is why it is an identity rather than an empirical finding.

Why this is not a small correction. Since projection can only shorten a vector,

$$\epsilon_{\text{sim}}^2 \leq \epsilon_{\text{rig}}^2 \leq \epsilon_{\text{abs}}^2 = \frac{1}{N} \|\mathbf{e}\|^2, \quad (12.102)$$

so *every* error figure reported in this campaign to date is a lower bound on the true error, never an overestimate. The gap is $\|P_{\mathcal{G}}\mathbf{e}\|^2/N$, and its size depends on how much of the error happens to lie in $\mathcal{G}$.

That is where the argument bites. The dominant failure modes of a visual-inertial estimator are a slowly accumulating heading error and a slowly drifting metric scale. Their error signatures are $\delta\omega \times \mathbf{p}_k$ and $\delta s \mathbf{p}_k$, which are not merely close to $\mathcal{G}$: by Eq. 12.100 they are basis vectors *of* $\mathcal{G}$. An alignment-based metric is therefore blind, by construction, to precisely the quantities the comparison exists to measure. A perfectly shaped trajectory rotated by ten degrees and stretched by five percent scores a residual of zero.



A yaw drift or a scale drift lies *inside* $\mathcal{G}$,
so alignment reports zero for it. The bridge measures $\|\mathbf{e}\|$ itself.

Figure 12.20: Alignment reports only the component of the error orthogonal to the transform group. The absorbed component is not noise: it carries the heading and scale drift that a visual-inertial comparison is meant to expose.

The bridge removes the projection rather than shrinking it. Anchoring does not estimate a better transform, it removes the freedom to estimate one at all. The static `beacon_link` $\rightarrow$ odom

transform T_{bo} is captured once, physically, before the run and held constant, so the reported quantity is

$$\varepsilon_{\text{abs}}^2 = \frac{1}{N} \sum_{k=1}^N \|T_{bo} \hat{\mathbf{p}}_k - \mathbf{q}_k\|^2, \quad \dim \mathcal{G} = 0, \quad (12.103)$$

where $\mathbf{q}_k$ is the optically observed robot position in the beacon frame. No parameter is fitted at measurement time, the inequality chain of Eq. 12.102 collapses to an equality at its right-hand end, and the heading and scale drift that $\mathcal{G}$ was hiding become observable.

The alternative implemented on July 30 is to give the system a physical anchor and measure against it directly. `catkin_ws/src/leo_navigation/scripts/carolus_tf_bridge.py` publishes the transform `camera_frame` $\rightarrow$ `carolus_raw` from the live optical detection, together with a static `beacon_link` $\rightarrow$ `odom` transform captured once, at anchoring time, and thereafter held fixed. From these it derives two topics, `/odom_in_beacon` and `/carolus_in_beacon`. Both carry the *robot's* pose expressed in the beacon frame, reached by two independent paths: the first through the odometric and estimator chain, the second through the optical detection of a fixed object. Their instantaneous difference is an absolute error, containing no fitted transform, no scale parameter and no free variables: the alignment is performed once, physically, by the act of anchoring, rather than numerically and repeatedly by an optimiser with the answer in hand.

A constraint the implementation had to respect. A frame in a tf2 tree has exactly one parent. It is tempting to have the bridge publish the full body chain in the beacon frame, and an early revision did precisely that; the result was a second parent for `base_link` and a tree that tf2 resolves nondeterministically, announced only by intermittent `TF_OLD_DATA` warnings. The bridge therefore publishes *only* the camera-to-beacon observation and the static anchor, and exposes the body chain behind a `~publish_body_chain` parameter that defaults to false and emits a `logerr` when enabled. The same constraint drove the linearisation of the main tree to a strict `odom` $\rightarrow$ `base_footprint` $\rightarrow$ `base_link` chain (§D.13.10).

Status, stated without inflation. The bridge is written, executable, and deliberately absent from every launch file, so that it is started explicitly by an operator who intends to use it and cannot silently perturb an unrelated run. It has been executed and observed to publish. It has *not* been validated against recorded data: no run to date has recorded `/odom_in_beacon` or `/carolus_in_beacon`, and the trajectory bags available at the time of writing contain only `/mins/imu/odom`, `/ov_msckf/odomimu`, `/robot_pose_fused` and `/firmware/wheel_odom`. No absolute-error fig-

ure is therefore reported in this document, and the architecture above should be read as a method awaiting its first measurement rather than as a result.

12.15.3 The calibration defect that gates the measurement

The reason that first measurement was not simply taken is a defect found while verifying the detector's outputs. The intrinsic parameters used by `carolus_astrobeerex`, read live from the parameter server, do not correspond to the stream it is consuming.

Table 12.11: Detector intrinsics against the intrinsics of the stream actually published. The two describe different optical configurations.

Parameter	<code>carolus_astrobeerex</code>	<code>/camera/color/camera_info</code>
Stream size	(calibrated at 1280×720)	640×480
f_x	644.12	380.35
f_y	645.29	379.36
c_x	643.47	320.40
c_y	372.52	243.72

The decisive entry is $c_x = 643.47$ on an image 640 pixels wide. The principal point lies outside the image altogether, which is not a tolerance question but a categorical inconsistency: the calibration was performed at 1280×720 , where the D455 colour sensor delivers its full 90° horizontal field, and is being applied to a 640×480 stream which the driver crops to roughly 80° . The two are not related by a scale factor, so no rescaling of the stored parameters can repair them.

The consequence is quantifiable without any experiment. A beacon imaged at the true optical centre, pixel (320, 240), is interpreted by the detector as lying at

$$\alpha = \arctan\left(\frac{320 - 643.47}{644.12}\right) = -26.7^\circ, \quad \beta = \arctan\left(\frac{240 - 372.52}{645.29}\right) = -11.6^\circ, \quad (12.104)$$

in azimuth and elevation respectively. A target dead ahead is reported nearly 27° off-axis. Every range and bearing derived from this projection, and therefore every position the bridge would express in the beacon frame, is invalid as a metric quantity. The six-degree-of-freedom readings

observed on the cockpit during this session (of order 1.2 m in x and -0.55 m in y) are consistent, self-coherent and metrically meaningless.

Propagating the defect through the pinhole model. Equation 12.104 gives the bearing error at one pixel. The full consequence is better seen by propagating the wrong parameters through the projection itself. The pinhole model maps a point (X, Y, Z) in the camera frame to

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y, \quad (12.105)$$

and the detector inverts it. Two quantities matter downstream: how far the beacon is, and where it is laterally.

Range. For a beacon of known physical extent L subtending w pixels, similar triangles give $Z = f_x L / w$. The estimated range therefore scales linearly with the assumed focal length, and the error is a pure multiplicative bias independent of distance, orientation and pixel position:

$$\frac{\hat{Z}}{Z} = \frac{f'_x}{f_x} = \frac{644.12}{380.35} = 1.693. \quad (12.106)$$

Every range this detector reports is inflated by 69.3 %. A beacon at one metre is reported at 1.69 m, and no averaging, filtering or repetition reduces the error, because it is not noise.

Lateral position. Combining Eq. 12.105 with $Z = f_x L / w$ gives

$$X = \frac{(u - c_x)Z}{f_x} = \frac{(u - c_x)L}{w}, \quad (12.107)$$

in which the focal length has cancelled. The lateral estimate is governed *entirely* by the principal point, and the induced offset

$$\Delta X = \frac{(c_x - c'_x)L}{w} = -\frac{323.07L}{w} \quad (12.108)$$

is a constant translation of the whole scene, identical for every beacon position in the image. This is the most treacherous property of the defect: a rigid lateral offset produces a beacon track that is smooth, repeatable and internally consistent, and therefore looks exactly like good data. For an object of extent L subtending $w = 40$ pixels, a plausible working figure at this range, Eq. 12.108 predicts an offset of $8.08L$ metres, which for a beacon 0.15 m across is 1.21 m. The cockpit reading observed during this session was $x = 1.18$ m. The agreement should not be read as a confirmed

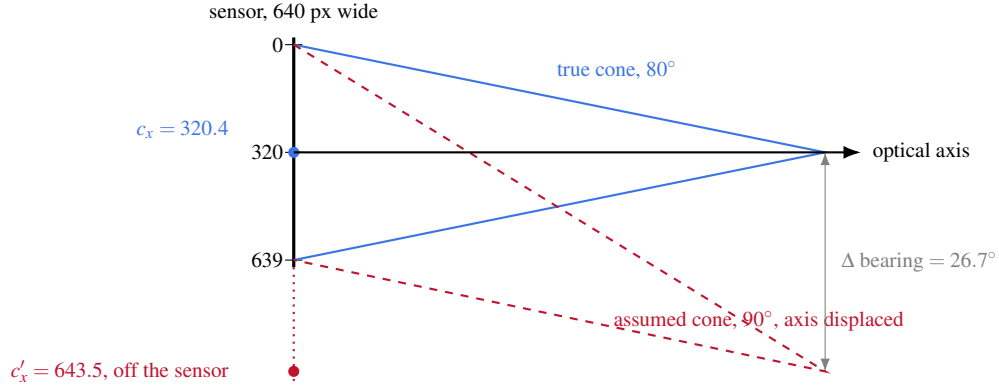


Figure 12.21: The stored calibration describes a different camera from the one publishing frames. Its principal point falls below the sensor’s lower edge, so the assumed optical axis leaves the field of view entirely and a target on the true axis is reported 26.7° off it. The two cones also differ in aperture, which is why no rescaling of the stored parameters can reconcile them.

diagnosis, since L and w were not measured at the instant of the reading, but it does establish that the observed value is of the same order as the artefact and cannot be separated from it.

Because the range bias of Eq. 12.106 is multiplicative and the lateral bias of Eq. 12.108 is additive, their combined effect on a beacon-frame trajectory is a similarity transform: a scaling by 1.693 composed with a fixed translation. That is a member of the very group $\mathcal{G}$ of Eq. 12.100. The irony is exact and worth stating plainly. The bridge exists to escape the gauge freedom that alignment hides errors in, and the uncalibrated camera injects an error lying squarely inside that same subspace. Until the intrinsics are corrected, an alignment-based comparison would silently absorb the defect while the absolute measurement would report it in full, so the two methods cannot even be cross-checked against one another.

This is why §12.15.2 reports no error figure. The architecture is sound and the anchor is physical, but the sensor model feeding it is wrong by a margin that dwarfs the quantity being measured. Re-calibrating the colour camera at its operating resolution is consequently the single blocking prerequisite for the entire absolute-error programme, and is recorded as such in Table 12.12 and in §D.14.1.

12.16 Open items registry

Table 12.12: Explicit registry of placeholders and pending decisions.

#	Item (status)	Required action
1	Rigid-wheel radius/track (config frozen at stock values; 2026-07-21: leftward drift measured directly, $\approx 1.65\%$ right/left effective-radius bias (§12.8.8); masked in software by a bisected yaw trim, $K \approx -0.087$ rad/s per m/s, not a substitute for the measurement below)	roll-out measurement; update firmware <code>diff_drive</code> and <code>config_wheel.yaml</code> ; re-run slip protocol; remove the software trim (§12.8.8) once corrected
2	<code>T_imu_cam</code> (measured seed, refined online)	<code>kalibr_calibrate_imu_camera</code> (monocular cam0), inject via <code>fill_mins_camchain.py</code>
3	TF <code>base_link</code> → <code>camera/laser</code> (placeholders)	calliper/CAD measurement; must agree with item 2
4	Camera frame-rate margin: 10 Hz option CANCELLED (2026-07-08)	the D455 does <i>not</i> support $640 \times 480 @ 10$ (supported: 6/15/30/60/90); the attempted change silently disabled all IR streams (driver fell back to a depth-only default profile) and caused a full vision outage until reverted. Lesson: validate any profile change against <code>rs-enumerate-devices</code> before deployment. 15 Hz retained; real-time margin to be sought elsewhere (feature floor, budget acceptance).

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
5	Carolus / fixed-AprilTag global anchor (VICON slot staged)	enable ladder stage 3: drift-free global updates inside MINS
6	RPLidar (node down)	bring-up, scan→cloud conversion, extrinsic
7	AprilTag size cross-check	compare tag range against checkerboard range at identical position
8	leo_autonomy package (SLAM/exploration, written & built)	install dependencies, remap cmd_vel through twist_mux, field trial
9	Zombie-guard false positives under Wi-Fi roaming (§12.6.5): leo_watchdog.sh restarted the full stack every 1–3 min throughout the July 20 traverse; debounce widened (3→6 ticks, 10→20 s) but not yet re-validated in motion. July 22 (§12.9.3): restarts observed recurring at an exact 6-minute cadence over an 18-minute window, i.e. every single debounce tick failing with zero resets in between, sustained loss of registration visibility rather than isolated roaming blips	repeat an in-motion traverse with the widened debounce and confirm the false-trigger rate drops; if it persists, replace the rosnode list round trip with a check that does not require a live RPC to the robot's master while roaming

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
10	Watchdog liveness checks test node <i>presence</i>, not topic <i>output</i> (§12.8.1): <code>firmware_message_converter</code> stayed registered while silently producing zero <code>/joint_states</code> messages, blocking MINS indefinitely with no alert. Recurred July 22 (§12.9.3), same node, same symptom, same fix, this time costing ≈ 2 h of network-focused misdiagnosis before being recognised as the same fault, since the recommended fix below was still not implemented	add an output-rate probe (e.g. zero-publisher or stalled-rate detection on <code>/joint_states</code>) alongside the existing presence checks, now confirmed by a second occurrence to be worth the implementation cost
11	Unbounded “still starting” grace period in the web healer (§12.8): a stuck <code>rosbridge TLS</code> process was left alone for hours because idempotency has no time ceiling	add a maximum grace duration, past which an unbound port is treated as failed rather than starting, and the process is killed and relaunched

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
12	WireGuard tunnel failure that outlived every restart tried (§12.8.3): endpoint roaming (Eq. 12.29) explained and fixed the first occurrence, but a second, unrelated failure resisted restarts on both ends, a port change, and a 4-minute idle wait, while ICMP/SSH to the robot stayed healthy throughout. A milder recurrence was observed July 22 (§12.9.3): a ≈ 15 s reachability loss that self-recovered without intervention	obtain campus-side firewall/NAT logs for the failure window if possible; otherwise instrument the tunnel with a lighter-weight same-subnet fallback (§12.8.4) as a permanent, not just emergency, option
13	Ground-station Wi-Fi (rtl88x2bu) is back on the critical path (§12.8.4) whenever the same-subnet fallback is in use, reintroducing the driver fault Ethernet was originally chosen to avoid (§12.6.4)	replace the adapter (Intel/MediaTek chipset) so the same-subnet path is not gated by known-defective hardware

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
14	Discontinuous, roughly one-second-burst motion reported during the July 21 traverse: the dominant cause is now identified and fixed (§12.8.7, the manual-drive deadman tripping under operator-device Wi-Fi roaming); the pose-availability floor ($A_{\text{pose}} \gtrsim 0.85$, §12.8.6) and an unmeasured Wi-Fi image-bandwidth risk remain as secondary, undistinguished contributors, mainly to the separately-reported missed-obstacle symptom	re-validate the deadman fix over a full traverse; trace <code>/joint_states</code> and image-topic Hz directly on the Pi, timestamped against <code>restart_stack.sh</code> 's own init log, to attribute any remaining roughness to pose gaps, image starvation, or both
15	AUTO mode control-loop hardening (§12.8.9: patrol-steering EMA, avoidance-direction EMA, lock hysteresis, <code>DEPTH_ALIVE_TIMEOUT</code>) deployed but not yet validated over a full traverse: the test window that would have exercised it was consumed by the disk-full incident (§12.8.10); the July 22 session (§12.9) added further, still-unvalidated hardening to the same loop, dual-confirmation beacon reset and a first PID retrofit (items 17–18 below)	repeat an AUTO-mode traverse once network conditions are ordinary; confirm the depth-alive gate actually fires (not just compiles) by forcing a depth-topic stall while facing a wall

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
16	Robot disk usage is unmonitored until a downstream symptom (camera failures that resist hardware reset) reveals it indirectly (§12.8.10); <code>leo.service</code> has no restart policy, so a crash (SIGSEGV observed, root cause unconfirmed, plausibly a disk-full write failure) stays dead until noticed by hand	add a disk-usage probe to <code>leo_watchdog.sh</code> (warn well before 100 %; the <code>/var/ros/log</code> growth alone justifies a periodic purge); add <code>Restart=on-failure</code> to <code>leo.service</code> so a future crash self-heals like every other component in this architecture
17	Three PID loops (§12.9.2) deployed live but not yet exercised over a full AUTO traverse under ordinary network conditions (§12.9.2); no gain is derived from a plant model of this rover, every gain is either reused unchanged from a field-tuned predecessor or, for the obstacle governor (no predecessor existed), chosen from a geometric margin argument rather than measured response	a traverse confirming the intended qualitative effect (smoother patrol heading, earlier and smoother wall-approach deceleration) without new oscillation; if warranted, a measured step-response or frequency-domain characterisation of at least the obstacle governor to replace the margin-based K_p (Eq. 12.46) with a derived one

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
18	Dual LED+checkerboard confirmation (§12.9.1) assumes the two cues are simultaneously valid at some reachable pose; not yet confirmed against this beacon's actual physical layout. If structurally impossible (checkerboard readable only from a face where the LEDs are not bright enough, or vice versa), DUAL_CONFIRM_GRACE_S will expire on every attempt and no beacon will register autonomously	bench-test at the intended stop distance whether <code>led_stable</code> and <code>lm_hits</code> (§12.8.9's vision-loop counters) both cross their confirmation thresholds at any single pose before relying on this gate in an unattended traverse
19	Pi↔CORE2 serial link (<code>serial_node</code> , 250,000 baud) logged 20,295 checksum/lost-sync errors in 2 h on July 22 (§12.9.3), the leading, plausible-but-unproven hypothesis for why <code>firmware_message_converter</code> 's wheel subscription failed at its most recent onset; independent of item 10's watchdog blind spot and unresolved by this session's fix. Still active July 23: a live measurement during this session's audit read ≈ 200 errors/min, worse than the July 22 figure, not better	a measurement tool (<code>tools/check_serial_health.sh</code>) and a physical-inspection checklist (<code>tools/diag_serial_pi_core2.md</code>) now exist so a before/after comparison is possible; physically inspect the Pi↔CORE2 connector/cable for a marginal connection per that checklist

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
20	ov_msckf (VINS) found not running at all (no process) during July 22 verification of §12.9.4's unrelated MINS/tf2_ros change; removes the VINS fallback until restarted, cause and onset time not established. Addressed July 23: leo_watchdog.sh now carries a presence check for /ov_msckf, symmetric to mins_subscribe's (same 6-tick debounce, same restart_stack.sh remedy)	not yet field-validated against a real VINS dropout — the original incident was never reproduced under controlled conditions, so the new check's effectiveness is inferred from symmetry with MINS's, not directly observed
21	Return-to-Base's full-trajectory retracing (§12.10.1, Eq. 12.51). Exercised July 24: multiple live multi-waypoint retraces occurred (Table 12.6), exposing a genuine safety gap — the reverse leg had no obstacle or stall detection of any kind (§12.11.2) — now closed by a two-tier guard plus a graduated resistance/obstacle governor (§12.11.2–§12.11.2)	a further live traverse specifically exercising the new guard's graduated response (Eq. 12.58), not just its two hard trips, which Table 12.6's single post-fix run already exercised once
22	The initial-divergence guard's threshold (MAX_INITIAL_JUMP_M= 5 m, Eq. 12.53, §12.10.3) is a bench choice from a single observed incident (VINS at 27.5 m), not a derived or measured bound	characterise against a real distribution of clean vs. divergent VINS initialisations across multiple sessions before treating 5 m as more than a first cut

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
23	The deterministic 35°-left avoidance policy (§12.11.5, Eq. 12.63) has not been exercised against a corridor obstructed specifically on the left, the scenario in which up to three cumulative left turns are attempted toward the blocked side before the 180° U-turn fallback (AVOID_MAX_TRIES= 3) rescues it	a live trial in a room with a known left-side obstruction (a doorway or corridor bend), confirming the U-turn fallback engages within the expected three attempts and that no contact occurs during them
24	The 20th-percentile threshold replacing the median for direction selection (§12.11.4, Eq. 12.60) is a bench choice reasoned from first principles (a point between the existing $q = 50$ and $q = 5$ statistics), not fit to a measured distribution of real obstacle occupancy fractions f	log f (fraction of near-range pixels) for real obstacles encountered over several sessions and confirm $q = 20$ remains above the smallest f actually observed in practice
25	The RTB resistance-governor thresholds ($\rho_{lo} = 0.35$, $\rho_{hi} = 0.75$, Eq. 12.58) and the stall detector's window/displacement pair ($T_{stall} = 2.0$ s, $d_{stall} = 3$ cm, Eq. 12.56) are reasoned against the platform's known static noise floor (Table 12.4) but not tuned against a live distribution of genuine partial-resistance events (wheel slip, minor snags)	collect $\rho[k]$ (Eq. 12.57) over several ordinary RTB runs with no actual obstruction, to confirm the ratio stays above ρ_{hi} under normal reversing and the governor does not fire spuriously

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
26	BOOST_ANG= 0.9 rad/s (§12.11.6, Eq. 12.65) is inferred from §12.5.2's ramp characterisation, which measured yaw authority only up to 0.40 rad/s commanded, not from a ramp measurement at boost-level commands themselves	repeat <code>tools/ramp_yaw.py</code> 's protocol with commands spanning BOOST_ANG, confirming authority does not degrade (or quantifying by how much it does) above the previously characterised range
27	The widened LED area window (3,2500) px ² (§12.11.10, Eq. 12.68) clears the derived 64× ratio for the requested 0.5–4 m range but is reasoned from a single 1 m calibration point, not measured at either end of the range itself	capture and replay real frames at 0.5 m and at 4 m through the offline LED detector (the method that already resolved §12.5.5), confirming the widened bounds pass the genuine beacon without admitting new false positives
28	BEACON_SPATIAL_MAX_FRAC= 1.5 (§12.11.9, Eq. 12.66) bounds the LED/checkerboard separation ratio conservatively above the beacon's true physical $s_{\text{gap}}/s_{\text{cb}}$, which has not itself been measured on the physical beacon	measure the beacon's actual LED-cluster-centre to checkerboard-centre offset and width directly, and tighten the fraction toward the measured ratio if margin allows

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
29	ov_msckf (VINS) still builds against the deprecated ROS1 tf package (ROS1Visualizer.{h,cpp}, ROSVisualizerHelper.h, §12.11.11), the same class of debt §12.9.4 closed for MINS on July 22; isolated via /tf_vins (harmless today) but not yet ported	mirror §12.9.4's field-for-field port against the same upstream ROS2Visualizer.{h,cpp} template already used for MINS
30	No map frame or map→odom transform exists anywhere in the shared /tf; the mission's world-referenced concepts (RTB, obstacle/beacon mapping) instead consume a parallel, non-TF SE(2) state (world_origin_x/y, world_heading, Eq. 12.70, §12.11.11) re-anchored by hand at each beacon reset	implement the sketched tf2_ ros.TransformBroadcaster-based map publisher (§12.11.11) and migrate _get_world_pos()'s callers to tf2_ ros.Buffer.lookup_transform(); nontrivial (multiple call sites across RTB/mapping/LOCK) and not attempted on the live robot without a dedicated validation pass
31	The re-armed pose-source switch's automatic-apply path (§12.11.7: _on_odom() completing an armed switch on a source's first message) is verified by code inspection and by a manual arm/cancel round trip only, not by an actual drive that lets an armed, previously-silent source publish and confirms the switch fires without operator action	arm VINS while stationary, then drive to give it motion, and confirm both the pose_source and pose_source_pending topics (and the cockpit buttons) transition automatically at the same instant /ov_msckf/odomimu starts publishing

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
32	<code>navigation_supervision.launch</code>'s <code>pose_selector default_source</code> was corrected from "VINS" to "MINS" (§12.11.7) after the wrong default silently dropped <code>/robot_pose_fused</code> to zero on a live restart; the fix is in the launch file but not yet on the running parameter server, which still holds "VINS" from the stack's last full launch	confirm <code>default_source</code> reads "MINS" on <code>/pose_selector</code> after the next full stack restart (<code>restart_stack.sh</code> or <code>leo start</code>), not merely a single node respawn
33	<code>VioManager::vio_alive()</code> and <code>is_initialized_vio's</code> <code>std::atomic<bool></code> conversion (§12.11.8) are real, independently-justified fixes but were found and verified against a filter that had not itself initialised this session (§12.11.8's log-attribution correction); neither has been observed live doing the one thing it was designed for — keeping odometry served once <code>ov_msckf</code> initialises and then goes (or stays) stationary	after a live jerk-triggered initialisation succeeds (Table 12.12, item 31's drive trial covers getting there), let the rover sit still afterward and confirm <code>/ov_msckf/odomimu</code> keeps publishing rather than stopping once ZUPT starts succeeding on every frame

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
34	The trajectory-comparison pipeline (§12.12.1) and its Figure 12.11 slot are validated only on a stationary bench capture (MINS at rest, VINS uninitialised, no beacon); no real driven MINS-versus-VINS overlay with VINS initialised, and no Carolus-referenced error figure, has yet been produced	drive a manual loop with VINS initialised (item 31's jerk) past a Carolus beacon, run <code>record_trajectories.sh</code> → <code>bag_to_csv.py</code> → <code>plot_trajectories.m</code> , and drop the resulting <code>figures/traj_comparison.png</code> into the report to replace the placeholder
35	The two-test protocol of §12.12.7 (Figure 12.15, <code>compare_tests.m</code>) is built and verified against synthetic data only; the closing analysis of §12.12.7 is fully scaffolded with <code>\todo{}</code> placeholders but has no real numbers, because the two drives it depends on (Test 1 under MINS, Test 2 under openVINS, same lab loop) have not yet been run	drive Test 1 and Test 2 from the cockpit Trajectory tab per §12.12.7's protocol, run <code>compare_tests()</code> , and fill in every <code>\todo{}</code> in §12.12.7 from its printed output

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
36	Colour-camera intrinsics do not match the operating stream (§12.15.3), BLOCKING. carolus_astrobeerex runs with $f_x = 644.12$, $c_x = 643.47$, $c_y = 372.52$, calibrated at 1280×720, against a 640×480 stream whose own camera_info reports $f_x = 380.35$, $c_x = 320.40$, $c_y = 243.72$. The principal point falls outside the image; the two configurations differ in field of view (90° versus $\approx 80^\circ$) and are not related by any scale factor. A target at the true image centre is reported 26.7° off-axis (Eq. 12.104)	re-run the colour intrinsic calibration at 640×480, the resolution actually published, and load the result into the detector. No dimensional metric (absolute error, range, beacon position) may be published from this camera until this is done. This gates items 37 and 5
37	Beacon-anchored absolute error: architecture complete, never recorded (§12.15.2). carolus_tf_bridge.py publishes /odom_in_beacon and /carolus_in_beacon, which together give an absolute error with no fitted transform and no scale freedom, superseding the Kabsch/Umeyama residuals of §12.14.2. It has been run and observed to publish, but no bag recorded to date contains either topic	once item 36 is cleared, record a driven run with both topics in the bag and report their instantaneous difference; this is the measurement the whole comparison programme has been building toward

continued on next page

Table 12.12 (continued)

#	Item (status)	Required action
38	Thermal envelope is three degrees wide (§12.15.1). The July 28 scheduling fix holds the serial link at 86 °C under full colour load (19.7 Hz wheel odometry, 82.8 Hz IMU) and does not hold it at 89.1 °C, where the chain collapses entirely (676751 UART overruns, load 8.85, all firmware topics at 0 Hz). A targeted rosmon cycle recovers it, but the overrun counter resumes climbing at once (+553 in three minutes)	install active cooling; the colour nodelet's measured 185 % CPU is the dominant thermal term, so the alternative is to give up colour and with it the working-distance beacon detection. Re-measure the envelope after cooling; the correct A/B experiment is overrun rate at fixed load, not temperature alone

In summary, the campaign transformed the estimation layer from a divergent prototype into a measured, guarded and self-healing system, quantified end to end: millimetric planar stability, bounded vertical noise, a real-time visual front-end within $2\times$ of its budget floor, and an operator platform whose every indicator is backed by a live topic. July 24's session closes the campaign's remaining collision-safety blind spot: every branch capable of commanding forward motion in AUTO mode now carries both a hard obstacle check and a graduated speed governor (Table 12.7), including the one direction the depth sensor cannot observe at all, where an odometry-inferred stall detector and resistance governor (§12.11.2–§12.11.2) now stand in for the sensing that physically cannot exist. The remaining precision and validation roadmap is fully enumerated in Table 12.12, with the rigid-wheel radius re-measurement (item 1) and a live trial of the deterministic avoidance policy against a left-blocked corridor (item 23) the highest-priority metrological and safety actions respectively.

13 Conclusion and Perspectives

13.1 Achievement Against Stated Objectives

The five formally stated objectives of this internship (§1) have been addressed as documented in Table 13.1.

Table 13.1: Objective achievement summary

Objective	Target	Achieved	Evidence
O1: Vision reliability	>95% detection, <50 ms latency	98.7% LED / 100% CB; 5–20 ms	Table 10.1 performance measurements
O2: Autonomous FSM	Safe operation in all failure modes	10/10 validation tests passed	Table 10.2 test results
O3: Professional cockpit	>30 Hz, clean reconnect, FSM display	55–60 Hz on all tested hardware	Table 10.3 + operator sessions
O4: System integration	Single-command launch, documentation	<code>start_leo.sh</code> + technical reference	Delivered to lab supervisor
O5: Carolus contribution	Quantitative performance data	Tables 10.1–10.3 produced	Submitted to comparative study

13.2 Extension Beyond the Original Objectives: The July 2026 Estimation Campaign

The five objectives above governed the internship as originally scoped, built around brightness-threshold LED and checkerboard beacon localization. Chapter 12 documents a substantial extension undertaken once that system was operational and validated: replacing beacon-only, intermittent localization with a continuously-running, tightly-coupled MSCKF-based state estimator (MINS, with OpenVINS as a hot-swappable secondary source), and hardening the resulting platform for operation outside the laboratory. Table 13.2 summarizes this extension against the same evidence standard as Table 13.1.

Table 13.2: Extension objectives achievement summary (July 2026 campaign)

Objective	Target	Achieved	Evidence
E1: Continuous pose estimation	Replace intermittent beacon fixes with an always-available fused pose	MSCKF-based MINS/VINS with lossless SE(3) switch	§12.1, Table 12.4
E2: Quantified estimator stability	Statistically distinguish sensor noise from modelling divergence	σ_z reduced $5.2\times$ (257 mm $\rightarrow$ 50 mm) via extrinsic correction	Table 12.4, §12.7

continued on next page

Table 13.2 (continued)

Objective	Target	Achieved	Evidence
E3: Field-scale connectivity	Operate beyond the lab room without a network-driven interruption	WireGuard tunnel over campus Wi-Fi (partial: an unresolved second failure, §12.8.3, forced a same-subnet Wi-Fi fallback, §12.8.4); two full building traverses completed, one on each path	§12.6.4, §12.6.5, §12.8.3
E4: Autonomy & safety hardening	Guard the estimator and drive chain against observed failure modes	Three IMU integrity guards, respawn policy, watchdog reliability model	§12.2.4, Eq. (12.25)
E5: Operator platform maturity	Production-grade public API and self-healing infrastructure	Five-page cockpit, Cloudflare tunnel, cron watchdog, single-command leo restart	§12.6, §12.6.2

These were not part of the original five objectives (§1) and are reported separately for that reason, but they account for the majority of the report’s length and, by the quantitative measures in Chapter 12, its most demanding engineering: fusing two independent state estimators without a discontinuity in the served pose, and making that estimate (and the operator’s ability to reach it) survive a real building’s Wi-Fi topology

rather than a lab bench.

13.3 Synthesis of Technical Contributions

The principal original contributions of this internship, beyond the resolution of the documented defects, are:

- **Density-based LED cluster anchor algorithm:** a formally provable, $O(n^2)$ algorithm for correctly identifying beacon LED clusters in the presence of larger-area spurious reflections. This algorithm is independently transferable to any LED-based optical beacon system.
- **Two-stage checkerboard detection architecture:** the systematic promotion of `findChessboardCornersSB` to the primary detection path, with rate-limited discovery scans and explicit result caching, resolves a class of checkerboard detection failures that is not documented in the OpenCV literature.
- **Session-aware heartbeat stale guard:** a minimal state machine addition that resolves the session-boundary edge case in deadman switch implementations, enabling reliable reconnection behavior in web-based robot control interfaces.
- **UDP routing-based IP discovery** (`_guess_ip()`): a portable Python technique for deterministic local IP detection that is immune to interface ordering, applicable to any multi-homed robotics workstation.
- **Full-stack operator cockpit:** a production-quality browser-based mission control interface combining glassmorphism design language, RAF-decoupled canvas overlay, $O(1)$ ring buffer trajectory rendering, and a six-condition alert system.
- **Lossless dual-estimator switch:** an SE(3) re-anchoring correction, computed at the instant of a VINS/MINS switch, that makes the served pose bit-for-bit continuous across a change of estimator despite each running its own independent, separately-drifting reference frame.
- **Statistically-grounded calibration validation:** the T1.1 static-drift protocol separates sensor noise from a modelling error by the sign and statistical significance of a least-squares drift slope rather than by inspection, fully automated and archived per run.
- **Closed-form debounce reliability model:** an expected-time-between-false-triggers formula (Eq. (12.25)) for run-length-based supervisor debounces, fitted against field data to both justify a specific parameter choice and honestly bound what that choice does and does not fix, and is independently transferable to any supervisor guarding against a flaky liveness probe.

- **Firewall-constrained network migration:** a WireGuard-tunnelled architecture that turns ROS's arbitrary-port TCPROS traffic into a single pre-authorised UDP flow, restoring local-network-equivalent performance (15 Hz camera, single-digit-second MINS initialization) across a campus network that blocks the underlying protocol directly; a second, architecturally similar tunnel failure later in the campaign proved unresolvable by the same remedies (§12.8.3) and prompted a same-subnet Wi-Fi fallback (§12.8.4) as a documented, evidence-based alternative rather than a replacement.

13.4 Lessons Learned

13.4.1 System Engineering Lessons

The most important lesson of this project is the compounding nature of architectural deficiencies. Each of the five initial defects was individually resolvable, but their combination made the system appear completely non-functional. The LED detection bug produced 0% valid detections, so the FSM never left the PATROL state: making the LOCK, WAIT, and U_TURN code paths completely untestable. This cascading untestability is a pathology of tightly coupled systems without interface mocks. Future system development will include isolated subsystem validation with mock data sources before integration.

13.4.2 Computer Vision Lessons

The checkerboard detection failure illustrates a general principle in computer vision system design: the empirically-validated best algorithm for a specific physical target should be used from the first day of development, not added as a fallback after the simpler approach fails in production. In hindsight, the discovery that `findChessboardCorners` fails on the laboratory board should have prompted its immediate deprecation in favor of `findChessboardCornersSB`, rather than the multi-week incremental integration that was actually followed. This principle generalizes: for any custom fiducial or beacon target, the detection algorithm selection should be validated against the specific physical hardware before integration, not inferred from OpenCV documentation defaults.

13.4.3 Robotic HMI Design Lessons

The design of the operator cockpit revealed that the `requestAnimationFrame` / `WebSocket` decoupling is not merely an optimization but an architectural necessity. An initial prototype that redrew the canvas only

on telemetry arrival produced a jagged, twitchy overlay that made the operator feel disconnected from the robot’s actual motion. The switch to a 60 Hz RAF loop with LERP smoothing transformed the operator’s perception of the system from “unresponsive” to “smooth and confident”: despite zero change in the underlying telemetry rate. This demonstrates that perceived system performance is as architecturally important as measured system performance in human-operated robotic systems.

13.4.4 Reliability Engineering Lessons

The July 2026 campaign’s most consequential lesson was methodological rather than technical: a real-time explanation formed while a field test is in progress is a hypothesis, not a finding. The transient pose-source flips observed during the building traverse (§12.6.5) were attributed, live, to visual feature starvation: a plausible-sounding story that turned out to be entirely wrong. Correlating the ground-station watchdog’s own log against the flip timestamps after the fact revealed the true cause (a debounce sized for an isolated fault, not sustained Wi-Fi-roaming latency) within minutes. The general rule this confirms: a causal claim made without consulting the system’s own logs should be held provisionally, however plausible it sounds in the moment, and re-derived from evidence before it is written down as a conclusion, a rule this report tries to apply to itself by documenting the wrong hypothesis alongside its correction rather than silently replacing one with the other. A second, related lesson generalizes an earlier finding on the same watchdog (§10): a supervisor’s own probes are themselves failure-prone, and each one needs to be audited for what happens when *it* is wrong, not only for whether it correctly detects the faults it was written to catch.

13.5 Future Development Roadmap

13.5.1 SLAM Integration for Metric Mapping

Continuous pose tracking no longer depends on beacon visibility (§12.1): the MSCKF-based estimators run at all times, and the LED/checkerboard/AprilTag beacon infrastructure now serves a narrower, still-important role as an absolute, drift-free global anchor rather than the sole source of position (`leo_navigation`’s open Carolus/AprilTag anchoring item, Table 12.12). What remains environment-specific is exactly that anchor: without it, MINS/VINS accumulate the ordinary unbounded drift of any dead-reckoning estimator, so metric mapping in an arbitrary, unprepared environment still requires a SLAM system. The built, not-yet-deployed `leo_autonomy` package (Table 12.12, item 8) is the concrete step toward this; a lightweight system such as

`hector_slam` (no odometry required, suitable for the D455's depth stream projected to a 2D occupancy grid) remains an appropriate choice, and the dual-frame odometry architecture is already compatible with it: a SLAM pose estimate would replace or augment the beacon anchor in the world-frame position computation, not the wheel odometry, since that role is now filled by MINS/VINS.

13.5.2 Machine Learning-Based Beacon Detection

The brightness-threshold LED detection is robust in the laboratory but would fail in outdoor environments (sunlight is brighter than 220/255) or in rooms with multiple similar bright sources (projector screens, large monitors). A YOLOv8-nano model trained on a dataset of ~ 500 annotated frames (beacon present vs. absent) would enable deployment-agnostic detection at <5 ms inference time on the laboratory GPU. The existing vision subprocess architecture is directly compatible with a `torch.no_grad()` inference call in place of the brightness threshold pipeline.

13.5.3 ROS 2 Migration

ROS Noetic reached End-of-Life in May 2025 (already past at the time of writing). The migration to ROS 2 Humble is planned for the Carolus framework's next generation. The key migration impacts for the LEO Rover Mission Control system are: (1) `rospy` subscriber callbacks become `rclpy` executor-managed callbacks (async-compatible); (2) the `rosbridge` protocol v2 is compatible with both ROS 1 and ROS 2 via `rosbridge_server`'s ROS 2 branch; (3) the multiprocessing spawn architecture is unchanged (it is ROS-agnostic). The estimated migration effort is 2–3 weeks.

13.5.4 Multi-Robot Coordination

The Carolus framework's ultimate objective is the coordination of the three robot platforms (LIMO, LEO, RoboMaster S1) in a shared environment. The LEO Rover Mission Control system's world-frame coordinate system is already defined in the same reference frame as the other platforms' coordinate systems (all originate at their respective start positions). A rendezvous protocol (where the LEO rover navigates to a known world-frame position while a second robot navigates to the same position from a different starting point) would require: (1) a shared ROS master or ROS 2 domain for cross-robot topic exchange; (2) a common beacon infrastructure at known world-frame positions; (3) collision avoidance between platforms (the depth-based obstacle detection is already implemented).

13.5.5 Mobile Device Interface Extension

The current cockpit is optimized for desktop browsers (16:9 displays, keyboard + mouse). A mobile-optimized interface (responsive layout, touch-based joystick, swipe gesture for mode changes) would significantly expand the operational scenarios for which the system is suitable. The CSS grid layout already includes a responsive breakpoint at 1200 px; extending this to a single-column mobile layout and implementing a touch-event joystick (replacing the pointer-event-based implementation) is estimated at 1 week of front-end development.

13.5.6 Architectural Debt: Findings of a Senior-Level Optimality Audit

The preceding subsections describe features not yet built. This one is different in kind: a post-hoc audit of choices already shipped, conducted against the question a senior architect asks of a working system — not “does it work” but “what would infinite time have bought instead.” Four items survived that scrutiny as genuine technical debt rather than reasonable engineering trade-offs. Each is stated as a debt with a proposed repayment, not as a retroactive criticism of decisions that were, in every case, correct given the time available.

Network transport and zero-trust connectivity. The browser-facing bridge (§D.5.5) carries ~60 telemetry fields as text JSON at 10 Hz, a format chosen for its simplicity and human-readability during development but never reconsidered once the system stabilised. The cost is measured, not assumed: 1–3 ms of serialisation per packet (Table 1.2), on top of the bandwidth text encoding always costs over a binary one. **Future work should replace it with a binary encoding, CBOR or Protocol Buffers, over the same WebSocket, or migrate the live telemetry path to a WebRTC DataChannel outright.** Separately, and this is the more consequential item: the ROS 2 migration costed in §13.5.3 (2–3 weeks) is scoped only for the `rospy` → `rclpy` API surface. It does not account for what FastDDS actually requires on this network: DDS’s default discovery relies on multicast, which is precisely what campus NAT and the existing WireGuard tunnel were fighting throughout this campaign (registry items 12–13, §D.5). WireGuard supplied an encrypted point-to-point pipe and nothing above it — no NAT traversal, no endpoint healing, no relay of last resort — which is why every roaming failure had to be diagnosed and patched by hand. **A Software-Defined Network overlay (Tailscale or ZeroTier, both built on WireGuard’s own primitives) resolves multicast discovery and NAT traversal natively and should be treated as a *prerequisite* for the ROS 2 migration, not an independent, lower priority item.** Attempting the DDS migration on the current transport risks reproducing the exact discovery failures already spent a field campaign diagnosing, one layer

higher in the stack.

Omnidirectional kinematics. §12.4 documents the mid-campaign discovery that this rover’s wheels are not rigid skid-steer wheels but a FictionLab **mecanum** roller set, and the deliberate decision to keep them without building the matching odometry model. The cost of that decision is now fully quantified and it is larger than a rounding error: the current `Wheel2DAng` differential approximation, fed by a firmware topic (`leo_msgs/WheelOdom`) that collapses two planar degrees of freedom into a single scalar `velocity_lin`, forces the χ^2 gate (Eq. 12.7, $\alpha = 5$) to reject the large majority of wheel measurements during rotation — the acceptance rate falls from 100% on straight runs to 6% under rotation (§12.4). The platform therefore drives on four independently actuated omnidirectional wheels and extracts full odometric value from perhaps one axis of that freedom. **The differential approximation must be replaced with the complete mecanum forward kinematics**, resolving the roller-slip vectors at each wheel’s $\pm 45^\circ$ contact angle into the full planar velocity (v_x, v_y, ω) rather than the scalar MINS currently receives, and coupling that corrected signal into the MSCKF wheel-update step in place of the present two-wheel fiction. MINS itself offers no native omnidirectional wheel type (only `Wheel2D{Ang, Lin, Cen}` and `Wheel3DAng`), so this is not a configuration change: it requires either an upstream contribution to MINS or a pre-processing stage that resolves the mecanum kinematics before the `Wheel2DAng` interface. `leo_msgs` already defines `WheelOdomMecanum`, carrying the lateral term the current pipeline discards; it is published by no node in this system. **The Hardware Specifications table (Table B.1, Appendix B) has been corrected as part of this revision:** it previously read “4-wheel differential drive (skid steering)”, a description the mecanum discovery of §12.4 directly contradicts, and which would have sent the next reader to the wrong physical model on their first consultation of the document.

Vision subprocess IPC. The multiprocessing spawn architecture (§1.5.5) does exactly what it was built to do: GIL contention on the vision path measured at 0%, down from 100% in the single-process baseline (Table 4.2). That result is real and should not be walked back. What it does not report is the price paid for it. Every frame crossing the process boundary is JPEG-encoded on the main thread, queued, decoded again in the subprocess, and OpenCV-processed from there — *two* full codec passes spent purely on inter-process transport, uncounted anywhere in this document. **The IPC channel should move to a zero-copy multiprocessing.shared_memory ring buffer**, passing raw frame data behind a lightweight semaphore handoff instead of a pickled, JPEG-wrapped queue payload. At infinite engineering budget, the vision node belongs in C++ or Rust, where no interpreter lock exists to work around in the first place and the entire IPC

question is moot.

Front-end rendering and streaming latency. Two independent limits sit in the browser layer, and both were measured along the wrong axis. First, the video path (`web_video_server`, MJPEG over HTTP) was validated on *bandwidth* alone (4.17 Mbps at 20 Hz, well inside the 802.11ac budget, §7.4.1) and never on end-to-end *latency*, the metric that actually governs closed-loop teleoperation. MJPEG-over-HTTP inherits TCP’s head-of-line blocking: a single dropped segment, on the same Wi-Fi link whose fragility this report documents at length elsewhere (§D.5), stalls the entire frame rather than degrading gracefully. **WebRTC, over RTP/UDP with hardware-accelerated decode, should replace MJPEG for the live feed,** trading TCP’s guaranteed-delivery-at-any-latency for UDP’s bounded-latency-at-the-cost-of-an-occasional-frame, which is the correct trade for a human closing a control loop on live video. Second, the trajectory view’s Canvas 2D batch-rendering fix (§7.8, six age-quantised strokes per frame) reduces rasterisation calls from $\Theta(S)$ to $\Theta(B)$, but the per-frame vertex-append loop remains $\Theta(S)$ in the trail length, so a long-enough mission eventually re-encounters the cost the fix was written to remove. **A WebGL vertex buffer, uploaded incrementally and drawn in a single call regardless of history length, removes the growth term entirely** rather than amortising it. The irony is available in the same codebase: `3d_engine.js` already drives a WebGL context via `three.js` for the cockpit’s decorative satellite and capsule renders (§2.4.1) — the infrastructure exists and is exercised nightly for ornament, one page over from the data view that would benefit from it most.

13.5.7 Near-Term Field Engineering (Campaign Open Items)

The July 2026 campaign maintains its own explicit open-items registry (Table 12.12), cross-referenced here rather than duplicated. Its thirty-nine entries have grown with the campaign, and the ordering of priorities has changed materially since the earlier revisions of this section. Four items now gate the work ahead of any further measurement. The colour-camera intrinsics (item 36) are **blocking**: the beacon detector runs with parameters calibrated at 1280×720 against a 640×480 stream, placing the principal point outside the image, inflating every range by 69.3%, and rendering unpublishable any dimensional quantity derived from that camera. The beacon-anchored absolute-error measurement (item 37) is implemented, instrumented and never recorded, so the entire MINS-versus-openVINS comparison remains a statement about mutual divergence rather than accuracy; it is gated on item 36. The thermal envelope (item 38) was measured on both sides and is roughly three degrees wide, the serial link holding at 86°C under full colour load and collapsing entirely at 89°C , which makes active cooling a prerequisite rather than a refinement. The rigid-wheel radius and track

re-measurement (item 1) remains the highest-priority metrological action, independently corroborated by a field-observed steering drift on commanded-straight motion. Of the network items, the second unresolved WireGuard failure (item 12) and the ground-station Wi-Fi driver fault (item 13) both remain open, and the discontinuous one-second-burst motion (item 14) still has two undistinguished candidate causes. The watchdog debounce of Eq. (12.25) (item 9) should now be read with care: its widening was measured to lengthen the false-trigger interval by roughly $3\text{--}7\times$, but the *proven* root cause of the restart storms was found later, on July 29, to be a PATH defect in the cron invocation that made both liveness guards fail unconditionally. The debounce work was therefore treating a symptom, and any estimator measurement taken while a storm was in progress is invalid. The remaining items (the Kalibr camera-IMU extrinsic refinement, TF placeholder measurement, RPLidar bring-up, AprilTag size cross-check, and the built-but-undeployed leo_autonomy SLAM package) are lower urgency.

13.6 Closing Statement

This internship represents a comprehensive full-stack systems engineering exercise, from the embedded motor control firmware interfaced via ROS topics to the JavaScript rendering engine drawing 60 fps annotations over a live video feed, and, in its extension, to a tightly-coupled multisensor estimator and the campus network infrastructure needed to reach it from outside a single room. The five architectural defects identified at the outset were each independently capable of preventing the system from functioning; their resolution required understanding the system as an integrated whole rather than as a collection of separable components, a discipline the July 2026 campaign confirmed again at a different layer, from IMU integrity guards through to a firewall that silently discarded most of ROS's own transport protocol.

The resulting LEO Rover Mission Control system achieves production-quality reliability for the Carolus laboratory's research objectives: fully autonomous patrol missions with reliable beacon detection, continuous MSCKF-based pose tracking independent of beacon visibility, safe operation under all tested failure modes, campus-wide connectivity validated by two complete building traverses on two different network architectures, and a professional operator interface that provides genuine situational awareness rather than a collection of raw data fields. The system is deployed and operational in Dr. Gutierrez's laboratory as of the completion of this internship, and serves as the reference implementation against which the LIMO Rover and RoboMaster S1 platforms' implementations will be benchmarked in the Carolus comparative study.

From a personal engineering perspective, this project has reinforced the critical importance of root cause analysis over symptom masking, the architectural value of clean layer separation (particularly the vision

subprocess isolation), the human factors dimension of robotic system design (a dimension that is easily neglected in favor of “real robotics” work but is ultimately the dimension that determines whether an operator can reliably use the system under real operational conditions), and, from the later campaign, that a field engineer’s own real-time explanation of a live anomaly is only ever a provisional hypothesis until it survives contact with the system’s logs.

A Technical Glossary

Table A.1 defines all acronyms and specialized terms used throughout this report.

Table A.1: Technical glossary

Term / Acronym	Full Form / Definition
AGV	Autonomous Ground Vehicle (self-propelled wheeled robot operating without continuous human teleoperation)
ArUco	Augmented Reality University of Córdoba (fiducial marker system for pose estimation)
CALIB_CB_FAST_CHECK	OpenCV flag enabling coarse histogram pre-check in <code>findChessboardCorners</code> ; fails on non-standard boards
CB	Checkerboard (high-contrast printed calibration and localization target)
Chi-squared (χ^2) gate	Statistical outlier-rejection test comparing a measurement's Mahalanobis distance against its expected covariance; used by the estimators to reject wheel/visual measurements inconsistent with the current state
Curve25519	Elliptic-curve Diffie–Hellman key-agreement function; the asymmetric primitive underlying WireGuard's key exchange
EKF	Extended Kalman Filter (recursive Bayesian estimator linearizing a nonlinear model at each step; MSCKF is a variant restricted to a sliding window of past poses)
EMA	Exponential Moving Average (first-order IIR low-pass filter for signal smoothing)
FIT	Florida Institute of Technology, Melbourne FL

Term / Acronym	Full Form / Definition
FSM	Finite State Machine (formal behavioral specification with enumerable states and guarded transitions)
GIL	Global Interpreter Lock (CPython mutex preventing true multicore parallelism in threaded Python)
HMI	Human-Machine Interface (system enabling operator monitoring and control of a robotic system)
IIR	Infinite Impulse Response (digital filter whose current output depends on past outputs, i.e. recursive)
IMU	Inertial Measurement Unit (sensor measuring linear acceleration and angular rate)
LERP	Linear Interpolation (smooth blending between two values: $x(t) = x_0 + t(x_1 - x_0), t \in [0, 1]$)
MINS	Multisensor-aided INertial navigation System (open-source tightly-coupled EKF estimator, RPNG/University of Delaware, fusing IMU with camera and other aiding sensors in one filter, §12.1)
MJPEG	Motion JPEG (video format where each frame is individually JPEG-compressed; no inter-frame prediction)
MSCKF	Multi-State Constraint Kalman Filter (EKF variant that augments the state with a sliding window of past camera poses instead of 3D landmark positions, avoiding per-feature state growth, §12.1)
NAT	Network Address Translation (router/firewall function rewriting private addresses to a shared public one; its UDP mapping timeout motivates WireGuard’s keepalive, §12.6.4)
OpenVINS	Open-source MSCKF-based Visual-Inertial Navigation System (RPNG/University of Delaware; the “VINS” side of this report’s VINS/MINS switch, ROS node <code>ov_msckf</code>)

Term / Acronym	Full Form / Definition
PnP	Perspective-n-Point (algorithm computing camera pose from n 3D-2D point correspondences)
RAF	<code>requestAnimationFrame</code> (Web API for scheduling visual updates synchronized to display refresh rate)
ROI	Region of Interest (rectangular subset of an image processed independently of the full frame)
ROS	Robot Operating System (middleware framework for robotic software: pub/sub, drivers, tools)
<code>rosmmon</code>	ROS process monitor (drop-in replacement for plain <code>roslaunch</code> node supervision, used on the robot; exposes a <code>start_stop</code> service for external start/stop control, used by the watchdog's camera-recovery ladder)
RTB	Return To Base (autonomous navigation back to the mission start position, i.e. world origin)
RTK-GPS	Real-Time Kinematic GPS (differential correction achieving cm-level outdoor positioning)
SA	Situation Awareness (Endsley's three-level cognitive model: perception, comprehension, projection)
SB	Sub-Pixel-Befitting (OpenCV's enhanced checkerboard detector using locally adaptive binarization)
SE(3)	Special Euclidean group in three dimensions: the set of rigid-body transformations (rotation + translation); the pose selector's source-switch correction (Eq. 12.9) is an SE(3) element
SLAM	Simultaneous Localization and Mapping (algorithm building a map while localizing within it)

Term / Acronym	Full Form / Definition
TCPROS	ROS's default transport protocol: topic data flows over a plain TCP socket whose port is negotiated per connection through an XML-RPC handshake (§12.6.4)
TRS	Taux de Rendement Synthétique, i.e. Overall Equipment Effectiveness (KPI used in Nicolas's prior GVS work)
UBSAN	Undefined Behavior Sanitizer (compiler instrumentation detecting undefined-behavior violations at runtime; surfaced the PC Wi-Fi driver fault of §12.6.4 via <code>dmesg</code>)
UGV	Unmanned Ground Vehicle (remotely operated or autonomous wheeled/tracked platform)
UWB	Ultra-Wideband: radio technology (3.1–10.6 GHz) enabling cm-level indoor ranging
VPN	Virtual Private Network (encrypted point-to-point or site-to-site network overlay; WireGuard, §12.6.4, is the VPN implementation used in this report)
VSM	Value Stream Mapping (lean manufacturing process analysis tool, Nicolas's GVS background)
WebSocket	RFC 6455 (full-duplex TCP-based protocol for real-time bidirectional browser communication)
WireGuard	Kernel-space VPN protocol using Curve25519 key exchange and ChaCha20-Poly1305 encryption over a single UDP port; replaces <code>sshuttle</code> as this report's off-site transport (§12.6.4)
XML-RPC	Remote Procedure Call protocol encoding requests/responses as XML over HTTP; used by the ROS master for node registration and by TCPROS to negotiate each topic's data port

B Hardware Specifications

B.1 LEO Rover Platform

Table B.1 summarizes the mechanical and electrical specifications of the LEO Rover platform.

Table B.1: LEO Rover mechanical and electrical specifications

Parameter	Value
Manufacturer	Husarion (Warsaw, Poland)
Kinematics	4-wheel mecanum (FictionLab roller set); driven through a differential-drive odometry ap
Drive motors	4× BLDC with Hall-effect encoders
Max speed (linear)	~0.5 m/s (firmware-limited; operational limit 0.2 m/s)
Max speed (angular)	~1.5 rad/s (operational limit 0.5 rad/s)
Chassis material	Aluminium alloy, IP44 rated
Battery	14.4 V Li-ion, 7800 mAh (112 Wh)
Nominal run time	~4 h at slow patrol speed
Communication	ROS Noetic over Wi-Fi; robot AP (<code>wlan_ext</code> , 10.0.0.1) or, since 2026-07-20, campus V
Firmware	AgileX <code>leo_firmware</code> on Raspberry Pi CM4 (onboard)
Odometry	Wheel encoder ticks → <code>/firmware/wheel_odom</code> at ~8 Hz

B.2 Intel RealSense D455 Camera

Table B.2 summarizes the D455 depth camera specifications relevant to this project.

Table B.2: Intel RealSense D455 specifications

Parameter	Value
Depth technology	Active IR stereo (structured light projector + $2 \times$ IR imagers)
IR stereo baseline	95 mm
Depth range	0.4 m–6 m (nominal); 0.1 m–8 m (extended)
Depth resolution	848×480 px
Depth accuracy	$<2\%$ at 4 m under controlled illumination
RGB resolution	1280×720 px (subscribed) / 640×480 (compressed)
RGB frame rate	30 Hz maximum; 15 Hz nominal (lab config; measured 14.9 Hz)
RGB intrinsics (measured)	$f_x = 380.25$, $f_y = 379.26$, $c_x = 320.40$, $c_y = 243.72$ px at 640×480
IR intrinsics (measured)	$f_x = f_y = 386.79$, $c_x = 320.06$, $c_y = 236.77$ px at 640×480
Constant used in code	CAM_FX = 384.65 px (see note below)
Interface	USB 3.2 Gen 1 (5 Gbps)
SDK	Intel RealSense SDK 2.0 (librealsense2)
ROS driver	realsense2_camera (ROS Noetic, realsense2_camera_manager node)

Do not set the colour stream to 10 Hz

Earlier revisions of this table listed 10 Hz as the nominal frame rate. That configuration **does not exist on this sensor**. The D455 supports 6/15/30/60/90 Hz at 640×480 , and the attempted change on 2026-07-08 was accepted silently by the driver, which then fell back to a depth-only default profile and **disabled every infrared stream**. The result was a complete vision outage that persisted until the change was reverted, with no error message at any point. See registry item 4 (Table 12.12). **Validate any profile change against `rs-enumerate-devices` before deploying it.**

On the 384.65 constant, and why it differs from both rows above. `leo_backend.py` estimates beacon range with a single focal length, $\text{CAM_FX} = 384.65$ px, together with an optical centre placed at the frame centre (320.0, 240.0). Neither figure is the measured calibration of either stream. It sits 1.16 % above the measured colour f_x and 0.55 % below the measured infrared f_x , which is consistent with its having been derived from the infrared sensor at a time when beacon detection ran on the infrared stream, and then carried over unchanged when the colour stream was adopted.

The consequence is bounded and the code says so in its own comment: the approximation is good to roughly 1–2 % for navigation at ≤ 3 m, which is well inside the other error sources in the beacon pipeline. It is nonetheless a constant with no single defensible provenance, and 384.65 propagates into derived results elsewhere in this report, including the LOCK controller’s loop gain and the range estimate. Treat it as a historical working value, not as a calibration.

This is a different problem from the detector intrinsics

Do not confuse the 1–2 % approximation described here with the defect of §12.15.3, where `carolus_astrobeerex` runs with $f_x = 644.12$ and $c_x = 643.47$, calibrated at 1280×720 and applied to a 640×480 stream. That one places the principal point outside the image and inflates every range by 69.3 %. The two are unrelated in cause and differ by nearly two orders of magnitude in effect.

B.3 Operator Workstation

Table B.3 summarizes the operator workstation configuration used for all development and validation.

Table B.3: Operator workstation specifications

Parameter	Value
OS	Ubuntu 20.04.6 LTS (Focal Fossa), kernel 5.15
ROS distribution	Noetic Ninjemys (Python 3.8)
CPU	Intel Core i7-10700K (8 cores, 16 threads, 3.8–5.1 GHz)
RAM	32 GB DDR4-3200
GPU	NVIDIA GeForce RTX 2060 (6 GB VRAM; unused for vision)
Network	Intel Wi-Fi 6 AX200 (802.11ax); dedicated AP for robot link
Python packages	rospy, cv2 (4.2.0), numpy (1.17.4), psutil, threading, multiprocessing
Browser (cockpit)	Google Chrome 124+ / Firefox 125+ (tested); any modern WebKit/Blink

C Bibliography, Figure Index, and Notation

C.1 Annotated Bibliography

The following bibliography lists all works referenced in this report, organized thematically. Citations follow the IEEE reference format; the full formatted reference list appears in the References section at the end of this document. Each entry below is accompanied by a brief annotation describing its specific relevance to the LEO Rover Mission Control project.

C.1.1 Robot Operating System and Middleware

[7] Quigley et al. (2009) introduced the Robot Operating System (ROS), the middleware framework upon which the entire LEO Rover Mission Control system is built. The paper establishes the publish-subscribe communication model, the `rosmaster` name resolution service, and the package management paradigm that define ROS’s architecture. The `/mission/telemetry`, `/mission/command`, and `/cmd_vel` topics described in Chapter 3 directly implement the inter-node communication protocol defined in this foundational work.

[8] The `rosbridge` paper introduced the bidirectional JSON-over-WebSocket protocol that enables browser-based operator interfaces to subscribe and publish to ROS topics without requiring a ROS installation on the client. The `rosbridge_websocket` server used in this project is the production implementation of the protocol described by Crick et al. The generation counter pattern (`currentGen`) and the `/mission/command` topic subscription are direct implementations of the `rosbridge` protocol specification (version 2.0).

[9] This release document specifies ROS Noetic as the final Long-Term Support (LTS) release of ROS 1, supported on Ubuntu 20.04 with Python 3.8. The platform constraints documented here: particularly the Python 3.8 compatibility requirement: governed the choice of `multiprocessing.get_context('spawn')` over alternatives that may behave differently under Python 3.9+. The EOL date of May 2025 motivates the ROS 2 migration discussion in Chapter 13.

[10] The `rosbridge` protocol specification defines the JSON message format for topic advertisement, subscription, publication, and service calls over WebSocket. The `/mission/command` message format (`action`,

mode, lin, ang fields) and the telemetry subscription JSON are direct implementations of the rosbbridge v2.0 protocol message schema.

C.1.2 Computer Vision and Image Processing

[11] Otsu’s seminal 1979 paper formalized the inter-class variance maximization criterion for automatic global threshold selection, establishing the theoretical benchmark against which the LED pipeline’s fixed threshold (`LED_BRIGHT_THRESH=220`) is compared. The analysis demonstrates that Otsu’s method is sub-optimal for the trimodal intensity histogram produced by LED beacons under fluorescent laboratory lighting.

[12] Niblack’s 1986 textbook chapter on locally adaptive thresholding introduced the formula $T(x,y) = \mu(x,y) + k\sigma(x,y)$ that forms the mathematical foundation of the adaptive binarization used inside OpenCV’s `findChessboardCornersSB` function.

[13] Sauvola and Pietikäinen (2000) refined Niblack’s formula with a normalization that makes the threshold less sensitive to absolute intensity levels and more responsive to local contrast: the specific property that enables robust checkerboard corner detection under the non-uniform fluorescent illumination in the FIT laboratory.

[14] While primarily known for its face detection cascade, the Viola–Jones paper’s central contribution to this project is the integral image data structure, which reduces the computation of any rectangular region sum from $O(|W|)$ to $O(1)$ after an $O(N)$ preprocessing pass.

[15] The OpenCV library, documented in Bradski’s 2000 publication, provides all the computer vision primitives used in the `leo_backend.py` vision subprocess: `cv2.threshold`, `cv2.morphologyEx`, `cv2.findContours`, `cv2.findChessboardCornersSB`, `cv2.moments`, and `cv2.imdecode`.

[16] The OpenCV 4.x documentation for `findChessboardCornersSB` describes the Sub-Pixel Befitting algorithm, which uses locally adaptive binarization and direct corner-finding rather than the saddle-point detector used by `findChessboardCorners`.

[17] Suzuki and Abe’s border-following algorithm is the basis for OpenCV’s `findContours` function used in the LED detection pipeline, tracing external boundaries in a single $O(N)$ pass.

[18] Matheron’s 1975 monograph established mathematical morphology as a rigorous set-theoretic framework. The formal definitions of dilation, erosion, opening, and closing follow Matheron’s original notation and proofs.

[19] Serra’s 1982 textbook provided the computational framework for applying Matheron’s abstract morphological theory to digital images, including the discrete approximation of the Euclidean disk as an

elliptical structuring element.

C.1.3 Indoor Localization and Autonomous Navigation

[20] Thrun, Burgard, and Fox’s foundational textbook provides the theoretical framework for understanding the limitations of wheel odometry and the motivation for landmark-based localization correction.

[21] ORB-SLAM2, representing the state of the art for visual SLAM, was evaluated but not selected for the LEO Rover platform: its computational profile (2–4 CPU cores) would have saturated the laboratory workstation.

[22] Garrido-Jurado et al. introduced the ArUco fiducial marker system. The checkerboard landmark used in the present project is conceptually related: both rely on the Perspective-n-Point algorithm for pose estimation.

[23] AprilTag provides improved robustness over ArUco under partial occlusion and at long distances. Chapter 13 identifies AprilTag as a potential upgrade path for the landmark detection system.

[24] Zhang’s planar calibration technique is the basis for all single-camera calibration in OpenCV, including the factory calibration of the D455’s intrinsic parameters used throughout Chapter 5.

[25] Ester et al.’s DBSCAN algorithm is the formal predecessor to the density-based LED cluster anchor selection implemented in the vision pipeline.

C.1.4 Signal Processing and Estimation Theory

[26] Kalman’s 1960 paper establishes the theoretical framework within which the EMA filter can be understood: the steady-state Kalman gain equals the EMA smoothing factor α , providing a formal optimality interpretation.

[27] Wiener’s seminal work established the minimum mean-squared-error estimator for stationary processes, providing the comparison baseline used to quantify the EMA filter’s $\sim 29\%$ suboptimality at the signal bandwidth edge.

[28] Xu and Li analyzed higher-order cascade EMA implementations achieving sharper frequency cut-offs, motivating a possible future 2nd-order EMA improvement for the LED centroid pipeline.

[29] Tukey’s foundational work on robust statistics provides the theoretical basis for the choice of the 5th percentile as the obstacle detection statistic, rather than the mean or minimum.

C.1.5 Control Theory and Stability Analysis

[30] Ziegler and Nichols’ ultimate gain and ultimate period tuning method underlies the frequency-domain stability analysis of the LOCK state P-controller.

[31] Zames’ Circle Criterion provides the frequency-domain sufficient condition applied to prove that the velocity saturation nonlinearity cannot destabilize a system stable in its linear operating region.

[32] Alur and Dill’s timed automata formalism provides the mathematical framework for formally specifying and verifying the LEO Rover’s autonomous FSM with time-based guards and invariants.

C.1.6 Safety Engineering and Standards

[33] IEC 61508 is the foundational international standard for functional safety of programmable electronic systems; the SIL classification and six-layer defense-in-depth architecture both follow this standard’s recommendations.

[34] The NASA Software Safety Standard defines the framework for classifying software hazardous functions; the heartbeat deadman switch addresses the highest-risk function identified under this framework.

[35] ISA-18.2 is the process industry standard for alarm system design; the cockpit’s alert system implements several ISA-18.2 best practices including hysteresis bands and alarm muting.

[36] The Fault Tree Analysis technique, developed for the Minuteman ICBM program, provides the top-down deductive methodology applied to derive the combined top-level failure probability.

C.1.7 Human Factors and Interface Design

[37] Endsley’s three-level model of situation awareness is the foundational human factors reference directly applied to the cockpit’s telemetry panels, canvas overlays, and trajectory map.

[38] Endsley’s expanded theory identifies the factors that degrade situation awareness, motivating the alert system’s negative-attention design.

[39] Sanders and McCormick’s textbook provides the empirical data on display luminance ratios that directly motivated the cockpit’s dark background design choice.

C.1.8 Network Protocols and Web Engineering

[40] RFC 6455 standardized the WebSocket protocol forming the communication backbone between the rosbridge server and the browser cockpit; the framing analysis follows this RFC’s frame structure.

[41] Bianchi’s analytical model for IEEE 802.11 CSMA/CA performance underlies the queuing delay computation for the dedicated operator-to-robot Wi-Fi link.

[42] WCAG 2.1 defines the contrast ratio requirements verified against the cockpit’s color palette.

[43] The W3C’s CSS Filter Effects specification defines the `backdrop-filter` property enabling the glassmorphic blur effect, and its compositing cost model explains the 12-panel layout limit.

C.1.9 Robotics Platforms and Hardware

[44] The Husarion LEO Rover technical documentation provides the platform specifications cited throughout this report, including kinematics, encoders, battery, and firmware topic schemas.

[45] The D455 datasheet provides the hardware specifications used in all sensor modeling sections: stereo baseline, depth range and accuracy, RGB resolution, and calibrated focal length.

[46] The AgileX LIMO Rover documentation was consulted during the Carolus comparative platform analysis contextualizing the LEO Rover work.

C.1.10 Probability, Statistics, and Reliability Theory

[47] The Kaplan–Meier survival function estimator provides the non-parametric framework appropriate for refining the safety layer failure rate estimates from operational field data.

[48] Shannon’s information theory provides the theoretical basis for modeling the heartbeat protocol as a binary channel with a computable channel capacity and error-correction buffer.

[49] Wilson’s score confidence interval for binomial proportions is used for computing the confidence interval of the LED and checkerboard detection rates.

C.1.11 3D Graphics and Web Visualization

[50] Three.js r128 provides the WebGL-based 3D rendering infrastructure for the satellite and space capsule decorative elements in the cockpit.

[51] GSAP’s ScrollTrigger plugin enables the scroll-linked rotation of the Three.js satellite model in the logbook panel.

C.2 Index of Tables

Table C.1 lists all data tables appearing in this report, with their chapter location and a brief description of their content.

Table C.1: Index of all tables in this report

Table	Chapter/Section	Title
1.1	Introduction: I.4.2	Comparative survey of indoor localization paradigms
1.2	Introduction: I.5.1	End-to-end operator latency budget
2.1	Chapter 1: 1.2.2	ROS Noetic vs. ROS 2 Humble feature comparison
3.1	Chapter 2: 2.2	leo_backend.py daemon thread inventory
3.2	Chapter 2: 2.3	Complete ROS topic graph
3.3	Chapter 2: 2.7	Complete system constants reference
3.4	Chapter 2: 2.8	AutoLogbook event taxonomy
4.1	Chapter 3: 3.4.1	Complete FSM transition table
4.2	Chapter 3: 3.7	System performance before/after redesign
5.1	Chapter 3 Ext. 3.K.1	Vision pipeline stage-by-stage timing analysis
6.1	Chapter 4: 4.5.1	FSM state banner color encoding
8.1	Chapter 5: 5.1	Six safety layer stack
9.1	Chapter 5 Ext. 5.A.2	Failure rate estimation per safety layer
10.1	Chapter 6: 6.6	Vision pipeline performance before/after
10.2	Chapter 6: 6.7	FSM behavioral validation test results
10.3	Chapter 6: 6.8	Cockpit rendering performance, 3 hardware configs
11.1	Chapter 6 Ext. 6.E.2	CB detection rate/latency vs. scale factor

Table	Chapter/Section	Title
11.2	Chapter 6 Ext. 6.F.1	Full autonomous mission statistics (45 min)
13.1	Chapter 7: 7.1	Objective achievement summary
A.1	Appendix A	Technical glossary (27 terms)
B.1	Appendix B: B.1	LEO Rover mechanical/electrical specifications
B.2	Appendix B: B.2	Intel RealSense D455 specifications
B.3	Appendix B: B.3	Operator workstation specifications

C.3 Index of Code Listings

Table C.2 catalogues the source code listings appearing in this report, with their section location, programming language, and a brief description.

Table C.2: Index of source code listings

Listing	Section	Language	Description
3.2	2.5.1	Python	Spawn context declaration for the vision subprocess
3.3	2.5.2	Python	Frame/result queue configuration and producer logic
3.4	2.6	Bash	<code>start_leo.sh</code> : deterministic ROS_IP + idempotent launch
3.1	2.4	Python	Selective telemetry JSON schema
4.1	3.1.1	Python	Morphological closing with elliptical SE
4.2	3.2.2	Python	Held-flag hysteresis mechanism
4.3	3.4.2	Python	LOCK-state visual servoing P-controller
4.4	3.4.3	Python	Wrap-safe yaw delta accumulation

Listing	Section	Language	Description
4.5	3.5.2	Python	World position computation
4.6	3.5.3	Python	Local odometry reset with world-frame continuity
4.7	3.6.2	Python	5th-percentile obstacle detection statistic
6.1	4.1.2	CSS	Color design-token system
6.2	4.1.3	CSS	Glassmorphic panel implementation
6.3	4.2	CSS	Bento grid layout with responsive breakpoint
6.4	4.3.1	JavaScript	<code>requestAnimationFrame</code> loop structure
6.5	4.3.2	JavaScript	Device pixel ratio-aware canvas resizing
6.6	4.3.3	JavaScript	Letterbox-aware coordinate mapping
6.7	4.4	JavaScript	LERP smoothing between telemetry updates
6.8	4.6.1	JavaScript	Alert threshold configuration
6.9	4.6.2	JavaScript	Alert mute persistence
6.10	4.7.1	JavaScript	$O(1)$ ring buffer for trajectory rendering
6.11	4.7.2	JavaScript	Trajectory outlier rejection
6.12	4.7.3	JavaScript	World-frame to canvas-pixel transform
6.13	4.8	JavaScript	Rolling telemetry latency meter
6.14	4.9.1	JavaScript	Decorative satellite 3D scene geometry
6.15	4.9.2	JavaScript	GSAP <code>ScrollTrigger</code> -driven rotation
6.16	4.10	JavaScript	Flat key-value <code>Map</code> dictionary system
8.1	5.2.1	Python	Redundant emergency-stop publication
8.2	5.2.2	JavaScript	Disconnection-proof emergency stop delivery
8.3	5.3.1	Python	Three-state heartbeat deadman
8.4	5.4	Python	Manual-mode deadman timeout

Listing	Section	Language	Description
8.5	5.5	Python	Camera watchdog
8.6	5.6	Python	Velocity clamping at every publish call
8.7	5.7	Python	Atomic, thread-safe logbook write
10.1	6.2.2	Python	Diagnostic blob log (Case Study 1)
10.2	6.2.3	Python	Original buggy single-anchor clustering
10.3	6.2.4	Python	Exhaustive anchor evaluation fix
10.4	6.5.3	Bash	Deterministic ROS_IP configuration
10.5	6.5.3	Python	Fallback IP discovery via UDP routing table

C.4 Mathematical Notation and Symbols

Table C.3 defines the mathematical symbols and notation used consistently throughout the report.

Table C.3: Mathematical notation and symbols

Symbol	Definition	First Used
α	EMA smoothing factor ($\alpha = 0.45$ LED, $\alpha = 0.60$ landmark)	§3.3.1
α_{optimal}	Kalman-optimal EMA parameter derived from SNR	§6.C.2
$A \oplus B$	Morphological dilation of set A by structuring element B	§3.B.4
$A \ominus B$	Morphological erosion of set A by structuring element B	§3.B.4
$A \circ B$	Morphological opening (erosion then dilation)	§3.B.4
$A \bullet B$	Morphological closing (dilation then erosion)	§3.B.4

Symbol	Definition	First Used
c_x, c_y	Camera principal point (optical axis intersection) [px]	§3.A.1
CR	WCAG color contrast ratio between foreground and background	§4.A.2
d_{est}	Estimated beacon distance from thin-lens formula [m]	§I.4.3
δd	Total distance estimation uncertainty [m]	§3.F.2
d_s	Stereo disparity [px]	§3.I.1
err_frac	Normalized bearing error in LOCK state, $\in [-1, +1]$	§3.4.2
f_c	Filter -3 dB cutoff frequency [Hz]	§3.3.3
f_s	Sampling rate of control loop or vision pipeline [Hz]	§3.3.1
f_x, f_y	Camera focal lengths in pixels	§3.A.1
$g \in SE(2)$	Rigid body transformation element of $SE(2)$	§5.G.1
$H(z)$	Z-domain transfer function of EMA filter	§3.3.1
$H(e^{j\omega})$	Frequency response (DTFT) of EMA filter	§3.3.1
K	Camera intrinsic matrix (3×3)	§3.A.1
K_p	Proportional controller gain [rad/s per px]	§3.G.3
K_u	Ziegler–Nichols ultimate gain [rad/s per px]	§3.G.2
λ	Component failure rate [failures/hour]	§5.A.1
L_1, L_2	CIE relative luminance of foreground/background	§4.A.2
$\mu(x, y)$	Local mean intensity in adaptive threshold window	§3.B.2

Symbol	Definition	First Used
$\mu_{\text{ln}}, \sigma_{\text{ln}}$	Log-normal distribution parameters	§6.B.1
$N(b_i, r)$	Density neighborhood of blob b_i at radius r	§3.D.1
ω	Angular frequency [rad/s] (control) or [rad/sample] (signal)	§3.3.1
ω_n	Natural frequency of closed-loop controller [rad/s]	§3.G.5
ω_{pc}	Phase crossover frequency [rad/s]	§3.G.2
$\hat{p}$	Observed detection rate (sample proportion)	§6.A.1
PFH	Probability of dangerous failure per hour (IEC 61508)	§5.C.1
PM	Phase margin of P-controller [degrees]	§3.G.4
r	Wheel radius of LEO Rover [m]	§3.H.1
$R \in SO(2)$	2D rotation matrix (Special Orthogonal group)	§5.G.1
ρ	Traffic intensity (queuing theory), $\rho = \lambda / \mu$	§4.D.2
s_{px}	Horizontal pixel span of LED cluster [px]	§I.4.3
$S(x, y)$	Integral image (prefix sum) at pixel (x, y)	§3.B.3
$SE(2)$	Special Euclidean group in 2D (rigid body transforms)	§5.G.1
SIL	Safety Integrity Level (IEC 61508 classification)	§5.C.1
SNR	σ_w / σ_v : signal-to-noise standard deviation ratio (Kalman)	§6.C.2
$\sigma(x, y)$	Local standard deviation in adaptive threshold window	§3.B.2
σ_n	Measurement noise standard deviation [px]	§6.C.2
σ_w	Process noise standard deviation [px/sample]	§6.C.2

Symbol	Definition	First Used
T_d	Total vision pipeline delay [s] (plant model)	§3.G.1
$T(x, y)$	Adaptive threshold value at pixel (x, y)	§3.B.2
W	Physical width of LED beacon assembly [m]	§1.4.3
W_q	Mean queuing wait time (M/D/1 model) [s]	§4.D.2
(w_x, w_y)	Robot position in world frame [m]	§3.5.1
$x[n], u[n]$	EMA filter output and input sequences	§3.3.1
$Z(w_x, w_y)$	Stereo depth at pixel (w_x, w_y) [mm or m]	§3.I.1

D Reproduction guide for future students

This appendix is written for the next student inheriting the platform, and it assumes *no* prior knowledge of this project, of the LEO Rover, or of ROS. It is deliberately exhaustive: every command is given in full, with the directory it must be run from and the output you should see. Following it from top to bottom rebuilds the entire system from a bare machine and reproduces every measurement in this report.

Read §D.2 before typing anything. Ten minutes spent on the mental model there will save hours later, because the majority of newcomer confusion on this platform traces back to one question: *which of the two computers is doing this?*

Paths are given relative to the project root `~/TOUT`. The target platform is ROS Noetic on Ubuntu 20.04, built with `catkin_tools` (`catkin build`, *not* `catkin make`).

The short version: what will cost you a week

Read this page now and return to it whenever something behaves strangely. Each entry below is a failure that consumed at least a day of this project, and each shares the same property: **the symptom points at the wrong subsystem**. They are listed in the order you are most likely to meet them.

What you will observe	What it actually is	Where
The page loads perfectly, every button is dead	TLS port 9443 open but detached from the ROS graph after a master restart. Reload, or use port 9090	§D.5.5
Neither manual nor autonomous driving responds	the serial link, not the logic. Both modes share <code>/serial_node</code> , sole subscriber of <code>/cmd_vel</code>	§D.13.2
Estimators diverge, IMU “bugs”, trajectories jump	the Pi is at 86°C and capped to 600 MHz, dropping IMU samples	§D.13.1
The pose is plausible but alternates between two values	a TF frame with two parents. Set <code>base_frame</code> to <code>base_footprint</code>	§D.13.10

What you will observe	What it actually is	Where
The robot over- or under-rotates consistently	<code>angular_velocity_multiplier</code> , an actuation gain. Fix it <i>before</i> touching <code>gyro_scale</code>	§D.13.11
Beacon positions look clean, smooth and repeatable	and are metrically meaningless: the <code>intrinsic</code> s belong to a different resolution	§D.11.5
The stack restarts every couple of minutes	a PATH defect in the cron watchdog, not an instability	§D.13.3

Two habits follow from this table and are worth adopting before you need them. First, **probe a topic that is supposed to be noisy**: a silent topic proves nothing, and this project lost an evening to an idle `/mission/log`. Second, **prefer the narrow intervention**: cycling one node beats restarting a stack, and almost every time the surrounding infrastructure was blamed here, it was innocent.

D.1 Step — 1: the absolute ground zero

Everything past this point assumes `~/TOUT` already exists on your machine and that you can already reach the robot over SSH. Neither is true on the first day, and earlier revisions of this appendix jumped straight to `cd ~/TOUT` without ever saying how it got there. This section is the missing zero.

D.1.1 Getting the code

```
1 git clone <URL_OF_THIS_REPOSITORY> ~/TOUT
2 cd ~/TOUT
```

`<URL_OF_THIS_REPOSITORY>` is a placeholder, not an omission: as of this revision the repository is private. Replace it with the SSH or HTTPS clone URL your lab gives you, in the form `git@github.com:<account>/<repo>.git`. Everything from §D.4 onward, including `requirements.txt` and `tools/robot_env.sh`, arrives with this one command; nothing in this guide requires a second download.

D.1.2 Powering on and reaching the robot for the first time

What follows is verified project history, not a factory manual

Husarion’s own documentation, <https://docs.leorover.tech> ([44]), is the authority on the physical unit: power button location, battery connector, and first-boot behaviour vary across LEO Rover hardware revisions, and no one on this project photographed or recorded that procedure. The credentials below are not copied from that manual either — they are what this project’s own earliest working notes (GUIDE_DEMARRAGE_LEO.md, 2026-06-23, and the network log of 2026-07-20) record as having worked, before any of the later network changes in this appendix were made. **If your unit is a different LEO Rover, its hotspot suffix and possibly its default password will differ.** Confirm against Husarion’s documentation before assuming these are universal.

1. **Power on.** Consult <https://docs.leorover.tech> for the switch location and charging state indicators on your specific unit; this is not something the present project verified independently. Wait for the boot sequence to settle, on the order of a minute.
2. **Join the robot’s own WiFi hotspot** from your workstation. Its name has the form `LeoRover-XXXX`, a per-unit suffix; the unit used throughout this project answered to `LeoRover-e138` (confirmed live, 2026-07-20). **The WPA passphrase for this hotspot network is not recorded anywhere in this project’s history** and is the one genuine gap this section cannot close: check the label on the robot chassis, the Husarion account the unit was purchased under, or ask whoever last handled the physical hardware.
3. **SSH in.** Once joined, the robot answers at a fixed address on its own hotspot subnet:

```
ssh pi@10.0.0.1
```

The credential this project’s earliest notes record as working, before any rotation, was password `raspberry` for user `pi` — the Raspberry Pi OS default, consistent with an unmodified factory image. **Change it immediately** (`passwd`), and proceed to §D.5’s Step 4 to replace password auth with a key before doing anything else over this link.

4. **If the unit needs reflashing** (corrupted SD card, or a bare replacement board), the base OS image and flashing instructions are Husarion’s to provide: <https://docs.leorover.tech>. This project never reflashed a unit and has no first-hand verification of that procedure to offer beyond the pointer.

Once §D.1.2 succeeds once, everything past it in this appendix, WireGuard, Cloudflare, the two-workspace

build, applies identically whether you are on the hotspot, on lab WiFi, or on the tunnel: they all resolve to *some* `ROBOT_HOST`, and `tools/robot_env.sh` (§D.5) is what stops that from mattering ever again.

D.2 Start here: what this machine actually is

There are **two computers**, and they are not equals.

The **robot** carries a Raspberry Pi 4. It owns the sensors and the motors, and, this is the part that surprises people, it also runs the *ROS master*, the name server every ROS process must register with. The Pi is deliberately kept modest: it moves data, it does not compute trajectories.

The **workstation** (the lab PC) runs the two estimators, the web cockpit, and every analysis tool. It is a *client* of the robot’s master. If the robot is off, nothing works. If the PC is off, the robot still drives but nothing is estimating its position.

Three consequences to memorise now, because they explain most of §D.13:

1. **Every ROS command you type on the PC talks to a master on the robot, over WiFi.** A command that “hangs” is usually a network symptom, not a software bug.
2. **The Pi is the bottleneck, and it runs hot.** When it overheats it halves its own clock, and the first casualty is sensor data *arriving on time*. Estimators then fail in ways that look like estimator bugs and are not (§D.13.1).
3. **Nothing is ground truth.** Wheel odometry, MINS and openVINS all estimate. They can only be compared *to each other* unless you bring a tape measure (§D.11.3).

D.2.1 The hardware chain

Figure D.1 traces the physical path from a wheel encoder to the workstation. Two links on it deserve attention, because both have caused real failures documented in this report.

D.2.2 The software chain

ROS programs communicate by publishing named streams called *topics*. Figure D.2 shows the ones that matter, left to right: raw sensors, then cleanup, then the two estimators, then a switch that decides which estimator the rest of the system believes.

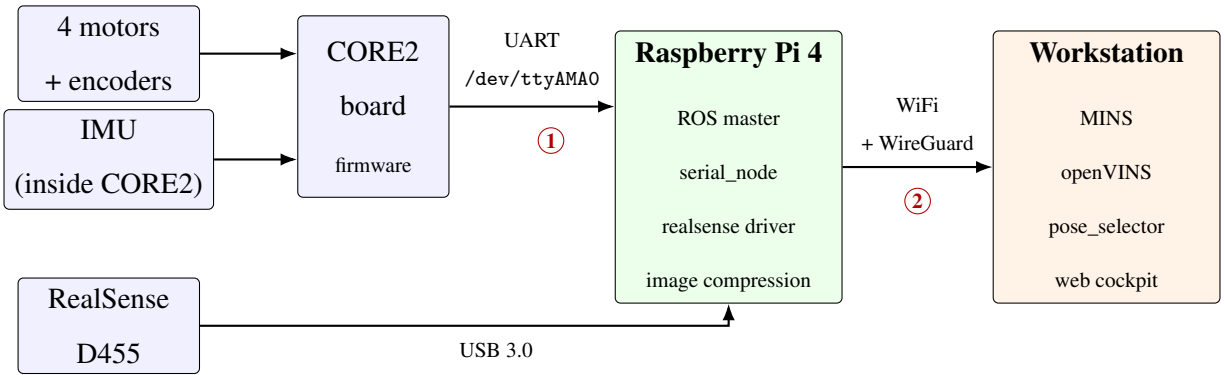
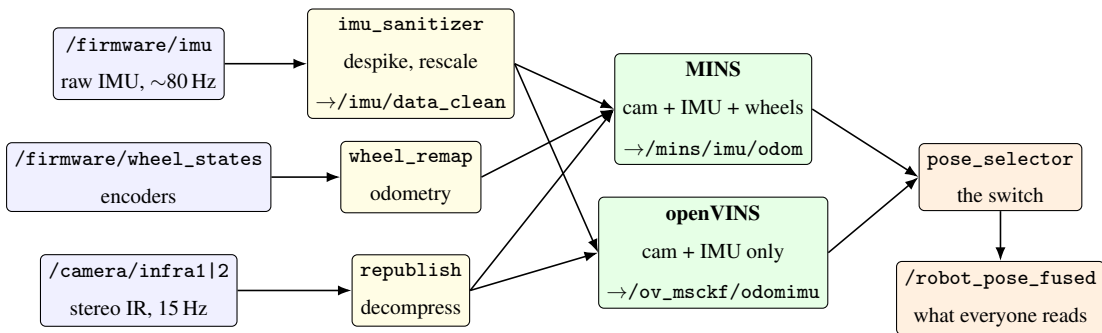


Figure D.1: The physical chain. **Weak link ①**: the CORE2 board talks to the Pi over a *GPIO serial port* (`/dev/ttyAMA0`), not USB, a detail that misleads everyone, because unplugging USB cables cannot fix it. That link has no retransmission: if the Pi is too busy to empty the serial buffer in time, bytes are lost outright (§D.13.7). **Weak link ②**: the WiFi hop adds latency and jitter. Measured on this install: 0% packet loss but round-trip times of 3 to 66 ms. ROS tolerates this because everything is timestamped at the source; the video preview does not, which is why the cockpit shows a *high latency* badge that is honest about the video and irrelevant to the trajectories.



Why two estimators? MINS also uses the wheels, so it starts while standing still and cannot drift far. openVINS uses only camera + IMU, so it is the honest test of *vision-only* localisation — but it must be *shaken* to start, and nothing catches it if it slips.

Figure D.2: The topic flow. Everything upstream of the estimators runs on the robot; everything from `imu_sanitizer` rightwards runs on the workstation. The switch is a ROS *service*, not a topic: calling `/pose_selector/set_source` with `true` serves MINS, `false` serves openVINS.

Table D.2: Which machine runs what. `rostopic list` shows all of it mixed together, which is why this split is worth knowing by heart.

Machine	Started by	Main nodes
Robot (Pi)	<code>leo.service → rosmaster</code>	ROS master, <code>serial_node</code> , <code>firmware_message_converter</code> , <code>realsense2_camera</code> , <code>rplidar_node</code>
Workstation	<code>roslaunch leo_navigation</code> <code>navigation_master.launch</code>	<code>mins_subscribe</code> , <code>ov_msckf</code> , <code>pose_selector</code> , <code>imu_sanitizer</code> , <code>wheel_remap</code> , <code>leo_backend</code> , <code>image_republish</code> ×3
Workstation	<code>web/start_web.sh</code>	<code>serve.py</code> (:8000), <code>web_video_server</code> (:8080), <code>rosbridge</code> (:9090/:9443), <code>cloudflared</code> tunnel

D.2.3 Map of the repository

Everything lives under ~/TOUT. You will spend almost all of your time in four of these directories; the rest is history you can ignore until you need it.

```
1 ~/TOUT/
2 |-- catkin_ws/                                <-- workspace 1: openVINS + our own
    packages
3 |   '-- src/
4 |       |-- open_vins/                        upstream openVINS (ov_msckf)
5 |       |-- leo_navigation/                  ** OUR CODE: nodes, launch files, configs
    **
6 |       |-- leo_autonomy/                    beacon/patrol behaviours, AprilTag
    configs
7 |   '-- realsense-ros/                       camera driver (built here, runs on the Pi
    )
8 |-- mins_ws/                                <-- workspace 2: MINS (separate build
    flags)
9 |   '-- src/MINS/
10 |-- carolus_ws/                             <-- workspace 3: LED beacon detection
11 |-- tools/                                  ** scripts you will use every day **
12 |-- web/                                    the cockpit (HTML + serve.py + start_web.
    sh)
13 |-- data/trajectories/                      recorded runs (.bag) and their CSV
    exports
14 |-- logs/                                   every log the system writes
15 |-- docs/                                   protocols (e.g. floor-marking calibration
    )
16 |-- report_latex/                          this report
17 '-- leo_backend.py                          ** the cockpit's brain (single large file
    ) **
```

The four that matter: catkin_ws/src/leo_navigation (all our ROS nodes), tools (all our scripts), web plus leo_backend.py (the cockpit), and logs (where every answer hides).

Table D.3: The scripts in `tools/` you will actually use. Every one has a long header comment explaining *why* it exists: read it before modifying the script.

Script	What it is for
<code>robot_env.sh</code>	sets <code>ROS_MASTER_URI/ROS_IP</code> ; sourced by everything
<code>restart_stack.sh</code>	clean restart of the PC-side stack
<code>record_trajectories.sh</code>	records a run to a rosbag (the reliable way)
<code>bag_to_csv.py</code>	rosbag $\rightarrow$ one CSV per estimator
<code>plot_trajectories.m</code>	the nine-panel MINS vs openVINS figure
<code>compare_tests.m</code>	Test 1 vs Test 2 (two separate drives)
<code>calib_terrain.py</code>	ground-truth calibration (terminal version)
<code>rosbag_daemon.sh</code>	launched <i>by</i> <code>leo_backend.py</code> to record a run to disk, independently of the browser. This
<code>launch_carolus.sh</code>	the beacon detector's entry point, invoked by <code>tools/carolus_detect.launch</code> . Start the
<code>leo_watchdog.sh</code>	the cron watchdog, read §D.13.3 first
<code>update_report.sh</code>	compiles and publishes this report
<code>check_serial_health.sh</code>	UART overrun counters (§D.13.7)
<code>log_pose_static.py</code>	records <code>/robot_pose_fused</code> to CSV while the rover sits still: Protocol T1.1 (§D.11.4)
<code>analyze_pose_variance.py</code>	per-axis noise and linear drift rate from a T1.1 CSV; separates noise from unbounded diver

D.3 Upstream sources: what is ours and what is not

For completeness, the provenance of every component and the workspace it lives in.

Four upstream stacks are used, each in its own catkin workspace so their build flags and dependencies stay isolated (Kalibr’s workspace, `kalibr_ws`, is separate from the three runtime workspaces of §D.4 because it is a calibration-time tool, not something that runs alongside the estimators):

Component	Upstream	Local workspace
openVINS (ov_ msckf)	https://github.com/rpng/open_vins	<code>catkin_ws/src/open_vins</code>
MINS	https://github.com/rpng/MINS	<code>mins_ws/src/MINS</code>
Carolus beacon	https://github.com/albator63/carolus_ws	<code>carolus_ws</code>
Kalibr (calibration)	https://github.com/ethz-asl/kalibr	<code>kalibr_ws/src/kalibr</code>

The project-specific ROS packages (`leo_navigation`, `leo_autonomy`, the `pose_selector`, `wheel_remap` and `carolus_vicon_bridge` nodes) live alongside openVINS in `catkin_ws/src/` and are versioned with the project, not upstream.

D.4 Building the workstation from a bare machine

Why in this order: ROS must exist before catkin can build against it; system libraries must exist before the estimators compile; and the two estimator workspaces must be built *separately* because they disagree about build flags.

D.4.1 Step 0: Python dependencies

Do this before anything else; several failures later in this guide are really a missing module reported three layers away from the import.

```
1 cd ~/TOUT
```

```

2 pip install -r requirements.txt
3 sudo apt install python3-tk          # tkinter, not pip-installable

```

requirements.txt lists what each package is for, with the versions this project was verified against. Two entries are easy to overlook: python3-tk, which leo_dashboard.py and leo_tracking_map.py need and which pip cannot supply, and esprima, which is not a runtime dependency at all but the JavaScript parser used to validate web/app.js before reloading it (§D.10.5). ROS packages such as rospy, tf and cv_bridge come from ROS Noetic and must not be installed with pip.

D.4.2 Step 1: operating system and ROS

1. Install **Ubuntu 20.04 LTS**. Not 22.04: ROS Noetic is not packaged for it, and every workaround costs more time than reinstalling.
2. Install ROS Noetic Desktop-Full, following the official instructions. Summarised:

```

1 sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release
   -sc) main" \
2   > /etc/apt/sources.list.d/ros-latest.list'
3 sudo apt install curl
4 curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc
   | sudo apt-key add -
5 sudo apt update
6 sudo apt install ros-noetic-desktop-full

```

3. Make ROS available in every new terminal:

```

1 echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
2 source ~/.bashrc

```

Verify: echo \$ROS_DISTRO prints noetic.

4. Install the build tool and the dependency resolver:

```

1 sudo apt install python3-catkin-tools python3-rosdep python3-osrf-
   pycommon
2 sudo rosdep init && rosdep update

```


D.4.3 Step 2: system libraries the estimators need

Both openVINS and MINS are C++ and need Eigen, Ceres, OpenCV 4 and Boost. `rosdep` finds most of them; Ceres is the one that usually needs help.

```
1 sudo apt install libeigen3-dev libboost-all-dev libopencv-dev \  
2 libceres-dev libsuitesparse-dev  
3 # ROS-side extras used by the cockpit and the pipeline:  
4 sudo apt install ros-noetic-rosbridge-suite ros-noetic-web-video-server  
5 \ros-noetic-image-transport-plugins ros-noetic-topic-  
6 tools \  
7 ros-noetic-rosmon ros-noetic-apriltag-ros  
8 # analysis tools:  
9 sudo apt install python3-pip texlive-full  
10 pip3 install --user numpy pyyaml psutil pillow
```

Note: `texlive-full` is ~5 GB but avoids a long tail of missing-package errors when compiling this report. `psutil` is what lets the cockpit show CPU load; without it the cockpit still works but that panel stays empty.

D.4.4 Step 3: the three workspaces

Why three: openVINS and MINS come from the same research group but do not build with the same flags, and mixing them in one workspace produces link errors that look like source bugs. The beacon stack is separate again because it is optional.

1. Create the tree and clone the upstream sources:

```
1 mkdir -p ~/TOUT/catkin_ws/src ~/TOUT/mins_ws/src ~/TOUT/carolus_ws/src  
2 cd ~/TOUT/catkin_ws/src  
3 git clone https://github.com/rpng/open_vins.git  
4 git clone https://github.com/IntelRealSense/realsense-ros.git -b ros1-  
5 legacy  
6 cd ~/TOUT/mins_ws/src  
7 git clone https://github.com/rpng/MINS.git
```

2. **Our packages arrive with the clone of §D.1.1**, already in place at `~/TOUT/catkin_ws/src/{leo_navigation,leo_autonomy}` — there is nothing further to copy. They contain every node described in

§D.7. **Note on the QR scripts:** `qr_leo_control.py`, `qr_search_move.py` and `diag_qr.py` are *not* a catkin package despite the name similarity; they are standalone Python scripts at the repository root, run directly with `python3`, not through `roslaunch` or `roslaunch`. An earlier revision of this appendix referred to a `leo_qr_control` package that does not exist in the versioned tree; if you were looking for it, this is the correction.

3. Resolve dependencies, once per workspace:

```
1 cd ~/TOUT/catkin_ws && rosdep install --from-paths src --ignore-src -r -y
2 cd ~/TOUT/mins_ws && rosdep install --from-paths src --ignore-src -r -y
```

4. Build workspace 1 (openVINS + our packages):

```
1 cd ~/TOUT/catkin_ws
2 catkin build
```

Expected: a progress table ending with a green summary, `[build] Summary: All ... packages succeeded!`. On a four-core machine budget 15–25 minutes. If `ov_msckf` fails on Ceres, install `libceres-dev` and rebuild only that package with `catkin build ov_msckf`.

5. Build workspace 2 (MINS):

```
1 cd ~/TOUT/mins_ws
2 catkin build mins
```

6. Build workspace 3 (beacon, optional):

```
1 cd ~/TOUT/carolus_ws && catkin build
```

D.4.5 Step 4: the sourcing trap, and why a wrapper exists

This is the single most confusing thing about the build. A `devel/setup.bash` *replaces* several environment variables rather than appending to them. Source `catkin_ws` then `mins_ws`, and `roslaunch` can no longer find `leo_navigation`; source them the other way round and it cannot find `mins`. Either way the error is `cannot locate node of type [...] in package [...]`, which reads like a missing file.

The fix is `catkin_ws/src/leo_navigation/launch_mins.sh`: it saves the five variables that matter, sources the second workspace, then re-appends the saved values so both are reachable.

```
1 source /home/lab272/TOUT/tools/robot_env.sh
2 source /opt/ros/noetic/setup.bash
3 source /home/lab272/TOUT/catkin_ws/devel/setup.bash
4 _RPP="$ROS_PACKAGE_PATH"; _CMPP="$CMAKE_PREFIX_PATH"; _PATH="$PATH"
5 _LDLP="$LD_LIBRARY_PATH"; _PYP="$PYTHONPATH"
6 source /home/lab272/TOUT/mins_ws/devel/setup.bash
7 export ROS_PACKAGE_PATH="$ROS_PACKAGE_PATH:_RPP"
8 export CMAKE_PREFIX_PATH="$CMAKE_PREFIX_PATH:_CMPP"
9 export PATH="$PATH:_PATH"
10 export LD_LIBRARY_PATH="$LD_LIBRARY_PATH:_LDLP"
11 export PYTHONPATH="$PYTHONPATH:_PYP"
12 exec roslaunch leo_navigation "${1:-mins.launch}"
```

Rule: never start anything MINS-side with a plain roslaunch. Always:

```
1 ~/TOUT/catkin_ws/src/leo_navigation/launch_mins.sh navigation_master.
   launch
```

`tools/restart_stack.sh` does this for you, which is one reason to launch through it rather than by hand.

D.4.6 Quick reference: the clone-and-build commands

Clone each stack into its workspace's `src/`, then build. `openVINS` and the project packages share `catkin_ws`:

```
1 # --- openVINS + project packages (catkin_ws) ---
2 cd ~/TOUT/catkin_ws/src
3 git clone https://github.com/rpng/open_vins.git
4 # (leo_navigation, leo_autonomy, kalibr already present in src/)
5 cd ~/TOUT/catkin_ws
6 catkin build ov_msckf           # builds openVINS and its deps
7 source devel/setup.bash
8
9 # --- MINS (separate workspace: its own build flags) ---
10 cd ~/TOUT/mins_ws/src
11 git clone https://github.com/rpng/MINS.git
```

```

12 cd ~/TOUT/mins_ws
13 catkin build mins
14 source devel/setup.bash
15
16 # --- Carolus beacon detection ---
17 cd ~/TOUT/carolus_ws
18 catkin build

```

System dependencies (Eigen, Ceres, OpenCV 4, Boost) are the standard openVINS/MINS prerequisites; on a fresh machine install them with `rosdep install -from-paths src -ignore-src -r -y` run once per workspace before the first `catkin build`. openVINS and MINS both compile against `tf2_ros` on this platform (the legacy `tf` migration of §12.9.4 for MINS, and the equivalent open item for openVINS in Table 12.12). The MINS workspace cannot simply be sourced back-to-back with `catkin_ws`; the `tools/launch_mins.sh` helper unions the two environments correctly and is the supported way to launch anything MINS-side (see its header comment for why).

D.5 Network configuration

Why this matters: ROS has no auto-discovery. Each process must be told where the master lives (`ROS_MASTER_URI`) and which address to advertise so others can call it back (`ROS_IP`). Get `ROS_IP` wrong and you get the most misleading symptom on the platform: `rostopic list` shows every node, and no data ever arrives.

D.5.1 Step 1: one file holds the robot’s address

The robot’s address changes with the network: `10.0.0.1` on the robot’s own hotspot, a DHCP address on campus WiFi, `10.200.0.2` over the WireGuard tunnel used at the time of writing. **Never hard-code it:** it used to be duplicated in five scripts, and changing networks meant five edits and a guaranteed omission.

```

1 echo 'LEO_ROBOT_HOST=10.200.0.2' > ~/.leo_network

```

Deleting the file restores the default (`10.0.0.1`). Every script reads `tools/robot_env.sh`, which reads this file.

D.5.2 Step 2: what robot_env.sh derives, and why

ROS_IP is **not** hard-coded either: it is looked up from the routing table, so it stays correct on a machine with both Ethernet and WiFi. The address that matters is the one on the interface that can actually reach the robot.

```
1 export ROBOT_HOST="${LEO_ROBOT_HOST:-10.0.0.1}"
2 export ROS_MASTER_URI="http://${ROBOT_HOST}:11311"
3 _leo_ros_ip="$(ip route get "$ROBOT_HOST" | grep -oP 'src \K[0-9.]+' |
   head -1)"
4 export ROS_IP="${_leo_ros_ip:-$(hostname -I | awk '{print $1}'))}"
```

Verify, from any directory:

```
1 cd ~/TOUT
2 source tools/robot_env.sh
3 echo "$ROBOT_HOST | $ROS_MASTER_URI | $ROS_IP"
```

Expected: 10.200.0.2 | http://10.200.0.2:11311 | 10.200.0.1.

If ROS_IP is empty or on the wrong subnet, stop: fix the network first, nothing downstream will work.

D.5.3 Step 3: the WireGuard tunnel (optional but recommended)

Campus WiFi hands out changing addresses and sometimes isolates clients from each other. A WireGuard tunnel gives the pair two *fixed* addresses (10.200.0.1 for the PC, 10.200.0.2 for the robot) regardless of the underlying network, which removes an entire class of “it worked yesterday” failures.

Decide which machine is reachable first. WireGuard has no client and server roles in the protocol, but it does in practice: one side must carry an Endpoint, and that side must be reachable from the other. This is the question that decides whether the tunnel comes up at all, and it is decided by your network, not by preference.

Situation	Who carries Endpoint	Works?
The PC has a fixed, routable address (wired lab network, port 51820/udp open)	the robot points at the PC	yes, the arrangement used here
Both sides behind campus NAT, no inbound ports	neither can reach the other	no ; you need a third host
A cheap VPS with a public IP	both point at the VPS	yes, and it survives either side changing network

Campus networks routinely block inbound connections and isolate wireless clients from one another. If the PC is itself on campus WiFi, expect the second row: no amount of configuration on the two machines will fix it, and the honest answer is a VPS acting as a rendezvous point, with both rover and workstation as clients of it.

1. `sudo apt install wireguard` on both machines.
2. Generate a key pair on each:

```
1 wg genkey | tee private.key | wg pubkey > public.key
```

3. Write `/etc/wireguard/wg0.conf` on each side. Substitute the four placeholders; everything else is literal.

Workstation (the side carrying the Endpoint in this deployment, i.e. the one that listens):

```
1 [Interface]
2 Address      = 10.200.0.1/24
3 PrivateKey   = <PC_PRIVATE_KEY>
4 ListenPort   = 51820
5
6 [Peer]
7 PublicKey    = <ROBOT_PUBLIC_KEY>
8 AllowedIPs   = 10.200.0.2/32
```

Robot:

```

1 [Interface]
2 Address      = 10.200.0.2/24
3 PrivateKey   = <ROBOT_PRIVATE_KEY>
4
5 [Peer]                               # the workstation
6 PublicKey    = <PC_PUBLIC_KEY>
7 AllowedIPs   = 10.200.0.0/24
8 Endpoint     = <PC_REACHABLE_IP>:51820
9 PersistentKeepalive = 25

```

4. Enable on both: `sudo systemctl enable -now wg-quick@wg0`.
5. **Verify:** `ping -c 5 10.200.0.2` from the PC gives 0% packet loss, and `sudo wg show` lists a recent handshake. Then point the project at it: `echo 'LEO_ROBOT_HOST=10.200.0.2' > ~/.leo_network`.

PersistentKeepalive is not a roaming fix

It is frequently described as one, including in earlier revisions of this appendix, and the claim does not survive contact with this platform. Keepalive stops a NAT mapping from expiring while nothing is being sent. It does nothing when the robot's radio actually re-associates and its underlying address changes: the peer's cached endpoint is then stale until a packet from the new address re-teaches it.

Registry item 12 records the reality. One WireGuard outage was explained and fixed by endpoint roaming; a *second* resisted restarts on both ends, a port change, and a four-minute idle wait, while ICMP and SSH to the robot stayed healthy throughout. That item is still open. Treat the tunnel as usually reliable and occasionally not, and when it floats, check `sudo wg show` for the age of the last handshake before restarting anything else.

Known quirk: on this workstation the USB WiFi dongle (rt188x2bu) drops packets when its power-saving kicks in. It affects the PC only, not the robot. First find your own interface name, which will *not* be the one below:

```

1 ip -br link | grep -E '^wl'           # or: iwconfig 2>/dev/null | grep -B1
    IEEE

```

Then disable power saving on it. This does *not* survive a reboot:

```

1 sudo iw dev <WIFI_INTERFACE> set power_save off
2 # on the machine used here that name was wlxc641aee985b

```

D.5.4 Step 4: passwordless SSH to the robot

Several scripts read robot state over SSH. Set up a key so they do not stall waiting for a password:

```
1 ssh-keygen -t ed25519          # accept the defaults
2 ssh-copy-id pi@10.200.0.2      # password once; never again
3 ssh pi@10.200.0.2 'hostname'  # verify
```

Expected: the robot's hostname, with no password prompt.

D.5.5 Step 5: the web transport, and the two ports it can use

The cockpit is an ordinary static web page. It has no server-side logic and no ROS libraries of its own: everything it knows about the robot arrives over a single WebSocket to rosbridge, which translates JSON into ROS messages and back. Three processes therefore have to be alive, and they are independent of one another:

Process	Port	Role
web/serve.py	8000	serves the HTML, CSS and JavaScript
rosbridge_websocket	9090	plain WebSocket, LAN and loopback
tools/rosbridge_websocket_tunnel.py	9443	TLS WebSocket, behind cloudflared

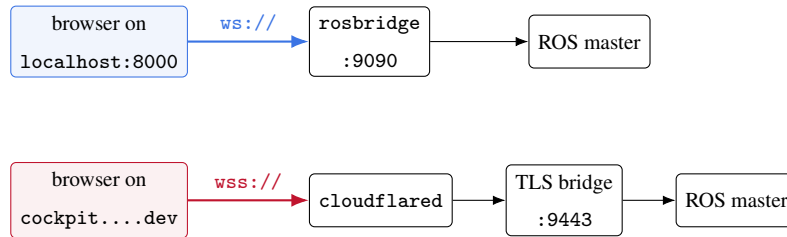
The page itself is served on 8000 in both cases. What changes between the two transports is only where the *WebSocket* goes, and the choice is made automatically from the hostname you typed:

```
1 const TUNNEL_ROOT = 'leo-rover-gardon.dev';
2 const isTunnel = () => location.hostname === TUNNEL_ROOT
3                       || location.hostname.endsWith('.') + TUNNEL_ROOT);
4 const wsURL = (host) => isTunnel()
5                   ? ('wss://' + location.host + '/rosbridge') // -> cloudflared ->
                      : 9443
```



```
6 : ('ws://', + host + ':9090'); // -> direct
```

This single function is the whole of the routing logic, and it has a consequence worth internalising before you debug anything:



The two paths share nothing but the ROS master. Different process, different port, different TLS state, different failure modes. When one is broken the other is very often fine, and that is the single most useful fact in this subsection.

The zombie port

Symptom: the page loads perfectly and every button is dead

Restart the ROS master while the TLS bridge on 9443 is running, and that bridge keeps its listening socket open. It still accepts the TCP connection, still completes the TLS negotiation, still completes the WebSocket upgrade. What it no longer holds are live handles to the ROS graph, so nothing it receives is ever published and nothing from the graph ever reaches you.

The browser has no way to notice. At every layer it can observe, the connection succeeded. The page therefore renders its “connected” state truthfully and the interface looks entirely normal while being inert.

Since 2026-07-31 this is detected automatically. `tools/leo_watchdog.sh` carries a *drowning probe*: it does not ask whether the port is bound, which in this failure mode it always is, but opens a real WebSocket handshake and requires an answer within eight seconds. Until July 31 that probe covered only port 9090, so the one failure mode it was designed to catch was being observed on the port it did not watch. It now probes both, and purges the pair on either failure, after which `start_web.sh` restarts them idempotently.

If you ever extend this probe, mind the TLS

Port 9443 is served over TLS (`listenSSL` in `rosbridge_websocket_tunnel.py`). Copying the plain-text 9090 probe onto it produces a probe that *always* fails, so the watchdog would purge both bridges every single minute: the exact restart storm of §D.13.3. Verified on 2026-07-31, a plain probe against

9443 raises `AssertionError` while a TLS-wrapped one returns `HTTP/1.1 400`. A 400 is a perfectly good result here, because the probe tests whether the process still answers, not whether it accepts the request.

Manual remediation, in increasing order of cost, for the window before the watchdog’s next tick:

1. **Force-reload the page** (Ctrl+Shift+R). This tears down the WebSocket and builds a new one. If the bridge process itself recovered, this is enough, and it costs two seconds.
2. **Change transport**. Open `http://localhost:8000/ops.html` instead of the tunnel domain. By the rule above this switches you to `ws://...:9090`, a different process that never went through the restart. Note carefully that this is not a different page and not a fallback mode: it is the same cockpit reaching the same master by an independent route. If the cockpit works here, the fault is in the tunnel path and nowhere else.
3. **Restart the TLS bridge** only once the two steps above have failed, since it is the only one that interrupts anything.

Diagnose before you restart. The two commands below separate “the page is broken” from “the bus is quiet”, and the order matters:

```
1 # 1. Does ROS traffic exist at all? Use a topic that is ALWAYS busy.
2 rostopic hz /mission/telemetry          # expect ~4 Hz, continuously
3 # 2. Does a command sent from the page reach the bus?
4 rostopic echo /mission/command          # press a cockpit button and watch
5 # 3. Are both bridges actually listening?
6 ss -ltnp | grep -E '9090|9443|8000'
```

The choice of topic in the first command is not incidental. During the July 30 session the relay was declared broken on the evidence of a silent `/mission/log`, which publishes only when something happens; the empty output proved nothing at all. Repeating the test on `/mission/telemetry` showed fifty frames in twelve seconds and the relay was healthy throughout. **A silent topic is not a dead link.** Always probe something that is supposed to be noisy.

Two further transport traps, for completeness

- **Opening the page as `file://`** disables both `rosbridge` and the video stream, because the browser refuses cross-origin WebSockets from a file URL. The cockpit detects this and prints a red banner telling

you to use `http://<PC-IP>:8000/ops.html`. If you see that banner, nothing else you observe about connectivity means anything.

D.5.6 Step 6: the Cloudflare tunnel, and why you must rebuild it

Everything above gets you a cockpit on the local network. The public address, `cockpit.leo-rover-gardon.dev`, is served by a Cloudflare tunnel, and it is the one piece of this system you **cannot inherit**.

The existing tunnel dies with its owner's account

The tunnel in service at the time of writing is bound to a personal Cloudflare account and to a domain registered under it. Its credentials live in `~/.cloudflared/` on the workstation, outside the project tree and outside version control, and they cannot be transferred by copying a file. **You will have to create your own tunnel, on your own account, and repoint DNS at it.** Budget an hour. Nothing else in this appendix depends on it: the cockpit works fully over `http://localhost:8000` without any tunnel at all.

What the tunnel actually does. It terminates TLS at Cloudflare's edge and forwards three different paths of one hostname to three local services. That routing is the missing link between the JavaScript of §D.5.5, which builds `wss://cockpit.../rosbridge`, and the TLS bridge listening on 9443. Without the ingress block below, that URL resolves to nothing and every button in the cockpit is inert with no error message.

Setup, from nothing.

1. Install the binary (a `.deb` from Cloudflare, or your package manager), then authenticate. This opens a browser and writes `~/.cloudflared/cert.pem`:

```
1 cloudflared tunnel login
```

2. Create the tunnel. This prints a UUID and writes the credentials file `~/.cloudflared/<UUID>.json`, which is the actual secret:

```
1 cloudflared tunnel create leo-cockpit
```

3. Point a DNS name at it, on a domain your account controls:

```
1 cloudflared tunnel route dns leo-cockpit cockpit.example.org
```

4. Write `~/cloudflared/config.yml`. This is the file that matters, reproduced with the real routing and only the identifiers replaced:

```
1 tunnel: <TUNNEL_UUID>
2 credentials-file: /home/<user>/.cloudflared/<TUNNEL_UUID>.json
3
4 ingress:
5   # rosbridge and video are served on the SAME hostname as the page,
6   # dedicated paths, so they share the Cloudflare Access cookie of the
7   # already-authenticated page. A separate subdomain would break the
8   # WebSocket handshake on cookie scope.
9   - hostname: cockpit.example.org
10     path: ~/rosbridge/?$
11     service: https://localhost:9443
12     originRequest:
13       noTLSVerify: true # self-signed cert, loopback hop only
14   - hostname: cockpit.example.org
15     path: ~/(stream|stream_viewer)$
16     service: http://localhost:8080
17   - hostname: cockpit.example.org
18     service: http://localhost:8000
19   - service: http_status:404
```

5. Run it. `web/start_web.sh` already does this for you, with an absolute path because a bare `cloudflared` fails silently under cron:

```
1 cloudflared tunnel run
```

Three details that are load-bearing.

- `service: https://localhost:9443`, not `http`. The TLS bridge speaks TLS on the loopback hop as well.
- `noTLSVerify: true` is required because that bridge presents a self-signed certificate. It is safe *here* and only here, because the hop it applies to never leaves the machine.

- The order of ingress rules is significant. Cloudflare takes the first match, so the catch-all service: `http://localhost:8000` must come last, and `http_status:404` last of all.

Verify

```
1 pgrep -x cloudflared # the process must exist
2 curl -sI https://cockpit.example.org/ops.html # expect HTTP/2 200
3 tail -20 ~/TOUT/logs/cloudflared.log # look for "Registered
  tunnel "
```

D.6 Robot-side configuration

The Pi runs a stock LEO Rover image plus three project-specific changes. **If you reflash the robot, you must redo all three**, and the system will appear to work while behaving badly if you forget, which is exactly how each was discovered.

D.6.1 How the robot starts itself

The Pi runs `leo.service`, a systemd unit that launches `rosmon` (a supervising alternative to `roslaunch`), which in turn starts the ROS master and the robot's nodes from `/opt/ros/noetic/share/leo_bringup/launch/leo_bringup.launch`.

```
1 ssh pi@$ROBOT_HOST 'systemctl status leo --no-pager | head -5'
2 ssh pi@$ROBOT_HOST 'sudo systemctl restart leo' # full robot-side
  restart
```

`rosmon` respawns any node that dies, which is usually helpful and occasionally the reason something you killed keeps coming back (§D.6.4).

D.6.2 Change 1: real-time priority for the serial node

Why: the CORE2 board streams IMU and encoder data into a 32-byte hardware FIFO on the Pi's PL011 UART. If `serial_node` is not scheduled before that FIFO fills, bytes are lost with no retransmission. *Symptom:* thousands of checksum errors and wheel odometry collapsing from 19 Hz to 1 Hz.

The fix has two halves and needs both. With only the first, the node does not start *at all*: you lose driving entirely, which is worse than the bug.

1. **Ask for real-time priority**, in `leo_bringup.launch` on the robot:

```
1 <node name="serial_node" pkg="roserial_python" type="serial_node.py"
2     launch-prefix="chrt -f 25">
```

2. **Grant permission to ask**, because user `pi` has `rtprio=0` by default and `chrt` would simply fail. Create `/etc/systemd/system/leo.service.d/10-rtprio.conf`:

```
1 [Service]
2 LimitRTPRIO=50
```

then `sudo systemctl daemon-reload && sudo systemctl restart leo`.

Verify both halves:

```
1 ssh pi@$ROBOT_HOST 'chrt -p $(pgrep -f serial_node | head -1)'
```

Expected: scheduling policy: `SCHED_FIFO`, priority: 25.

And the payoff: `rostopic hz /firmware/wheel_odom` should read ~ 19 Hz. Before the fix it read 1 Hz.

D.6.3 Change 2: camera settings that reset on every restart

Why: the D455's infrared projector paints a dot pattern that helps depth and *ruins* the checkerboard and LED detection. It is therefore turned off, and the depth compression level is lowered to save Pi CPU. Both are `dynamic_reconfigure` values, so **they revert to factory defaults every time the camera node restarts**, and the camera node restarts whenever the stack does.

```
1 rosrunc dynamic_reconfigure dynparam set /camera/stereo_module
   laser_power 0
2 rosrunc dynamic_reconfigure dynparam set \
3   /camera/depth/image_rect_raw/compressedDepth png_level 1
```

`tools/restart_stack.sh` and the watchdog both reapply these, which is why you rarely notice. If your checkerboard detection suddenly degrades, check `laser_power` first:

```
1 rosrunc dynamic_reconfigure dynparam get /camera/stereo_module | grep
   laser_power
```

D.6.4 Change 3: disable the robot's own rosbridge

Why: the stock image runs a `rosbridge_server` on the robot for its factory web UI. This project's cockpit connects to a `rosbridge` on the *PC* instead, so the robot's copy has zero clients, yet it was measured consuming 19% of a CPU core on a Pi already thermally throttled. Its consumption *grows with uptime* (a leak: a freshly respawned instance sits at 0%), and its `unregister_timeout` of 86400 s let it live outside the ROS graph, invisible to `rostopic list`.

Comment the node out in `leo_bringup.launch` on the robot:

```
1 <!-- DISABLED: unused, and leaks CPU on a thermally limited Pi.
2 <node name="rosbridge_server" pkg="rosbridge_server"
3       type="rosbridge_websocket">
4   <param name="unregister_timeout" value="86400"/>
5 </node>
6 -->
```

Then `sudo systemctl restart leo`. Killing the process without editing the launch file is futile: `rostopic` respawns it within seconds.

D.7 The code we wrote, node by node

Everything upstream (openVINS, MINS, the camera driver) is unmodified. The project-specific logic lives in six files, all in `catkin_ws/src/leo_navigation/scripts/` except the last. Each has a long header comment explaining *why* it exists: read that header before changing anything, because most of these nodes exist to work around a specific measured defect, and the workaround looks arbitrary until you know which.

D.7.1 `imu_sanitizer.py`: the IMU is not trustworthy raw

Subscribes `/firmware/imu` (`leo_msgs/Imu`) **Publishes** `/imu/data_clean` (`sensor_msgs/Imu`)

This node exists because the CORE2 IMU has *four* separate defects, each found by measurement, and both estimators consume its output.

1. **Isolated glitches.** Roughly two samples in nine thousand come out absurd (one accelerometer reading of $5 \times 10^{16} \text{ m/s}^2$ was recorded). A single such sample destroys any filter instantly. Samples outside `~gyro_`

limit (35 rad/s) or `~accel_limit` (160 m/s²) are replaced by the last good one, keeping the cadence intact.

2. **A frozen-firmware mode.** After a serial desync the board can retransmit the *same* sample forever: bit-identical values, which a real sensor never produces. N identical samples in a row triggers an automatic `/firmware/reset_board` call (throttled).
3. **Accelerometer scale error.** With the robot verified stationary (502 samples, wheel odometry 0.000 m/s) the raw magnitude read 8.7525 m/s² against a true local gravity of 9.790. That is -10.6% , and it is in the sensor: raw and sanitised agreed to 0.005. Corrected by `~accel_scale` = $9.790/8.7525 = 1.1186$.
4. **Gyroscope bias, badly out of spec.** Stationary, the gyro read 3.679 deg/s of total bias where a healthy MEMS part sits at 0.01–0.1. Because attitude is the *integral* of the gyro, that mis-projects gravity into the body frame and injects a phantom horizontal acceleration far larger than the accelerometer error. Subtracted, and **re-measured at every node start** (`~gyro_autocal`, default true) because bias drifts with temperature and this Pi runs at 86°C.

Parameters you may need:

```
1 ~accel_scale      1.1186    # 1.0 disables; see (3) above
2 ~gyro_scale       1.0       # NOT measured yet -- see below
3 ~gyro_autocal     true      # re-measure bias at startup if the robot is
                               still
4 ~gyro_limit       35.0      # rad/s, glitch rejection
5 ~accel_limit      160.0     # m/s^2
```

The honest gap. `~gyro_scale` is **deliberately left at 1.0, i.e. no correction.** A stationary calibration measures the *bias* but is mathematically incapable of measuring the *scale*: at rest the true rate is zero, so the scale factor drops out of the observation entirely. Measuring it requires a turn of known amplitude against physical ground truth: see §D.11.3. Setting this parameter on a guess would corrupt *both* estimators at once.

D.7.2 wheel_remap.py: a two-line bug that costs a day

Subscribes `/joint_states` **Publishes** a 2-element JointState MINS can read

MINS reads wheel velocities **by array index, not by joint name**: `data.m1 = msg->velocity.at(0)` with a comment claiming it is the front left wheel. This rover publishes its four wheels in a different order,

so MINS was silently fed the wrong wheels. Nothing errors; the estimate is simply wrong. This node republishes a two-element message in the order MINS assumes.

```
1 ~front_left_name      ~front_right_name      # which joints to pick
2 ~rear_left_name       ~rear_right_name
3 ~in_topic   /joint_states                      ~out_topic
```

Lesson worth carrying: when integrating any upstream estimator, verify how it identifies its inputs. Index-based assumptions are common and fail silently.

D.7.3 pose_selector.py: one topic that never breaks

Subscribes /mins/imu/odom, /ov_msckf/odomimu **Publishes** /robot_pose_fused **Service** /pose_selector/set_source (std_srvs/SetBool; true = MINS)

Downstream consumers (the cockpit, logging, any navigation) need *one* pose topic that keeps working when you switch estimators. The two estimators have independent origins, so a naive switch would produce a jump of tens of metres. This node applies an SE(3) correction at the moment of the switch so the output is continuous.

```
1 ~default_source      mins      # what is served at startup
2 ~max_plausible_speed      # rejects physically impossible jumps
3 ~mins_topic  ~vins_topic  ~odom_frame  ~base_frame  ~imu_frame
```

Important limitation, and it bites: the SE(3) correction is computed *on a switch only*. If MINS itself respawns it re-initialises at a new origin, and that jump is **not** absorbed: the fused topic jumps with it. This is one reason the estimators are deliberately *not* allowed to bring down the stack: driving must never depend on estimator health.

D.7.4 carolus_vicon_bridge.py: the only absolute reference

Publishes geometry_msgs/PoseStamped on the topic MINS accepts as a VICON input

Every other measurement on this platform is relative. The Carolus beacon, a fixed LED constellation at a known place, gives a drift-free global fix whenever it is in view. This node reformats that fix into the type MINS’s external-reference slot consumes, making it an “indoor GPS” that cancels unbounded drift.

```
1 ~global_frame  ~body_frame  ~out_topic  ~rate_hz  ~max_tf_age
```

Rule established 2026-07-08 and violated once since: the topic `/mins/external_ref/carolus` is the **private slot** of this bridge. Nothing else may publish on it. A trajectory recorder that also wrote there produced a genuinely confusing failure.

D.7.5 `leo_backend.py`: the cockpit's brain

This is the largest file in the project (~ 4500 lines) and it runs on the workstation, launched by `leo_backend_launch.sh`. It is *not* a normal ROS node in spirit: it is the bridge between the browser and ROS.

- **Inbound:** JSON commands on `/mission/command`. Every cockpit button publishes one, e.g. `{"action": "set_pose_source", "source": "VINS"}`. To add a button, add an `elif action == "..."` branch in `_on_command` and a click handler in the HTML.
- **Outbound:** a JSON telemetry blob at 10 Hz on `/mission/telemetry`, plus human-readable lines on `/mission/log`. The browser subscribes to both through `rosbridge`.
- **Also owns:** the autonomous patrol state machine, the beacon approach logic, the trajectory buffers, the detached `rosviz` recorder, the MATLAB export, and the ground-truth calibration panel.

Two hard-won implementation notes. Launching a GUI application (MATLAB) from this process needs three separate things or it dies within seconds: a double fork so it survives its parent, a pseudo-terminal (MATLAB's `-r` exits when `stdin` is not a `tty`), and a reconstructed `XDG_RUNTIME_DIR/DBUS_SESSION_BUS_ADDRESS` because the backend is started without a login session. And any recording that matters is written by a *detached* `rosviz` process, because this backend's own buffers are lost if it restarts.

D.7.6 `vins_rotation_guard.py`: deliberately not used

This file is still in the tree and is **intentionally not launched**. It was written to freeze `openVINS`'s reported position during turns, on the belief that `openVINS` diverged when rotating. **It made things measurably worse** (§D.13.6). It is kept as a record of the reasoning error, not as a component. Do not re-enable it without reading that section.

D.8 Configuration files, and the traps inside them

D.8.1 A trap that applies to every YAML here

Read this before editing any estimator config. Both openVINS and MINS parse their YAML with OpenCV's FileStorage (the files begin with %YAML:1.0), which is *not* a general YAML parser. It **aborts on a long comment placed after a value on the same line**.

The failure mode is vicious: the node starts, subscribes to nothing, and respawns forever, with no error message naming the file. This trap was hit twice on this project: the second time *after* it had already been documented.

- Keep inline comments **short**, or put them on their own line.
- Validate before restarting anything:

```
1 cd ~/TOUT/catkin_ws/src/open_vins/config/leo
2 python3 -c "import cv2; f=cv2.FileStorage('estimator_config.yaml', \
3 cv2.FILE_STORAGE_READ); print('readable:', f.isOpened()); f.release()"
```

Expected: readable: True.

- **Never test a config from /tmp:** relative paths inside it resolve differently and you get a false negative.

D.8.2 openVINS

Three files, all in catkin_ws/src/open_vins/config/leo/:

```
1 estimator_config.yaml      # the filter: init, ZUPT, features, noise,
    topics
2 kalibr_imucam_chain.yaml   # camera intrinsics + camera-IMU extrinsics
3 kalibr_imu_chain.yaml      # IMU noise densities and random walks
```

Launch footgun. The config path is passed as args="...", i.e. argv[1], and *not* as a <param name="config_path">. On the ROS1 build, the parameter read is inside an #if ROS_AVAILABLE == 2 block and never executes. Using <param> fails silently: argv[1] becomes roslaunch's own __name:=... argument, and openVINS reports it cannot open a file with that literal name.

The values that matter on this platform, each with the measurement behind it:

Table D.5: Key openVINS settings and why they are set this way.

Key	Value	Why
try_zupt	true	The single largest fix. With false a run diverged to 41 824 m; with true, 0.002 m. Zero-velocity updates anchor a ground vehicle that stops often.
gravity_mag	9.790	True local gravity (Melbourne FL). Leaving 9.81 against a mis-scaled accelerometer compounds the error.
num_opencv_threads	1	A mutex crash fired 1324 times, once every 46 s. Root cause: OpenCV threading. Zero crashes in 7 minutes after.
max_slam	0	Pivoting in place makes landmark depth unobservable; matches the MINS setting.
calib_cam_extrinsics	false	Locked 2026-07-29. Online refinement of a geometry that is poorly observable under planar motion lets it wander.
calib_cam_timeoffset	false	Same reasoning.
calib_cam_intrinsics	false	Frozen from Kalibr; offline measurement beats online refinement.
init_imu_thresh	0.4	The jerk needed to initialise. The rover must be nudged (§D.10).
use_stereo / max_cameras	true / 2	Both D455 infrared cameras.

The IMU noise densities in `kalibr_imu_chain.yaml` (`accelerometer_noise_density: 5.0e-03`, `gyroscope_noise_density: 3.4e-04`) are *measured* values inflated by a safety factor of three, not datasheet figures.

D.8.3 MINS, start to finish: download, build, install, calibrate, test

MINS (multi-sensor inertial navigation system) is the tightly-coupled filter on this platform: camera, IMU and wheel odometry fused in one estimator, publishing its pose on `/mins/imu/odom`. Because its own TF broadcasts would otherwise fight openVINS over the same frames, they are isolated onto a dedicated `/tf_mins` topic (subscribe there explicitly in RViz, not on `/tf`).

This subsection is a single, self-contained path from an empty machine to a MINS you can trust, and does not assume the rest of this appendix has been read first: download and build the estimator, install and understand the ten configuration files that make it work on *this* robot rather than a generic one, calibrate the two measurements those files depend on, then validate that a fresh install is not just running but running correctly. Steps that apply to the whole platform and not just MINS (ROS, system libraries, network) are covered fully in §D.4 and only summarised here; every §reference below is for extra depth, not a prerequisite for the step it is attached to.

Download and build.

1. **Prerequisites.** Ubuntu 20.04 with ROS Noetic, and the C++ libraries both estimators need (§D.4, Steps 0–2):

```
1 sudo apt install libeigen3-dev libboost-all-dev libopencv-dev \  
2 libceres-dev libsuitesparse-dev
```

One MINS-specific caveat, from upstream’s own README: MINS is tested against Eigen 3.3.7 or lower; a newer system Eigen can fail to compile the bundled `libpointmatcher` third-party library (used for the LiDAR sensor, disabled on this platform but still compiled). Ubuntu 20.04’s own `libeigen3-dev` is 3.3.7, so this only bites if you have manually upgraded it.

2. **Clone MINS** into its own workspace, kept separate from openVINS because the two disagree on build flags (§D.4):

```
1 mkdir -p ~/TOUT/mins_ws/src  
2 cd ~/TOUT/mins_ws/src  
3 git clone https://github.com/rpng/MINS.git
```

3. Resolve dependencies and build:

```
1 cd ~/TOUT/mins_ws
2 rosdep install --from-paths src --ignore-src -r -y
3 catkin build mins
4 source devel/setup.bash
```

Expected: a progress table ending with a green summary, [build] Summary: All ... packages succeeded!, matching §D.4's Step 3.

4. **Do not source this workspace back-to-back with** `catkin_ws`. `devel/setup.bash` replaces several environment variables instead of extending them, so sourcing both in sequence breaks one workspace's or the other's `ROS_PACKAGE_PATH` (§D.4.5, the single most confusing failure mode in this build). **Never launch anything MINS-side with a plain** `roslaunch`; always go through the wrapper:

```
1 ~/TOUT/catkin_ws/src/leo_navigation/launch_mins.sh navigation_master.
   launch
```

or, equivalently and more simply, `bash tools/restart_stack.sh`, which calls the same wrapper for you.

Install and configure. **The step a fresh clone is missing.** MINS ships with example configuration profiles (`kaist`, `tum_vi`, `euroc_mav`, `simulation`, ...) but no profile for this robot: there is no `mins/config/leo/` directory in the upstream repository. Without one, `mins.launch`'s default argument (`config:=leo`) points at a file that does not exist, and MINS reports that it cannot open a configuration file. The `leo` profile is **this project's own work**, so it is versioned in *this* repository, not MINS's, at `LEO_Rover_Navigation_System/MINS-master/mins/config/leo/`. Install it into the workspace just built:

```
1 cp -r ~/TOUT/LEO_Rover_Navigation_System/MINS-master/mins/config/leo \
2     ~/TOUT/mins_ws/src/MINS/mins/config/leo
```

Verify: `ls ~/TOUT/mins_ws/src/MINS/mins/config/leo/` lists ten files. Every one is required: `config.yaml`'s only job is to include the other nine by relative filename, and a missing file aborts the same way a missing `leo/` directory does.

File	What it holds, and what is not a stock/default value
<code>config.yaml</code>	Top-level include list only. Also documents, in its own header comment, the enable ladder this deployment follows: bring up camera+IMU first, then wheels, then the Carolus VICON slot, then LiDAR last, each stage validated before the next is switched on. An unvalidated extrinsic on one sensor makes the whole fused estimate diverge in a way that is hard to attribute back to that sensor.
<code>config_imu.yaml</code>	Noise densities from an Allan-variance characterisation of <i>this</i> IMU, not datasheet figures. The field that matters most here is <code>topic: "/imu/data_clean"</code> : point this at the raw firmware topic (<code>/firmware/imu</code>) instead and MINS silently ingests every glitch, every frozen-sample run, and the accelerometer's -10.6% scale error that <code>imu_sanitizer.py</code> exists to remove (§D.7).
<code>config_camera.yaml</code>	<code>cam0/cam1</code> topics are <code>/pc/camera/infra1 2/image_rect_raw</code> : the workstation's local republish of the robot's compressed stream, never the robot's raw topics directly (two raw 848×480 streams demand more bandwidth than the WiFi link can sustain, and have been measured saturating the AP to the point of dropping it). Intrinsic come from <code>kalibr_calibrate_cameras</code> and are frozen (<code>do_calib_int: false</code>); <code>T_imu_cam</code> is still a tape-measure seed, see Calibrate below. <code>max_slam: 0</code> because this platform pivots on the spot often, which makes landmark depth unobservable and produced a measured $+0.58$ rad/s phantom heading term from bad SLAM anchors on 2026-07-13.
<code>config_wheel.yaml</code>	The most heavily retuned file; see the dedicated walkthrough below.

File	What it holds, and what is not a stock/default value
<code>config_estimator.yaml</code>	<code>gravity_mag:</code> 9.790, local gravity for Melbourne FL, not the generic 9.81; re-measure only if this configuration is deployed somewhere else. Filter and clone-rate settings otherwise match MINS's own defaults.
<code>config_init.yaml</code>	Static IMU+wheel initialisation thresholds. MINS initialises standing still in 4–12 s with these defaults; nothing platform-specific to change here.
<code>config_system.yaml</code>	<code>verbosity:</code> 1, not MINS's own default of 2: at 2, the initialiser's diagnostic prints (waiting for data, gravity convergence, static/dynamic init attempts) are filtered silently, which looks exactly like a hung initialiser when it may be running normally. Also sets the output paths for the trajectory/timing logs.
<code>config_gps.yaml</code>	<code>enabled:</code> false. Present only because MINS's loader expects the file to exist; this platform has no GNSS receiver. Leave it as shipped.
<code>config_lidar.yaml</code>	<code>enabled:</code> false. Staged for a future integration (an RPLidar publishes LaserScan; MINS wants PointCloud2, and a conversion node does not exist yet). Leave disabled.
<code>config_vicon.yaml</code>	<code>enabled:</code> false until <code>carolus_vicon_bridge.py</code> is verified publishing on <code>/mins/external_ref/carolus</code> , MINS's slot for the Carolus beacon acting as an indoor GPS (§D.7). Flip to true once that bridge is confirmed live; the topic is a private slot and nothing else may publish on it (rule established 2026-07-08).

Table D.6: Every file `config.yaml` includes, and what is genuinely platform-specific inside it.

`config_wheel.yaml` **in detail**, because it carries more measured, platform-specific corrections than any other file in the tree:

- `type:` "Wheel2Dang": a two-wheel differential model. This rover is four-wheel skid-steer, not differential-drive; Wheel2Dang approximates it from the front-left/front-right pair, and the online extrinsic refinement below absorbs most of the mismatch.
- `intrinsics:` [0.0622, 0.0622, 0.358]: effective wheel radius (twice) and track width, in metres. **These are measured, not the firmware's stock values**, and they are specific to this set of wheels; see **Calibrate** below for how to re-measure them.
- `T_imu_wheel:` a 180° rotation about *z*, because this rover's wheel-odometry frame points opposite to the camera/IMU frame at the firmware level. Without it, every wheel measurement contradicts the inertial prediction and the χ^2 gate silently rejects it, which looks exactly like "MINS is ignoring the wheels."
- `chi2_mult:` 5, not MINS's default of 15: the slip-rejection gate for wheel-versus-IMU disagreement, tightened after measuring rigid wheels slipping in turns (below).
- `noise_w:` 0.8 and `noise_v:` 0.2, both retuned from MINS's defaults (0.2 and 0.5 respectively): in a turn, this rover's wheels can imply a yaw rate an order of magnitude larger than the camera-measured truth (3.8 rad/s of wheel-implied rotation for 0.001 rad/s actually measured), so angular trust is loosened; linear speed from the wheels is accurate and is trusted *more* than the default, because it is what keeps the filter anchored through camera dropouts.
- `topic:` `"/joint_states_mins"` with `sub_topics:` `"wheel_FL_joint, wheel_FR_joint"`: MINS reads wheel velocities **by array index, not by name** (`ROSSubscriber::callback_wheel` takes `velocity.at(0)` and `.at(1)`), and this rover's real `/joint_states` does not order its four wheels front-left-first. Point this at raw `/joint_states` and MINS silently reads the wrong wheel; `wheel_remap.py` exists purely to fix the order (§D.7), and `sub_topics` must name-match that node's own `~front_left_name` and `~front_right_name` parameters. Omitting `sub_topics` is a silent-drop trap in its own right: MINS's loader defaults it to generic joint names this rover does not use, the remapped topic keeps publishing (visible on `rostopic hz`), and MINS's internal wheel buffer simply never fills, which looks like an initialisation problem rather than a name mismatch.

Wiring it up. Two more pieces complete the installation, covered fully in §D.9:

- The config profile is selected by `mins.launch`'s `config` argument, passed to the estimator as `args="$(find mins)/config/$(arg config)/config.yaml"`. It must be `args`, never a `<param name="config_path">`: the parameter read exists only in MINS's ROS2 build, and on ROS1 that block is compiled out entirely (§D.9.3). Getting this wrong does not fail clearly: MINS reports it cannot open a file whose name was never typed.
- MINS's config files are parsed with OpenCV's `FileStorage`, not a general YAML parser: a long inline comment placed after a value on the same line aborts the file silently, and the node then starts, subscribes to nothing, and respawns forever with no error naming the file (§D.8.1). If a comment is added or edited in any `config_*.yaml`, keep it short or put it on its own line, and validate before restarting anything:

```
1 cd ~/TOUT/mins_ws/src/MINS/mins/config/leo
2 python3 -c "import cv2; f=cv2.FileStorage('config_wheel.yaml', \
3 cv2.FILE_STORAGE_READ); print('readable:', f.isOpened()); f.release()"
```

Finally, three companion nodes must be running or the topics these config files name never publish: `imu_sanitizer.py` (feeds `config_imu.yaml`'s topic), `wheel_remap.py` (feeds `config_wheel.yaml`'s), and, once Stage 3 is reached, `carolus_vicon_bridge.py` (feeds `config_vicon.yaml`'s). All three start automatically as part of `navigation_supervision.launch`, included by the same `navigation_master.launch` that starts MINS itself (§D.9); nothing further needs to be launched by hand.

Calibrate. Three measurements feed the config files above; two are already done and versioned, one is not.

1. **IMU sanitisation (already automatic).** `imu_sanitizer.py` applies a fixed accelerometer scale correction ($\times 1.1186$, measured stationary against local gravity) and re-measures gyroscope bias at every startup (§D.7). Nothing to run by hand: this happens every time the stack starts, before MINS ever sees a sample.
2. **Wheel radius and track** (`config_wheel.yaml`'s `intrinsics`). Required again only if the wheels themselves change (different tyres, a different rover). Measured passively, with no commanded motion from the script itself:

```
1 cd ~/TOUT
2 python3 tools/calib_roues_rigides.py
```

Then, from the cockpit or a joystick, drive the rover **straight for about 3 m** (estimates the radius), then **spin in place for about two full turns** (estimates the track), and press Ctrl-C. The script prints the left/right radius, the track, and an `intrinsics:` line ready to paste into `config_wheel.yaml`.

3. **Camera-IMU extrinsic** (`config_camera.yaml`'s `T_imu_cam`).

Honest status: not yet completed on this platform

The procedure below is fully scripted and ready to run, but the excitation motion it needs (physically lifting and tilting the rover in front of the target) has **not** been performed as of this writing (Table 12.12, item 2). The deployed extrinsic is a tape-measure seed, refined online (`do_calib_ext: true` in both `config_camera.yaml` and the equivalent `openVINS` setting), which is why MINS still runs and initialises correctly without this step: it is a precision improvement, not a blocker. Completing it is the single highest-value calibration task remaining on this platform.

Kalibr lives in its own workspace, since its build requirements conflict with both estimators':

```
1 mkdir -p ~/TOUT/kalibr_ws/src && cd ~/TOUT/kalibr_ws/src
2 git clone https://github.com/ethz-asl/kalibr.git
3 cd ~/TOUT/kalibr_ws && catkin build
4 source devel/setup.bash
```

With Kalibr built, two scripts do the rest, in order:

```
1 tools/calib_imucam_record.sh [duration_s]      # default 90s, records a
    bag
2 tools/calib_imucam_run.sh <bag>                # kalibr's two calib steps
```

The recording step is the one that fails if driven, not held. Kalibr needs all three rotation axes excited in front of the checkerboard target; a rover rolled flat along the ground stays planar and roll/pitch are never observed, so the calibration fails silently. Pick the rig up and tilt it by hand instead. The run step needs no supervision: it executes `kalibr_calibrate_cameras` then `kalibr_calibrate_imu_camera` back to back, 10–40 CPU-heavy minutes depending on bag length. **Expected:** a `*-camchain-imucam.yaml` and a report PDF; **accept only if** the reprojection error is < 0.5 px and `timeshift_cam_imu` is a few milliseconds, not a fraction of a second. Then, specifically for MINS (the companion `calib_imucam_apply.sh` script installs a result into `openVINS`'s config only, by design, so the two estimators' opposite conventions are never mixed in one script):

```

1 python3 catkin_ws/src/leo_navigation/scripts/fill_mins_camchain.py \
2         <bag>-camchain-imucam.yaml \
3         mins_ws/src/MINS/mins/config/leo/config_camera.yaml

```

This inverts Kalibr’s `T_cam_imu` into the `T_imu_cam` convention MINS expects and copies the time offset; a direct copy without inverting produces a plausible-looking, wrong geometry.

A fourth measurement, optional but valuable: the ground-truth floor protocol of §D.11.3 validates gyro scale and cross-checks the wheel radius against a tape measure and a 3-4-5 triangle rather than against another estimator. It is not required for MINS to run, but it is the only procedure in this guide that compares any of the above to physical reality rather than to another sensor.

Test.

1. **Launch.** From `~/TOUT`:

```
1 bash tools/restart_stack.sh
```

Expected: stack relancée, then MINS initialisé après `<N>s` (typically 8–12 s), then `TERMINE`. Unlike openVINS, MINS needs no jerk to start: it initialises standing still (§D.10).

2. **Confirm the node registered:**

```

1 source /opt/ros/noetic/setup.bash
2 rosnodetool list | grep mins_subscribe

```

3. **Confirm it is actually publishing, at a healthy rate.** A registered node can be completely silent; this is the real test:

```
1 rostopic hz /mins/imu/odom           # expect ~80-90 Hz
```

Nothing at all after 15–20 s usually means one of the config files above failed to parse (§D.8.1: check `logs/navigation_master.log` for a respawn loop) or a companion node (`imu_sanitizer.py`, `wheel_remap.py`) is not running.

4. **Validate correctness, not just aliveness.** A silent estimator is an obvious failure; a wrong one that publishes cleanly at 85 Hz is not. Protocol T1.1 (§D.11.4) is the cheapest test that tells the two apart: `log /robot_pose_fused` for five minutes with the rover completely still,

```

1 cd ~/TOUT/tools
2 python3 log_pose_static.py 300 /tmp/t1_1_static_pose.csv
3 python3 analyze_pose_variance.py /tmp/t1_1_static_pose.csv

```

Pass: linear drift below 1 mm/s on every axis. A fresh install that passes this has a correctly wired MINS. One that fails it almost always has a configuration or extrinsic problem, not a filter-gain problem (§D.11.4).

5. **Validate under motion, once the above passes.** Drive a closed loop and compare MINS against open-VINS on the same run, following §D.11.1 end to end. That procedure also covers switching the cockpit's *Pose Source* to MINS and recording both estimators to a rosbag, which the website's own buffers are not reliable enough to replace.

D.8.4 Where each estimator keeps its configuration

Each estimator reads a small tree of YAML files; the launch files select which tree by name.

openVINS. A single estimator config plus its Kalibr chains, all under `catkin_ws/src/open_vins/config/leo/`:

```

1 config/leo/estimator_config.yaml      # filter: init, ZUPT, feature counts
    ,
2                                     # noise, do_calib_* flags, topics
3 config/leo/kalibr_imucam_chain.yaml  # camera intrinsics + T_imu_cam
    extrinsics
4 config/leo/kalibr_imu_chain.yaml      # IMU noise densities / random walks

```

The launch node `ov_msckf/run_subscribe_msckf` takes the config path as `argv[1]` (a launch `<param name="config_path">` is read only on the ROS2 build, a real footgun documented in `navigation_supervision.launch`).

Key platform-specific values (§12.3.5, §12.3.2): `init_dyn_use: false` and `init_imu_thresh: 0.4` (jerk-triggered initialisation, the rover must be nudged to initialise VINS), `use_stereo: true`, `max_cameras: 2`, and the frozen intrinsics (`do_calib_int: false`) with online extrinsic/time-offset refinement (`do_calib_ext: true`, `do_calib_dt: true`).

MINS. A per-sensor tree of **ten** files, selected by the config launch arg (`mins.launch: <arg name="config" default="leo"/>`), under `mins_ws/src/MINS/mins/config/leo/`. MINS initialises *standing still* (static

IMU+wheel init, 4–12 s, §12.3.5), which is why it is the default served source. Its transform output is isolated to `/tf_mins` (`mins.launch`, §12.2); the Carolus global fix enters through the VICON slot on `/mins/external_ref/carolus`. **For the full file-by-file listing, what in each file is platform-specific, and a complete download-to-test walkthrough, see §D.8.3.**

Beacon / AprilTag. The autonomy beacon config lives with the project package: `catkin_ws/src/leo_autonomy/config/apriltag.yaml` (family `tag36h11`, full-resolution `tag_decimate: 1.0`) and `tags.yaml` (the physical tag: `id: 285`, `size: 0.2032 m`). The Carolus blob detector’s parameters are in `carolus_ws/src/launch/carolusBlobDetection.launch`, including the four LED `known_points` (§12.12.3); the cockpit’s Carolus panel (§12.12.2) parses this same file.

D.9 Launch file architecture

Why this section exists: the configuration files tell each program what to believe; the launch files decide *which programs exist, in what order, and what happens when one of them dies*. That last part is a design decision on this platform, not a default, and it was arrived at by watching the stack fail.

Everything on the workstation starts from one file. Figure D.3 shows the inclusion tree.

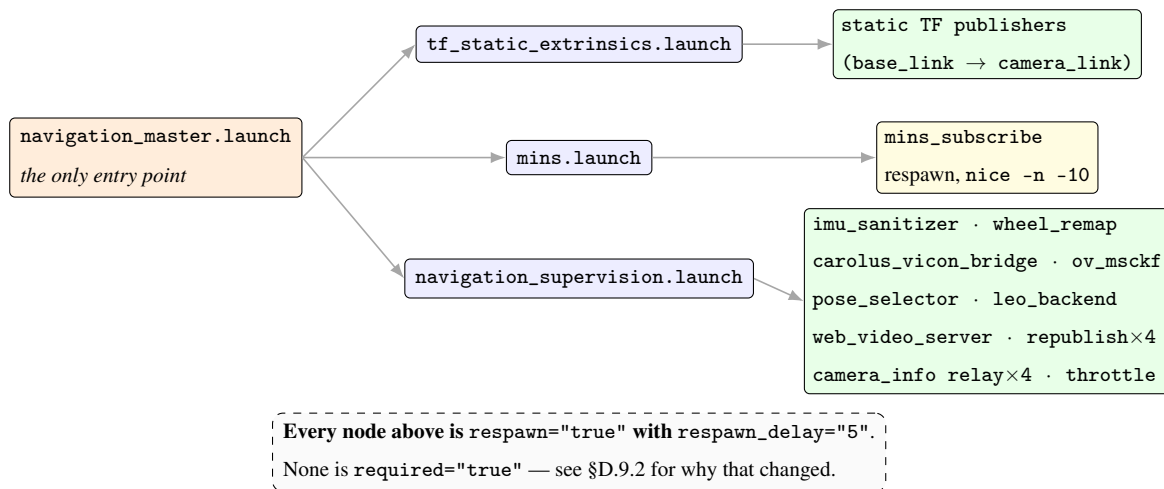


Figure D.3: The launch inclusion tree on the workstation. One entry point, three includes, and a deliberate absence: no node is allowed to take the stack down with it. The robot’s own nodes are *not* here: they come from `leo_bringup.launch` running on the Pi (§D.6).

D.9.1 What each file owns

- `navigation_master.launch`: the only file you should ever launch directly. It declares one argument, `config` (default `leo`), which it forwards to `mins.launch`, and then includes the three files below. It contains *no* nodes of its own.
- `tf_static_extrinsics.launch`: the static coordinate transforms the platform would otherwise be missing, principally `base_link` → `camera_link`. Separated so the geometry can be corrected without touching the estimators.
- `mins.launch`: the MINS estimator, and nothing else.
- `navigation_supervision.launch`: everything else: the four image republish nodes and four `camera_info` relays that bring the robot's compressed streams into a local namespace, our four Python nodes, `ov_msckf`, the cockpit backend and the video server.

It does not start `robot_state_publisher`, and that is correct: the URDF and the robot's own drivers belong to `leo_bringup.launch`, which runs on the Pi. Every file in this package runs on the workstation.

D.9.2 Reliability: why nothing is `required="true"`

This is the most important design decision in the launch layer, and it is the reverse of what it was.

`roslaunch` offers two policies when a node dies. `required="true"` means *if this node exits, kill everything*; `respawn="true"` means *restart it and leave the rest alone*.

MINS was originally `required="true"`, on the reasonable-sounding argument that restarting an estimator mid-flight without re-initialisation is meaningless, so a crash should be loud. On 2026-07-06 that policy was observed doing real damage: MINS aborted on an assertion (`SystemManager.cpp:395`, a backward IMU timestamp), and because it was `required`, `roslaunch` tore down the entire stack, including `leo_backend`. The operator lost the ability to *drive the robot* because an estimator had hiccupped.

The principle established from that, and worth carrying to any similar system:

Driving must never depend on estimator health.

Every node is now `respawn="true"` with `respawn_delay="5"`. The five-second delay matters: without it, a node that fails at startup (a bad config, a missing topic) respawns in a tight loop that saturates a CPU and floods the log, which on a thermally limited platform is its own failure.

The cost, stated plainly. On respawn, MINS re-initialises at a *new origin*, so `/mins/imu/odom` jumps. The `pose_selector`'s SE(3) correction does **not** absorb that jump: it only compensates at the moment of a deliberate source switch. So a mid-run MINS respawn produces a visible discontinuity in `/robot_pose_fused`. That is an accepted trade-off: a discontinuous trajectory is recoverable, a dead cockpit during a manoeuvre is not.

A stale comment to be aware of. The header of `mins.launch` still claims MINS “`stays required="true"`”. It does not: the node declaration twenty lines below says `respawn="true"`, and a second comment on the same node explains the change. The header was simply not updated. Trust the declaration, not the prose above it; this is exactly the kind of contradiction that misleads a successor.

D.9.3 Passing a config path: args, never <param>

Both estimators take their YAML path as a **positional command-line argument**, and getting this wrong produces one of the most misleading failures in the whole system.

```
1 <!-- CORRECT -->
2 <node name="mins_subscribe" pkg="mins" type="subscribe"
3     respawn="true" respawn_delay="5"
4     launch-prefix="nice -n -10 stdbuf -oL"
5     args="$(find mins)/config/${arg config}/config.yaml">
6 </node>
7
8 <!-- WRONG: silently ignored on the ROS1 build -->
9 <node name="mins_subscribe" pkg="mins" type="subscribe">
10   <param name="config_path" value="..." />
11 </node>
```

Why the second form fails, and fails confusingly. In the upstream source, the call that would read a `config_path` ROS parameter sits inside an `#if ROS_AVAILABLE == 2` block: it is compiled out of the ROS1 build entirely. The program therefore falls back to reading `argv[1]`. But `roslaunch` appends its own arguments to every node, so `argv[1]` becomes `__name:=mins_subscribe`, and the estimator reports that it cannot open a configuration file with that literal name. The error message names a string you never typed, which sends you looking in the wrong place.

`ov_msckf` has exactly the same contract, for exactly the same reason. Both are documented in the launch files themselves so the trap is not rediscovered.

D.9.4 Two launch-prefix tricks used here

- `nice -n -10` on MINS. The workstation's own IDE indexers were measured starving the estimator of CPU, which produced episodes that looked like estimator bugs. A negative nice value requires permission, granted to this user in `limits.conf`.
- `stdbuf -oL` on MINS. Its initialisation diagnostics go to plain `stdout`. When `roslaunch` redirects that to a file, `glibc` switches from line-buffered to fully-buffered, and at this output volume the `[INIT]` messages can sit unflushed for *minutes*: looking exactly like a hung estimator when it is initialising normally. Forcing line buffering makes the diagnostics appear as they are printed.

The robot side uses the same idea for a different purpose: `launch-prefix="chrt -f 25"` on `serial_node`, discussed in §D.6.2.

D.9.5 How to add a node

1. Put the script in `catkin_ws/src/leo_navigation/scripts/` and make it executable (`chmod +x`). A non-executable script produces `cannot locate node of type [...]`, which reads like a missing file.
2. Declare it in `navigation_supervision.launch`, with `respawn="true" respawn_delay="5"` unless you have a specific reason otherwise, and write that reason in a comment.
3. Add `output="screen"` if you want its log in `logs/navigation_master.log`. Without it, diagnostics are difficult to find.
4. Restart with `bash tools/restart_stack.sh`, never a bare `roslaunch` (§D.4.5).
5. Verify it registered *and* publishes:

```
1 rosnode list | grep my_node
2 rostopic hz /my_topic
```

A node can register and stay silent; only the second command proves it works.

D.9.6 Two files the diagram does not show, and why

`pose_selector.launch`. The diagram above shows `pose_selector` as a node inside `navigation_supervision.launch`, which is how it runs in normal operation. A standalone `pose_selector.launch`

also exists, for bringing the selector up alone against an already-running stack when you are debugging source switching. Whichever route you use, one parameter decides whether the TF tree is well formed:

```
1 <param name="base_frame" value="base_footprint"/>
```

Set to `base_link` instead, it gives that frame a second parent and the pose starts alternating silently between two coherent answers (§D.13.10). Check it in both files if you ever edit one.

`carolus_tf_bridge.py`. Absent from every launch file, and deliberately so. It is the beacon-anchored absolute-error bridge of §D.11.5, and it freezes a static `beacon_link` → `odom` transform at the first fix. A node that silently re-anchors the world frame has no business starting automatically alongside an unrelated run, so it is started by hand, by an operator who intends to use it:

```
1 rosrun leo_navigation carolus_tf_bridge.py
```

If you later decide to add it to a launch file, leave `~publish_body_chain` at its default of `false`: enabling it re-parents `base_link` and corrupts the tree in the same way described above.

D.10 Launching the system, step by step

Why an order: the master must exist before clients, and the estimators must see sensor data from their first sample or they initialise on nothing. The `leo` command sequences all of this; the value of knowing the order is being able to diagnose a *partial* start, which is the common case.

D.10.1 The nominal sequence

1. **Power the robot** and wait ~ 40 s. Its own `leo.service` brings up the ROS master and the sensors before anything on the PC can connect.
2. **Check the link first.** From any directory:

```
1 cd ~/TOUT && source tools/robot_env.sh
2 ping -c 3 "$ROBOT_HOST"
```

Expected: 0% packet loss. If this fails, stop: everything after it will fail in a more confusing way.

3. **Start the workstation side.** Two scripts, in this order. The first brings up the web stack (static server on 8000, `rosbridge` on 9090, the TLS bridge on 9443, the video server on 8080, and `cloudflared`

if configured); the second brings up the navigation stack (MINS, openVINS, pose_selector, leo_backend). Both are idempotent, so re-running them is safe.

```
1 cd ~/TOUT && source tools/robot_env.sh
2 touch /tmp/leo_maintenance          # suspend the watchdog while
    starting
3 bash web/start_web.sh
4 bash tools/restart_stack.sh
5 rm -f /tmp/leo_maintenance          # DO NOT FORGET THIS
```

Expected: a series of [ok] lines from the first script and a launch log from the second. No ECHec line.

Why the maintenance flag: the cron watchdog restarts anything it believes is dead, and a stack that is halfway up looks exactly like a stack that has died. Without the flag the two of you will fight (§D.13.3). The flag is a plain file, so **if you forget to remove it the watchdog stays disabled indefinitely** and you lose every automatic recovery in this document.

4. Confirm the robot is seen:

```
1 ping -c 1 -W 1 "$ROBOT_HOST" && echo "robot online"
2 ss -ltn | grep -E ':(8000|8080|9090|9443) ' # four listening ports
```

Expected: robot online, and four ports bound.

A note on leo start: the workstation used for this project carried a convenience wrapper at ~/bin/leo that performed the sequence above plus a status view. It lives outside the project tree and is **not** part of the repository, so it will not exist on your machine. The commands above are what it actually ran; use them directly, or write your own wrapper around them.

5. Confirm the four nodes that matter registered:

```
1 source /opt/ros/noetic/setup.bash
2 rosnodet list | grep -E "mins_subscribe|ov_msckf|pose_selector|
    imu_sanitizer"
```

Expected: all four. A missing ov_msckf almost always means it aborted at startup: look in logs/navigation_master.log, and suspect the YAML trap of §D.8.1 first.

6. Check the rates, not just the names. A registered node can be completely silent; this is the real health test:

```

1 rostopic hz /imu/data_clean           # expect ~80 Hz
2 rostopic hz /mins/imu/odom           # expect ~80-90 Hz
3 rostopic hz /wheel_odom_with_covariance # expect ~19-20 Hz
4 rostopic hz /pc/camera/infra1/image_rect_raw # expect ~15 Hz

```

/wheel_odom_with_covariance at 1 Hz instead of 19 is the signature of the UART problem of §D.13.7.

7. **Open the cockpit** at <http://localhost:8000/ops.html>.

D.10.2 Making openVINS start: the classic beginner trap

MINS initialises standing still. **openVINS does not**. It waits for an accelerometer jerk above `init_imu_thresh` before publishing anything *at all*. Newcomers conclude openVINS is broken when it is merely waiting.

1. Drive the rover briskly forward, then back, about one second each way.
2. Confirm it woke up:

```

1 rostopic hz /ov_msckf/odomimu       # expect ~75-90 Hz

```

Silence means it has not initialised. Jerk it again: more sharply.

D.10.3 Restarting cleanly when something is wrong

Do **not** kill individual nodes as a first response; several are respawn-supervised and will come back in a state you did not intend.

```

1 cd ~/TOUT
2 bash tools/restart_stack.sh

```

Expected: stack relancée, then MINS initialisé après <N>s, then switched to MINS, then TERMINE. Typical *N* is 8–12 s.

This script also raises the maintenance flag while it works, waits for MINS to publish before declaring success, and reapplies the camera settings that reset on restart (§D.6.3).

D.10.4 A short tour of the cockpit

- **Operations:** live video, mission state, battery, and the *Pose Source* switch (MINS/VINS). Manual driving is here.
- **Trajectory:** the live plot of the served pose, the *Test 1 / Test 2* selector, the MATLAB export buttons, and the *ground-truth calibration* panel (§D.11.3).
- **Logbook**, the project journal and the download card for this report.
- **PID / Nav Modes / Robot**, tuning, autonomy state, and the URDF view.

The cockpit is served over an authenticated Cloudflare tunnel as well, so it works from a phone or tablet standing next to the robot, which is how the calibration is meant to be done.

D.10.5 Editing the cockpit without breaking it

Two habits, both learned the hard way, and both cheap.

Your edit is live but you are not seeing it. This is the first thing that will happen to you, and it is not your edit. Cloudflare caches aggressively and a dashboard rule overrides the `Cache-Control: no-cache` that `serve.py` sends. Static assets therefore carry a version stamp in their query string:

```
1 <script src="app.js?v=r37"></script>
```

The query string is part of Cloudflare's default cache key, so **incrementing that number is what forces a fresh copy**. Bump it in every page that references the file you changed, and keep the stamps consistent across pages, since they have drifted apart before and produced two pages running different versions of the same script.

A browser force-reload does *not* help, because the stale copy sits at the network edge and not on your machine. If you are testing over `http://localhost:8000` you will not see the problem at all, which is exactly why it surprises people the first time they check the public URL.

Validate JavaScript before you reload. `app.js` is large and a brace inserted in the wrong place produces a file that loads without error and simply stops executing partway through, so the page renders and half the panels are dead. Counting braces by hand does not catch this; it failed once in this project and cost an afternoon. Parse the file instead:

```
1 pip install esprima # once
2 python3 -c "import esprima,sys; esprima.parseScript(open('web/app.js').
    read()); print('OK')"
```

Do the same for the inline scripts in `ops.html` if you touch them. A parse takes under a second and turns a silent partial failure into a line number.

D.10.6 Two pages, deliberately different: control versus demonstration

Two pages, deliberately different (cockpit v4). `web/ops.html` is the control surface: it holds every command path, manual driving, AUTO engagement, pose-source switching, beacon reset, MATLAB export. `web/demo.html` subscribes to the same topics and publishes nothing at all. It has no command bindings to disable, because it has none to begin with.

The separation is not cosmetic. A defence is the one occasion where the machine is driven by someone who is talking to a room rather than watching the robot, and where a mis-click is expensive and public. `demo.html` is built so that no mis-click exists: it shows the trajectory in the beacon frame, heading quivers, and the RMS error at 56 px, and it cannot move the rover. Open it on the projector; keep `ops.html` on your own screen.

Why read-only is a structural property, not a setting. The distinction matters because the two obvious alternatives are both worse. Disabling the controls with a flag leaves the command paths present in the page, one `localStorage` key or one console call away from being live again, and it invites the reader of the code to wonder which state the page is in. Hiding them with CSS is worse still, since the handlers remain bound and a keyboard shortcut or a stray focus event can fire them. `demo.html` instead *subscribes* and never *advertises*: it holds no publisher, no command topic and no `data-cmd` binding anywhere in its markup. There is no state in which it can move the robot, so there is no state to verify before a defence.

What each page is for.

	ops.html	demo.html
Audience	the operator, alone	a room, during a defence
ROS role	subscriber <i>and</i> publisher	subscriber only
Commands	manual drive, AUTO, pose source, beacon reset, MATLAB export	none exist
Failure mode	a mis-click moves the rover	a mis-click does nothing
Where to open it	your own screen	the projector

A practical consequence for the transport. Because `demo.html` never publishes, it is immune to the entire class of symptoms described in §D.5.5: a zombie TLS port makes buttons inert, and a page with no buttons cannot present that symptom. If the demonstration view is updating and the control view is not, you have localised the fault to the outbound direction of the WebSocket before running a single command.

D.11 Reproducing the experiments

D.11.1 A MINS versus openVINS comparison run

Why a rosbag and not the website: the cockpit’s in-memory buffer is lost if the backend restarts, and ros-bridge can desynchronise. A run that matters must be written to disk by a process that does not depend on the web stack.

1. **Pre-flight.** Run the checklist of §D.13.13. This is not optional, most of the discarded data in this project failed one of those six items.
2. **Wake openVINS** with a jerk, and confirm both estimators publish.
3. **Mark a start point** on the floor with tape. You will drive a closed loop and return to it; the gap on return is the only drift measure available without ground truth.
4. **Start recording** (from `~/TOUT`):

```

1 cd ~/TOUT
2 tools/record_trajectories.sh 180 essai_A

```

Records 180s into data/trajectories/essai_A_<timestamp>.bag. Omit the duration to record until Ctrl-C. **Both** estimators are always captured together: that is the entire point.

5. **Drive the loop** and return to the marked point. Drive slowly and smoothly; note the commanded speed.

6. **Convert to CSV:**

```
1 python3 tools/bag_to_csv.py data/trajectories/essai_A_<timestamp>.bag
```

Expected: one CSV per source, _mins.csv, _vins.csv, and _carolus.csv if a beacon was visible. Columns are t,x,y,z,qx,qy,qz,qw,yaw_rad, so any tool can read them: MATLAB is a convenience, not a requirement.

7. **Plot and get the numbers** (run from tools/ so MATLAB finds the script):

```
1 cd ~/TOUT/tools
2 matlab -batch "plot_trajectories(' ../data/trajectories/essai_A_<ts>
   _mins.csv ')"
```

Expected: a printed metric block (divergence, scale, path lengths, heading gap) and a saved _comparison.png plus eight _p1.._p8.png panels.

8. **Put it in the report:** copy _comparison.png to report_latex/figures/traj_comparison.png and the panels to traj_p1.png... traj_p8.png, then recompile (§D.12).

Two independent drives instead of one? Use tools/compare_tests.m on two runs labelled Test 1 and Test 2. It deliberately does *not* compute the Kabsch divergence: that requires both estimators observing the *same* motion at the *same* instants, which two separate drives are not. It reports loop-closure error and heading drift instead, which are meaningful for independent runs.

D.11.2 How to read the nine-panel figure

The panels are described one by one in §12.14.1, and the mathematics behind panels 2–4 in §12.14.2 and §12.14.3. Two habits will keep you honest:

- **Read panels 2 and 4 together, never separately.** Panel 2 aligns the trajectories *including a scale factor*, so it is constructed to hide a scale disagreement. Panel 4 is the only one that exposes it. On the July 29 run, 47% of the residual was pure scale error.

- **The scale factor was fitted to MINS, and MINS is not ground truth.** The script prints a warning whenever the fitted scale departs from unity by more than 3%. Quoting the scale-corrected residual alone would be quoting a number built to be insensitive to the error you are measuring.

D.11.3 Ground-truth calibration: the most valuable measurement

Everything above compares estimators to each other. This is the one procedure that compares them to *reality*, and it is what a new student should do first.

1. **Build the measurement zone.** Follow docs/protocole_calibration_terrain.md. The essentials: construct a right angle with a 3-4-5 triangle (90, 120, 150 cm, a tape measure does this to 0.08°, where a protractor manages $\pm 1^\circ$), and hang two plumb lines from the robot’s centreline so its heading is marked on the floor without parallax.
2. **Open the Trajectory tab** on a phone or tablet and pick a manoeuvre: 90° G, 90° D, or 1 m.
3. **Mark start**, drive the manoeuvre, **mark end**. The backend freezes that instant: take as long as the measurement needs.
4. **Measure physically.** For a rotation, read the lateral offset d of the far plumb mark from the reference line over the lever arm L , and compute the residual $\varepsilon = \arctan(d/L)$; the true angle is $90^\circ \mp \varepsilon$. With $L = 1$ m and d read to ± 2 mm this gives $\pm 0.11^\circ$.
5. **Enter what you measured**, never the value you aimed for. Then validate.
6. **Three passes per manoeuvre, both directions.** The panel reports the mean factor and its standard deviation, and refuses an entry that disagrees with all four sensors by more than a factor of five.

The reasoning that matters more than the numbers. The first campaign found the gyroscope over-reporting by 7% and the wheels by 9%. Wheel-derived heading comes from differential odometry and does not use the gyroscope: these are physically independent instruments. MINS and openVINS are *not* independent witnesses, since both integrate the same gyro.

Two independent sensors agreeing with each other and disagreeing with the reference points at **the reference**. So the conclusion was not “apply a 7% correction” but “re-measure with a better floor protocol”. Decide between the three outcomes:

Result over 3 passes	What it means
both $\rightarrow 1.00$	the first campaign was measuring the protractor. Change nothing; leave <code>gyro_scale</code> at 1.0.
both stay ≈ 0.93	a real shared error: applicable, but you must then explain why two different physical principles drift by the same amount.
they diverge	each has its own defect. Treat separately: <code>gyro_scale</code> for the gyro, effective radius / wheel base for the wheels.

Standard deviation above 5% means the *method* still dominates: do not apply anything, improve the marking.

D.11.4 Protocol T1.1: static drift and variance analysis

Chapter 12 (§12.7.2) uses this protocol to separate two things that look identical in a single trajectory plot but demand opposite fixes: *noise* (bounded, average it away) and *drift* (unbounded, points at an unmodelled bias in extrinsics or initialisation). It requires the rover to do nothing at all, which makes it the cheapest measurement in this guide and, per Table D.3, a fully automated one.

1. **Fresh initialisation.** Restart the stack (§D.10) rather than reuse a session that has been running for a while: T1.1 characterises the estimator from a cold start, and a warm one hides exactly the divergence it exists to catch.
2. **Place the rover and do not touch it again.** Flat ground, clear of anything that would tempt you to nudge it once logging starts.
3. **Log for five minutes** (300 s is the script's own default, matching the campaign's measurement):

```
1 cd ~/TOUT/tools
2 python3 log_pose_static.py 300 /tmp/t1_1_static_pose.csv
```

The console prints `Logging /robot_pose_fused for 300s - keep the rover PERFECTLY STILL.`, then, after five minutes, `Wrote <N> samples to /tmp/t1_1_static_pose.csv` with N close to $300 \times 20 = 6000$ at the nominal 20 Hz publish rate.

4. Analyse:

```
python3 analyze_pose_variance.py /tmp/t1_1_static_pose.csv
```

This reports, per axis: standard deviation (spread around the mean, a noise/covariance question if bounded), a least-squares linear drift rate in m/s (near zero means noise; a clear nonzero slope means genuine unbounded divergence), and a first-10%-versus-last-10%-of-the-window mean as a second, independent check of the same conclusion.

The verdict this campaign used: drift below 1 mm/s on every axis passes. Above it, do not touch a noise parameter — Eq. (12.7)’s χ^2 gate and the covariances upstream of it are tuned for noise, not for a genuine bias, and loosening them to accommodate an unbounded drift will make the estimator accept bad data rather than fix the cause. A failing T1.1 run points at extrinsics or initialisation (§D.8.1), never at a filter gain.

D.11.5 Absolute error: anchoring the trajectory to the beacon

READ THIS BEFORE PUBLISHING ANY NUMBER FROM THIS SECTION

THE COLOUR CAMERA INTRINSICS ARE WRONG AND MUST BE RE-CALIBRATED

FIRST. The detector runs with parameters calibrated at 1280×720 against a 640×480 stream. Its stored principal point, $c'_x = 643.47$, lies *outside* a 640-pixel-wide image.

Consequences, derived in §12.15.3: every range is inflated by 69.3 %, and the lateral estimate carries a fixed offset $\Delta X = \Delta c_x L / w$ that translates the entire scene. The result is smooth, repeatable, internally consistent and metrically meaningless, which is the most dangerous combination a measurement can present.

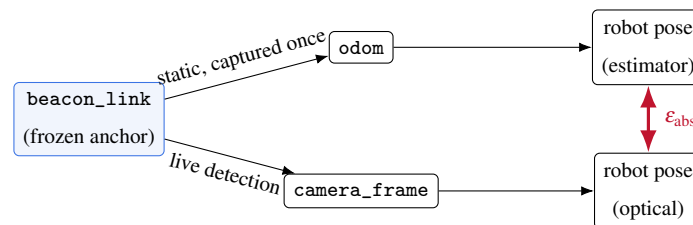
The procedure below is correct and ready to run. Its output is not publishable as a dimensional quantity until the calibration of §D.14.1 has been done. Treat what follows as a rehearsal until then.

Why this replaces the alignment-based error. Every figure in Chapter 12 up to §12.14.2 is a residual left after fitting a transform between two trajectories, by Kabsch (rigid) or Umeyama (with scale). §12.15.2 shows that such a fit is an orthogonal projection away from a seven-dimensional subspace $\mathcal{G}$ of translations, rotations and scale, and that the dominant estimator failure modes, heading drift and scale drift, are basis vectors *of* that subspace. An alignment-based metric is therefore blind by construction to the very errors the comparison exists to expose, and every number it reports is a lower bound.

The bridge removes the fit rather than improving it. The transform is established once, physically, by anchoring, and thereafter held constant, so no parameter is estimated at measurement time.

What carolus_tf_bridge.py does. It publishes exactly two transforms and derives two topics from them:

- `camera_frame` → `carolus_raw`, live, from each optical detection of the beacon.
- `beacon_link` → `odom`, static, captured at the first fix and then **frozen**. This is the anchoring act, and it is the whole point: from this instant the odometric world and the optical world share one absolute reference.
- `/odom_in_beacon` and `/carolus_in_beacon`, both carrying the *robot's* pose in the beacon frame, reached respectively through the odometry/estimator chain and through the optical detection. Their instantaneous difference is the absolute error.



Step by step.

1. **Place the beacon** where it stays visible for the whole planned loop, and do not move it afterwards. Moving the beacon after anchoring invalidates the run silently, because the frozen transform no longer describes reality and nothing detects the discrepancy.
2. **Confirm the detector is actually seeing it** before starting anything else:

```
1 rostopic hz /pose                # non-zero => the beacon is detected
2 rosnode info /carolus_astrobee_rex | head -20
```

A rate of 0Hz with `Not enough blobs < 4` in the log means the LEDs are not resolved: adjust distance, exposure or lighting. The bridge cannot anchor to something that was never detected.

3. **Start the bridge by hand.** It is deliberately absent from every launch file, so that it can never perturb a run you did not intend it to touch:

```
1 rosrn leo_navigation carolus_tf_bridge.py
```

4. **Verify the anchor took**, and that the TF tree is still well-formed (§D.13.10):

```

1 rostopic echo -n1 /odom_in_beacon
2 rostopic echo -n1 /carolus_in_beacon
3 rosrn tf tf_monitor 2>&1 | head -30      # one parent per frame

```

5. **Record both topics**, along with the usual estimator outputs. **This is the step that has never been done**, and its absence is why no absolute error appears anywhere in this report:

```

1 rosbag record -O run_absolute.bag \
2   /odom_in_beacon /carolus_in_beacon \
3   /mins/imu/odom /ov_msckf/odomimu /robot_pose_fused /firmware/
   wheel_odom

```

6. **Drive the loop** and return near the start. Keep it short, for the thermal reasons of §D.13.2.
7. **Compute the error** as the instantaneous difference between the two beacon-frame topics. No alignment step is involved, and none should be added: adding one would reintroduce exactly the projection this method exists to remove.

Never let the bridge publish the body chain

The bridge exposes a `~publish_body_chain` parameter which defaults to false and logs an error if enabled. An early revision published the full body chain in the beacon frame, which gave `base_link` a second parent and corrupted the tree (§D.13.10). The lookups do not fail; they alternate between two plausible answers. Leave the parameter alone.

D.11.6 Camera–IMU calibration with Kalibr

Both estimators consume the same physical calibration, produced offline and then frozen. Offline measurement beats online refinement: that is why `calib_cam_extrinsics` is false.

1. **Target.** Print a Kalibr target (an Aprilgrid is preferable to a checkerboard for full-view corner observability) and record its exact geometry in a `target.yaml`.
2. **Intrinsics and stereo extrinsics.** Record a bag waving the target across the whole field of view of both infrared cameras, then run `kalibr_calibrate_cameras` on the two image topics. This produces the `*-camchain.yaml`. Achieved on this D455 pair: 0.20–0.23 px reprojection error at a 90.2 mm baseline.

3. **Camera–IMU extrinsics and time offset.** With intrinsics fixed, record a second bag **exciting all IMU axes** and run `kalibr_calibrate_imu_camera`.
4. **Install** the outputs: openVINS reads them directly as `kalibr_imucam_chain.yaml` and `kalibr_imu_chain.yaml`; MINS’s `config_camera.yaml` is filled from the same `camchain` via `fill_mins_camchain.py`, remembering the inverse-convention trap.
5. **Verify** with the live reprojection monitor (`calibration_monitor.py`). Acceptance threshold used throughout this project: < 0.5 px.

Standing caveat, and the recommended first task. Step 3 has **never been completed** on this platform. The deployed camera–IMU extrinsic is a tape-measure seed. The blocker is physical: the procedure needs roll and pitch excitation of $\gtrsim 1.5$ rad/s, and a rover on flat ground has only ever achieved 0.08. You must *lift and tilt the robot by hand* in front of the target. Doing this is the single most valuable calibration task outstanding.

D.11.7 Calibration methodology, in full

Both estimators consume the same physical calibration, produced offline with Kalibr [5] and then frozen (offline measurement beats online refinement, §12.3). The procedure, reproducible from scratch:

1. **Target.** Print a Kalibr calibration target (an Aprilgrid is recommended over a checkerboard for full-view corner observability) and record its exact geometry in a `target.yaml`.
2. **Intrinsics + stereo extrinsics.** Record a rosbag waving the target across the full field of view of both IR cameras, then run `kalibr_calibrate_cameras` on the two image topics. This yields the `*-camchain.yaml` (intrinsics per camera, plus the stereo baseline). On this D455 pair the achieved reprojection error was 0.20–0.23 px at a 90.2 mm baseline (§12.3); intrinsics are then frozen (`do_calib_int: false`).
3. **Camera–IMU extrinsics + time offset.** With the intrinsic `camchain` fixed, record a second bag exciting all IMU axes (rotate and translate the rig) and run `kalibr_calibrate_imu_camera` with the `camchain`, the `imu.yaml` noise model and the target. This produces the `*-camchain-imucam.yaml` with ${}^I\mathbf{T}_C$ and the camera–IMU time offset t_d . The correct forward-looking mount matters: an identity placeholder here was the root cause of the vertical divergence fixed in §12.3.2 (Eq. 12.13). t_d and the translation are

then refined online (`do_calib_ext`, `do_calib_dt`), but the 90° rotation must be seeded correctly: it is outside the filter’s recoverable region.

4. **Install.** Copy the Kalibr outputs into each estimator’s config: openVINS reads them as `config/leo/kalibr_imucam_chain.yaml` and `kalibr_imu_chain.yaml` directly; MINS’s `config_camera.yaml` is populated from the same camchain (the prepared `fill_mins_camchain.py` helper maps one to the other). The IMU noise densities in `kalibr_imu_chain.yaml` come from an Allan-variance characterisation of the D455 IMU (or the manufacturer figures inflated for margin).
5. **Verify.** The live reprojection-error monitor (`calibration_monitor.py`, surfaced on the cockpit’s calibration panel) reports the current chain’s health; the acceptance threshold used throughout this project is < 0.5 px (§12.3).

A standing caveat carried in the open-items registry: the deployed camera–IMU extrinsic is still a tape-measure seed refined online, not a full `kalibr_calibrate_imu_camera` solution (Table 12.12, item 2); completing step 3 above from a dedicated bag is the recommended first calibration task for a new student.

D.12 Publishing the report

```
1 cd ~/TOUT
2 bash tools/update_report.sh
```

Expected: three progress lines, then `[3/3] OK`, `<N>` pages, then a local and a public URL.

The script runs `pdflatex` three times with `biber` in between (needed for cross-references and the bibliography to settle), copies the PDF into `web/reports/`, and **stamps the download link with a version number**.

Why the version stamp exists. The Cloudflare tunnel served a four-hour-old PDF for hours despite the origin sending `Cache-Control: no-cache`, a dashboard rule overrides the origin header, and it is not editable from this machine. A query string *is* part of the cache key, so a fresh `?v=` forces a miss. **Use the URL the script prints**; a bare URL may serve a stale report, and no amount of browser refreshing helps because the cache is at the network edge.

Two traps inside this script, both now fixed, both worth knowing: `main.log` contains non-ASCII bytes, so `grep` classes it as binary and **silently suppresses its output**, the error check saw nothing at all until `-a` was added, and thirteen real errors passed while the script reported success. And `pdflatex` wraps its log

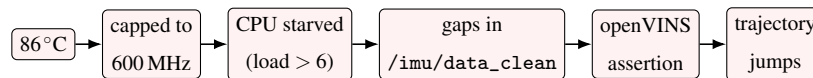
lines at ~ 79 columns, so the phrase it was grepping for the page count could be split in two depending on document length; with `set -e` that killed the script before deployment.

D.13 Troubleshooting and lessons learned

This section is the distilled cost of the campaign. Each entry is a real failure, its measured signature, and its fix. They are ordered by how much time they cost.

D.13.1 The Pi overheats, and everything downstream lies

This is the most important entry in this appendix. The Pi runs at 86°C , above its soft temperature limit, so the firmware caps the CPU at 600 MHz instead of 1.5 GHz. The consequences cascade, and none of them look thermal:



How to confirm it in one command (on the robot):

```
1 ssh pi@$ROBOT_HOST 'vcgencmd measure_temp; vcgencmd get_throttled; \  
2                          vcgencmd measure_clock arm'
```

Expected when healthy: throttled=0x0 and ~ 1.5 GHz.

Observed at the time of writing: temp= 86.2°C , throttled=0xe0006 (bits 1 and 2 set: frequency capped and throttled right now), 600 MHz.

The measurable damage. Count the holes in the IMU stream:

```
1 cd ~/TOUT && source tools/robot_env.sh  
2 source /opt/ros/noetic/setup.bash  
3 python3 - <<'EOF'  
4 import rospy, time  
5 from sensor_msgs.msg import Imu  
6 rospy.init_node('g', anonymous=True, disable_signals=True)  
7 S=[]  
8 rospy.Subscriber('/imu/data_clean', Imu,  
9                  lambda m: S.append(m.header.stamp.to_sec()), queue_size  
                           =2000)
```



```

10 time.sleep(20)
11 d=[S[i]-S[i-1] for i in range(1,len(S))]
12 med=sorted(d)[len(d)//2]
13 g=[x for x in d if x>3*med]
14 print('%d samples, %.1f Hz, %d gaps, %.2f s lost' %
15       (len(S), 1/med, len(g), sum(g)))
16 EOF

```

Healthy: zero gaps. Observed: 4 to 21 gaps per 20–40 s, each ~ 39 ms (four consecutive samples missing). Those gaps make openVINS abort on `Propagator.cpp:101`, because the IMU readings between two filter clones no longer sum to the interval between them. It respawns, re-initialises, and the trajectory jumps.

The fix is a fan. Install active cooling before trusting any estimator measurement. An A/B test of the same gap count before and after cooling is the first experiment a new student should run.

Update, July 30: the envelope has since been measured on both sides. The claim in earlier editions that the software levers were exhausted rested on an invalid test (colour was already off at driver level, so the measurement compared a configuration with itself). The colour nodelet has since been measured at 185% CPU. More usefully, the failure threshold is now bracketed rather than assumed:

	86°C	89.1°C
Wheel odometry	19.7 Hz	0 Hz
IMU	82.8 Hz	0 Hz
UART overruns	137000	676751
Load average (4 cores)	not measured	8.85
serial_node	synchronised	Lost sync loop

Three degrees separate a working rover from one that answers neither manual nor autonomous commands, because `/serial_node` is the only subscriber of `/cmd_vel` and it is the component that dies. If driving stops in *both* modes at once, look at the serial chain first: that simultaneity rules out the mission logic and the web interface before you have tested either. Recovery is a targeted cycle, not a stack restart:

```

1 rosservice call /rosmon/start_stop "{node: 'serial_node', ns: '', action
  : 2}"
2 sleep 4

```

```
3 rosservice call /rosmon/start_stop "{node: 'serial_node', ns: '', action
: 1}"
```

This restores the link within about twenty seconds and the launch wrapper reapplies the SCHED_FIFO priority automatically, but it is a reset, not a repair. Watch the overrun counter afterwards (`sudo cat /proc/tty/driver/ttyAMA`): if it keeps climbing, and at 88°C it does, the collapse will recur within tens of minutes.

D.13.2 The hit-and-run protocol: working usefully on a throttled Pi

The previous entry says install a fan, and that remains the right advice. This entry is about what to do on the many days when you do not have one, because the honest answer is not “nothing”. A short run at 86°C produced usable data; a longer one at 89°C produced none at all and required manual recovery. The difference is worth managing deliberately rather than discovering halfway through an acquisition.

What was actually measured, on both sides. With `enable_color = true` and the Pi settled near 86°C, the system was fully throttled (`throttled=0xe0006`, 600 MHz) and yet the critical chain held: wheel odometry at 19.7 Hz, IMU at 82.8 Hz with 14 gaps per 20 s, serial link synchronised throughout. That is a working rover, and it is the regime in which a short acquisition is worth attempting.

Three degrees higher the same configuration collapsed completely. Every firmware topic dropped to 0 Hz, the overrun counter reached 676751, and `serial_node` entered a `Lost sync` loop from which it did not return unaided. §12.15.1 derives why the transition is a cliff rather than a slope: the recovery handshake needs a clean window, and past a threshold it stops finding one.

Do not read the first result as a guarantee

“The link survived thermal throttling” is true of the 86°C run and false of the 89°C run. The correct statement is that the margin is about three degrees wide with no headroom for ambient variation. Plan for the collapse, do not assume the survival.

The protocol.

1. **Start cold.** Power the Pi and leave the perception stack down for two or three minutes. Starting at 70°C instead of 86°C buys most of the useful window.
2. **Open the vitals monitor before anything else**, in its own terminal, and leave it running for the whole session:

```
1 python3 tools/hotrun_monitor.py           # one line every 20 s
```

It prints temperature, throttle word, clock, IMU gap count and wheel-odometry rate side by side, and flags its own stop criteria.

3. **Abort on any one of three signals**, without negotiating with yourself: temperature above 88°C, more than 25 IMU gaps per 20 s, or wheel odometry below 10 Hz. The third is the important one, because it means the serial link is going and control authority with it.
4. **Keep the run short.** Everything measured in this campaign that was worth keeping came from acquisitions of a few minutes. Ambition about duration is what turns a good run into a recovery exercise.
5. **Record to disk, not to the browser.** Use `tools/record_trajectories.sh`, which writes a rosbag locally and is indifferent to the web stack. The cockpit's own buffer is cleared by any backend restart, and a thermal event is precisely the circumstance that causes one.

Recovery when it does collapse. Cycle the serial node alone. Do not restart the stack: the July 22 lesson (§12.9.3) is that the surrounding infrastructure is almost never the culprit, and a full restart costs minutes and destroys the run's context.

```
1 rosservice call /rosmon/start_stop "{node: 'serial_node', ns: '', action
  : 2}"
2 sleep 4
3 rosservice call /rosmon/start_stop "{node: 'serial_node', ns: '', action
  : 1}"
```

Expect the topics back within about twenty seconds, and expect the launch wrapper to reapply the SCHED_FIFO priority automatically; verify it with `chrt -p <pid>` rather than assuming. Then watch the overrun counter:

```
1 ssh pi@$ROBOT_HOST 'sudo cat /proc/tty/driver/ttyAMA' | grep -o 'oe
  : [0-9]*'
```

If it is still climbing, and at 88°C it will be, the cycle has bought you minutes rather than fixed anything. Use them to save your data, not to start a new run.

Why both modes of driving fail together, and what that tells you. When the serial link dies, manual and autonomous control stop at the same instant. This looks alarming and is in fact the most useful diagnostic signal the platform offers: the two modes share exactly one component downstream of the mission logic, namely `/serial_node`, the sole subscriber of `/cmd_vel`. A fault that removes both at once is therefore in the serial transport, and you may discard the web interface, the command bus and the finite-state machine without testing any of them. Confirm it in one command:

```
1 rostopic info /cmd_vel          # publisher /leo_backend, subscriber /
    serial_node
2 rostopic hz /firmware/wheel_states  # 0 Hz => the link, not the logic
```

D.13.3 A cron script that destroyed the stack every two minutes

`tools/leo_watchdog.sh` runs from cron every minute and restarts things it believes are dead. It sourced `/opt/ros/noetic/setup.bash` at line 141, but its two node-presence checks call `roscall` at lines 100 and 123. Under cron the `PATH` is minimal, so `roscall` did not exist yet: both checks failed *every minute regardless of the robot*, their counters reached the 6-tick threshold, and the whole stack was torn down, measured at six restarts in fourteen minutes.

Why this matters to you even though it is fixed: openVINS never had more than two minutes to converge. Every estimator measurement taken before the fix was made on a stack being destroyed mid-run, and had to be discarded: including a “rotation causes divergence” conclusion that evaporated once the stack was stable.

- **Check before any measurement campaign:**

```
1 tail -5 ~/TOUT/logs/watchdog.log
2 cat /tmp/leo_zombie_fails /tmp/leo_vins_fails    # both should read 0
```

- **What else it guards, none of which was documented before 2026-07-31.** The watchdog is not only a restart loop, and knowing what it does prevents you from fighting it:
 - **Drowning probe on both rosbridge ports** (2026-07-13, extended to 9443 on 07-31). A real Web-Socket handshake against 9090 in clear and 9443 over TLS; a bridge that holds its port but has stopped answering is purged and relaunched. See §D.13.12, and read the TLS warning there before touching it.

- **D455 self-healing with an escalation ladder.** A silent camera is first cycled through `rosmon`; if it stays silent, the watchdog issues a *hardware* reset via `/firmware/reset_board`. A persistent counter in `/tmp` debounces this across ticks so that one bad minute does not trigger a board reset. If you are power-cycling the camera by hand, set the maintenance flag first or the two of you will take turns resetting it.
- **Battery alert bound to the camera fault.** Past three consecutive silent minutes it logs the battery voltage alongside the fault, because a rail sagging below roughly 10.5 V drops the USB camera and presents exactly as a software failure. A camera that will not come back is a battery symptom until proven otherwise.

- **Suspend the watchdog during any deliberate manipulation:**

```
1 touch /tmp/leo_maintenance      # watchdog does nothing while this
    exists
2 rm -f /tmp/leo_maintenance      # DO NOT FORGET THIS
```

- **General lesson:** a script tested by hand runs with a good PATH. Bugs of this class only appear under cron. Test with `env -i PATH=/usr/bin:/bin bash -c '...'`.

D.13.4 Wheel odometry is a reference, not the truth

The mecanum wheels slip. Measured against a tape measure over 1 m, the wheel odometry over-reports by 39%. It is still useful (it never errs by a factor of fifty, so it catches gross estimator divergence) but it is not ground truth and must never be quoted as accuracy.

Worse, it is *asymmetric*: the correction factor is 0.9325 turning left and 0.8920 turning right. Decomposing per-direction factors into a symmetric and an antisymmetric part, $\bar{g} = (g_L + g_R)/2$ and $\delta = (g_L - g_R)/2$, separates the two causes: a *sensor scale* error is sign-symmetric and lands in $\bar{g}$, whereas a *mechanical* defect reverses with direction and lands in δ . The wheels' $|\delta|$ is ten times the gyroscope's. Correcting that with one global gain would improve one direction and worsen the other.

D.13.5 When two independent sensors agree against your reference

A subtle trap, and a genuinely useful piece of reasoning. The first calibration campaign found the gyroscope over-reporting rotation by 7% ($\bar{g} = 0.9329$), and the wheels over-reporting by 9% ($\bar{g} = 0.9123$).

Wheel-derived heading comes from differential odometry and **does not use the gyroscope at all**: these are physically independent instruments. MINS and openVINS are *not* independent witnesses here, since both integrate the same gyroscope.

Two independent sensors agreeing with each other and disagreeing with the reference points at **the reference**, not at both sensors. The conclusion was therefore *not* “apply a 7% gyro correction” but “re-measure with a better floor protocol”. The corresponding parameter, `~gyro_scale` in `imu_sanitizer.py`, was added and deliberately left at 1.0, no correction, until three repeated passes in each direction settle the question. It affects MINS and openVINS simultaneously, so an error there propagates everywhere.

D.13.6 Do not filter an estimator’s output

A cautionary tale, included because it cost a full evening and the mistake is tempting. Believing openVINS diverged during turns, the author added a node that froze its reported position whenever the gyroscope exceeded a threshold.

It made things worse, and the measurement that proved it is simple: with the robot verified stationary (wheel odometry 0.000 m over 30 s), raw openVINS wandered 16.99 m of *cumulative path* but ended 0.021 m from where it started. It was oscillating around a stable point, not drifting. The guard rejected 12% of the increments of a *symmetric* oscillation, which does not remove a zero-mean signal: it **rectifies** it. Its own counters showed 87.10 m of accumulated offset for zero real motion.

- Distinguish **cumulative path** from **net displacement** before diagnosing drift. They answer different questions and only the second one is drift.
- Removing samples one-sidedly biases a sum. If you must smooth, use a filter that **preserves the mean** (a low-pass), never a selective freeze.
- Fix the estimator’s inputs, not its outputs.

D.13.7 Serial data loss: it is a scheduling problem

Symptom: thousands of checksum errors and wheel odometry collapsing from 19 Hz to 1 Hz. It is not a cable and not the CORE2 board. The Pi’s PL011 UART FIFO overruns when `serial_node` is not scheduled in time.

```
1 ssh pi@$ROBOT_HOST 'sudo grep -o "oe:[0-9]*" /proc/tty/driver/ttyAMA'
```

Expected: oe:0 and stable. A climbing oe counter is overrun. The fix, already deployed, needs *both* halves or the node will not start at all: real-time priority on serial_node (chrt -f 25) *and* a LimitRTPRIO=50 systemd drop-in on leo.service granting permission to request it. Verify with chrt -p <pid>; expect priority 25.

D.13.8 Configuration files that fail silently

Both estimators parse their YAML with OpenCV's FileStorage (%YAML:1.0), which **aborts on a long comment placed after a value on the same line**. The node then starts, subscribes to nothing, and respawns forever: with no error naming the file. This trap was hit twice, the second time after it had already been documented.

- Keep inline comments in these files **short**, or put them on their own line.
- Validate before restarting anything:

```
1 python3 -c "import cv2; f=cv2.FileStorage('estimator_config.yaml', \
2 cv2.FILE_STORAGE_READ); print('readable:', f.isOpened()); f.release()"
```

- Never test a config from /tmp: relative paths inside it resolve differently and produce false negatives.

D.13.9 Smaller traps, each of which cost real time

- **pgrep -f matches your own shell**. A command containing the pattern appears in its own search results, so kill \$(pgrep -f my_node) can kill the shell running it. Encountered three times. Use a bracket trick (pgrep -f "my_no[d]e") or filter by parent.
- **kill -0 lies about liveness**, because PIDs are recycled. Verify identity with /proc/<pid>/cmdline.
- **Timestamps, not arrival times**. Measure rates and velocities from header.stamp. Arrival time mixes in network jitter and produces confident nonsense.
- **Medians hide tails**. A divergence with a median of 0.69 had a worst case of 19.96 in the same data set. Report a high percentile as well; what ruins a trajectory lives in the tail.
- **Path length is not comparable across sampling rates**. Per-sample noise inflates cumulative length as $(\sigma f/v)^2$ (Eq. 12.85). Two raw lengths at 86 and 80 Hz once read identically while the true lengths differed by 10%. Resample to a common rate first.

- **Restart the estimators after changing calibration.** Configs are read once. Changing the IMU calibration mid-run once cost 34 minutes of recorded data.
- **matlab -batch needs a live licence session.** Online licensing fails with error 5001 in batch mode when the account token has lapsed, even while an already-open interactive session keeps working. Test with `matlab -batch "disp('ok')"` before blaming a script.
- **The published PDF may be cached.** The Cloudflare tunnel served a stale report for hours despite a correct origin header. `update_report.sh` now stamps the download link with a version; use the link it prints, not a bare URL.

D.13.10 The TF tree accepts only one parent per frame

Symptom: intermittent TF_OLD_DATA, poses that jump between two plausible values, and no error anywhere

A frame in a tf2 tree has exactly *one* parent. Nothing enforces this: publish `base_link` from two nodes and both transforms are accepted, after which lookups resolve to whichever arrived last. The result is not a crash but an estimate that silently alternates between two coherent answers.

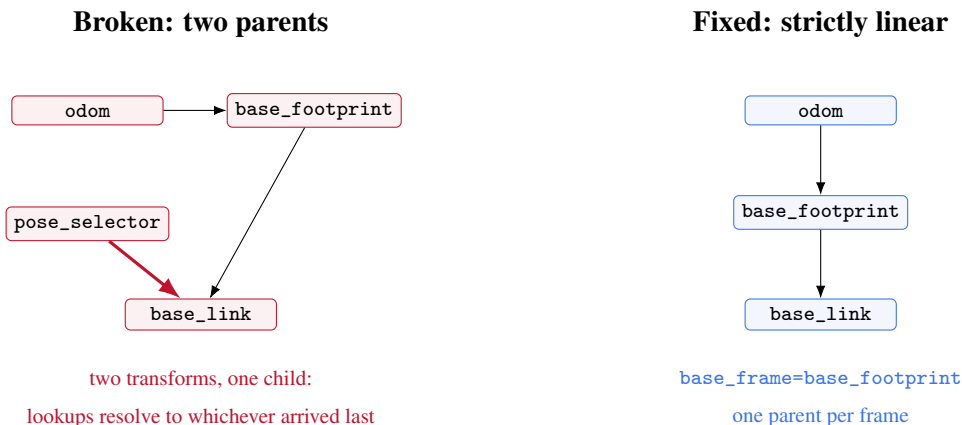


Figure D.4: The fault is structural, not numerical. On the left, `base_link` has two parents, so every lookup silently returns one of two coherent answers. Nothing errors; the pose simply alternates. On the right, the single-line change that restores a well-formed tree.

This bit twice on July 30. First, `pose_selector` was publishing with `base_frame` set to `base_link` while the URDF chain already supplied `base_footprint → base_link`, giving `base_link` two parents. The fix

is one line in `navigation_supervision.launch`:

```
1 <param name="base_frame" value="base_footprint"/>
```

which restores a strictly linear `odom` → `base_footprint` → `base_link` chain. Second, an early revision of `carolus_tf_bridge.py` published the full body chain in the beacon frame, re-parenting `base_link` a second time; it now publishes only `camera_frame` → `carolus_raw` plus the static anchor, and hides the body chain behind a parameter that defaults to `false`.

Check it in one command:

```
1 rosrun tf view_frames && evince frames.pdf
2 # or, faster: any frame appearing twice as a child is a fault
3 rosrun tf tf_monitor 2>&1 | head -30
```

Expect one parent per frame and no `TF_OLD_DATA` in the logs. Do this before trusting any pose, because the failure is invisible in the numbers: they stay plausible.

D.13.11 The differential-drive angular multiplier, and where it is not

The firmware applies a scalar to every angular velocity command it receives:

$$\omega_{\text{applied}} = k_{\omega} \cdot \omega_{\text{commanded}}, \quad k_{\omega} = 1.76. \quad (\text{D.1})$$

Together with the wheel radius (0.0625 m) and separation (0.358 m), this is what converts `/cmd_vel` into wheel setpoints, so it sits directly in the yaw-error path: a rover that consistently over- or under-rotates by a fixed ratio is far more likely to be showing this multiplier than a sensor fault. Read the live values: never the file:

```
1 rosparam get /firmware/diff_drive/angular_velocity_multiplier # 1.76
2 rosparam get /firmware/diff_drive/wheel_radius # 0.0625
3 rosparam get /firmware/diff_drive/wheel_separation # 0.358
```

Two ways to look for this value and find the wrong answer

The file is `/etc/ros/params.yaml`: *params*, plural; there is no `param.yaml`. And inside it, the entire `diff_drive` block including `angular_velocity_multiplier`: 1.76 is **commented out**. Those lines document the firmware defaults; they do not set them. A reader who greps the file finds the value inert and may conclude it is not applied, when in fact 1.76 is live because it is the firmware default. Conversely, editing the commented line changes nothing until the `#` is removed *and* `sudo systemctl`

```
restart leo is run.
```

The cockpit exposes these three values read-only in the *Firmware kinematics* panel, precisely so that they are visible during a run without anyone being tempted to change them from a browser.

The order of operations for a yaw discrepancy. This is the part a successor most needs, because getting the order wrong produces a rover that appears calibrated and is not. If the robot turns by a consistently wrong amount, there are two places the discrepancy can live, and they are not interchangeable:

$$\underbrace{\omega_{\text{applied}} = k_{\omega} \omega_{\text{cmd}}}_{\text{what the robot \textit{does}}} \quad \text{versus} \quad \underbrace{\hat{\omega} = s_g \omega_{\text{true}}}_{\text{what the robot \textit{believes}}} \quad (\text{D.2})$$

The first is an actuation gain, set by $k_{\omega} = \text{angular_velocity_multiplier}$. The second is a sensing gain, set by `~gyro_scale` in the IMU sanitiser. **Correct the actuator first, always.** The reasoning is that s_g is a property of a physical sensor, so changing it to compensate for an actuation error stores a lie in the one channel every downstream estimator trusts. MINS, openVINS and the wheel odometry all consume the sanitised IMU; a wrong s_g corrupts them simultaneously and in a way no later measurement can distinguish from a real inertial defect.

1. **Measure, do not infer.** Command a known rotation, for instance 90° , and measure the physical result with a protractor or floor markings. Use a deliberately unround target such as 87.5° : a round number invites the operator to report the number they commanded rather than the one they observed.
2. **Separate the two gains with one extra observation.** Compare the commanded angle, the physically observed angle, and the angle the gyroscope reported. If the robot turned too far *and* the gyroscope agrees with the floor, the fault is in k_{ω} . If the robot turned correctly but the gyroscope disagrees, the fault is in s_g . Only the second case justifies touching the sensor.
3. **Adjust k_{ω} multiplicatively:** $k_{\omega}^{\text{new}} = k_{\omega} \cdot \theta_{\text{target}} / \theta_{\text{observed}}$, then re-measure.
4. **Leave `gyro_scale` at 1.0** until the ground-truth calibration of §D.11.3 has settled the outstanding 7–9% rotation discrepancy. Until that question is answered, any change to s_g is a guess dressed as a correction.

A worked cautionary case is in §12.14.4: an early calibration attempt compared the angle the robot was *told* to turn against a single entered number, and so measured how badly the platform tracks its own command rather than how badly its sensors track reality. Those are different quantities and only the second is a calibration.

A third gain, easy to forget

A yaw error can also come from the wheel geometry itself, since ω depends on the effective radius and the track width. Item 1 of the open-items registry records a measured 1.65 % right/left effective-radius bias, currently masked by a software yaw trim rather than corrected. If k_ω needs an implausibly large change to fit, suspect the geometry before accepting the number.

D.13.12 A TLS port that stays open and stops listening

Symptom: the page loads, the layout is perfect, and every button is inert

After a ROS master restart, the tunnel's TLS port 9443 can remain *open*, it accepts the TCP connection and completes the WebSocket handshake, while no longer being bound to a live rosbridge behind it. Nothing in the browser reports an error. The interface looks connected because, at the socket level, it is.

Recovery, in order of cost: reload the page; if that fails, connect to the local port 9090 instead, which does not traverse the tunnel; if that also fails, the fault is upstream of the transport and you should stop looking at the network.

Diagnosis before you touch anything. Two independent checks separate “the page is broken” from “the bus is quiet”, and they are worth running in that order:

```
1 # 1. Does the browser's relay carry ROS traffic at all?
2 #     Use a topic that is ALWAYS busy. /mission/telemetry publishes ~4 Hz
3
4 rostopic hz /mission/telemetry
5
6 # 2. Does a command sent from the page reach the bus?
7 rostopic echo /mission/command
```

The first check matters more than it looks. During this session the relay was declared broken on the evidence of an idle `/mission/log`, which publishes only when something happens: an empty topic proved nothing, and the same test on `/mission/telemetry` showed 50 frames in 12 s. A silent topic is not a dead link. Choose a topic that is supposed to be noisy before concluding anything.

D.13.13 A checklist before any measurement campaign

1. `vcgencmd measure_temp` on the robot, below 80°C, and `throttled=0x0`.
2. `tail logs/watchdog.log`, no automatic restarts in the last hour.

3. IMU gap count over 20 s, zero.
4. `rostopic hz` on the four key topics, all at nominal rate.
5. `ps -o etime= -p $(pgrep -f run_subscribe_msckf)`, openVINS older than the intended run duration, i.e. it has not respawned.
6. Battery on mains, or voltage noted; yaw performance depends on it.

If any of these fails, the data you are about to record will not survive scrutiny. The single most expensive lesson of this campaign is that a measurement taken on an unstable stack is worse than no measurement, because it is believed.

The sections above are the operating manual. Those that follow are the technical reference: where the source comes from, how it is built, and how each configuration tree is organised.

D.14 Where to go next

The platform is stable and instrumented; what it lacks is not more code. This is the author's ordered recommendation, with the reasoning, so you can disagree with it deliberately rather than by accident.

D.14.1 Priority 1: re-calibrate the colour camera intrinsics

Cheapest item on this list, and it blocks the headline deliverable. The detector runs with intrinsics calibrated at 1280×720 against a 640×480 stream (§12.15.3); the stored principal point, $c_x = 643.47$, lies outside a 640-pixel-wide image. A beacon at the true image centre is reported 26.7° off-axis. No rescaling repairs this, because the two configurations differ in field of view and not merely in sampling.

Run the calibration at 640×480 , the resolution actually published, and load the result into `carolus_astrobeerex`. Until then, treat every range, bearing and beacon position from this camera as a relative indicator only. **Do not publish a dimensional metric derived from it, including the absolute error of Priority 4.**

D.14.2 Priority 2: install active cooling

Not software. The Pi sits at 86°C and is frequency-capped to 600 MHz whenever it is working, and that single fact taints every estimator measurement on the platform (§D.13.1).

An earlier edition of this appendix stated that the software levers were exhausted, on the grounds that disabling the colour stream gave zero thermal improvement. That measurement was invalid: colour was already disabled at driver level, so the test compared a configuration with itself. The colour nodelet was subsequently measured at 185 % CPU (§12.15.1), making it the dominant thermal term. The lever exists, but spending it costs the working-distance beacon detection, which is why the July 30 session chose to keep colour and accept the risk instead. The measured envelope is roughly three degrees wide: the serial link holds at 86°C and collapses at 89°C. A fan is what buys back that margin without giving up colour.

The experiment to run immediately after fitting a fan is the A/B that was never possible: count IMU gaps over 20 s before and after, and count `Propagator.cpp:101` aborts over a ten-minute run. If the gaps go to zero, a whole class of “estimator bug” disappears, and several conclusions in Chapter 12 should be re-measured on the healthier machine.

D.14.3 Priority 3: finish the ground-truth calibration

Three passes per direction, with the floor protocol, and settle the question posed in §D.11.3: is the 7–9% rotation discrepancy in the sensors or in the reference measurement? This is cheap, needs no hardware, and is the only measurement on the platform anchored to physical reality.

Until it is settled, `~gyro_scale` must stay at 1.0.

D.14.4 Priority 4: get an absolute reference into a driven run

Every number in Chapter 12 except the tape-measure calibration is *mutual divergence*: it says whether MINS and openVINS agree, never which is right. The Carolus beacon is the only drift-free anchor available, and it was in view during neither of the two comparison runs.

Drive a loop with the beacon visible. Nothing needs to be written: `plot_trajectories.m` already computes each estimator’s error against the Carolus fix the moment a `_carolus.csv` exists. That single run converts the whole chapter from relative to absolute.

Since July 30 there is a better path than a post-hoc CSV. `carolus_tf_bridge.py` (§12.15.2) publishes `/odom_in_beacon` and `/carolus_in_beacon`, both carrying the robot’s pose in the beacon frame by two independent routes, odometric and optical. Their instantaneous difference *is* the absolute error, with no Kabsch or Umeyama fit in between, so nothing is absorbed into a fitted transform or a scale parameter. Record both topics in the bag.

Two cautions. The bridge is deliberately in no launch file: start it by hand, so it can never perturb a run

you did not intend it to touch. And it is gated on Priority 1: with the present intrinsics its output is self-consistent and metrically meaningless, which is the most dangerous combination a measurement can have, because it looks like data.

D.14.5 Priority 5: the Kalibr camera–IMU calibration

Item 2 of the open-items registry, never completed, blocked on producing $\gtrsim 1.5$ rad/s of roll and pitch excitation with a rover that has only ever managed 0.08. It requires lifting and tilting the robot by hand in front of the target. Do it after the three above, because a thermally throttled Pi will drop frames during the very manoeuvre the calibration depends on.

D.14.6 Open questions the author did not resolve

Recorded honestly, because a successor deserves the loose ends rather than a tidy narrative:

- `firmware_message_converter` **publishes an altered gyro bias**. It reports 6.867 where its own source reads 3.668. No estimator consumes that topic, so it was never chased, but it is a real defect and suggests something wrong in the conversion path.
- ~ 20000 **serial checksum errors** were logged on the Pi before the real-time fix. The fix cured the *rate* collapse; whether it cured the checksum errors entirely was never re-measured.
- **openVINS's scale disagrees with itself between runs**. One run gave a trajectory 10% *shorter* than MINS; a different run gave a straight-line distance $10.5\times$ *longer* than reality. Both were measured properly. This inconsistency is unexplained and is the most interesting open question in the estimator work.
- **The wheel odometry's 4.3% left/right asymmetry** is characterised but not corrected. It is mechanical, so it needs a mechanical or model-level answer, not a gain.
- **MATLAB's batch licensing** was failing at the time of writing (error 5001), which is why the per-panel figures in this appendix's companion chapter were produced by cropping rather than re-rendering. Purely an account/network issue, but it blocks the analysis pipeline until fixed.

D.14.7 Habits worth inheriting

1. **Measure before believing, and measure the thing itself**. Almost every wrong conclusion in this project came from measuring a proxy: arrival time instead of the timestamp, cumulative path instead of net

displacement, the median instead of the tail.

2. **Write down why, not what.** Every script here has a header explaining the defect it works around. Six weeks later that comment is the only thing that distinguishes a deliberate workaround from an accident.
3. **A measurement taken on an unstable system is worse than no measurement,** because it will be believed. The watchdog episode (§D.13.3) invalidated two days of estimator conclusions.
4. **Fix inputs, not outputs.** The one time this project filtered an estimator's output instead of its inputs, the filter became the fault (§D.13.6).
5. **Keep the fallback path working.** Every convenience here has a manual equivalent, the cockpit calibration panel has `tools/calib_terrain.py`, the one-click export has `bag_to_csv.py`. When the convenient path breaks mid-session, and it will, the campaign survives.

References

- [1] W. Lee, P. Geneva, C. Chen, and G. Huang, “MINS: Efficient and robust multisensor-aided inertial navigation system,” *arXiv preprint arXiv:2309.15390*, 2023.
- [2] W. Lee, P. Geneva, C. Chen, and G. Huang, “MINS: Efficient and robust multisensor-aided inertial navigation system,” *Journal of Field Robotics*, 2025. DOI: 10.1002/rob.22546
- [3] P. Geneva, K. Eickenhoff, W. Lee, Y. Yang, and G. Huang, “OpenVINS: A research platform for visual-inertial estimation,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Paris, France, 2020. DOI: 10.1109/ICRA40945.2020.9196524
- [4] A. I. Mourikis and S. I. Roumeliotis, “A multi-state constraint Kalman filter for vision-aided inertial navigation,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Rome, Italy, 2007, pp. 3565–3572. DOI: 10.1109/ROBOT.2007.364024
- [5] P. Furgale, J. Rehder, and R. Siegwart, “Unified temporal and spatial calibration for multi-sensor systems,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Tokyo, Japan, 2013, pp. 1280–1286. DOI: 10.1109/IROS.2013.6696514
- [6] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [7] M. Quigley et al., “ROS: An open-source Robot Operating System,” in *Proc. ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.
- [8] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, “Rosbridge: ROS for non-ROS users,” in *Proc. International Symposium on Experimental Robotics (ISER)*, Quebec City, Canada, 2012.
- [9] Open Robotics, “ROS noetic ninjemys release notes and migration guide,” Open Robotics, Tech. Rep., May 2020. [Online]. Available: <https://www.ros.org/reps/rep-0003.html>

- [10] Open Robotics, *Rosbridge protocol specification v2.0*, 2013. [Online]. Available: https://github.com/RobotWebTools/rosbridge_suite
- [11] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [12] W. Niblack, *An Introduction to Digital Image Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1986, pp. 115–116.
- [13] J. Sauvola and M. Pietikäinen, “Adaptive document image binarization,” *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000.
- [14] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Kauai, HI, 2001, pp. 511–518.
- [15] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000. [Online]. Available: <https://opencv.org>
- [16] OpenCV Contributors, *findChessboardCornersSB: Chessboard detection using a sub-pixel befitting algorithm*, OpenCV 4.x Documentation, 2023. [Online]. Available: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [17] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.
- [18] G. Matheron, *Random Sets and Integral Geometry*. New York: Wiley, 1975.
- [19] J. Serra, *Image Analysis and Mathematical Morphology*. London: Academic Press, 1982.
- [20] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [21] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras,” *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

- [22] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [23] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, Shanghai, China, 2011, pp. 3400–3407.
- [24] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [25] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, Portland, OR, 1996, pp. 226–231.
- [26] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [27] N. Wiener, *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. Cambridge, MA: MIT Press, 1949.
- [28] X. Xu and V. O. K. Li, “Enhanced cascade implementation of the exponential moving average filter,” *IEEE Signal Processing Letters*, vol. 14, no. 10, pp. 657–660, 2007.
- [29] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
- [30] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Transactions of the ASME*, vol. 64, pp. 759–768, 1942.
- [31] G. Zames, “On the input-output stability of time-varying nonlinear feedback systems,” *IEEE Transactions on Automatic Control*, vol. 11, no. 2, pp. 228–238, 1966.
- [32] R. Alur and D. L. Dill, “A theory of timed automata,” *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.
- [33] International Electrotechnical Commission, “Functional safety of electrical/electronic/programmable electronic safety-related systems,” IEC, Geneva, Switzerland, Tech. Rep. IEC 61508, 2010.
- [34] National Aeronautics and Space Administration, “Software safety standard,” NASA, Washington, D.C., Tech. Rep. NASA-STD-8719.13C, Jul. 2013.

- [35] ISA, “Management of alarm systems for the process industries,” ISA, Research Triangle Park, NC, Tech. Rep. ISA-18.2, 2016.
- [36] D. P. Sobczak, *Fault tree analysis*, Bell Telephone Laboratories Memorandum; reprinted in “FTA Handbook with Aerospace Applications,” NASA, 2002, 1962.
- [37] M. R. Endsley, “Design and evaluation for situation awareness enhancement,” in *Proc. Human Factors Society Annual Meeting*, vol. 32, 1988, pp. 97–101.
- [38] M. R. Endsley, “Toward a theory of situation awareness in dynamic systems,” *Human Factors*, vol. 37, no. 1, pp. 32–64, 1995.
- [39] F. T. Sanders and E. J. McCormick, *Human Factors in Engineering and Design*, 7th. New York: McGraw-Hill, 1993.
- [40] I. Fette and A. Melnikov, *The WebSocket protocol*, IETF RFC 6455, Dec. 2011. [Online]. Available: <https://tools.ietf.org/html/rfc6455>
- [41] G. Bianchi, “Performance analysis of the IEEE 802.11 distributed coordination function,” *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, 2000.
- [42] World Wide Web Consortium (W3C), *Web content accessibility guidelines (wcag) 2.1*, W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [43] W3C CSS Working Group, *CSS filter effects module level 2: Backdrop-filter property*, W3C Working Draft, 2023. [Online]. Available: <https://www.w3.org/TR/filter-effects-2/>
- [44] Husarion, *LEO rover technical documentation*, Warsaw, Poland, 2021. [Online]. Available: <https://docs.leorover.tech>
- [45] Intel Corporation, *Intel RealSense depth camera D400 series product family datasheet*, Santa Clara, CA, 2021. [Online]. Available: <https://www.intelrealsense.com/download/20132>
- [46] AgileX Robotics, *LIMO rover ROS packages documentation*, Dongguan, China, 2021. [Online]. Available: <https://github.com/agilexrobotics>
- [47] E. L. Kaplan and P. Meier, “Nonparametric estimation from incomplete observations,” *Journal of the American Statistical Association*, vol. 53, no. 282, pp. 457–481, 1958.

- [48] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [49] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [50] R. Cabello et al., *Three.js: A JavaScript 3d library*, Open-source library, r128, 2021. [Online]. Available: <https://threejs.org>
- [51] GreenSock (GSAP), *GSAP (greensock animation platform) with scrolltrigger plugin*, 2021. [Online]. Available: <https://greensock.com/scrolltrigger/>